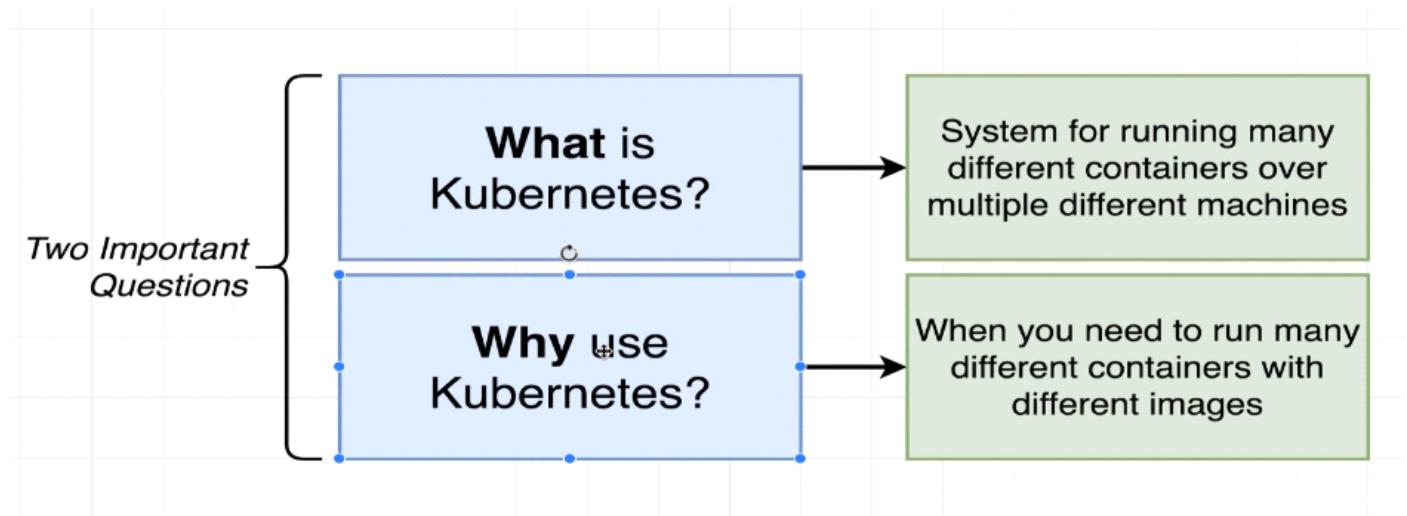


What is Kubernetes

Thursday, April 6, 2023

1:45 PM

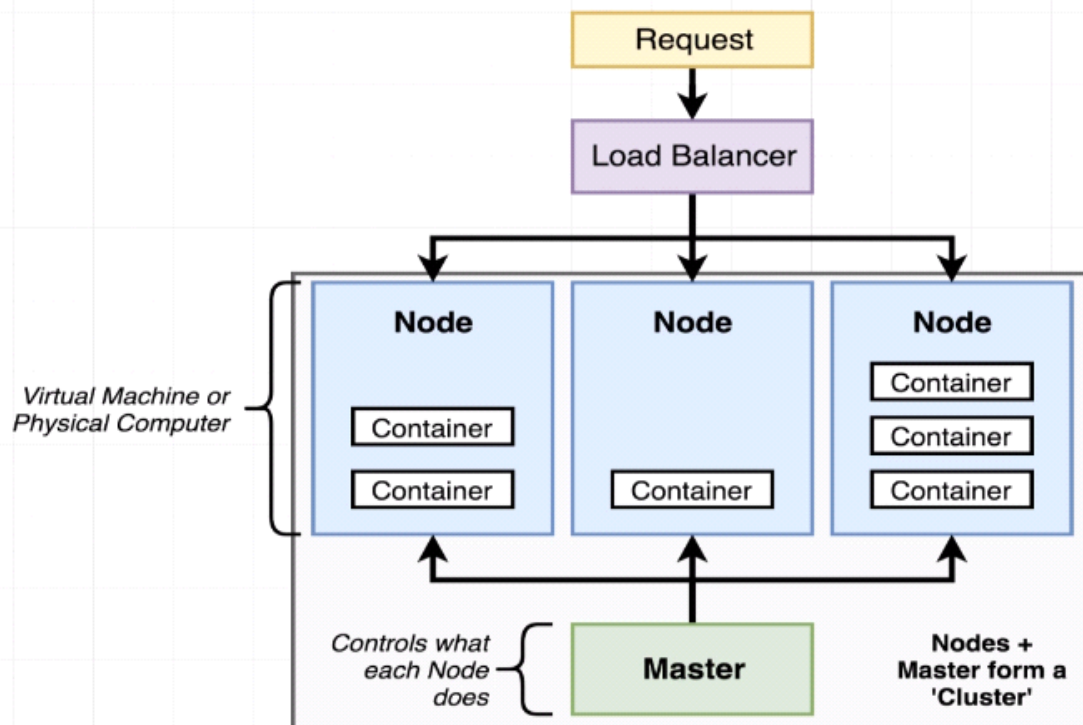
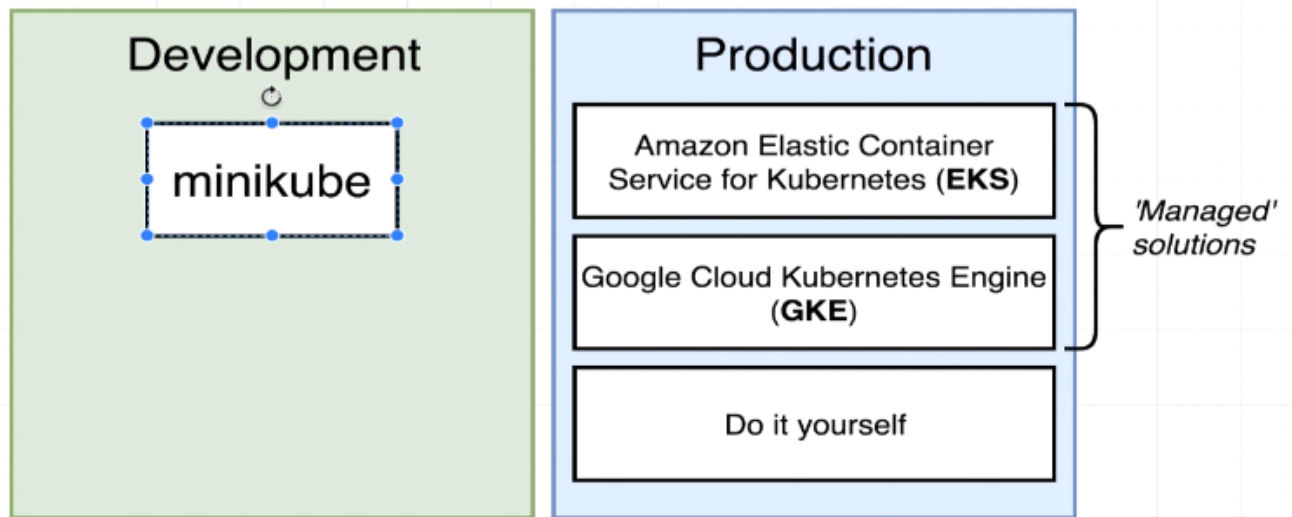


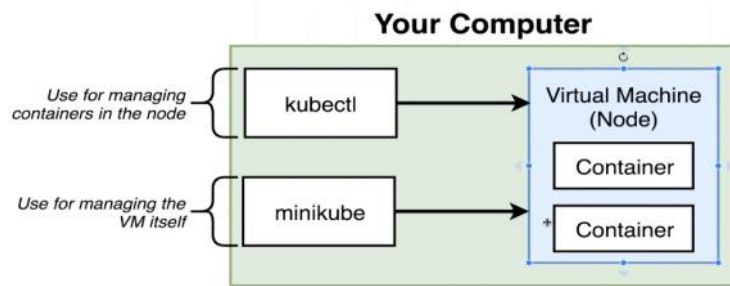
Kubernetes in Development and Production

Thursday, April 6, 2023

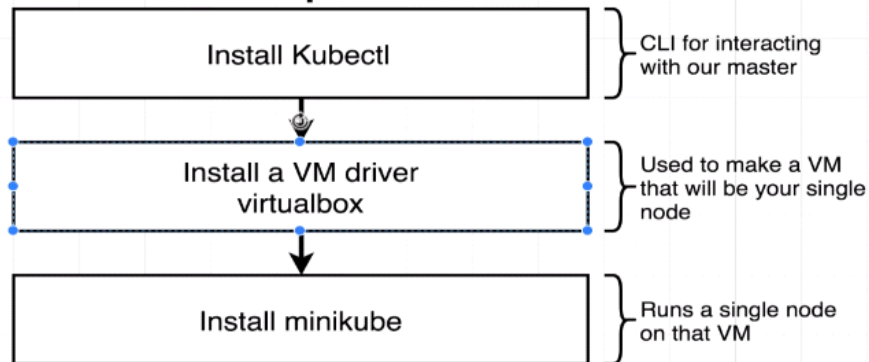
1:56 PM

Working with Kubernetes



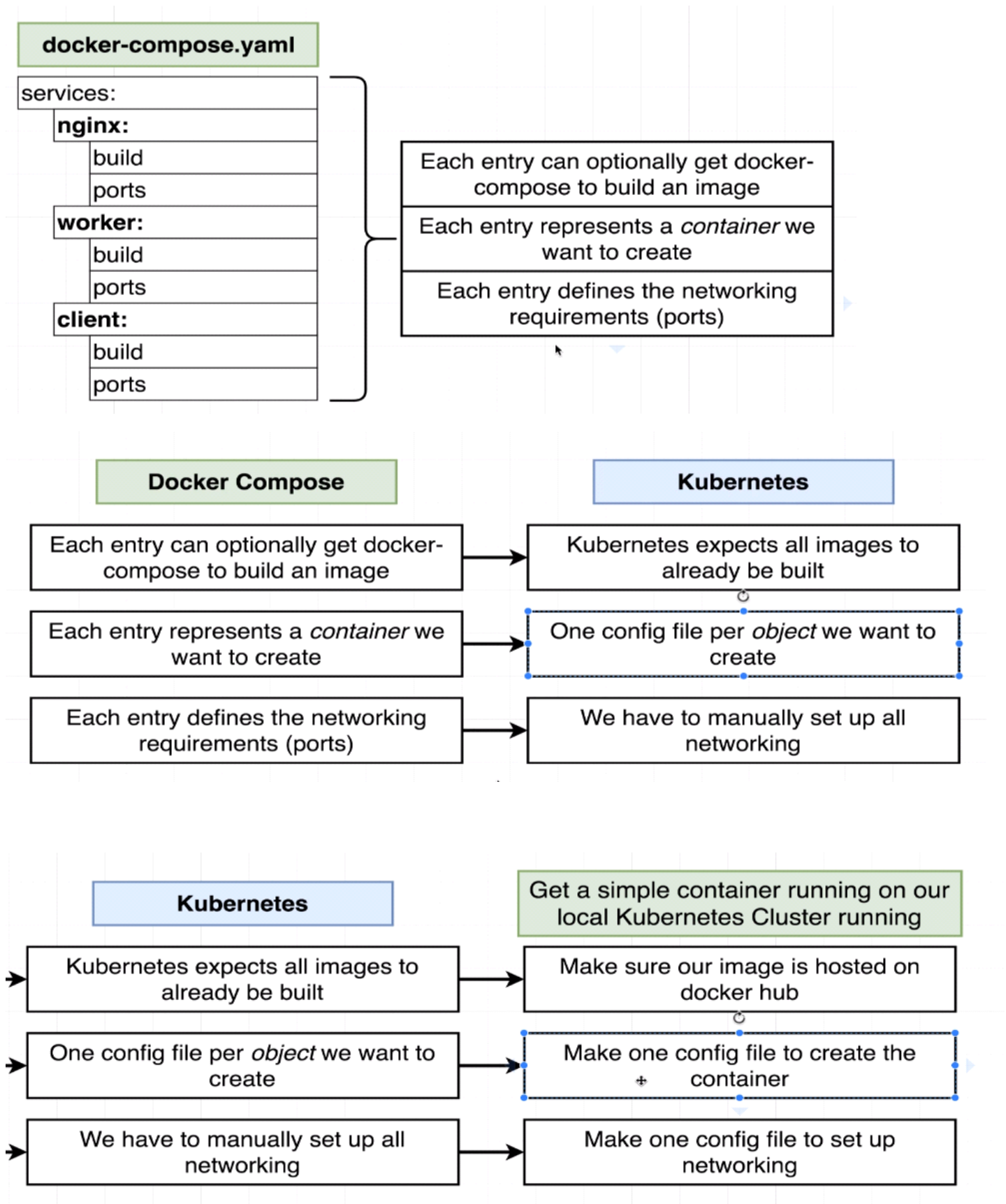


Local Kubernetes Development



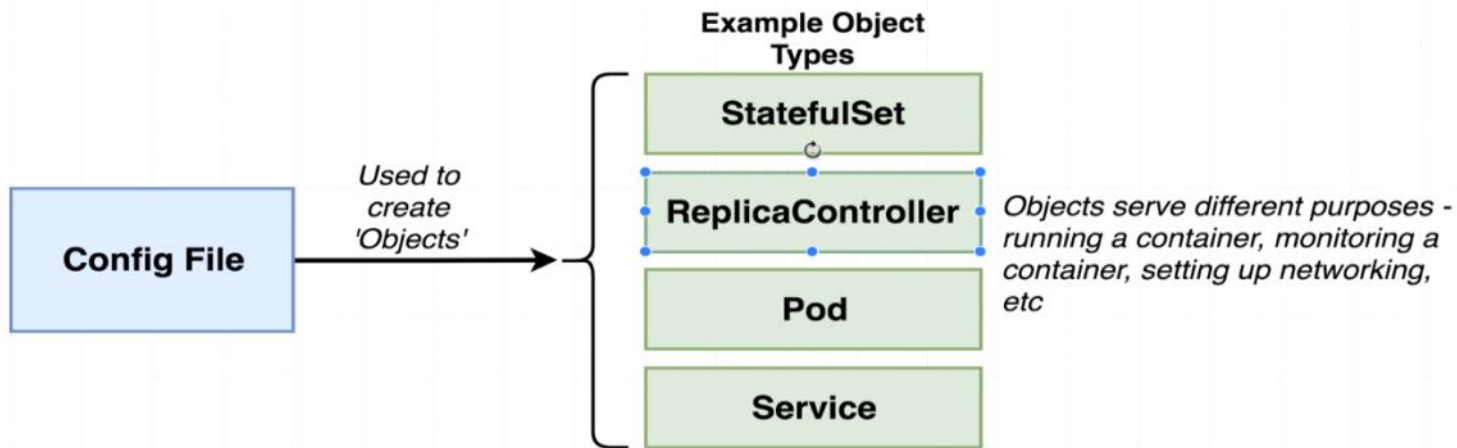
Mapping existing knowledge

Thursday, April 6, 2023 1:57 PM



Object Type and API

Thursday, April 6, 2023 2:22 PM

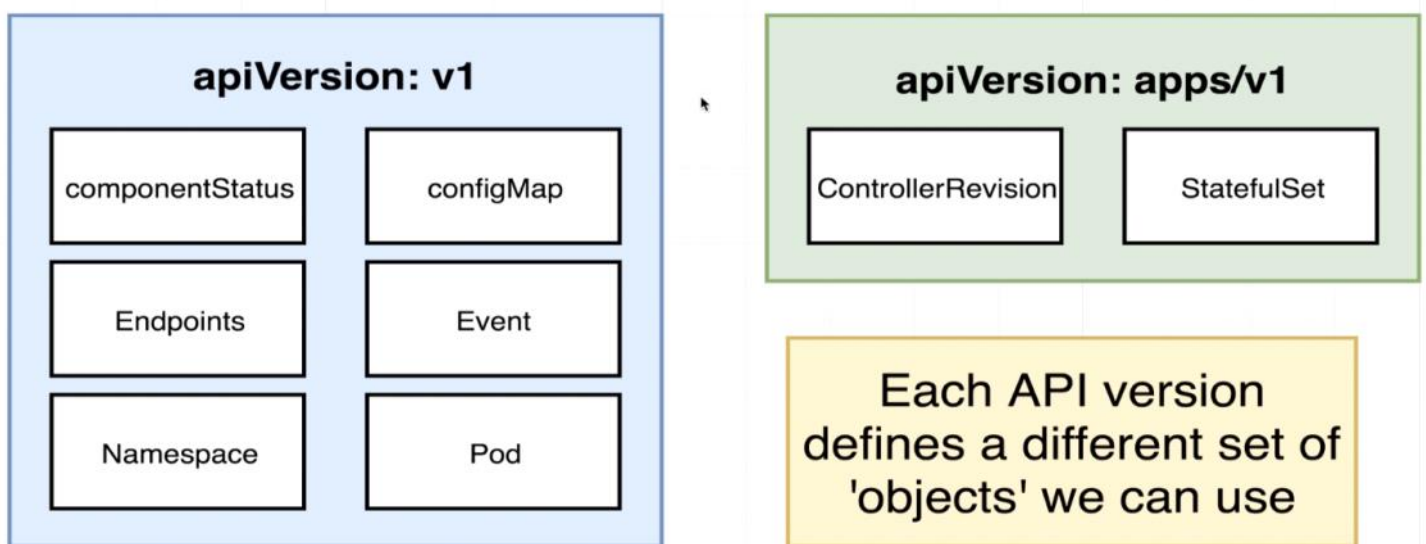


Pod to run container
Service - To setup networking
Deployments - Used to maintain sets of container

FYI,
The replication controllers helps us running multiple instance of a single pod and ensures that the specified no of pods runs al the time

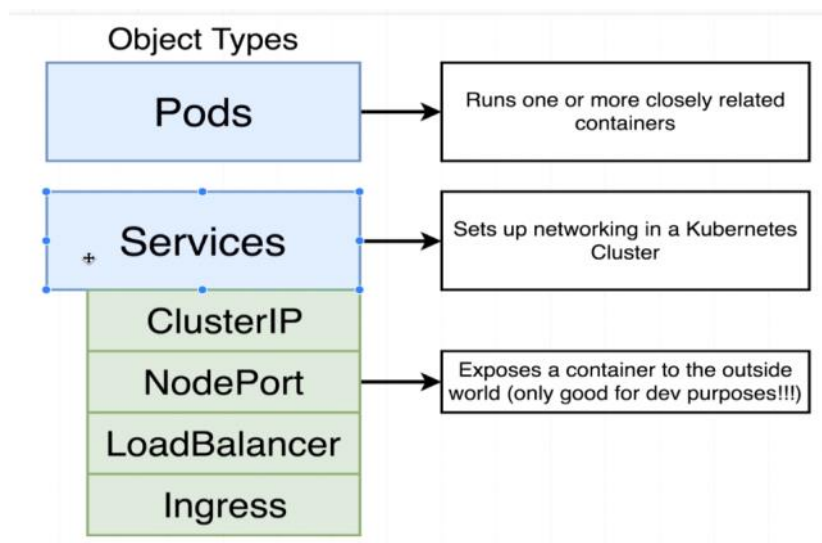
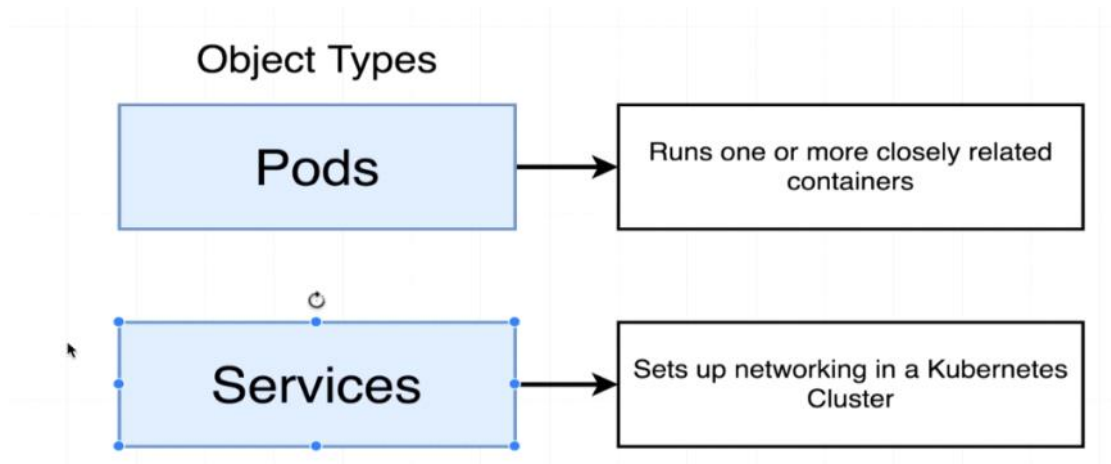
It spans across multiple nodes in a cluster

Where Replica Sets is its New version

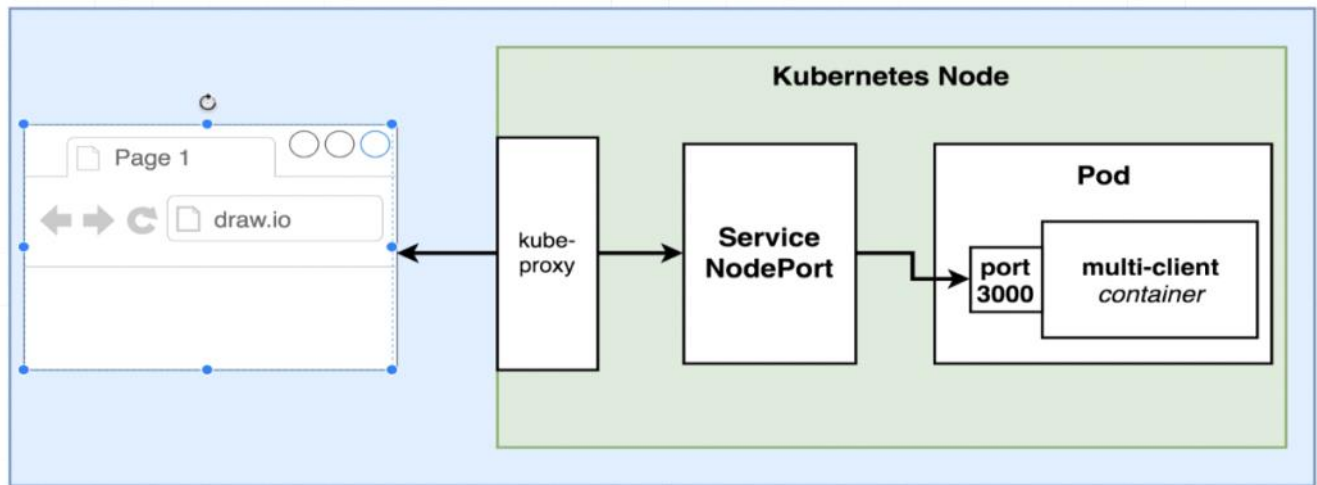


Object Type - Service - To setup networking

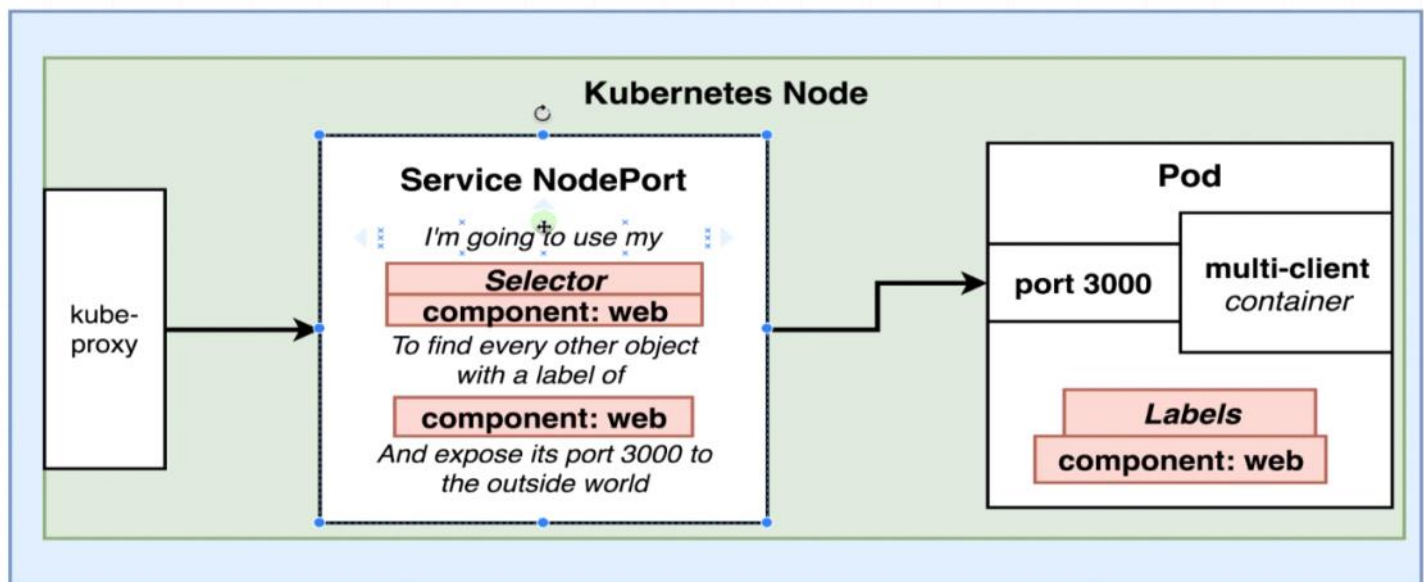
Thursday, April 6, 2023 2:22 PM

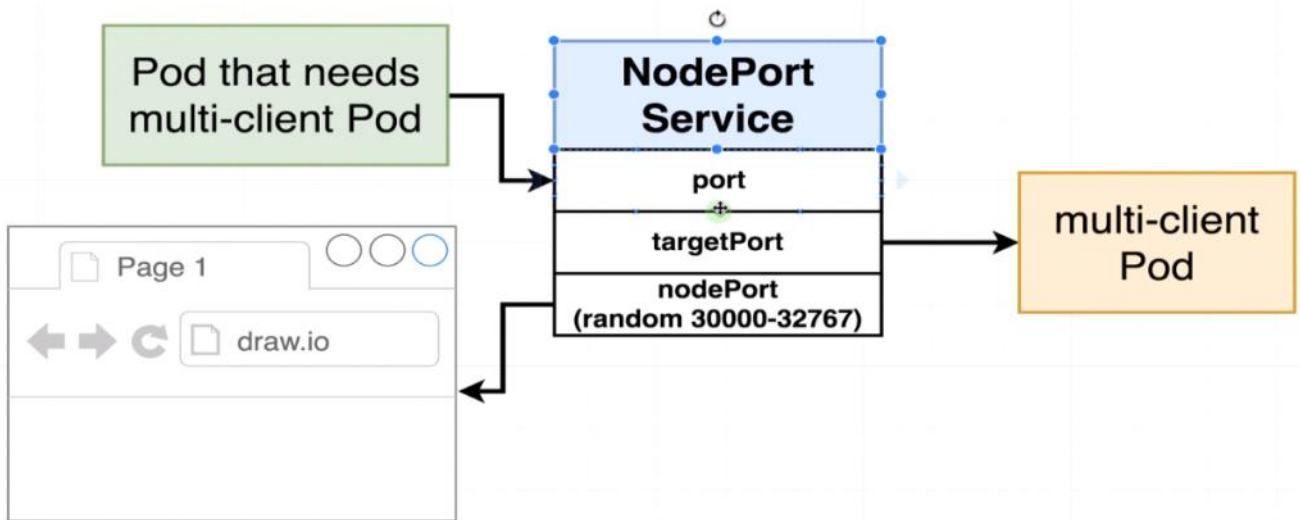


Your Computer



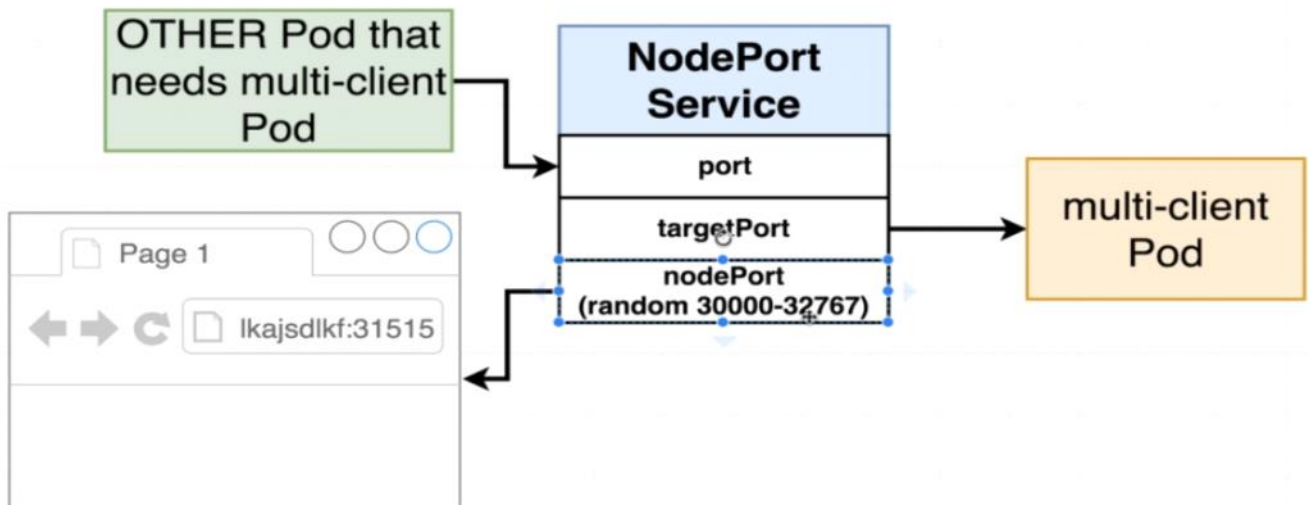
Your Computer





Target Port in Service = Container port in Pod
 Port is used for Internal connection
 Node Port is for external connection

Summary :

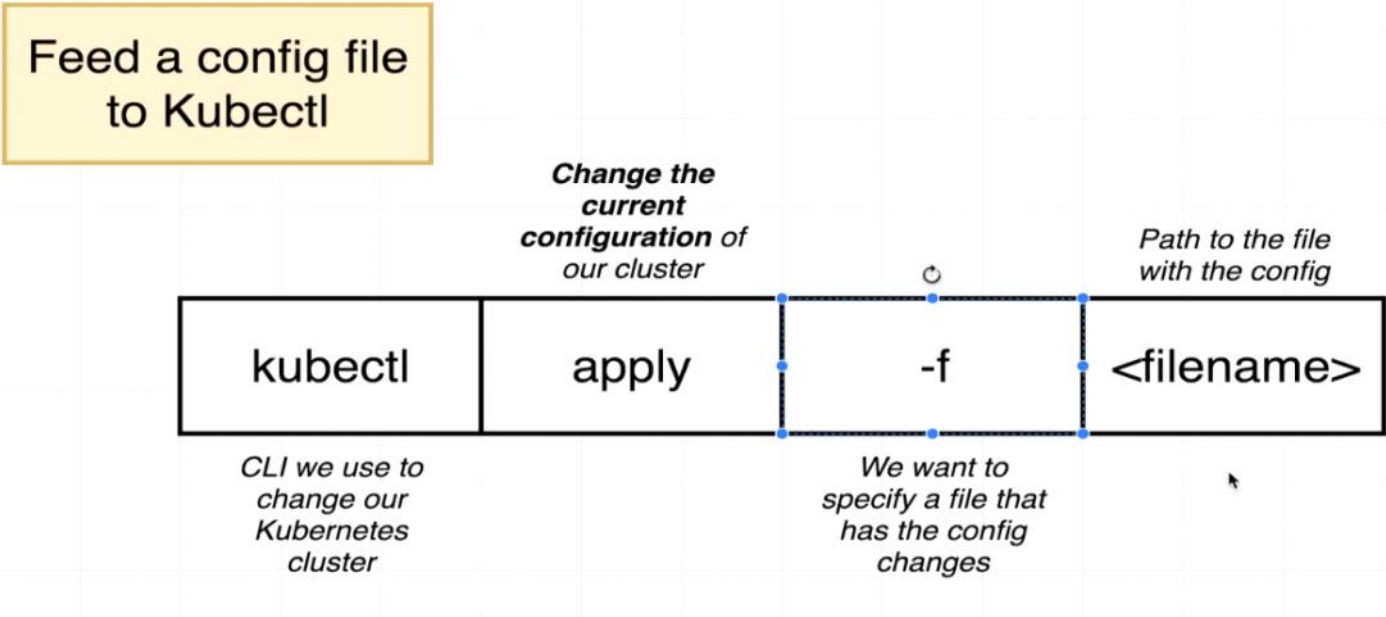


```
apiVersion: v1
kind: Service
metadata:
  name: client-node-port
spec:
  type: NodePort
  ports:
    - port: 3050
      targetPort: 3000
      nodePort: 31515
  selector:
    component: web
```

```
apiVersion: v1
kind: Pod
metadata:
  name: client-pod
  labels:
    component: web
spec:
  containers:
    - name: client
      image: stephengrider/multi-client
      ports:
        - containerPort: 3000
```

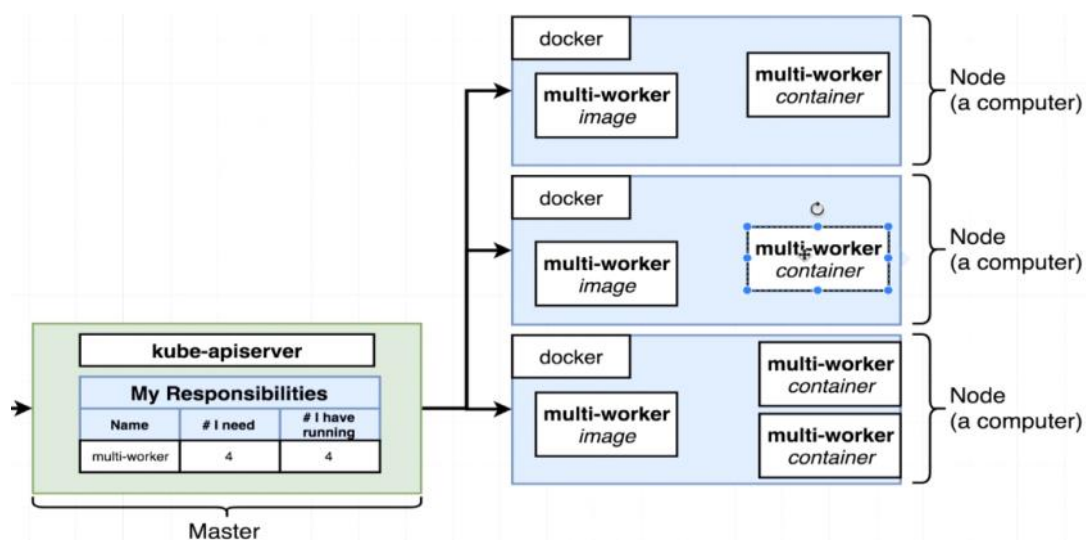
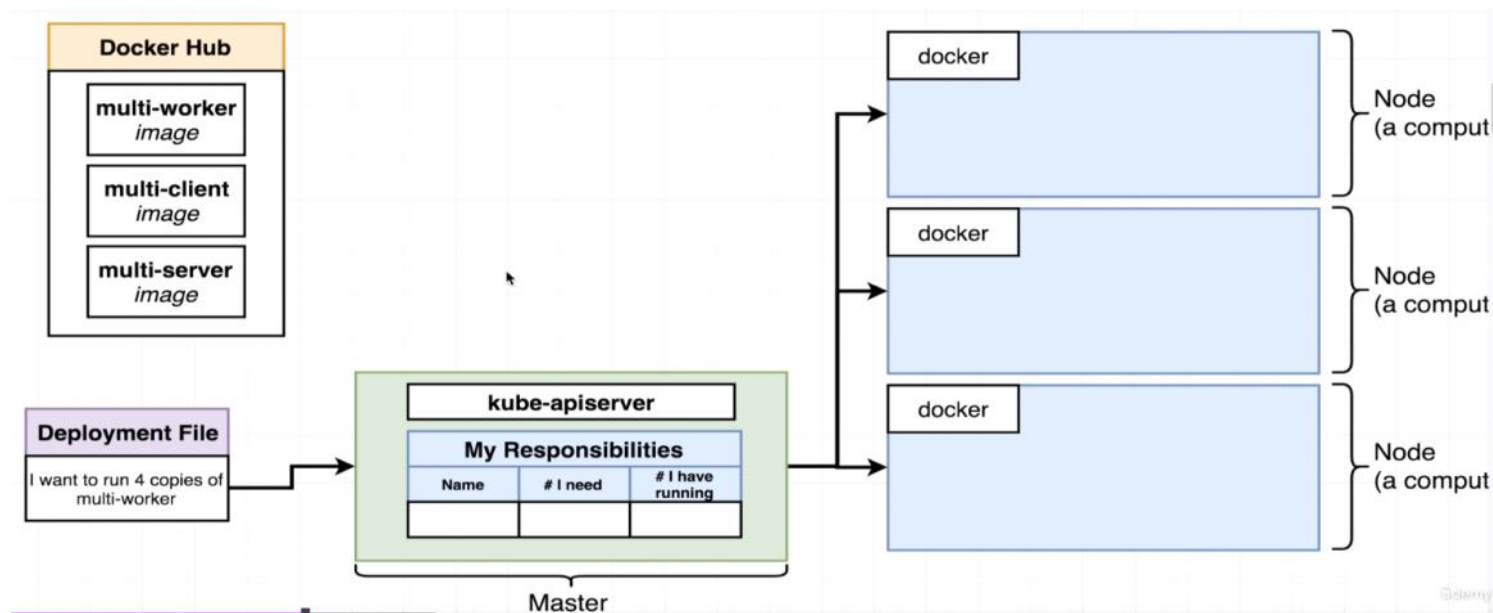

Connecting to Running container

Thursday, April 6, 2023 10:58 PM



Complete deployment flow

Thursday, April 6, 2023 11:15 PM



Imperative vs Declarative Deployments

Friday, April 7, 2023 12:46 AM

Important Takeaways

Kubernetes is a system to deploy containerized apps	The important stuff
Nodes are individual machines (or vm's) that run containers	
Masters are machines (or vm's) with a set of programs to manage nodes	
Kubernetes didn't build our images - it got them from somewhere else	
Kubernetes (the master) decided where to run each container - each node can run a dissimilar set of containers	
To deploy something, we update the desired state of the master with a config file	
The master works constantly to meet your desired state	

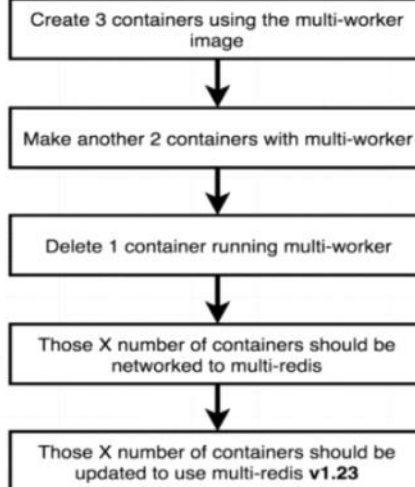
Imperative Deployments

'Do exactly these steps to arrive at this container setup'

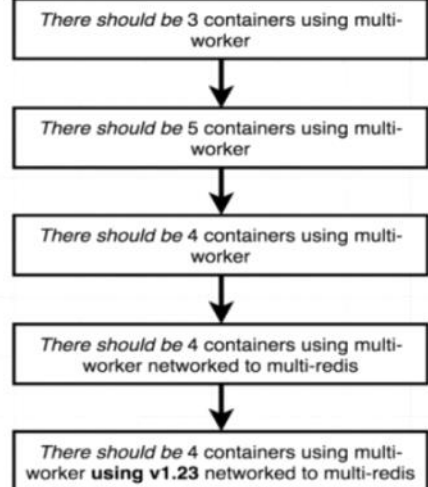
Declarative Deployments

'Our container setup should look like this, make it happen'

Imperative Approach



Declarative Approach

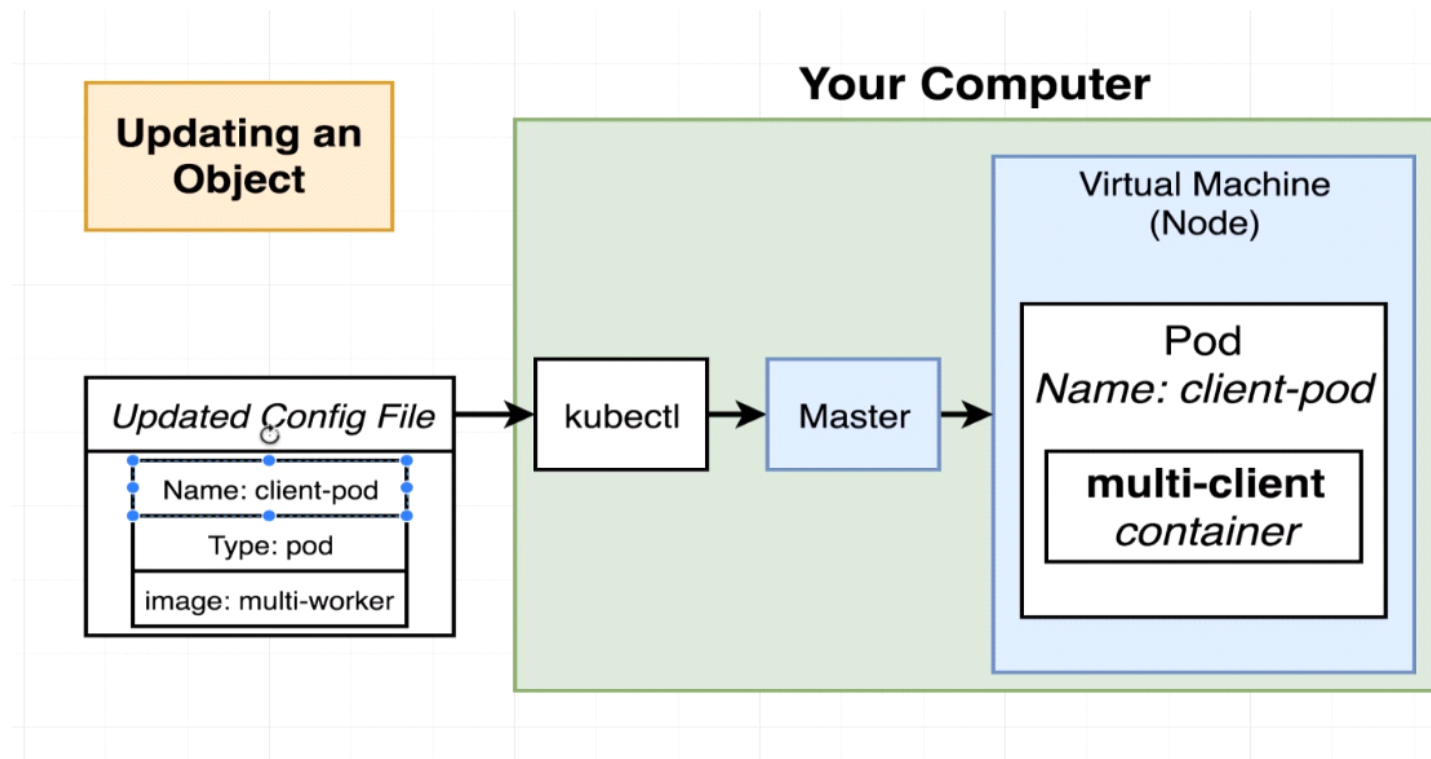


Kubernetes has commands to do things this way

Kubernetes also has commands to do things this way

Updating Existing Objects

Friday, April 7, 2023 2:20 AM



Running Containers with Deployments

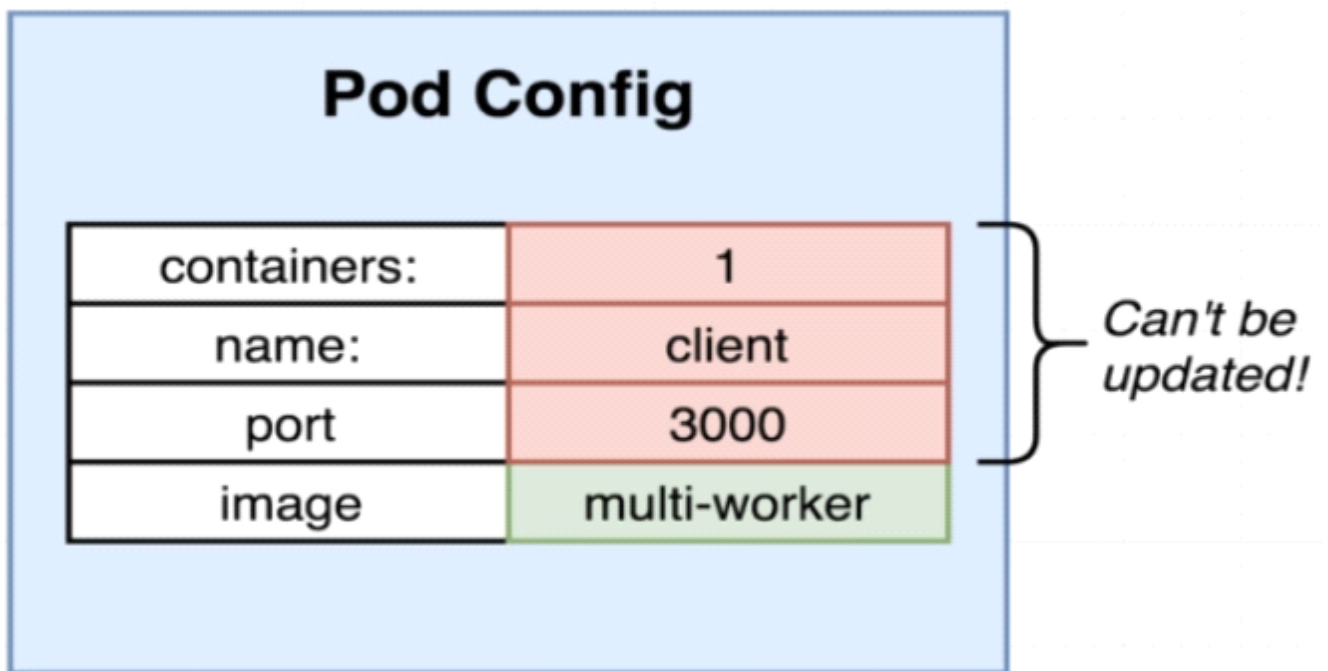
Friday, April 7, 2023 2:27 AM

Pod to run container

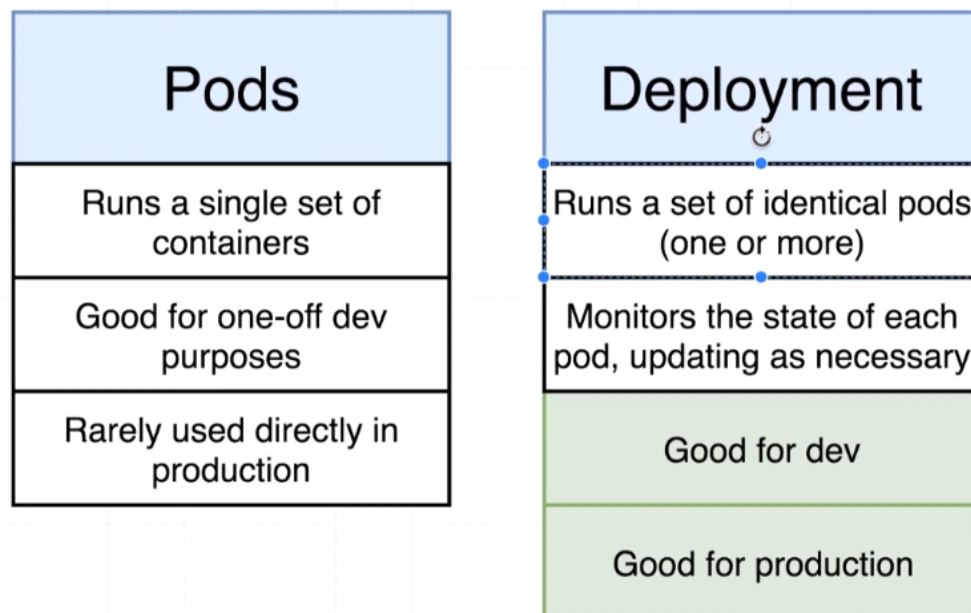
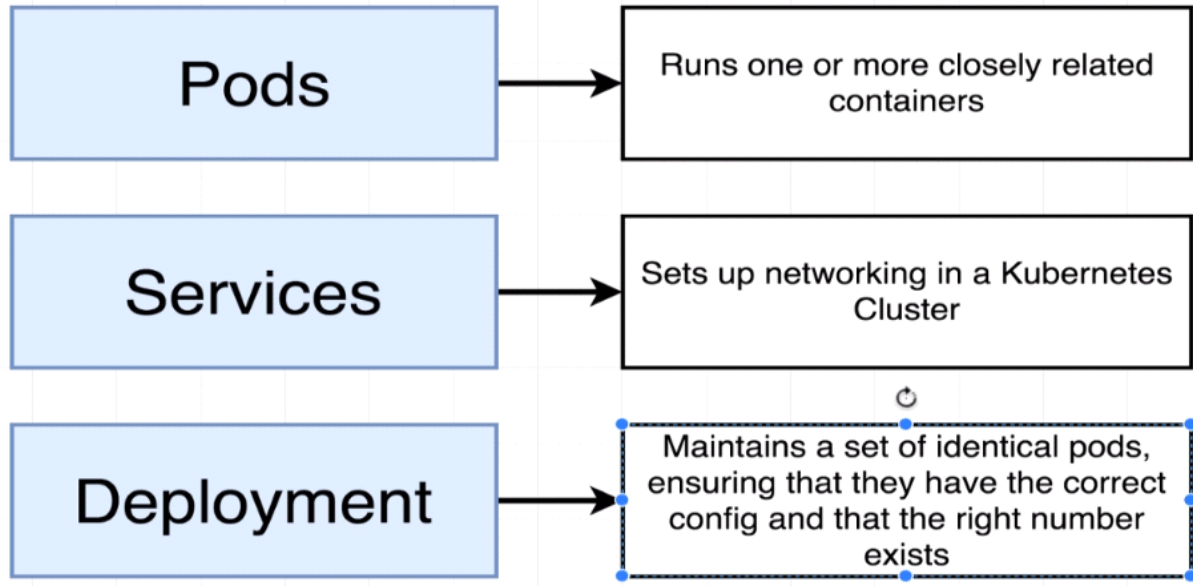
Service - To setup networking

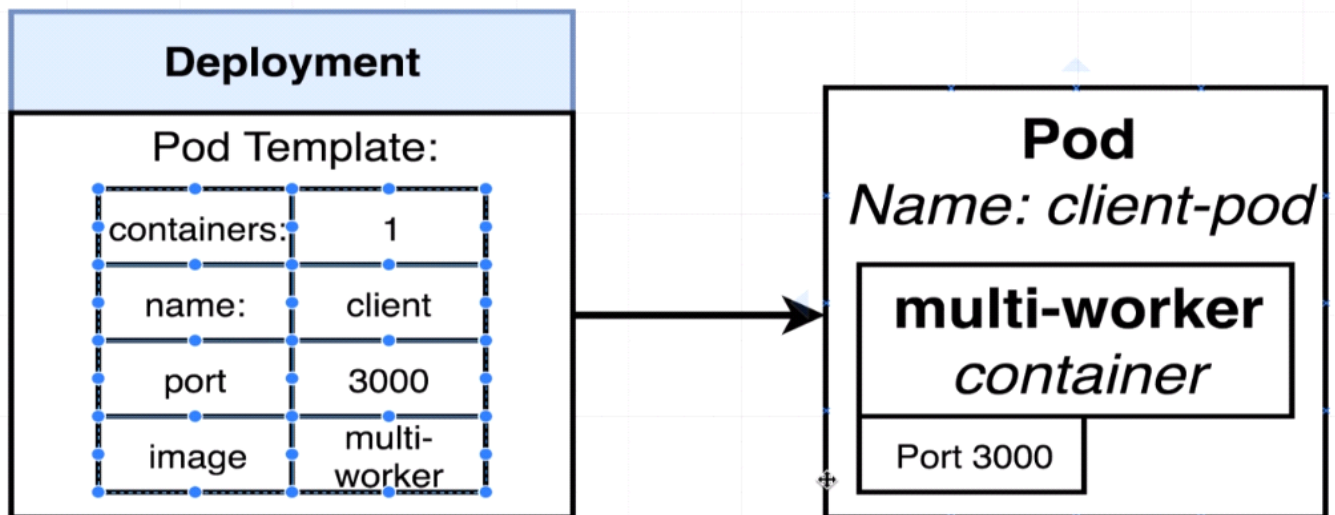
Deployments - Used to maintain sets o container

The limitation with Pods is that, you can only edit the image name but you can't edit any other configurations such as port no etc. This is also one of the reason we go for Deployments



Object Types

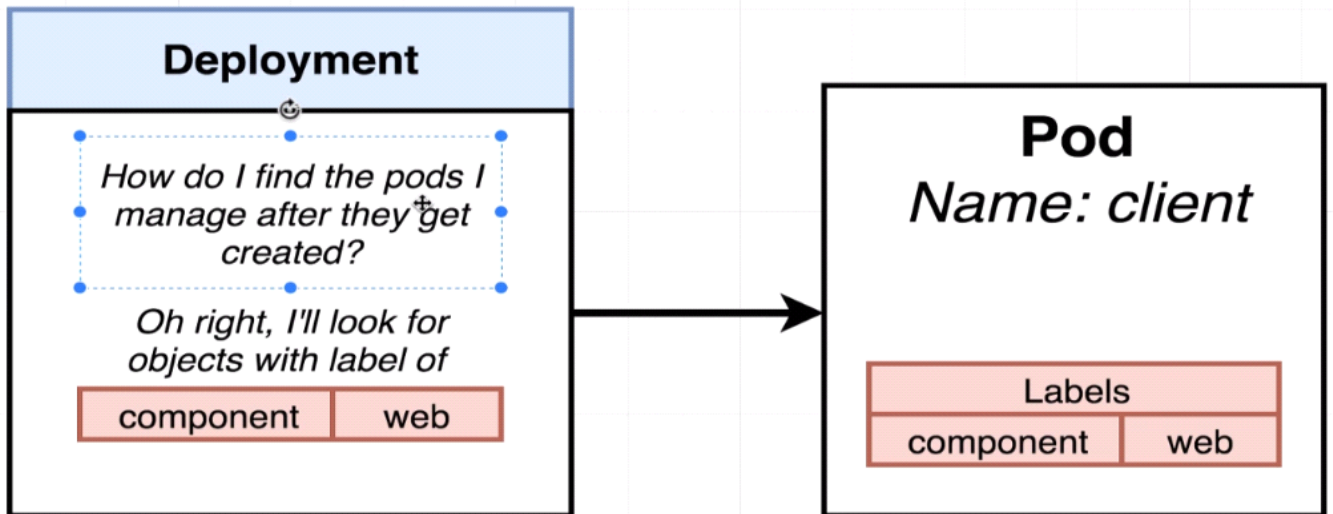




Sample deployment file: So why the need for two labels

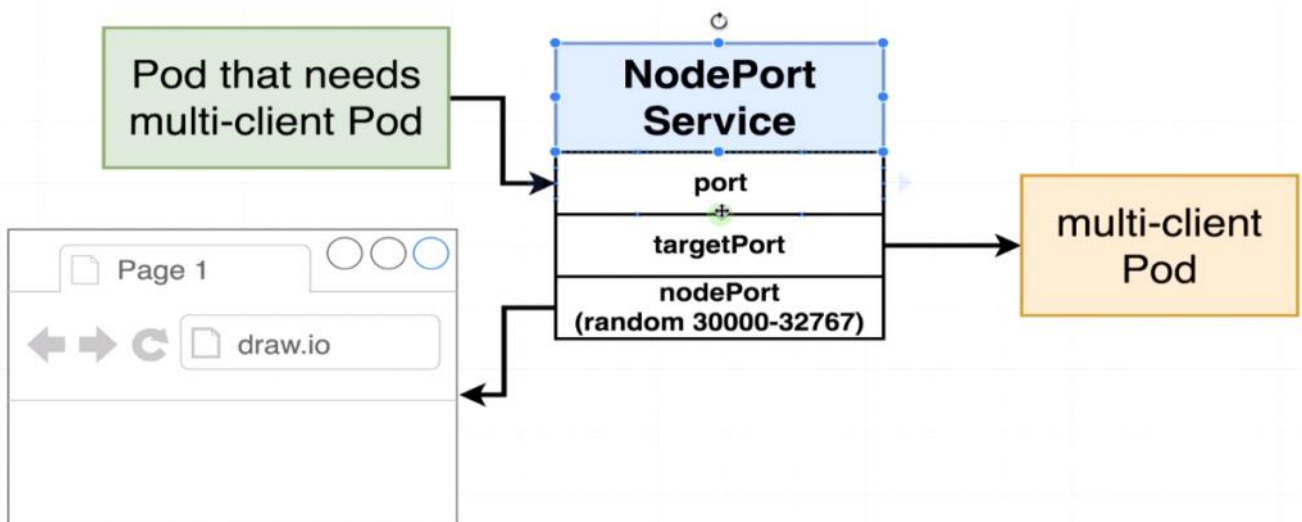
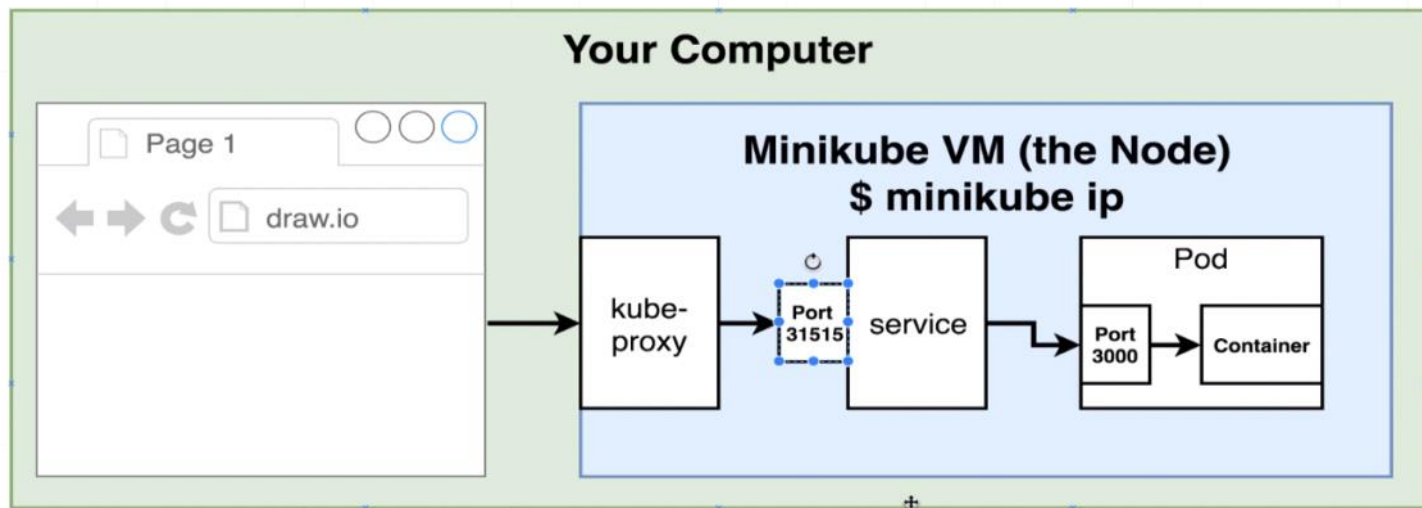
```
! client-deployment.yaml x
metadata:
  name: client-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      component: web
  template:
    metadata:
      labels:
        component: web
    spec:
      containers:
        - name: client
          image: stephengrider/multi-client
          ports:
            - containerPort: 3000
```

Basically deployment file reaches out to master for creating pods, which in turn use the deployment file to create one, and now for the Deployment to get hold of the pod, it uses the matchLabel to get the pod created



Why Use Services?

Friday, April 7, 2023 2:47 AM



Imperatively Updating a Deployment's Image

Friday, April 7, 2023 3:08 AM

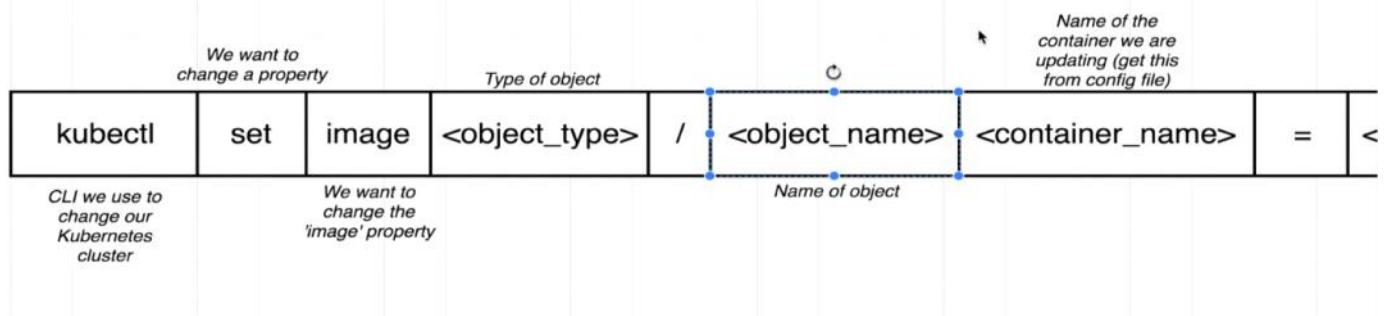
Problem

Force pods to re-pull an image without changing the image tag #33664

```
docker build -t stephengrider/multi-client:v5 .
```

```
docker push stephengrider/multi-client:v5
```

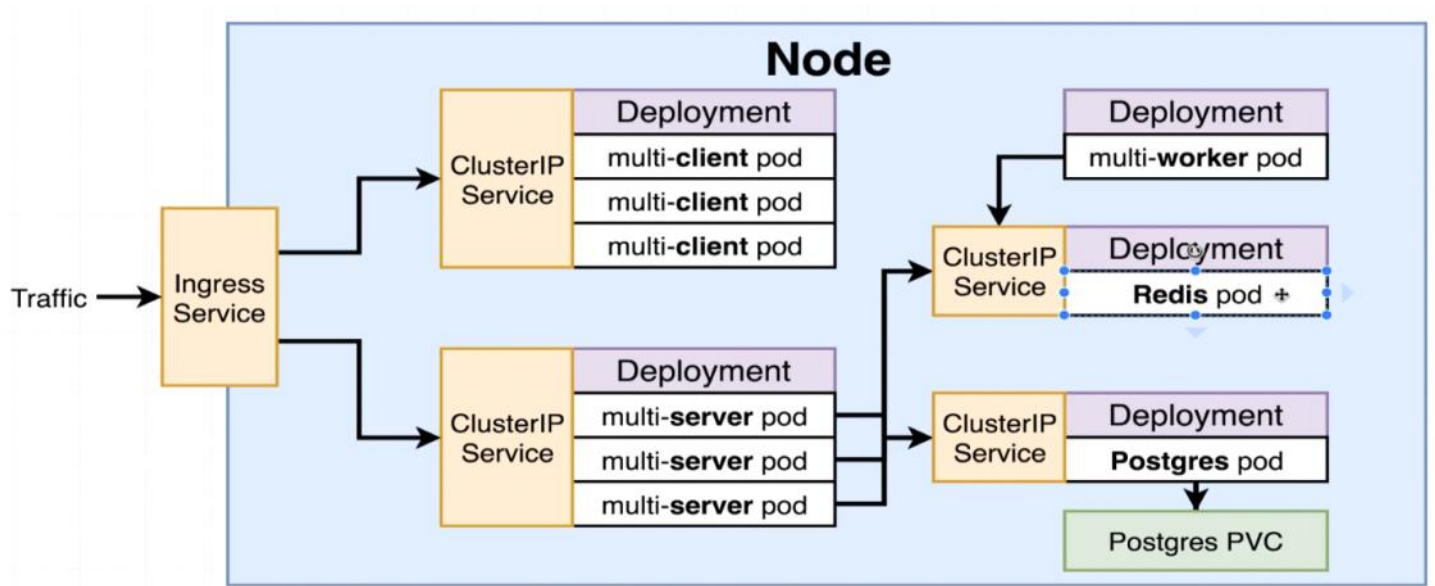
Imperative
command (ugh) to
update image



```
→ simplek8s git:(master) ✗ kubectl set image deployment/client-deployment client=stephengrider/multi-client:v5
```

The Path to Production

Friday, April 7, 2023 3:12 AM

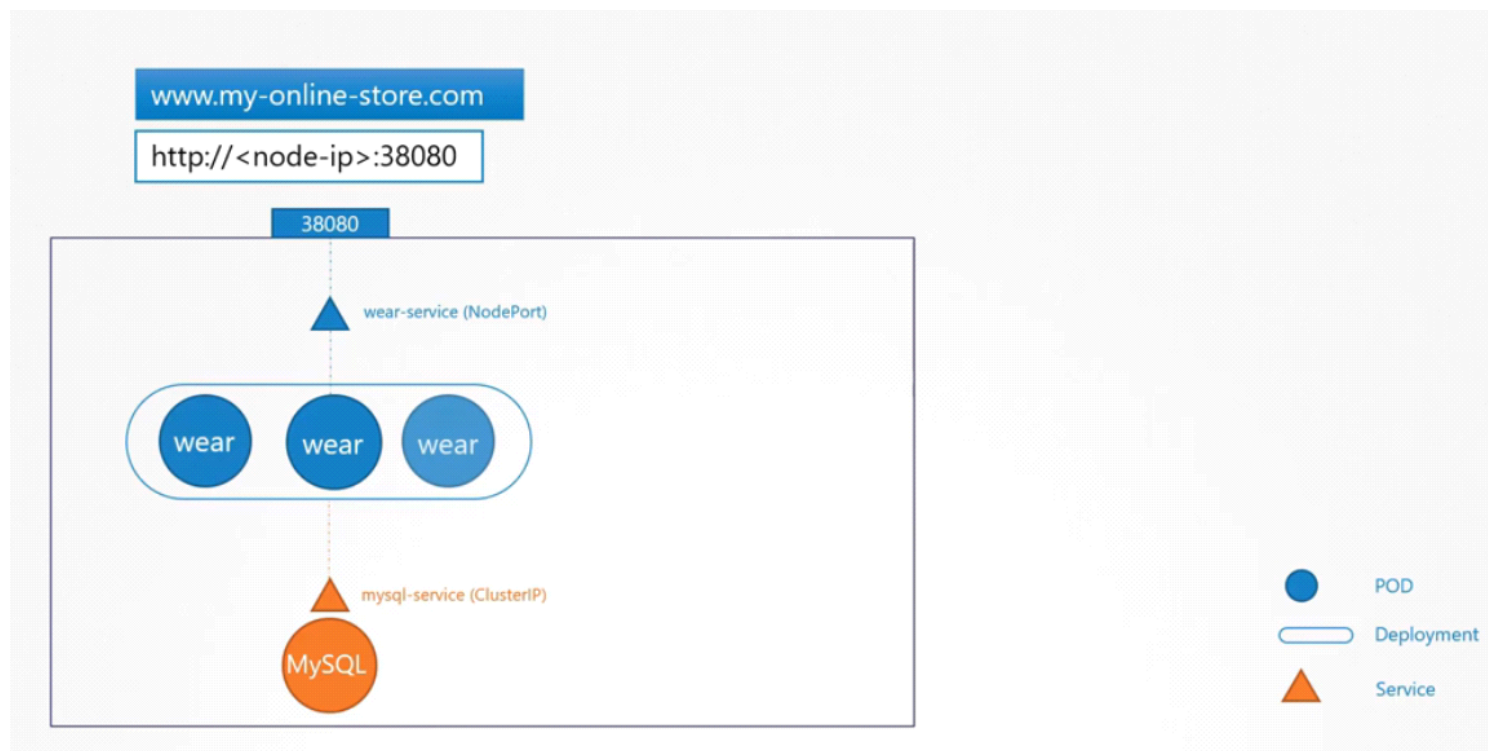


Services Types

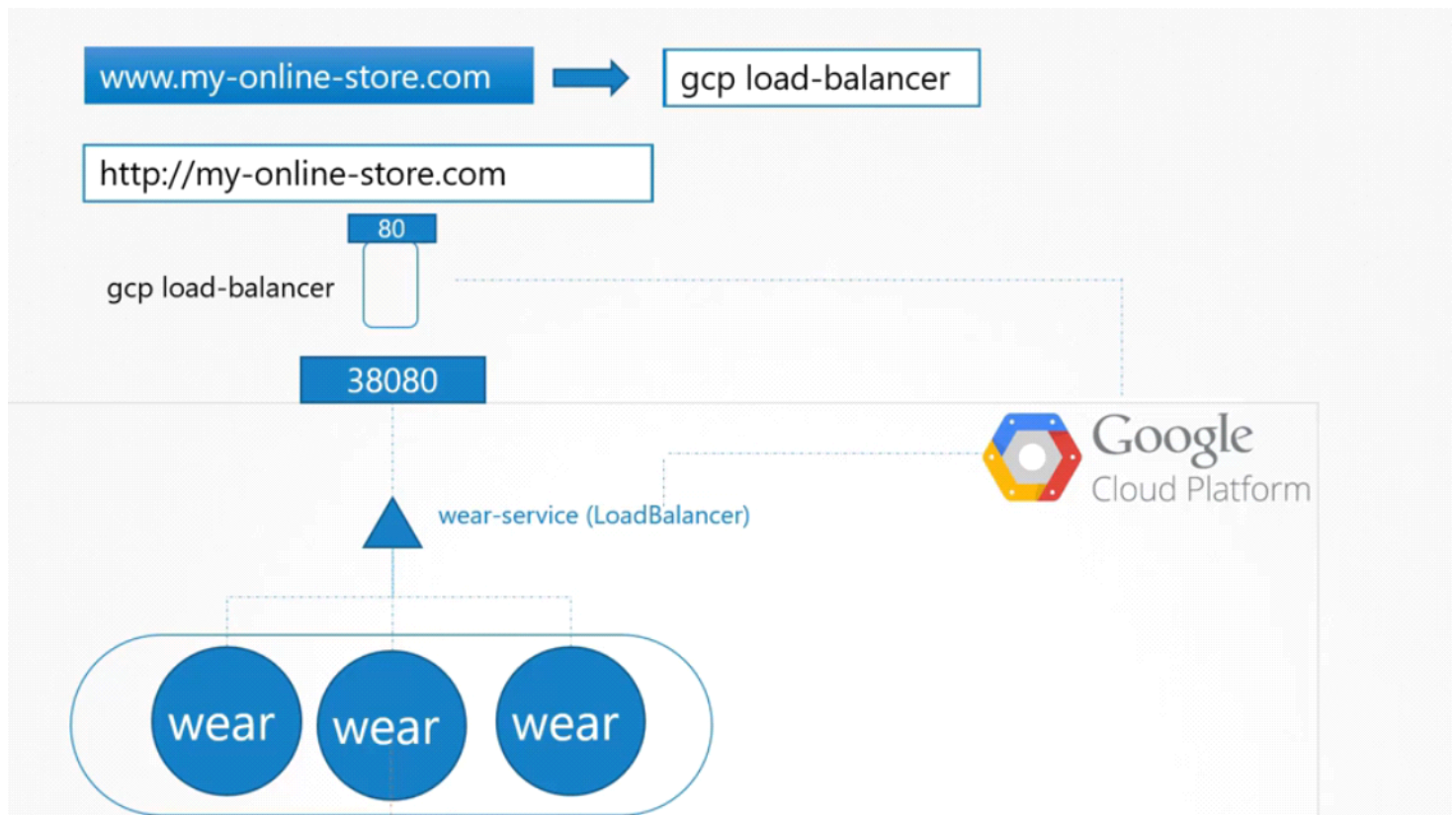


Node Port s something that provides external access
ClusterIP is used for internal access

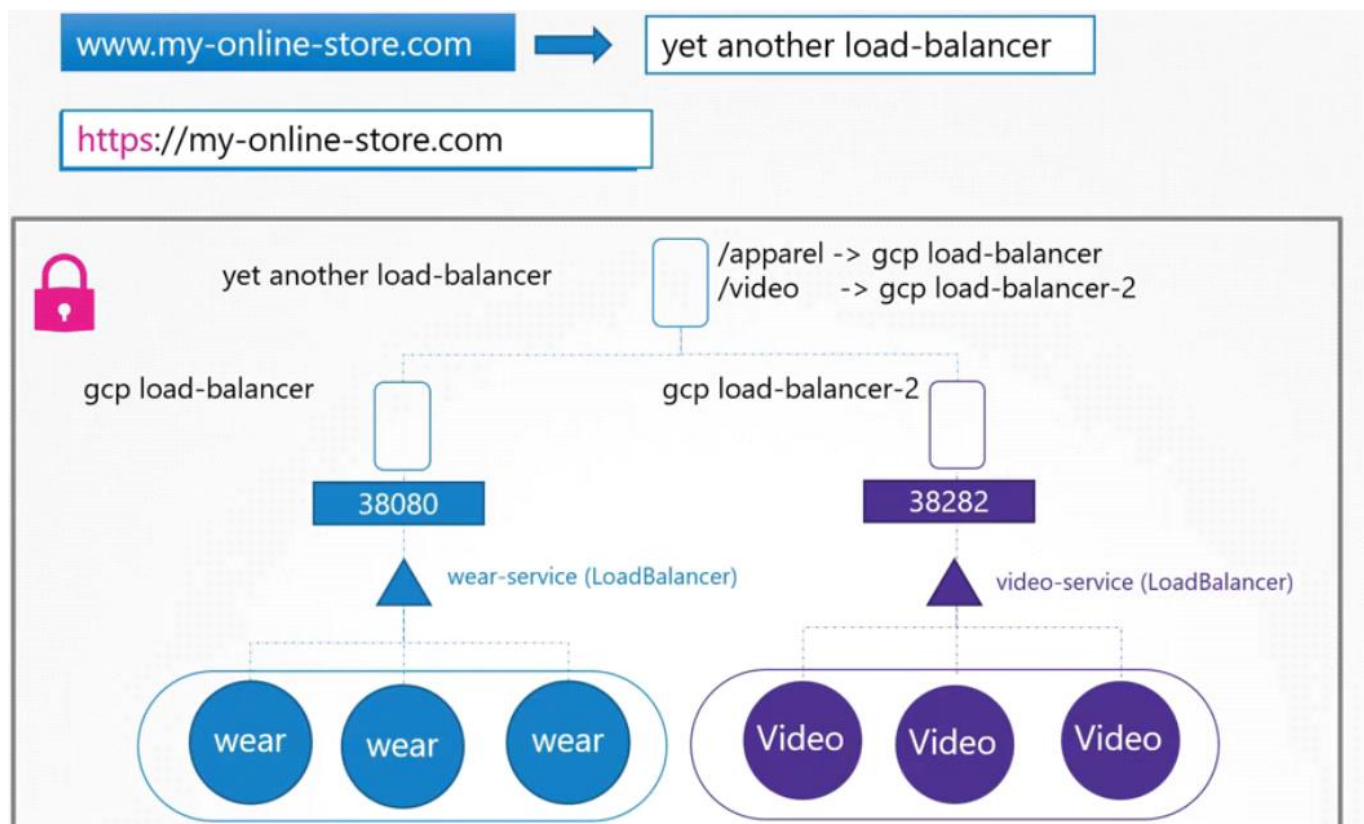
Cluster IP is something that can be used when you are in Onprem



But if your K8s is in a cloud provider, then instead of NodePort, you tend to use LoadBalancer

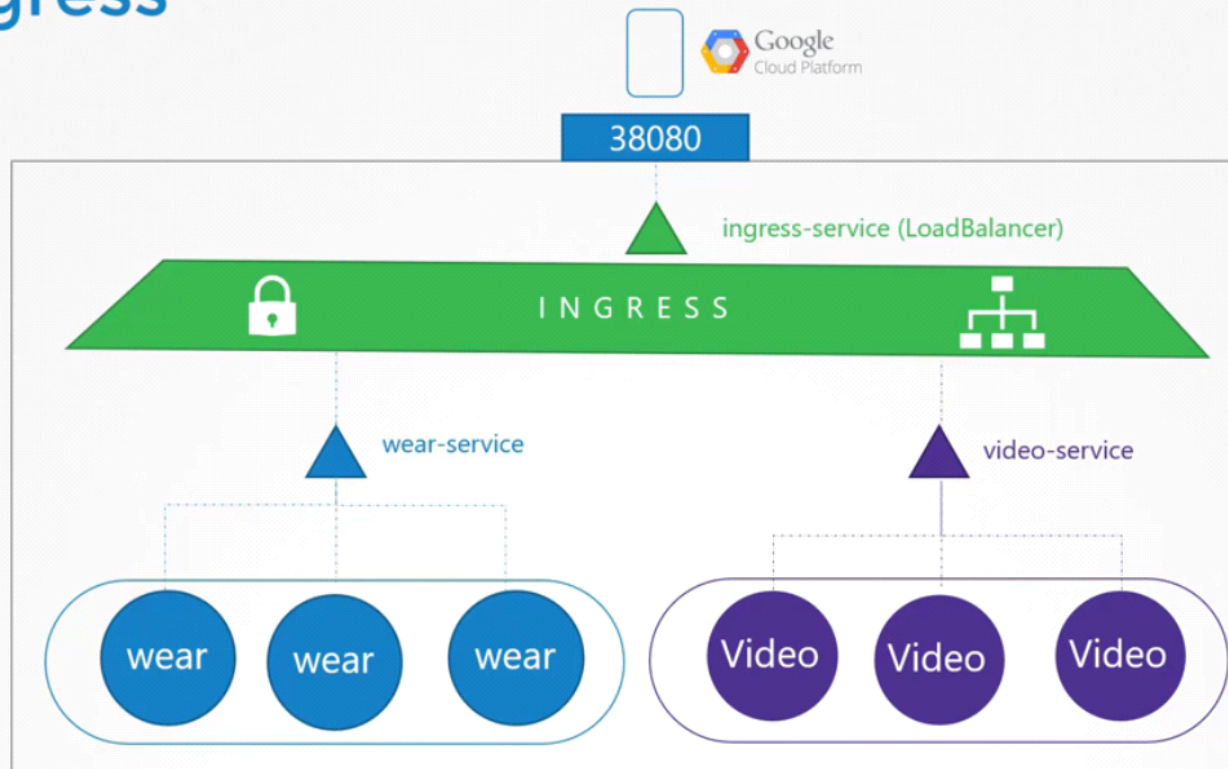


But if your app expose more than one app, then it would cost and configuration consuming to have more than one LB and configure https in all the apps as follows

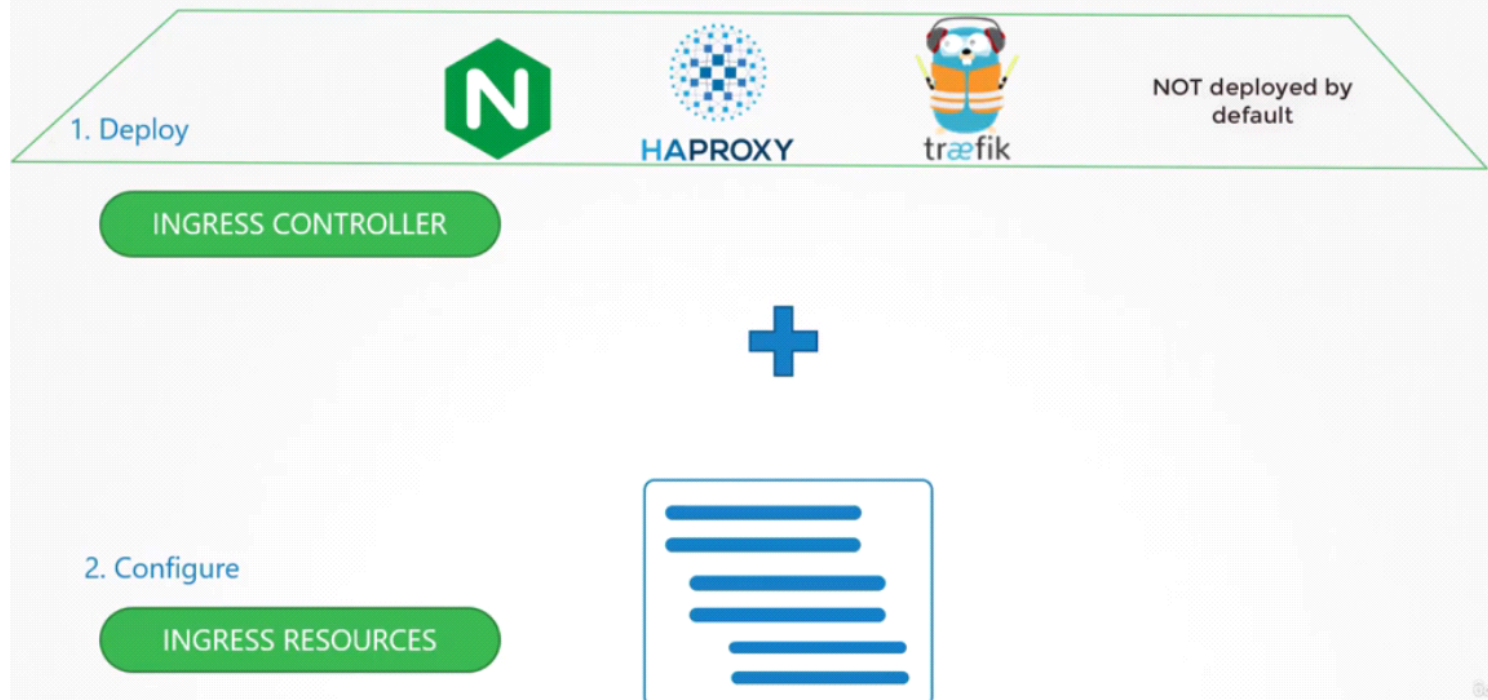


Now to solve this problem is were Ingress comes in

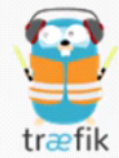
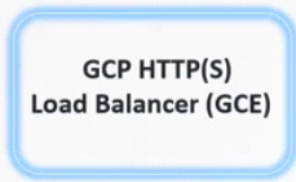
Ingress



Ingress

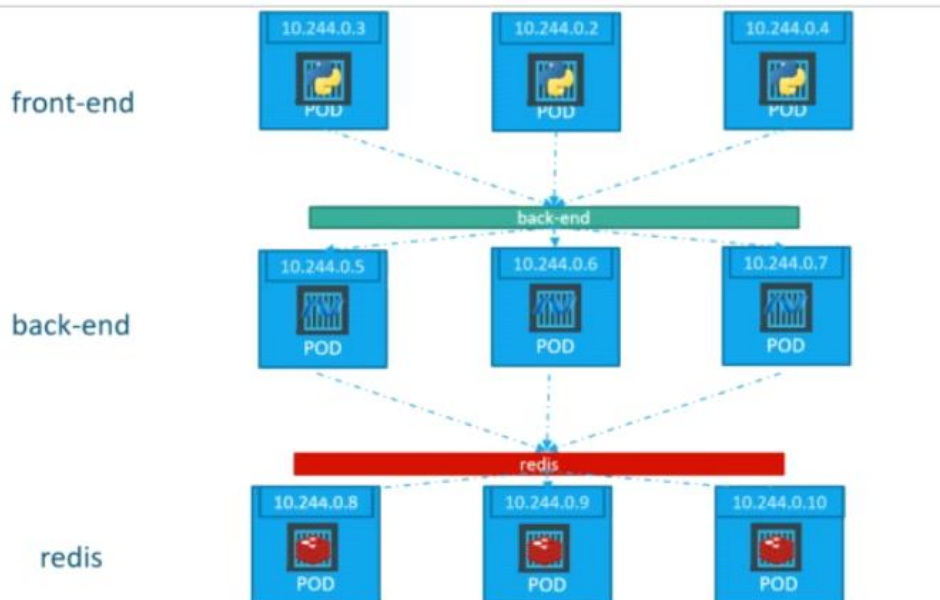


INGRESS CONTROLLER



Cluster IP

Tuesday, November 21, 2023 12:31 AM



service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: back-end
spec:
  type: ClusterIP
  ports:
    - targetPort: 80
      port: 80
  selector:
    app: myapp
    type: back-end
```

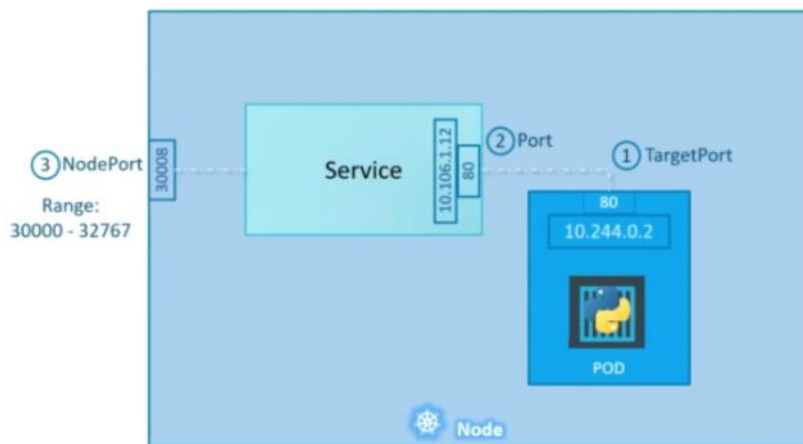
```
> kubectl create -f service-definition.yml
```

```
service "back-end" created
```

```
> kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	16d
back-end	ClusterIP	10.106.127.123	<none>	80/TCP	2m

Service - NodePort



```
service-definition.yml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
```

Link a Service to a pod via Labels

Service - NodePort

```
service-definition.yml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
    app: myapp
    type: front-end
```

```
> kubectl create -f service-definition.yml
service "myapp-service" created
```

```
> kubectl get services
```

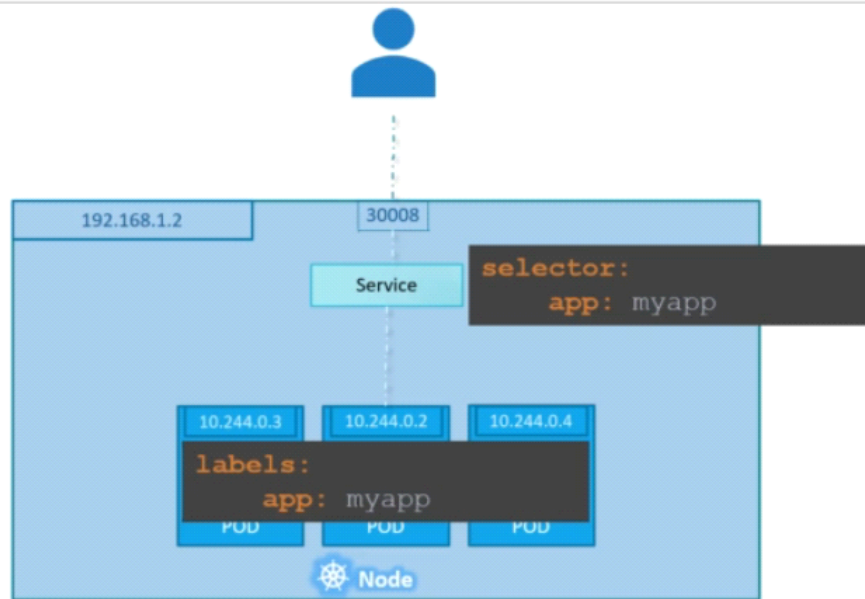
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	16d
myapp-service	NodePort	10.106.127.123	<none>	80:30008/TCP	5m

```
> curl http://192.168.1.2:30008
```

```
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
```

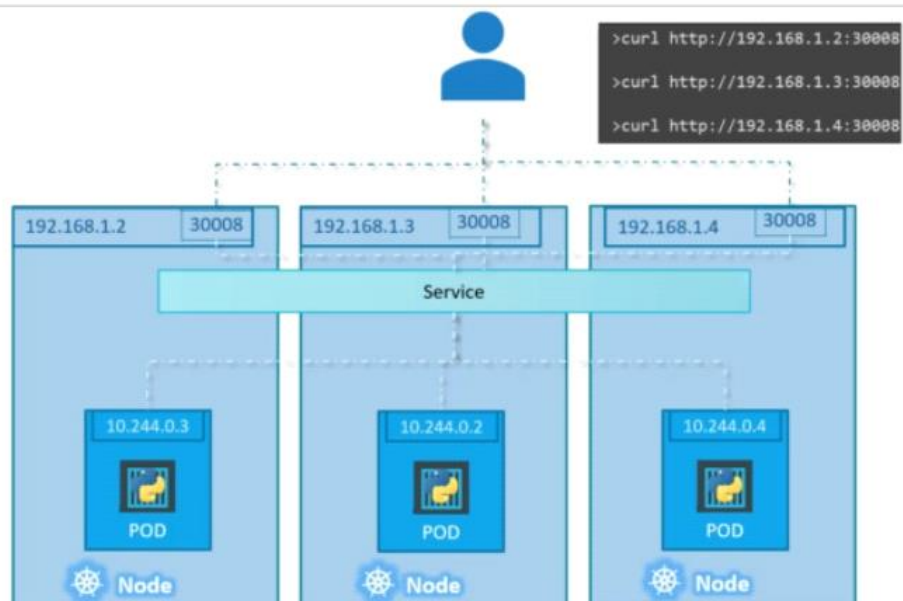
This can be used to map multiple instance

Service - NodePort



Its done even when they are in multiple nodes as service as scaled across multiple nodes

Service - NodePort



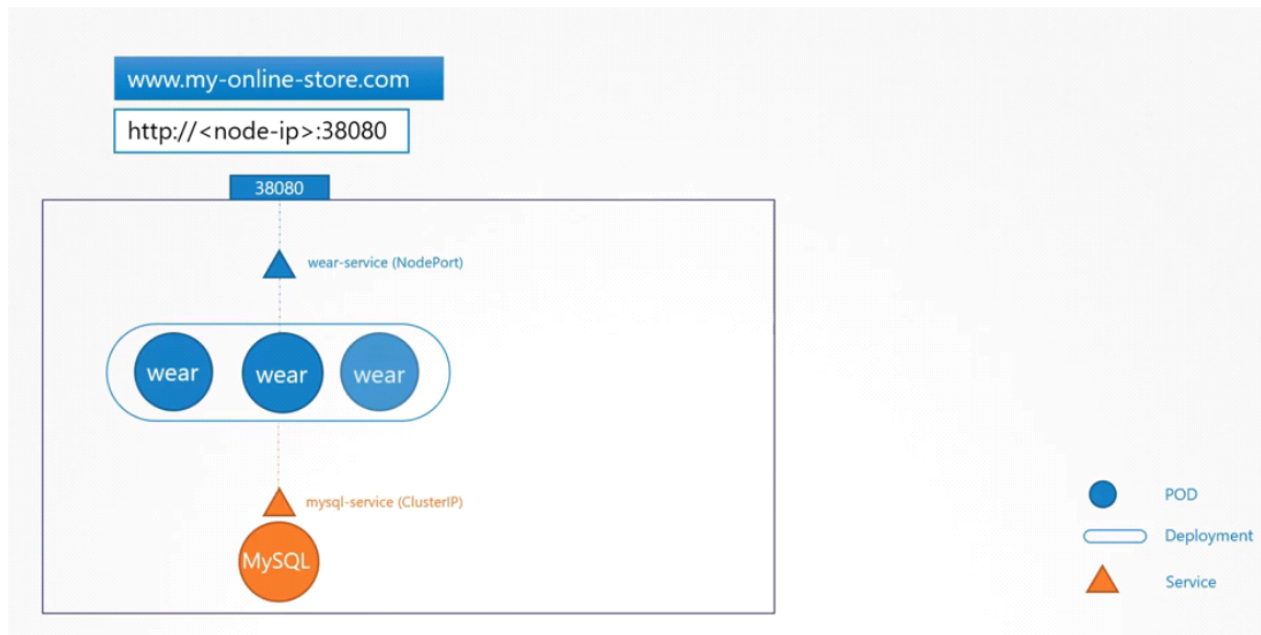
Node Port s something that provides external access

Ingress

Tuesday, November 21, 2023 12:34 AM

Node Port is something that provides external access
Cluster IP is used for internal access

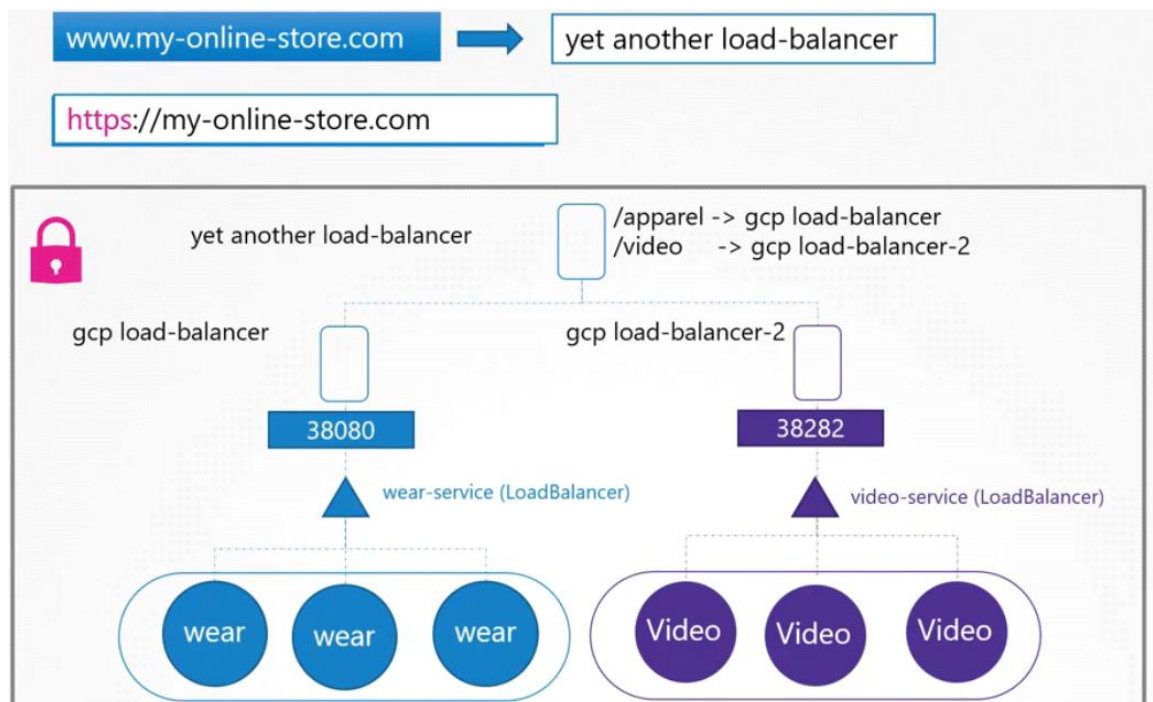
Cluster IP is something that can be used when you are in Onprem



But if your K8s is in a cloud provider, then instead of NodePort, you tend to use **Load Balancer**



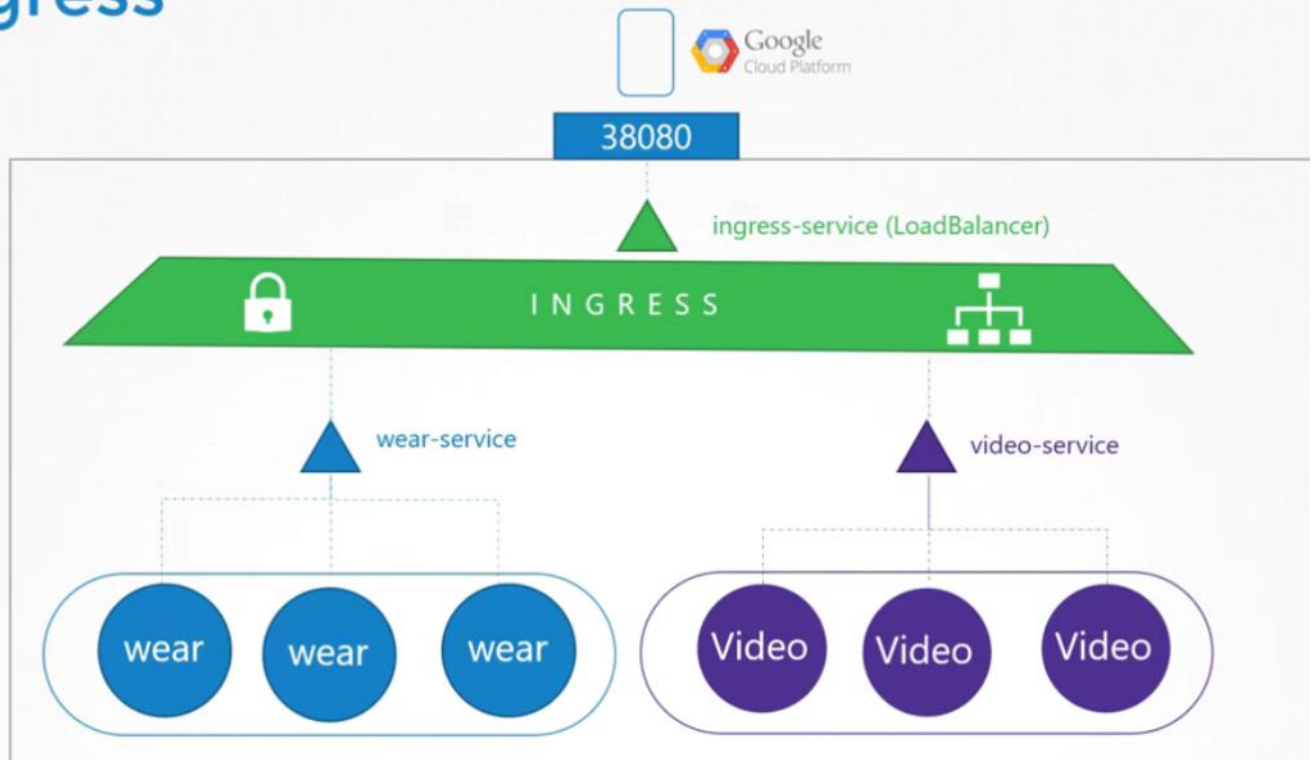
But if your app expose more than one app, then it would cost and configuration consuming to have more than one LB and configure https in all the apps as follows



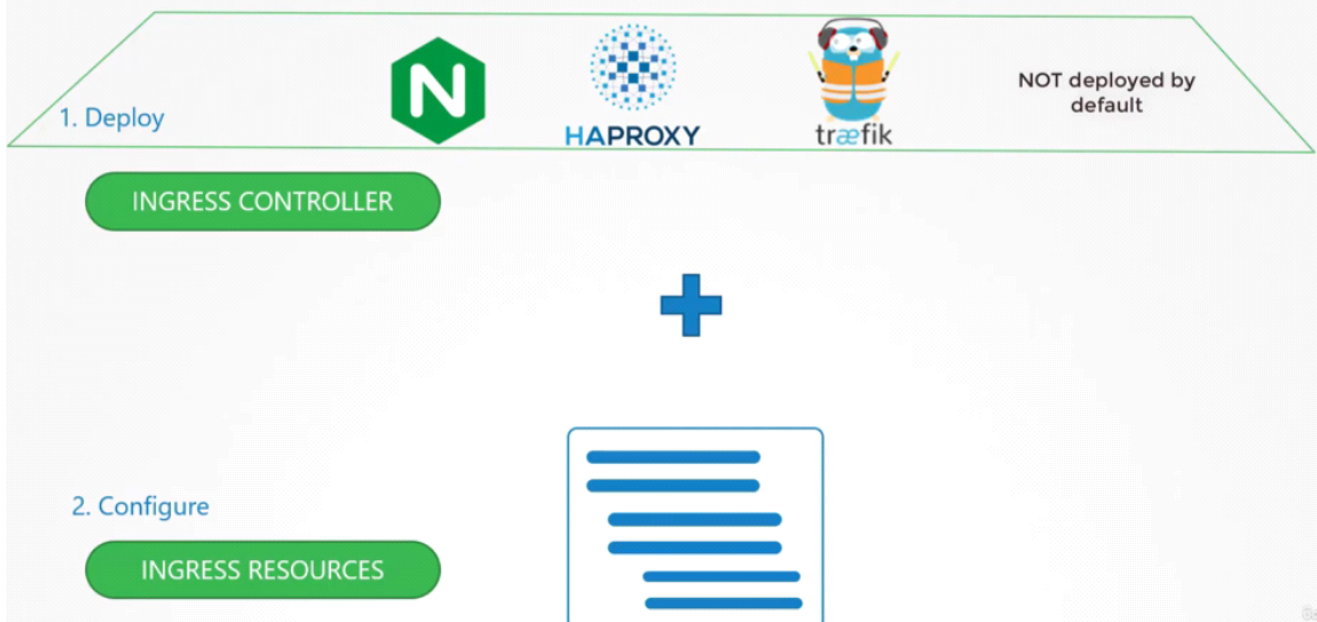
Now to solve this problem is where Ingress comes in

Ingress is a Layer 7 LB built into K8S cluster

Ingress

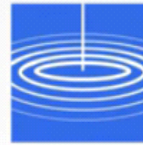


Ingress



INGRESS CONTROLLER

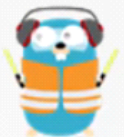
GCP HTTP(S)
Load Balancer (GCE)



Contour



HAPROXY



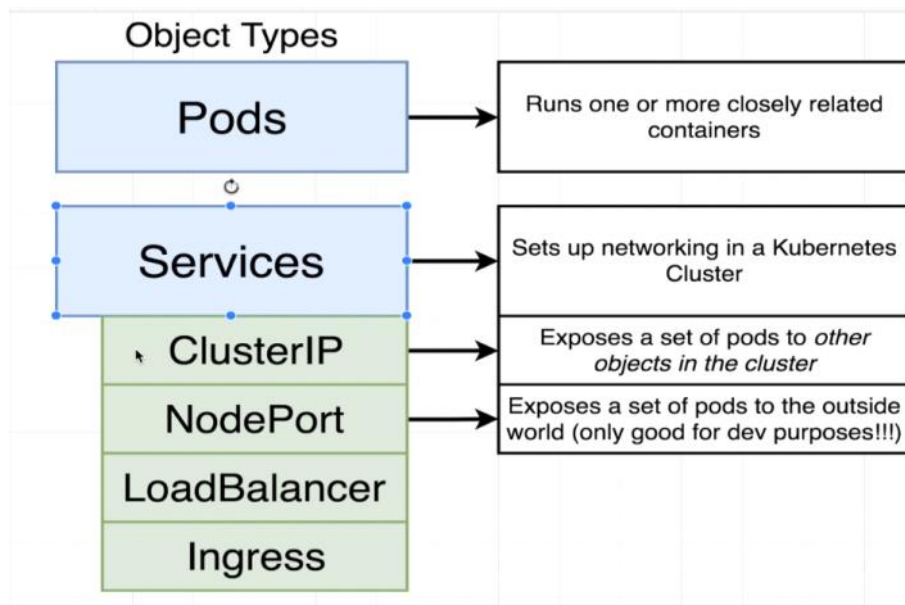
traefik



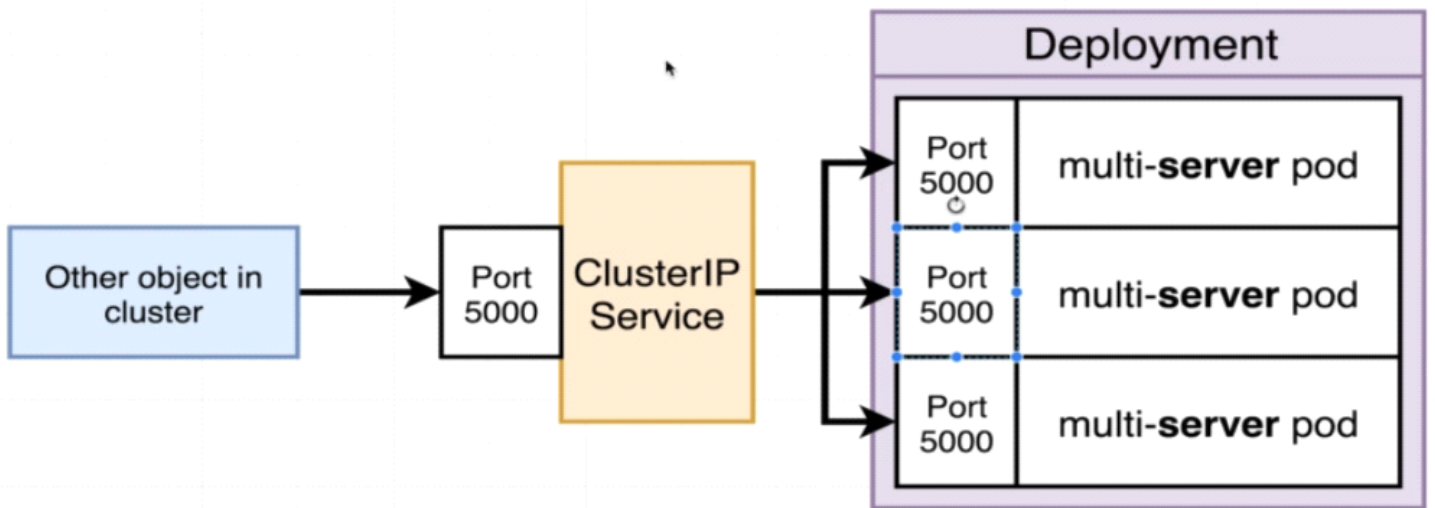
Istio

Node Port vs Cluster Ip

Friday, April 7, 2023 4:10 AM

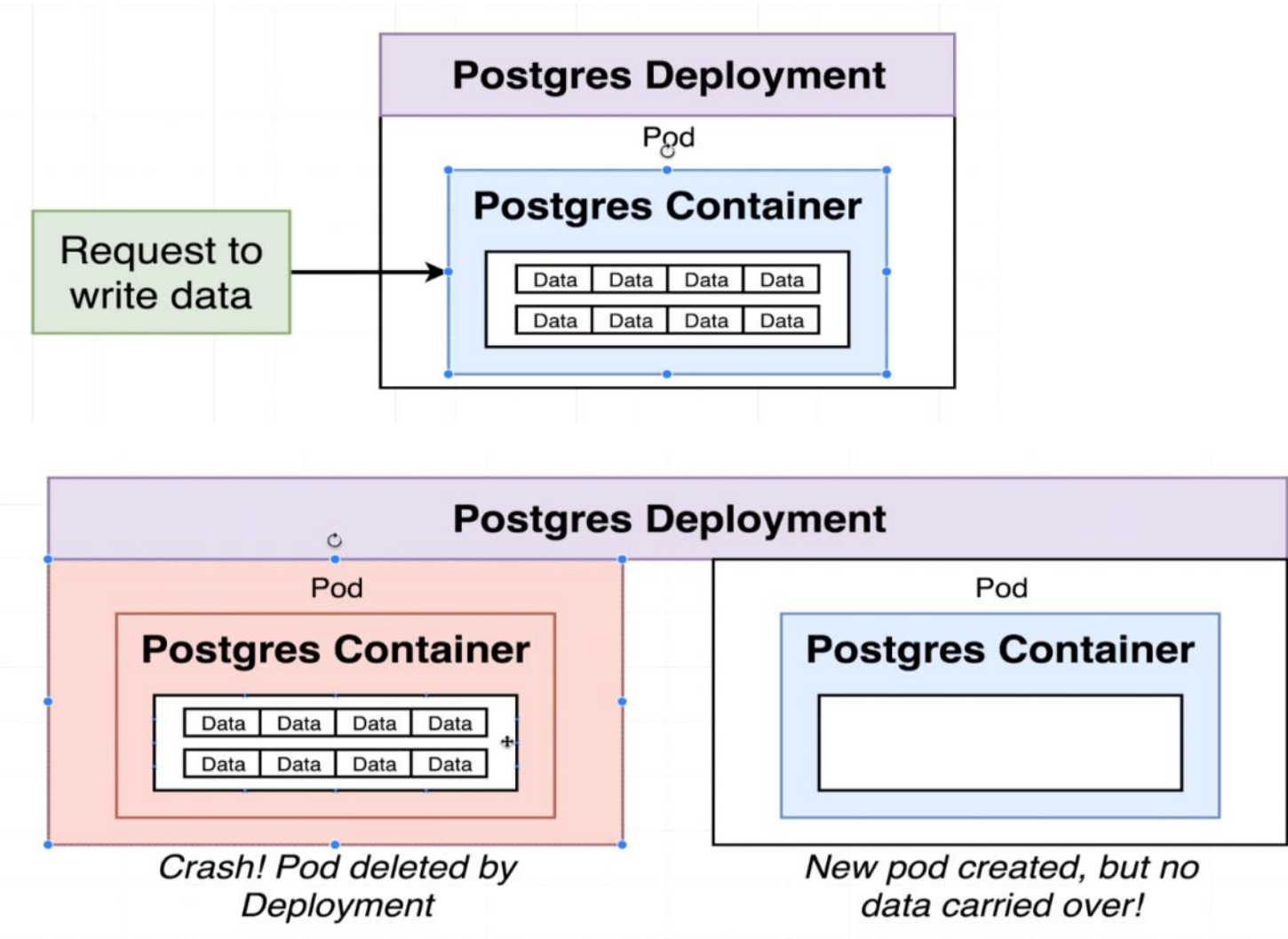


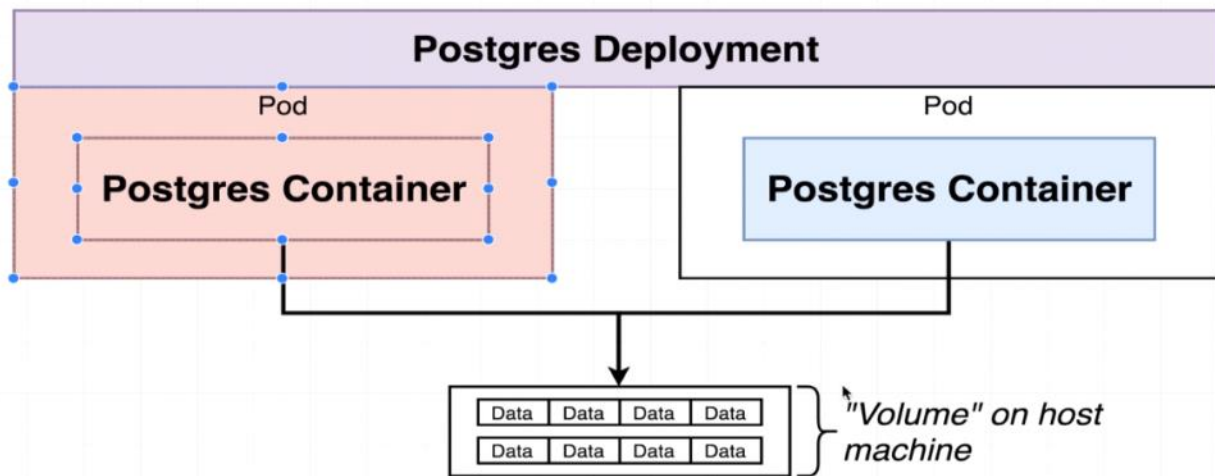
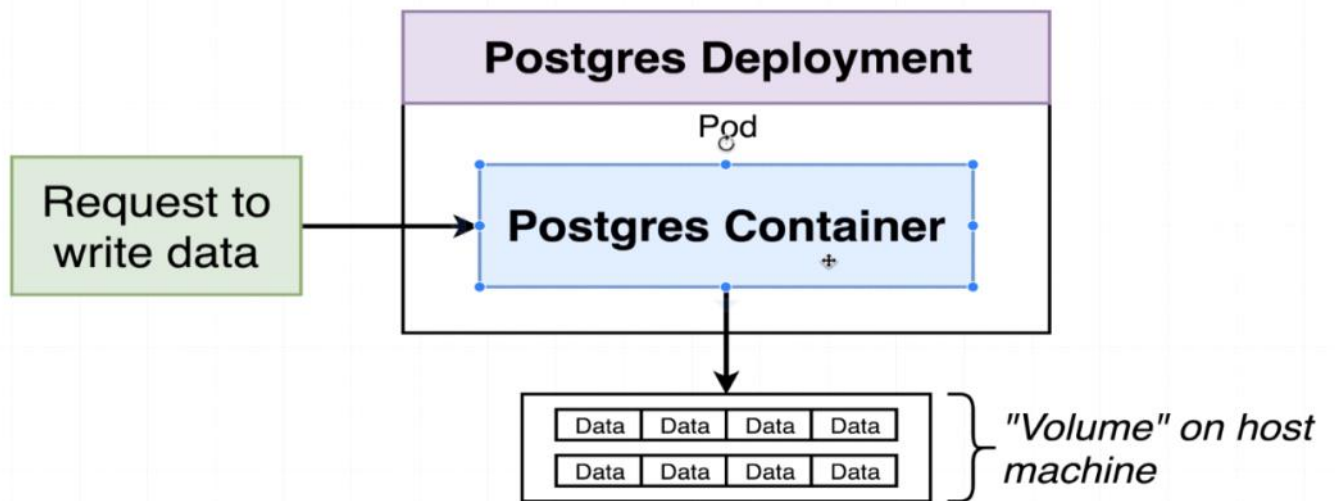
```
apiVersion: v1
kind: Service
metadata:
  name: client-cluster-ip-service
spec:
  type: ClusterIP
  selector:
    component: web
  ports:
    - port: 3000
      targetPort: 3000
```

The Need for Volumes with Databases

Friday, April 7, 2023 11:51 AM



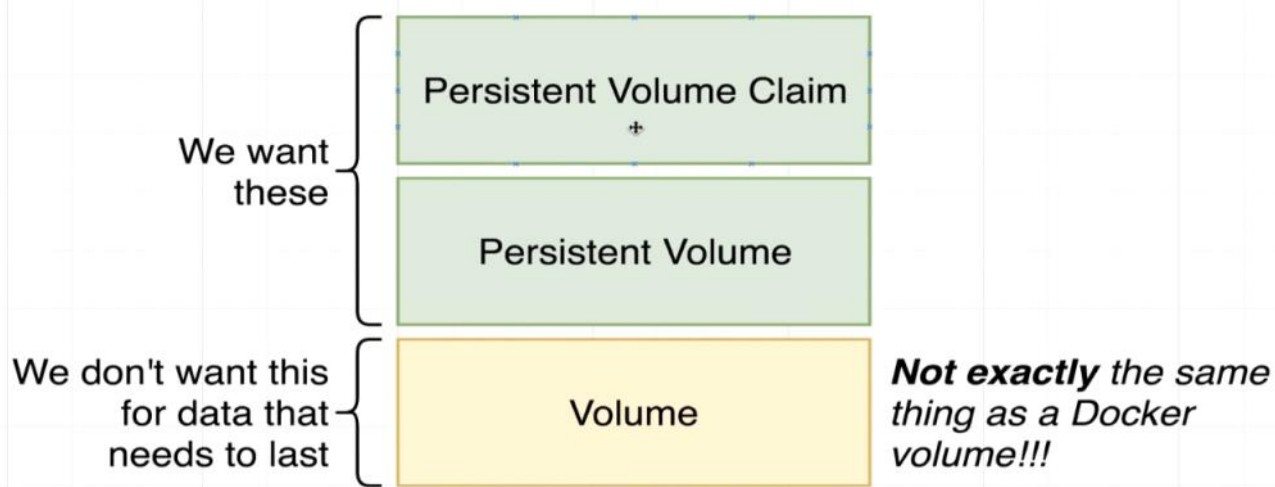


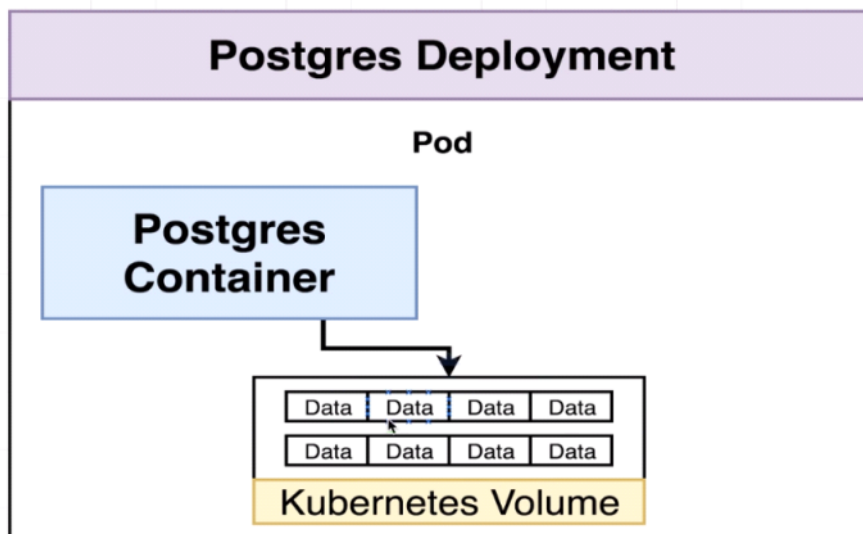
"Volume" in generic container terminology

Some type of mechanism that allows a container to access a filesystem outside itself

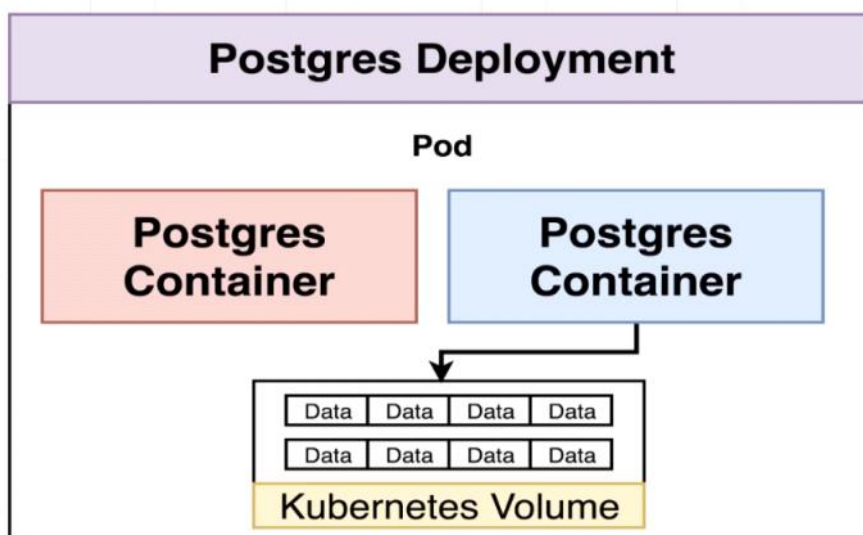
"Volume" in Kubernetes

An **object** that allows a container to store data at the pod level



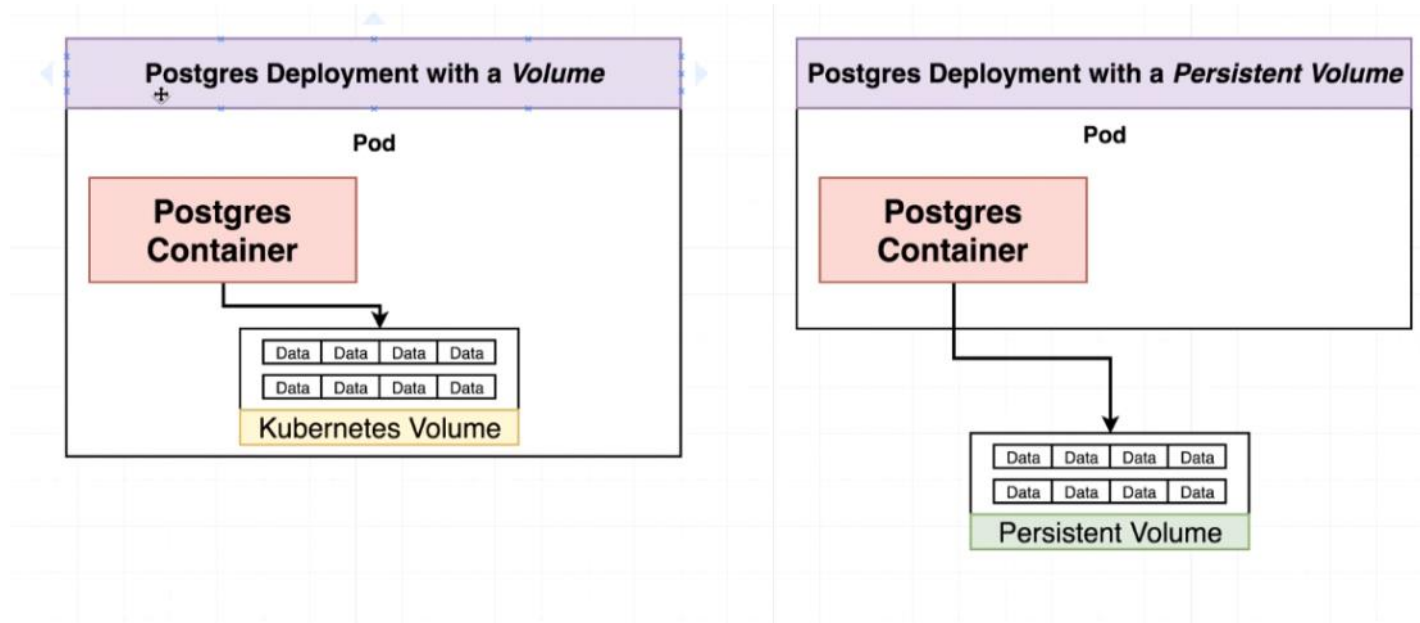


Here in docker the volume is tied to the pod, so if the pod dies, then the volume goes away as well.



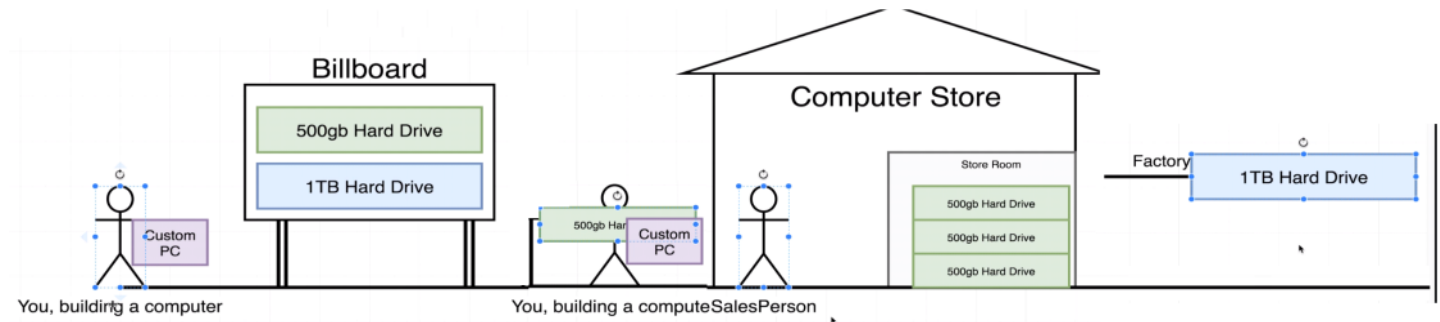
Volumes vs Persistent Volumes

Friday, April 7, 2023 12:08 PM

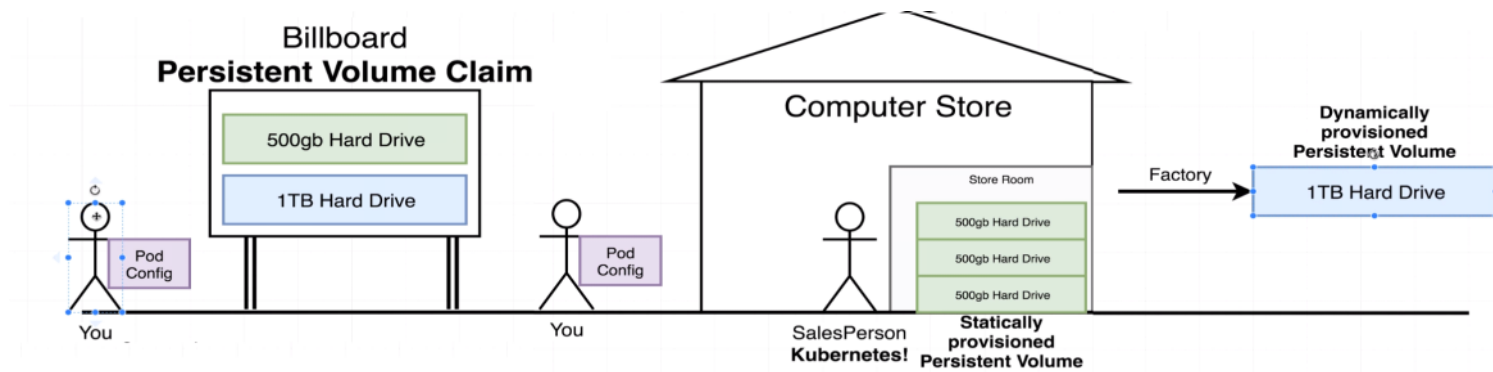


Persistent Volumes vs Persistent Volume Claims

Friday, April 7, 2023 12:09 PM



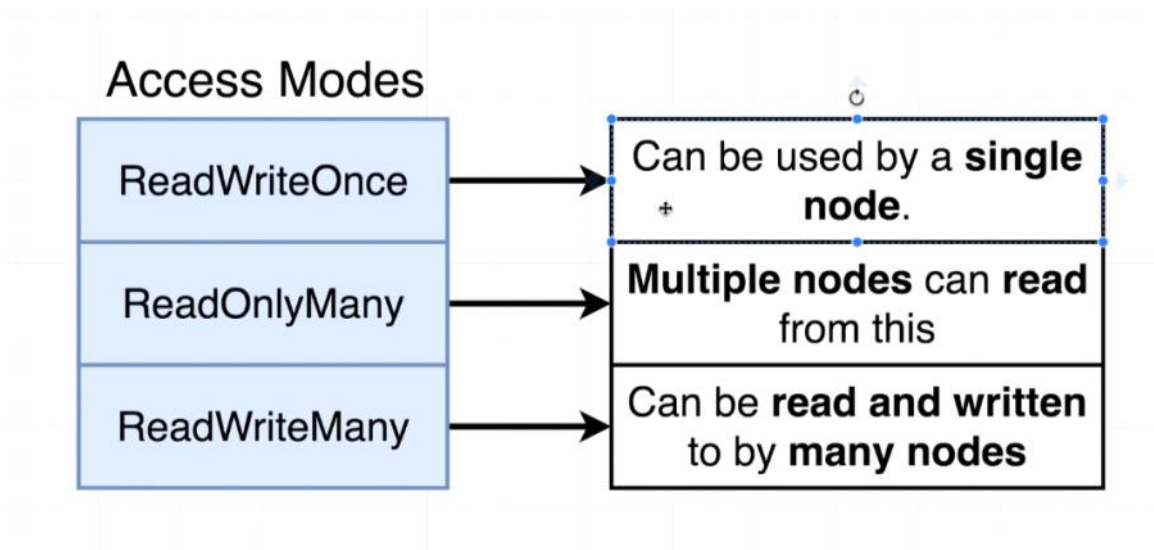
PVC = Advertisement



Persistent Volume Claims & Access modes

Friday, April 7, 2023 12:24 PM

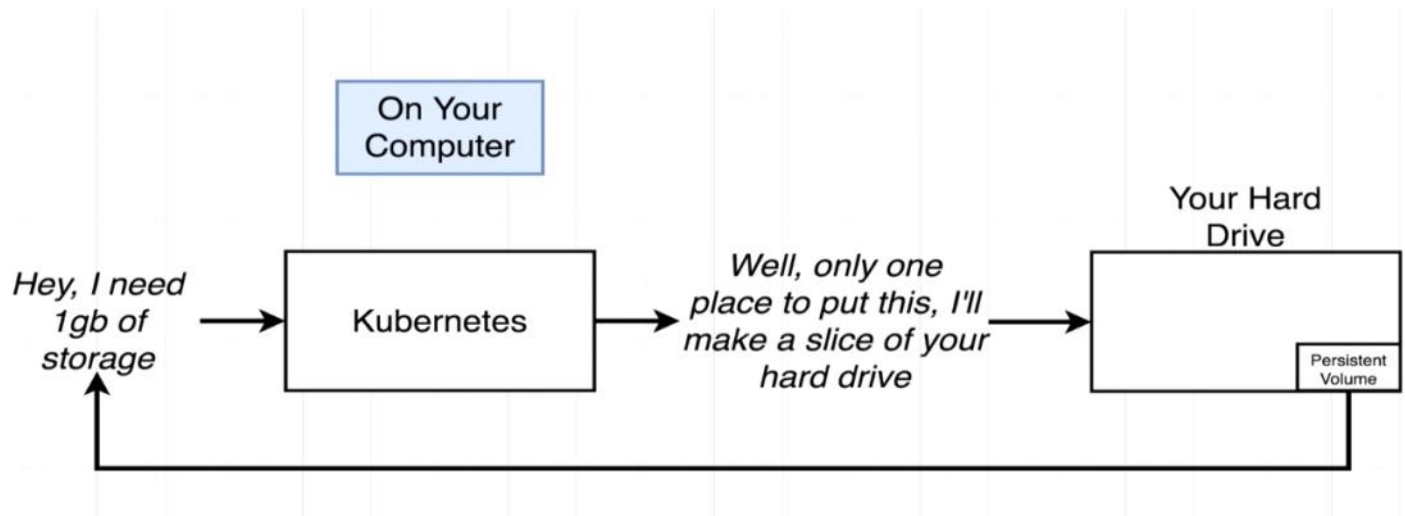
```
! database-persistent-volume-claim.yaml ✕
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: database-persistent-volume-claim
5  spec:
6    accessModes:
7      - ReadWriteOnce
8    resources:
9      requests:
10       storage: 2Gi
11
```



```
! postgres-deployment.yaml • ! database-persistent-volume-claim.yaml
14 spec:
15   volumes:
16     - name: postgres-storage
17       persistentVolumeClaim:
18         claimName: database-persistent-volume-claim
19   containers:
20     - name: postgres
21       image: postgres
22       ports:
23         - containerPort: 5432
24       volumeMounts:
25         - name: postgres-storage
26           mountPath: /var/lib/postgresql/data
27           subPath: postgres
28
```

Where Does Kubernetes Allocate Persistent Volumes?

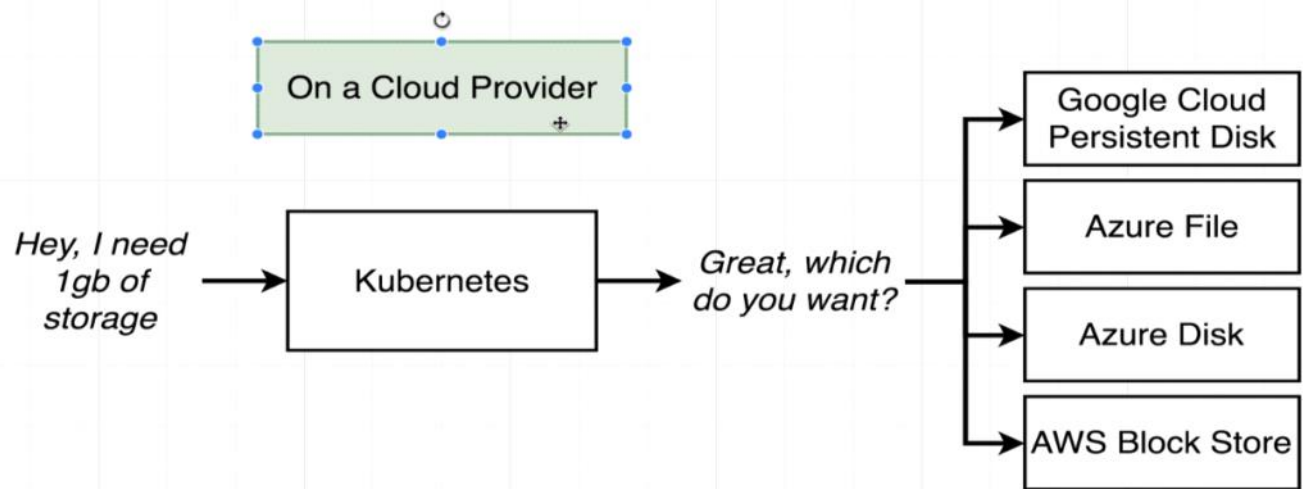
Friday, April 7, 2023 12:29 PM



To list all the storage options available in K8S, with minicabs there is only one which is the default one.

```
→ complex git:(master) ✗ kubectl get storageclass
NAME                                PROVISIONER          AGE
standard (default)                 k8s.io/minikube-hostpath 7d
→ complex git:(master) ✗
```

Minikube host path is something which does a slice as in the first diagram



Designating a PVC in a Pod Template

Friday, April 7, 2023 5:08 PM

```
! postgres-deployment.yaml • ! database-persistent-volume-claim.yaml
9      component: postgres
10     template:
11       metadata:
12         labels:
13           component: postgres
14       spec:
15         volumes:
16           - name: postgres-storage
17             persistentVolumeClaim:
18               claimName: database-persistent-volume-claim
19         containers:
20           - name: postgres
21             image: postgres
22             ports:
23               - containerPort: 5432
24
```

spec:

volumes:

- name: postgres-storage
persistentVolumeClaim:

claimName: database-persistent-volume-claim

Allocate the storage



containers:

- name: postgres
image: postgres

ports:

- containerPort: 5432

volumeMounts:

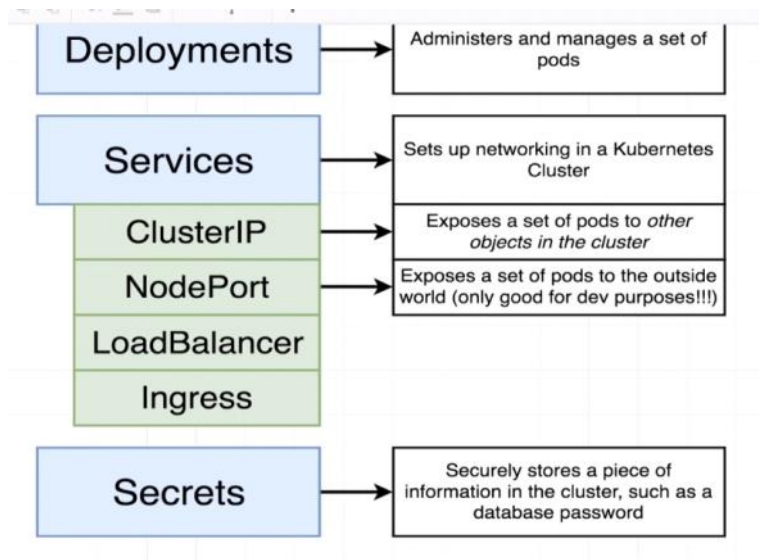
- name: postgres-storage
mountPath: /var/lib/postgresql/data
subPath: postgres

Assign the allocated Storage



Creating an Encoded Secret

Saturday, April 8, 2023 7:14 AM



Secrets will be created via imperative commands and not as objects, as creating in object would require configuring the ID and PWD in plaintext, which avoids the purpose of being secrets

```
→ complex git:(master) ✗ kubectl create secret generic pgpassword --from-literal PGPASSWORD=12345asdf
secret/pgpassword created
→ complex git:(master) ✗ kubectl get secrets
```

NAME	TYPE	DATA	AGE
default-token-2c228	kubernetes.io/service-account-token	3	8d
pgpassword	Opaque	1	11s

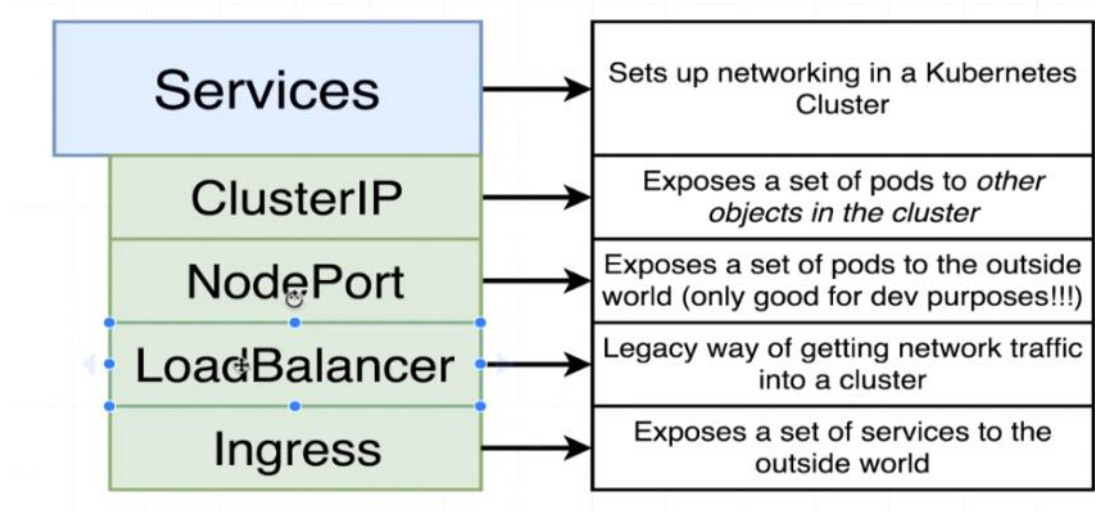
```
spec:
  containers:
    - name: server
      image: stephengrider/multi-server
      ports:
        - containerPort: 5000
      env:
        - name: REDIS_HOST
          value: redis-cluster-ip-service
```

```
- name: PGDATABASE
  value: postgres
- name: PGPASSWORD
  valueFrom:
    secretKeyRef:
      name: pgpassword
      key: PGPASSWORD
```

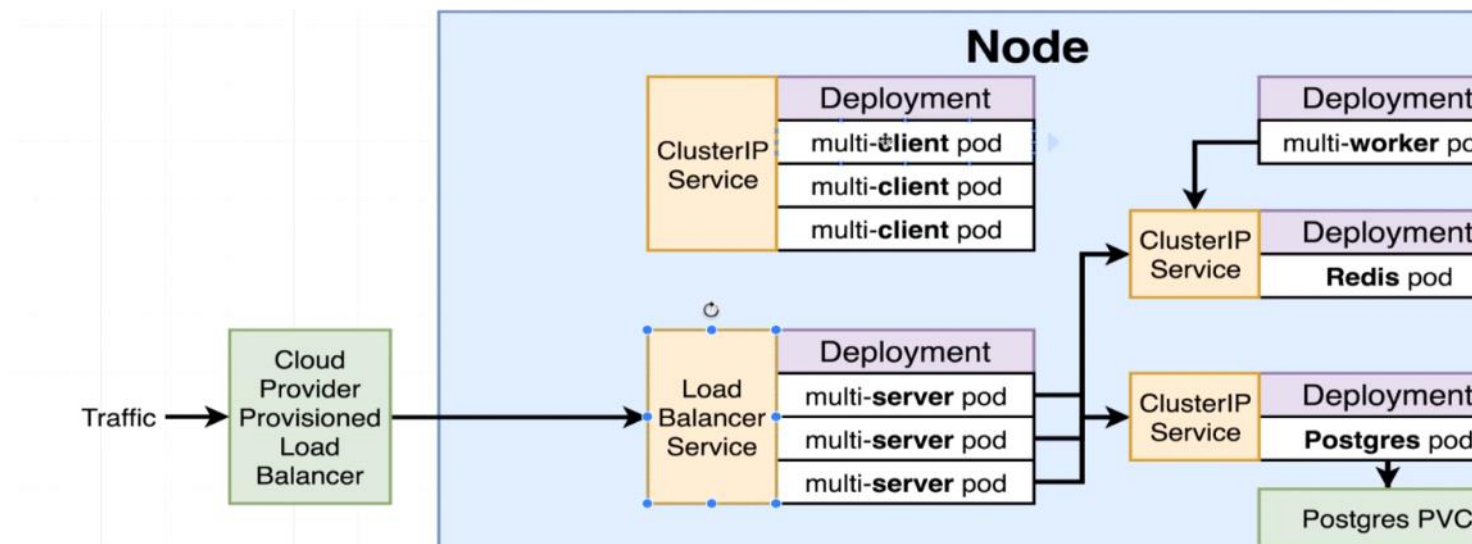
I

Load Balancer Services

Saturday, April 8, 2023 8:00 AM



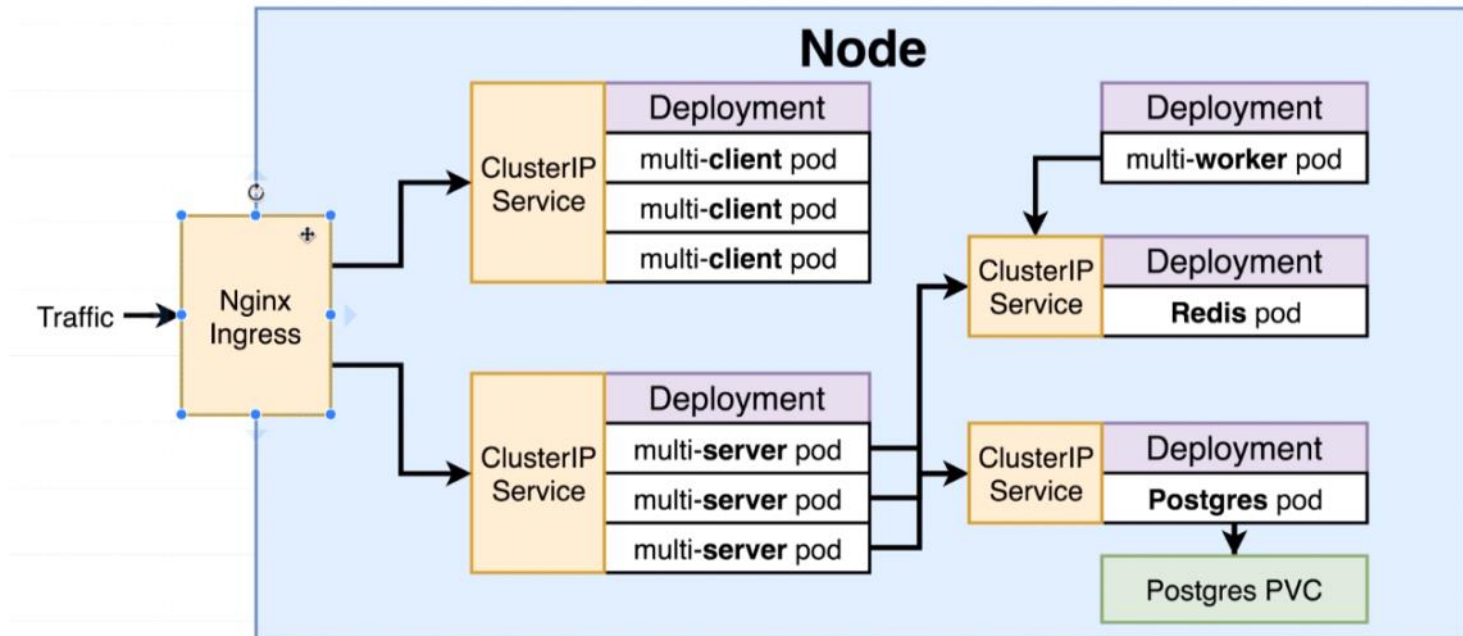
Load Balancer Limitation : It can give access to only one sets of pods and not pods of multiple types



A Quick Note on Ingresses

Saturday, April 8, 2023 8:05 AM

Disadvantage of using Load balancer is it allows the request only for one sets of pods, but in our case we need external access to 2 sets of pods, so the better option is to use ingress



We are using **ingress-nginx**, a community led project
+
github.com/kubernetes/ingress-nginx

We are **not** using **kubernetes-ingress**, a project led by the company nginx
github.com/nginxinc/kubernetes-ingress

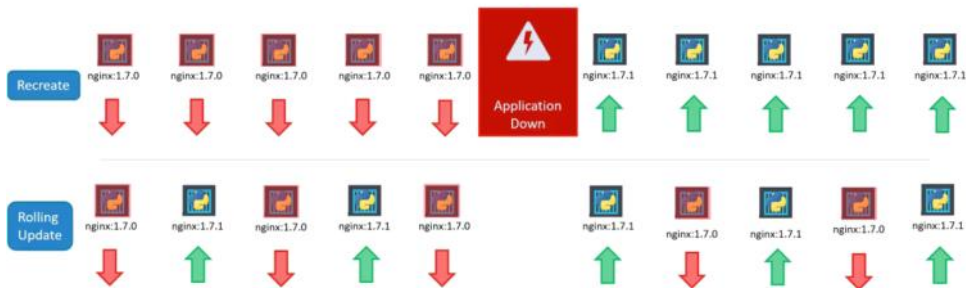
Setup of ingress-nginx changes depending on your environment (local, GC, AWS, Azure)

We are going to set up ingress-nginx on local and GC

Rollout and Deployment

Saturday, November 18, 2023 12:13 PM

Deployment Strategy



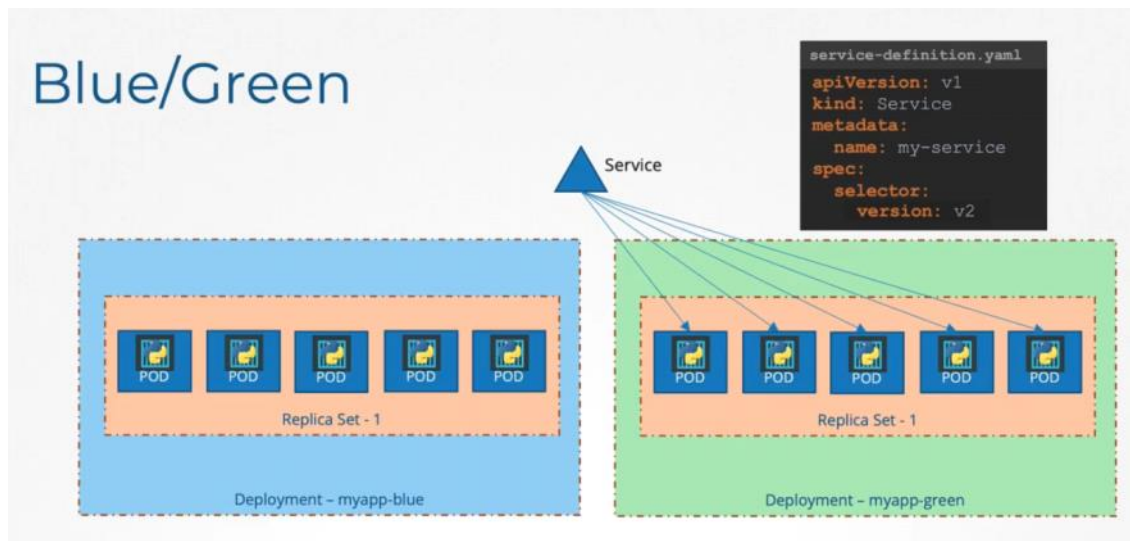
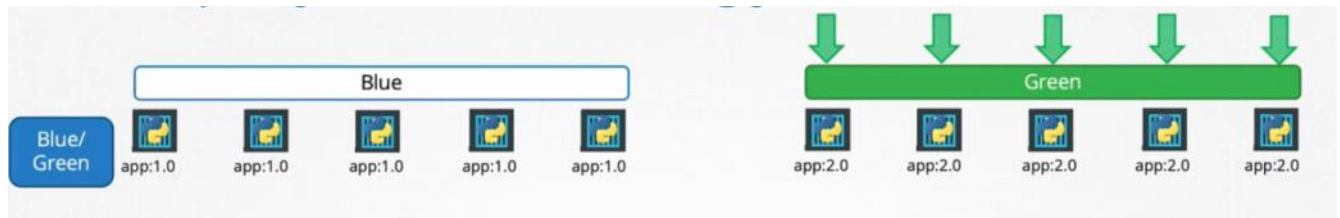
Rolling update is the default

Summarize Commands

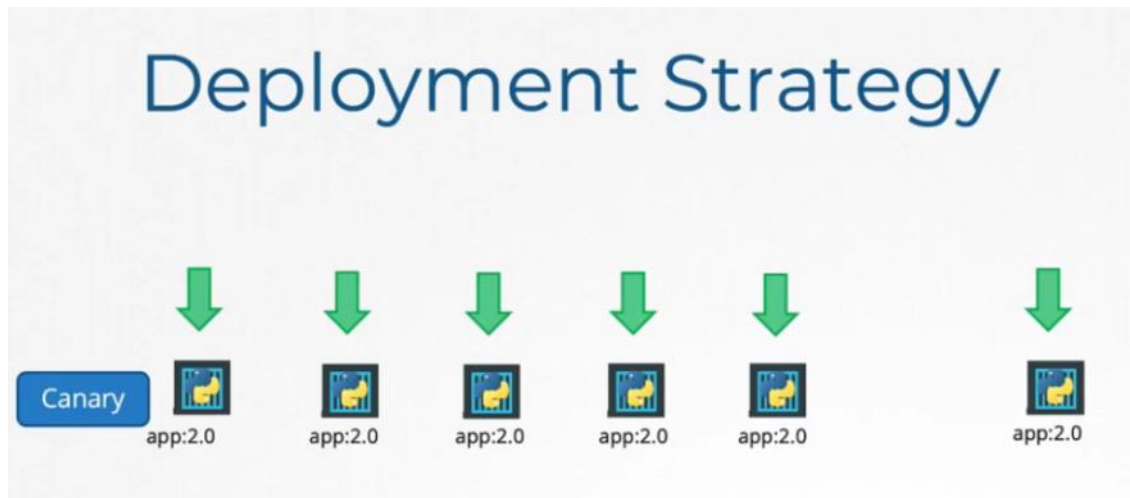
Create	> kubectl create -f deployment-definition.yml
Get	> kubectl get deployments
Update	> kubectl apply -f deployment-definition.yml > kubectl set image deployment/myapp-deployment nginx=nginx:1.9.1
Status	> kubectl rollout status deployment/myapp-deployment
Rollback	> kubectl rollout history deployment/myapp-deployment > kubectl rollout undo deployment/myapp-deploy

There are couple of other strategies Aswell, which is not really a strategies which you can specify as an option in deployment but they can be implemented in a different way

Blue green



Blue/green is best implemented via Service mesh



1. Blue-Green Deployment:

- **Overview:** In a Blue-Green deployment, you maintain two identical environments, referred to as "Blue" and "Green." At any given time, only one of these environments serves production traffic while the other remains idle.

2. Canary Deployment:

- **Overview:** In a Canary deployment, you release a new version of your application or service incrementally to a small subset of users before rolling it out to the entire user base.

Types of Workload

Saturday, November 18, 2023 12:52 PM

Job: Job is a onetime run, so its restart policy should be set to never

Create, View & Delete

```
job-definition.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: math-add-job
spec:
  template:
    spec:
      containers:
        - name: math-add
          image: ubuntu
          command: ['expr', '3', '+', '2']

      restartPolicy: Never
```

```
> kubectl create -f job-definition.yaml

> kubectl get jobs
```

NAME	DESIRED	SUCCESSFUL	AGE
math-add-job	1	1	38s

```
> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
math-add-job-187pn	0/1	Completed	0	2m

```
> kubectl logs math-add-job-1d87pn
5

> kubectl delete job math-add-job
job.batch "math-add-job" deleted
```

To create more than one same job. And add options if you want to run in parallel

```
job-definition.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: random-error-job
spec:
  completions: 3
  parallelism: 3
  template:
    spec:
      containers:
        - name: random-error
          image: kodekloud/random-error

      restartPolicy: Never
```

CRON Jobs

Create CronJob

```
kubectl create -f cron-job-definition.yaml
```

```
kubectl get cronjob
```

NAME	SCHEDULE	SUSPEND	ACTIVE
reporting-cron-job	*/* * * * *	False	0

cron-job-definition.yaml

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: reporting-cron-job
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      completions: 3
      parallelism: 3
      template:
        spec:
          containers:
            - name: reporting-tool
              image: reporting-tool
          restartPolicy: Never
```

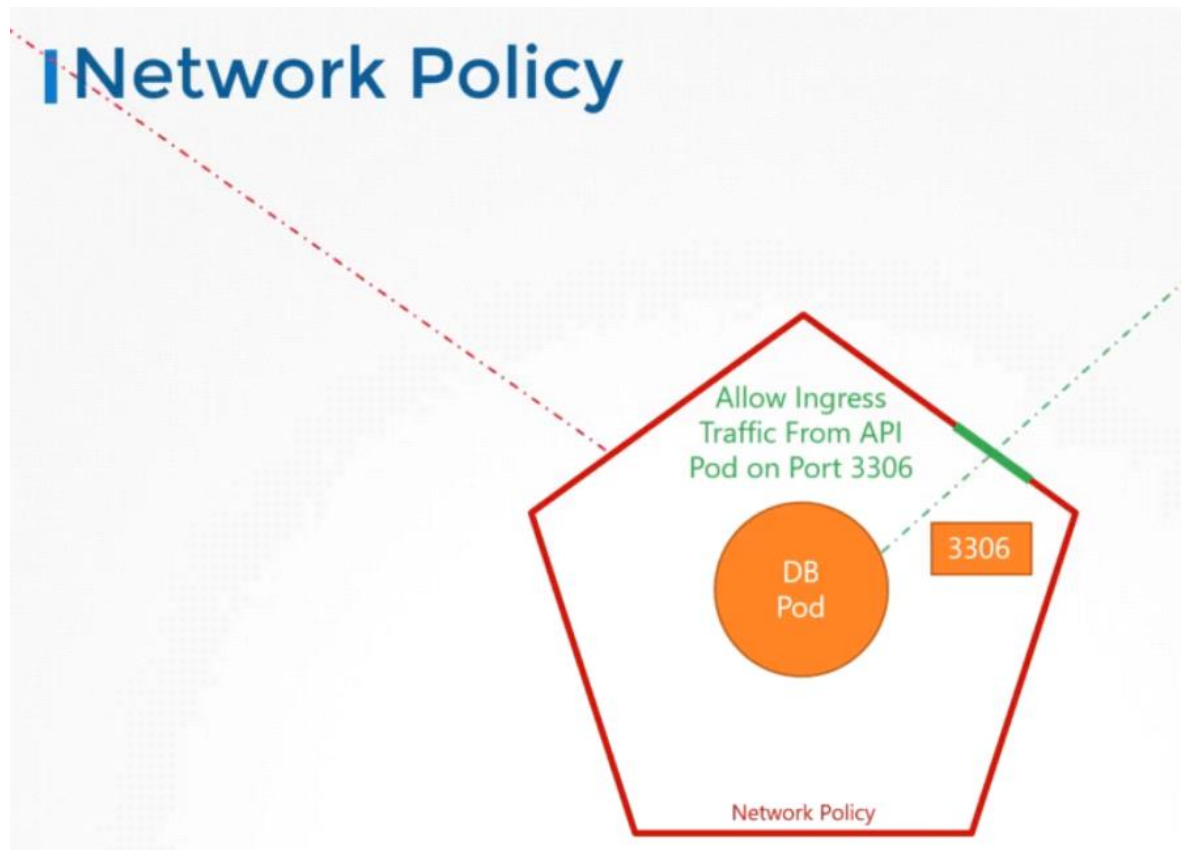

Network Policies - Ingress & Egress

Tuesday, November 21, 2023 12:52 AM

Ingress - Incoming traffic rule
Egress - Outgoing traffic rule



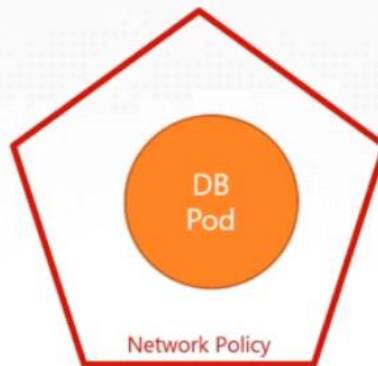
In which you can create network policy



How to create

Network Policy - Selectors

Allow Ingress
Traffic From API
Pod on Port 3306



```
podSelector:  
  matchLabels:  
    role: db
```

```
labels:  
  role: db
```

Network Policy - Rules

```
policyTypes:  
- Ingress  
ingress:  
- from:  
  - podSelector:  
    matchLabels:  
      name: api-pod  
  ports:  
  - protocol: TCP  
    port: 3306
```

Allow
Ingress
Traffic
From
API Pod
on
Port 3306

Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
    ports:
    - protocol: TCP
      port: 3306
```

Kubernetes API Primitives

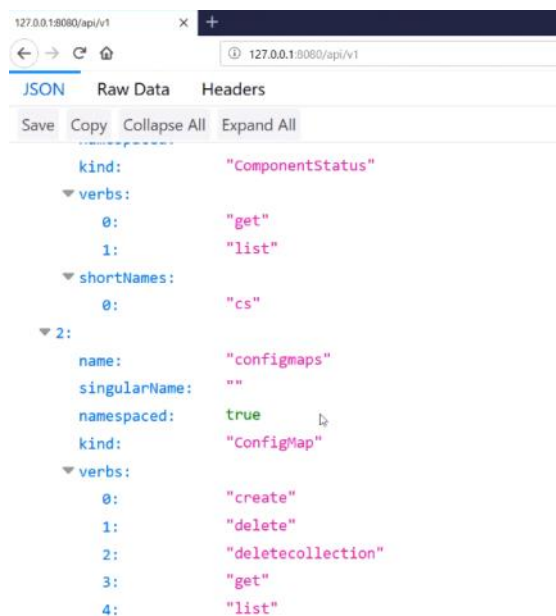
Monday, April 10, 2023 8:53 PM

To get all api resources

Kubectl api-resources

To access api server in localhost

```
C:\Users\Zeal Vora\Desktop\k8s\Configuration>kubectl proxy --port 8080
Starting to serve on 127.0.0.1:8080
```



kubectl get pods --show-label

kubectl scale deployments dpname --replicas=3

to get the public ip of worker node

kubectl gt nodes -o wide

Shortcut to create

kubectl expose deployment --help

--dry-run -o yaml

Docker Expose vs K8s Expose

Tuesday, April 11, 2023

12:32 AM

Overview of EXPOSE instruction

The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime.

The EXPOSE instruction does not actually publish the port.

It functions as a type of documentation between the person who builds the image and the person who runs the container, about which ports are intended to be published.

Example if a spring properties file is set to port 8090, then indicate it in EXPOSE command so as to aid any developers to who uses this app to use this port.

```
pod-expose-port.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - image: nginx
    name: democontainer
    ports:
    - containerPort: 8080
```

ports <[]Object>

List of ports to expose from the container. Exposing a port here gives the system additional information about the network connections a container uses, but is primarily informational. Not specifying a port here DOES NOT prevent that port from being exposed. Any port which is listening on the default "0.0.0.0" address inside a container will be accessible from the network. Cannot be updated.

readOnlyRootFilesystem <boolean>

Generating Pod Manifests via CLI

Tuesday, April 11, 2023 12:34 AM

1. Create a Pod from Nginx Image

Syntax: `kubectl run [NAME-OF-POD] --image=[IMAGE=NAME]`

Command: `kubectl run nginx --image=nginx`

2. Create a Pod and Expose a Port

`kubectl run nginx --image=nginx --port=80`

3. Output the Manifest File

`kubectl run nginx --image=nginx --port=80 --dry-run=client -o yaml`

Output the Manifest File

`kubectl run nginx --image=nginx --port=80 --dry-run=client -o yaml`

To find the memory and CPU

Monday, April 17, 2023 8:44 PM

Kubectl top pods
Kubectl top nodes

Filed selector

```
a>kubectl get pods --all-namespaces --field-selector metadata.namespace!=default
```

Output to json to identify the fields

Kubectl get events -o json

```
C:\Users\Zeal Vora>kubectl get events -o json
```

```
C:\Users\Zeal Vora>kubectl get events --field-selector involvedObject.name=event-pod
LAST SEEN   TYPE      REASON      OBJECT          MESSAGE
14m         Normal    Scheduled    pod/event-pod   Successfully assigned default/event-pod to worke
14m         Normal    Pulling      pod/event-pod   Pulling image "nginx"
14m         Normal    Pulled       pod/event-pod   Successfully pulled image "nginx" in 2.091702674
14m         Normal    Created      pod/event-pod   Created container event-pod
14m         Normal    Started      pod/event-pod   Started container event-pod

C:\Users\Zeal Vora>
```


PVC vs PV

Thursday, May 4, 2023 11:00 PM

PersistentVolume and PersistentVolumeClaim explain in simple words

In simple terms, a PersistentVolume (PV) is a piece of storage in a Kubernetes cluster that can be used by applications running in the cluster. Think of it as a virtual hard drive that applications can access. A PersistentVolumeClaim (PVC) is a request made by an application to use a portion of a PV. When a PVC is created, Kubernetes finds an available PV that meets the requested storage capacity and access mode, and binds the PV to the PVC. The application can then use the storage provided by the PVC.

The use of PVs and PVCs enables applications to be more portable and flexible. They allow applications to request the storage they need without having to know the underlying infrastructure details. This allows for easier management and migration of applications across different environments and cloud providers.

what if a PVC is not available in the provided PV

If a PVC requests storage that is not available in any of the PVs in the Kubernetes cluster, the PVC will remain in a pending state until an appropriate PV becomes available.

This can happen if the PVC requests more storage than any of the available PVs, or if the access mode of the PVC does not match any of the available PVs. In this case, you may need to create a new PV that meets the requirements of the PVC or adjust the size or access mode of the PVC to match an available PV.

It is important to note that the lifecycle of a PVC is independent of the lifecycle of the pod that uses it. If a pod is deleted, the PVC and PV continue to exist until they are explicitly deleted by an administrator. This means that data stored in a PV can persist between pod restarts, making it useful for stateful applications that require persistent storage.

How to Fix Kubernetes 'Service 503' (Service Unavailable) Error

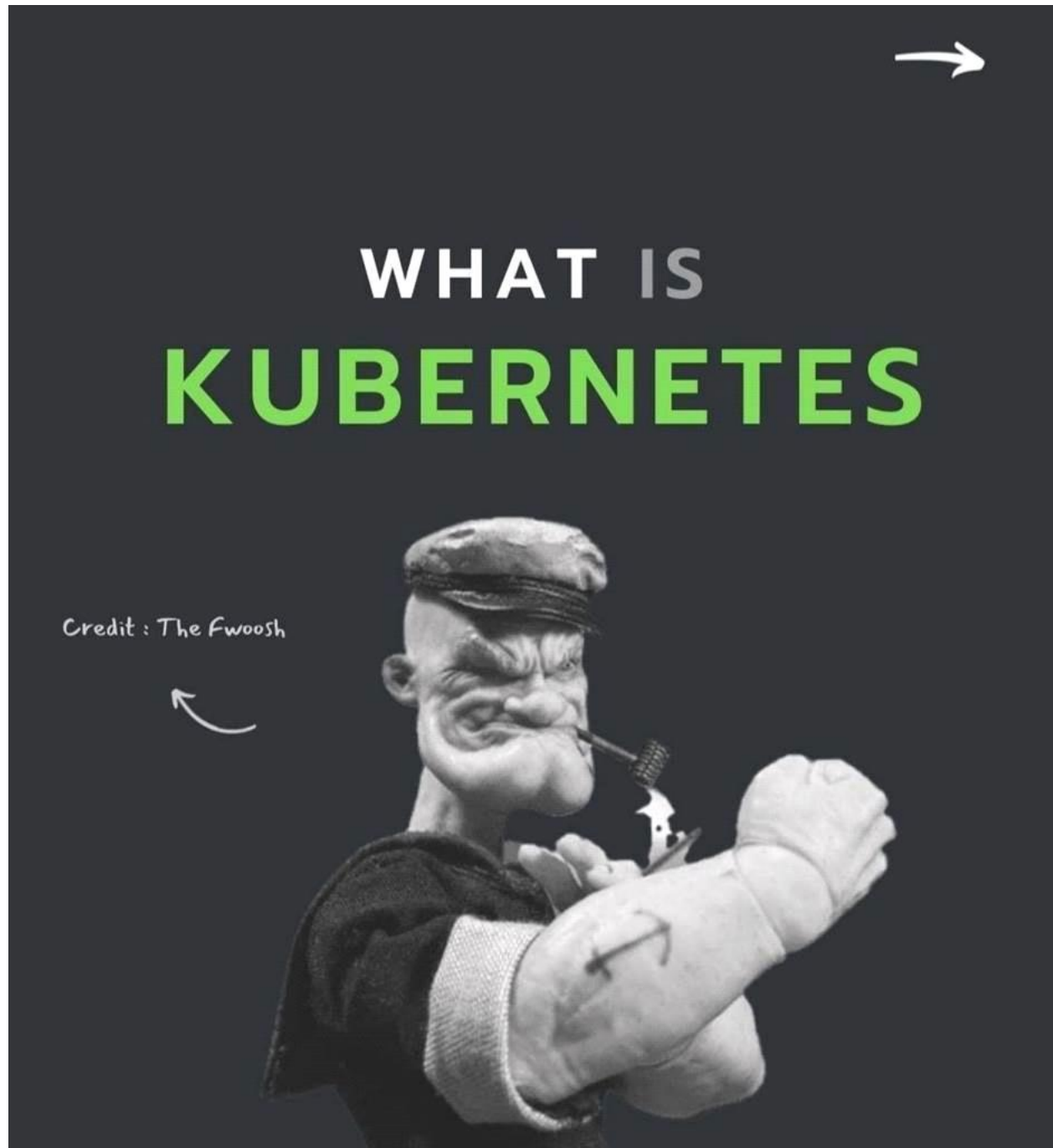
Monday, April 17, 2023 12:59 AM

<https://komodor.com/learn/how-to-fix-kubernetes-service-503-service-unavailable-error/>

What is Kubernetes

Monday, May 15, 2023

12:20 AM



APPLICATION



Lets say you have
created an application





And used **Docker containers**
to package the application

*Docker should make my application work
the same regardless of the environment*



DEPLOYED



Say you have deployed on 3
different servers using Docker

and ...

Your application starts getting massive traffic

Wow my application is doing
better than I thought it would

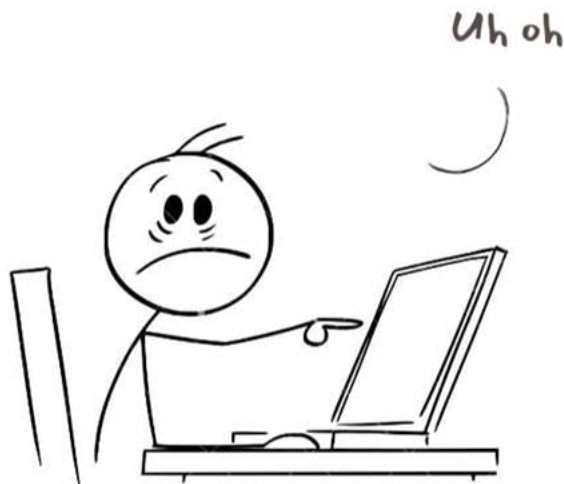


SCALING



Now you need to scale up fast; how will you go from 3 servers to 40 servers that you may require?

How to decide which container should go where?
Monitor all containers ? & make sure they restart if they die?



OUT OF CONTROL



How am I going to
manage all this?

ah I need to
restart them



huh I need to create
more instances

Wouldn't it be easier if
this behavior was
handled by a system?

KUBERNETES



This is where Kubernetes comes into play

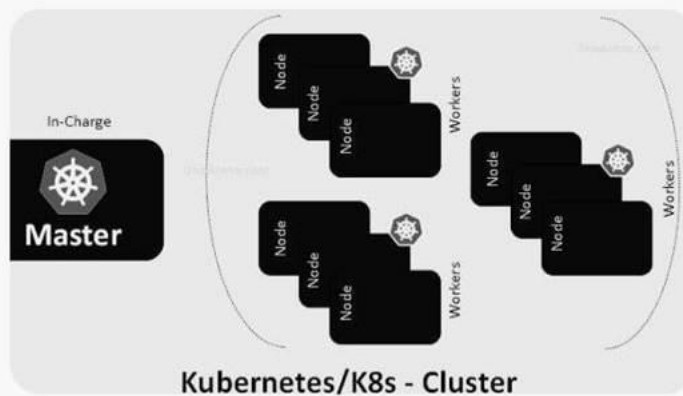
Kubernetes (aka k8s or “kube”) is an open source **container orchestration** platform that automates deploying, managing, and scaling containerized applications.



HOW IT WORKS ?

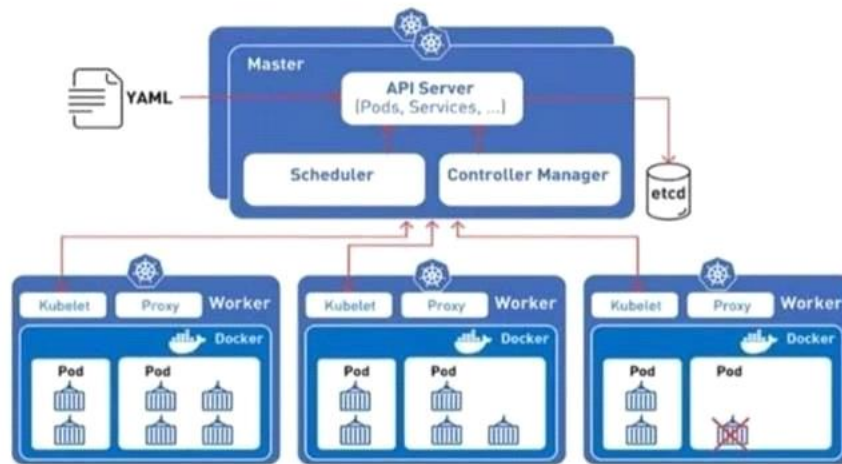


A Kubernetes cluster consists of a set of worker machines, called **nodes**, that run containerized applications



Every cluster has at least one **worker node**. Hence if a node fails, your application will still be accessible from the other nodes as in a cluster, **multiple** nodes are grouped.

ARCHITECTURE



Every node contains a container runtime, **Kubelet** (for starting, stopping, and managing individual containers by requests from the Kubernetes control plane), and **kube-proxy** (for networking and load balancing).

Kubectl Commands

Monday, December 19, 2022 3:03 AM

Kubectl get nodes -> to get all nodes
kubectl get pods

#Gives more details
Kubectl get pods -o wide

Kubectl edit deployment deploymentName
Kubectl get replicaset

Kubectl logs podName

Kubectl describe pod podName -> to get pod description

kubectl exec -it -podName -- bin/bash > to check what is going on inside a pod

kubectl delete deployment deploymentName

FYI, All the CRUD operations of create, delete happens at deployment level

To create deployment with multiple configurations, you can use apply command

kubectl apply -f ConigFileName.yaml

This same command can be used with services and volumes

kubectl get pods -o wide -> This command is used to get IP address of the pods
Kubectl get deployment deploymentName -o yaml

The image shows a summary card with a light blue background and rounded corners. It is divided into three sections, each with a title in a light gray rounded rectangle. The first section, 'CRUD commands', lists three actions: 'Create deployment' (in blue), 'Edit deployment' (in blue), and 'Delete deployment' (in blue), each followed by its corresponding kubectl command. The second section, 'Status of different K8s components', shows the command 'kubectl get nodes | pod | services | replicaset | deployment'. The third section, 'Debugging pods', is currently empty.

CRUD commands	
Create deployment	kubectl create deployment [name]
Edit deployment	kubectl edit deployment [name]
Delete deployment	kubectl delete deployment [name]

Status of different K8s components

kubectl get nodes | pod | services | replicaset | deployment

Debugging pods

Debugging pods

Log to console

```
kubectl logs [pod name]
```

Debugging pods

Log to console

Get **Interactive Terminal**

Get **info** about pod

```
kubectl logs [pod name]
```

```
kubectl exec -it [pod name] -- bin/bash
```

```
kubectl describe pod [pod name]
```

Use configuration file for CRUD

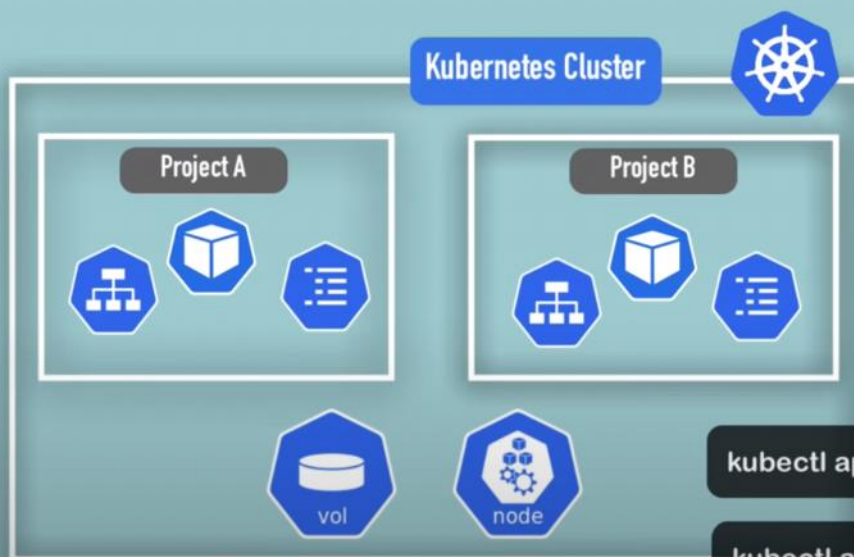
Apply a configuration file

```
kubectl apply -f [file name]
```

Delete with configuration file

```
kubectl delete -f [file name]
```

Components, which can't be created within a Namespace



- live globally in a cluster

- you can't isolate them

```
kubectl api-resources --namespaced=false
```

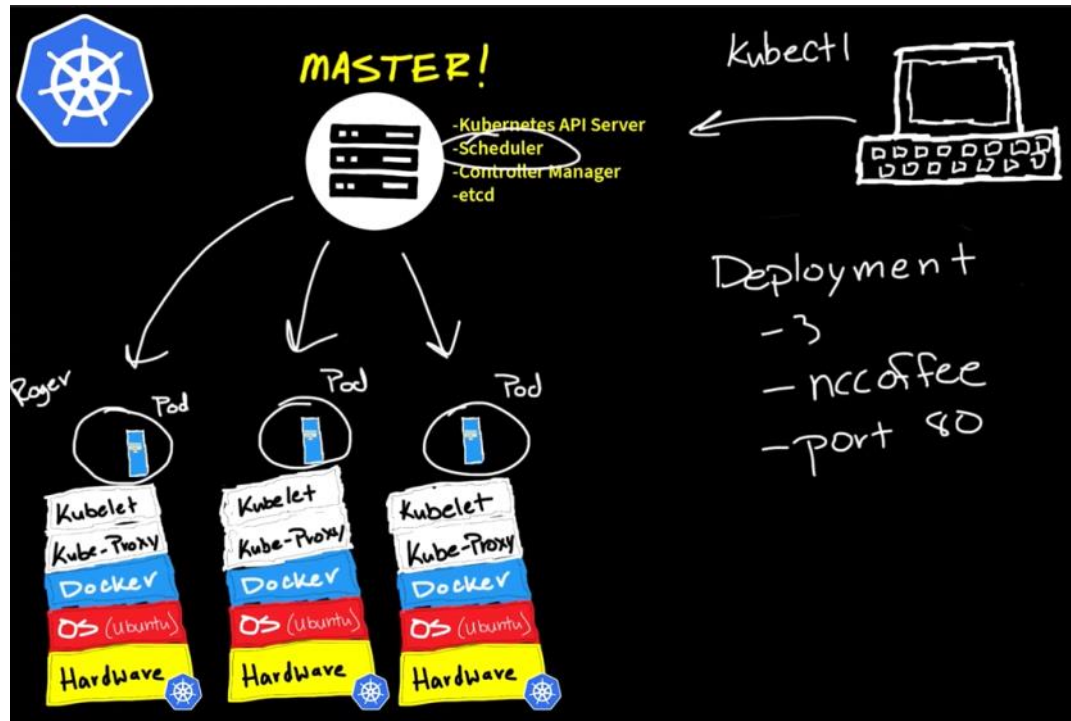
```
kubectl api-resources --namespaced=true
```

`kubectl api-resources -namespace=false` -> this would list all the resource that are not bounded to a namespace like volumes and nodes

As volume and node live globally and can't be isolated

Kubernetes in 30 minutes

Wednesday, March 15, 2023 1:27 PM



```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: networkchuckcoffee-deployment
5    labels:
6      app: nccoffee
7  spec:
8    replicas: 3
9    selector:
10     matchLabels:
11       app: nccoffee
12   template:
13     metadata:
14       labels:
15         app: nccoffee
16     spec:
17       containers:
18         - name: nccoffee
19           image: thenetworkchuck/nccoffee:pourover
20           imagePullPolicy: Always
21           ports:
22             - containerPort: 80
```

YAML
mani

Deployment yaml is used to create pods

```
networkchuck@voldemort:~$ kubectl apply -f nccoffeedeployment.yaml
```

```
kubectl get pods
```

```
#Gives more details
```

```
Kubectl get pods -o wide
```


Service yaml is used to expose multiple pods via a load balancer to an internet network

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: coffee-service
5    annotations:
6      service.beta.kubernetes.io/linode-loadbalancer-throttle: "4"
7    labels:
8      app: coffee-service
9  spec:
10   type: LoadBalancer
11   ports:
12   - name: http
13     port: 80
14     protocol: TCP
15     targetPort: 80
16   selector:
17     app: nccoffee
18   sessionAffinity: None
```

```
kubectl apply -f coffee-service.yaml
```

Kubectl get services

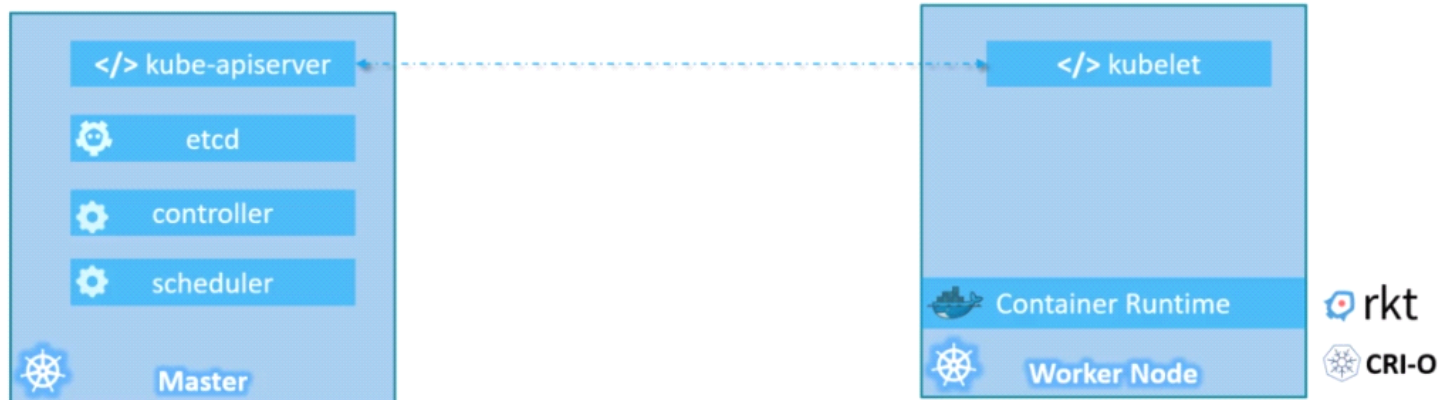
Describe command is used to get more details

```
networkchuck@voldemort:~$ kubectl get services coffee-service
NAME                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
coffee-service     LoadBalancer       10.128.162.242  45.79.242.206    80:30080->80
networkchuck@voldemort:~$ kubectl describe services coffee-service
```

To edit deployment - used to upscale or downscale and to deploy a new code

```
kubectl edit deployment networkchuckcoffee-deployment
```


Master vs Worker Nodes



kubectl

```
kubectl run hello-minikube
```



```
kubectl cluster-info
```



```
kubectl get nodes
```



K8s Commands

Sunday, March 19, 2023 9:20 PM

Create a new pod with the `nginx` image.

```
kubectl run nginx --image=nginx
```

Watch `kubectl get pods --` refreshes every 2 seconds

Which namespace has the `blue` pod in it?

```
kubectl get pods --all-namespaces |grep blue marketing
```

commands

```
> kubectl create -f replicaset-definition.yml
```

```
> kubectl get replicaset
```

```
> kubectl delete replicaset myapp-replicaset
```

*Also deletes all underlying PODs

```
> kubectl replace -f replicaset-definition.yml
```

```
> kubectl scale --replicas=6 -f replicaset-definition.yml
```

Deployment



Definition

```
> kubectl create -f deployment-definition.yml
deployment "myapp-deployment" created
```

```
> kubectl get deployments
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
myapp-deployment	3	3	3	3	21s

```
> kubectl get replicaset
```

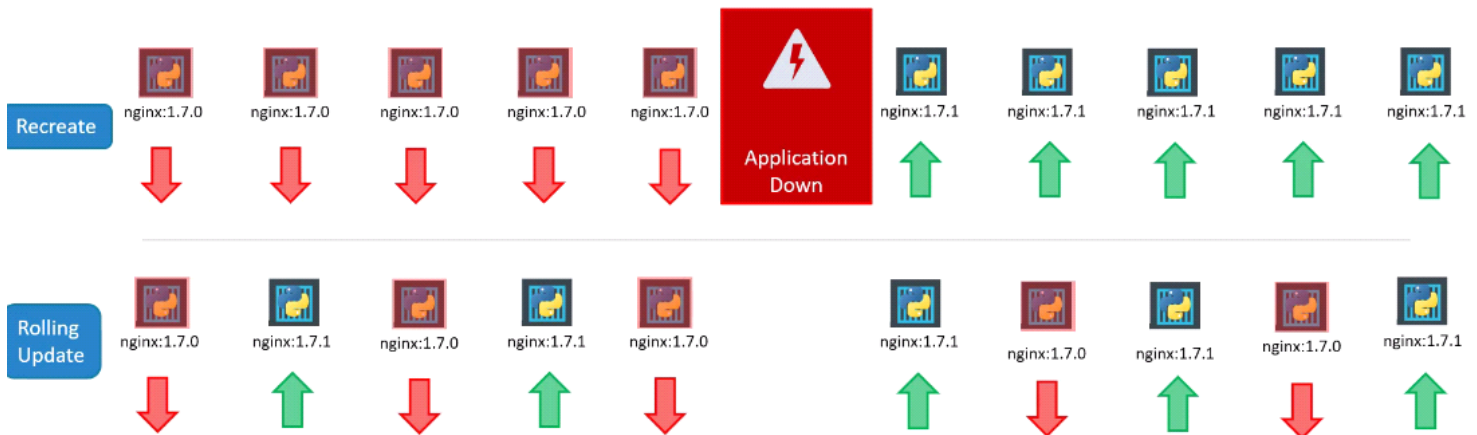
NAME	DESIRED	CURRENT	READY	AGE
myapp-deployment-6795844b58	3	3	3	2m

```
> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-deployment-6795844b58-5rbjl	1/1	Running	0	2m
myapp-deployment-6795844b58-h4w55	1/1	Running	0	2m
myapp-deployment-6795844b58-1fjvh	1/1	Running	0	2m

```
deployment-definition.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

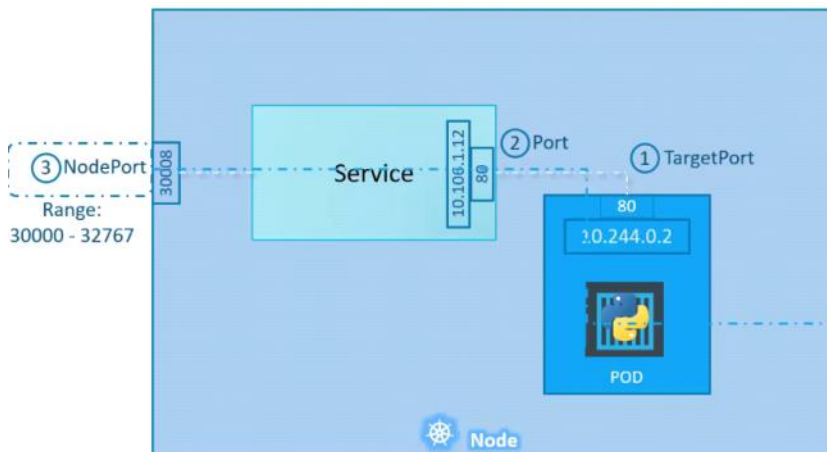
Deployment Strategy



Summarize Commands

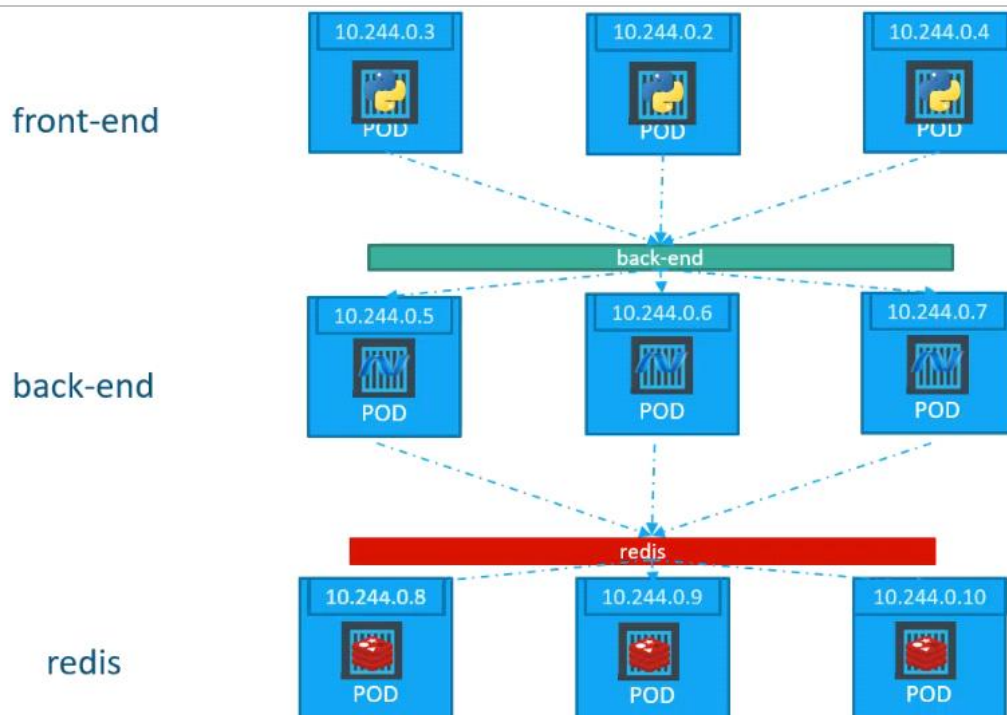
Create	> kubectl create -f deployment-definition.yml
Get	> kubectl get deployments
Update	> kubectl apply -f deployment-definition.yml
	> kubectl set image deployment/myapp-deployment nginx=nginx:1.9.1
Status	> kubectl rollout status deployment/myapp-deployment
	> kubectl rollout history deployment/myapp-deployment
Rollback	> kubectl rollout undo deployment/myapp-deployment

Service - NodePort



```
service-definition.yml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
```

ClusterIP



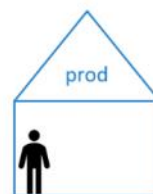
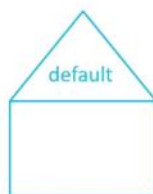
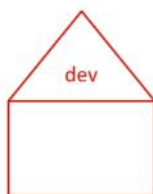
service-definition.yml

```
apiVersion: v1
kind: Service
metadata:
  name: back-end
spec:
  type: ClusterIP
  ports:
    - targetPort: 80
      port: 80
  selector:
    app: myapp
    type: back-end
```

Example voting app



Switch



```
> kubectl get pods --namespace=dev
```

```
> kubectl get pods
```

```
> kubectl get pods --namespace=prod
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=dev
```

```
> kubectl get pods
```

```
> kubectl get pods --namespace=default
```

```
> kubectl get pods --namespace=prod
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=prod
```

```
> kubectl get pods --namespace=dev
```

```
> kubectl get pods --namespace=default
```

```
> kubectl get pods
```

```
> kubectl get pods --all-namespaces
```

4. Use Shortcuts/Aliases

po for PODs
rs for ReplicaSets
deploy for Deployments
svc for Services
ns for Namespaces
netpol for Network policies
pv for Persistent Volumes
pvc for PersistentVolumeClaims
sa for service accounts

Command

Sunday, May 21, 2023

12:08 PM

To check current Zone of Pods

```
kubectrl exec -it dev-digitalstore-1-0-0-c684fd4f5-9j8dq -- date
```


Probe Handling in Kubernetes

Wednesday, June 14, 2023 1:41 PM

QQ : What happens when liveness and readiness probe URL are down(500 or 503 error code).

In Kubernetes, liveness and readiness probes are used to determine the health and availability of a pod. The behavior of Kubernetes when the probe URL returns a 500 or 503 error code depends on the configuration of the probes and the overall pod management policy.

Liveness Probe:

- If the liveness probe fails, indicating that the pod is not functioning properly, Kubernetes will automatically restart the pod. The probe is usually configured with an initial delay before it starts, so the pod has some time to stabilize before the probe begins.
- If the liveness probe URL returns a 500 or 503 error code, Kubernetes interprets it as a failure and triggers a pod restart.

Readiness Probe:

- The readiness probe determines whether a pod is ready to receive traffic. If the readiness probe fails, Kubernetes will stop sending traffic to that pod, but it will not automatically restart the pod. The pod remains in its current state until the readiness probe succeeds again.
- If the readiness probe URL returns a 500 or 503 error code, Kubernetes considers the probe as a failure, and the pod is marked as not ready. However, the pod will not be restarted solely based on the readiness probe failure.

It's important to note that while Kubernetes can handle the automatic restart of a pod based on liveness probe failures, it does not handle the automatic recovery of a failing service. If the probe URL consistently returns 500 or 503 errors, it indicates an issue within the application or service itself, which needs to be investigated and resolved separately from Kubernetes.

Kubernetes POD Troubleshooting techniques

Monday, August 21, 2023 11:44 AM

1. Check logs:

Use 'kubectl logs <pod_name>'

2. Analyze Pod Status:

Use 'kubectl get pod <pod_name>' and examine the status fields

3. Describe Pod:

Execute 'kubectl describe pod <pod_name>'

4. Verify Pod Configuration:

Review the pod's YAML configuration using 'kubectl get pod <pod_name> -o yaml'

5. Check Events:

Run 'kubectl get events'

6. Validate Container Images:

Check image availability and version in the pod's YAML using "kubectl get pod <pod_name> -o=jsonpath='{.spec.containers[*].image}'."

7. Restart Pod:

Use 'kubectl rollout restart deployment/<deployment_name>'

8. Review Service Dependencies:

Analyze dependencies in the YAML or documentation to ensure required services are available and running

9. Check Network Connectivity:

Get a shell in the running container:

'kubectl exec -it <pod_name> -- sh'

Use 'ping' or 'curl' to test network connectivity:

'ping <target_host>'

'curl <target_url>'

10. Inspect Resource Usage:

Utilize 'kubectl top pod <pod_name>'

11. Check Probes:

Execute 'kubectl describe pod <pod_name>'

Look for the "LivenessProbe", "ReadinessProbe", and "StartupProbe" sections to see if the probes are passing or failing.

12. Compare Recent Deployment with Current Version:

Use the following commands:

'kubectl rollout history deployment/<deployment_name> --revision=<old_revision_number>'

'kubectl rollout history deployment/<deployment_name> --revision=<new_revision_number>'

K8s Munshad

Monday, November 6, 2023 4:07 PM

Pods - Deploys single instance of the application

RS - Deploys multiple instance of one application

Deployments - Provides us the capability to upgrade the instances seamlessly using rolling update, undo change and pause and resume changes as required

Replica sets

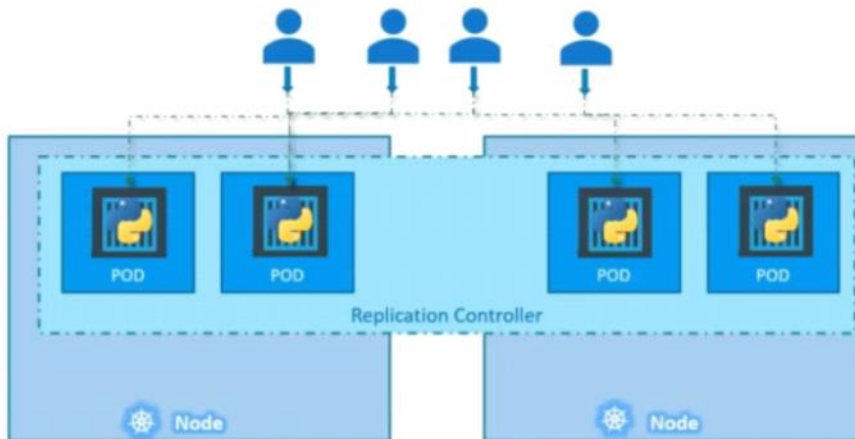
Monday, November 6, 2023 4:35 PM

The replication controllers helps us running multiple instance of a single pod and ensures that the specified no of pods runs al the time

It spans across multiple nodes in a cluster

Where Replica Sets is its New version

Load Balancing & Scaling



ReplicationController - Old version

```
rc-definition.yml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
```

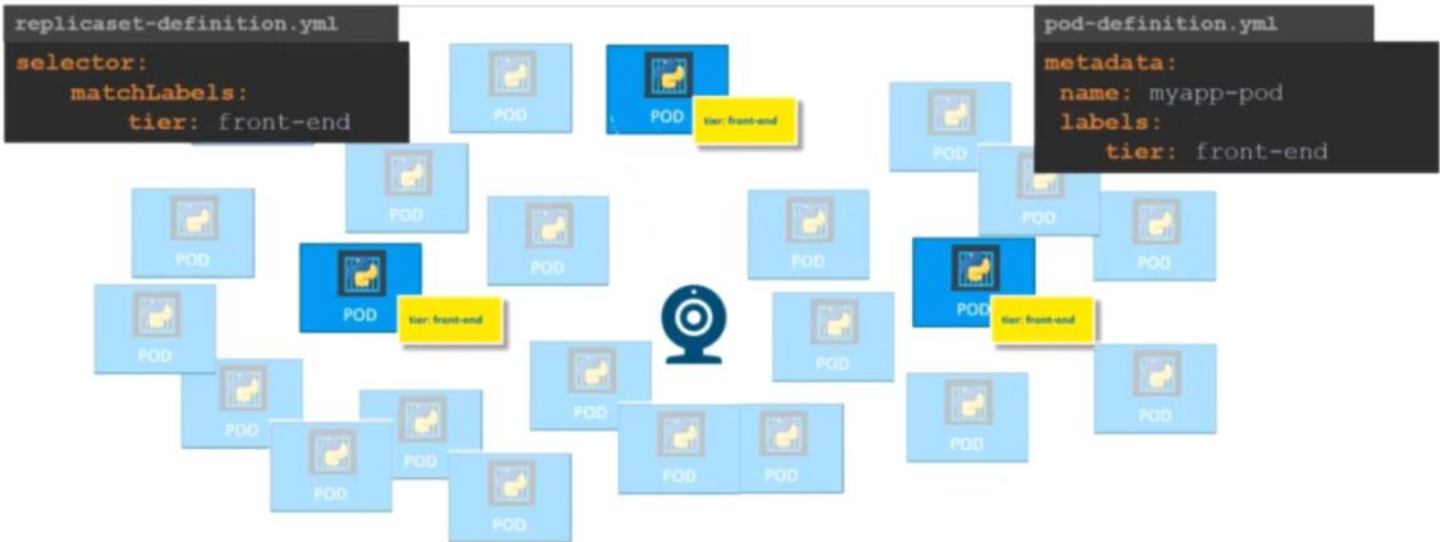
```
> kubectl create -f rc-definition.yml
replicationcontroller "myapp-rc" created
```

```
replicaset-definition.yml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

The major diff between replication controller vs replica set is the selector tag in replicas set

Why we need it? It can be used to manage the pods which are created before the creation of replicas set

Labels and Selectors



How to scale a replica set

1st way

Scale

```
> kubectl replace -f replicaset-definition.yml
```

```
replicaset-definition.yml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 6
  selector:
    matchLabels:
```

2nd way

```
> kubectl scale --replicas=6 -f replicaset-definition.yml
```

```
> kubectl scale --replicas=6 replicaset myapp-replicaset
```

↓ ↓
TYPE NAME

This wont be updating the file automatically

commands

```
> kubectl create -f replicaset-definition.yml
```

```
> kubectl get replicaset
```

```
> kubectl delete replicaset myapp-replicaset
```

*Also deletes all underlying PODs

```
> kubectl replace -f replicaset-definition.yml
```

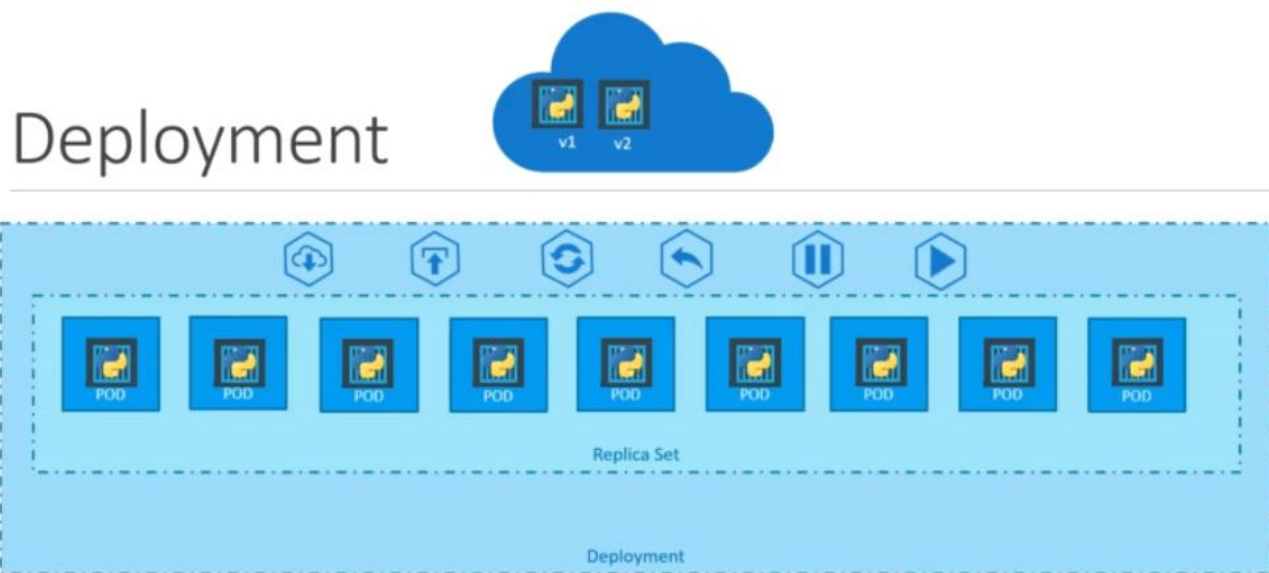
```
> kubectl scale --replicas=6 -f replicaset-definition.yml
```

kubectl scale rs new-replica-set --replicas=5

```
bash Copy code
kubectl get replicaset --all-namespaces
```

Deployments

Tuesday, November 7, 2023 1:59 PM



Deployments comes higher in the hierarchy above Replica Sets

Pods - Deploys single instance of the application

RS - Deploys multiple instance of one application

Deployments - Provides us the capability to upgrade the instances seamlessly using rolling update, undo change and pause and resume changes as required

Definition

```
> kubectl create -f deployment-definition.yml
```

```
deployment "myapp-deployment" created
```

```
> kubectl get deployments
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
myapp-deployment	3	3	3	3	21s

```
> kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
myapp-deployment-6795844b58	3	3	3	2m

```
> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-deployment-6795844b58-5rbj1	1/1	Running	0	2m
myapp-deployment-6795844b58-h4w55	1/1	Running	0	2m
myapp-deployment-6795844b58-1fjvh	1/1	Running	0	2m

```
deployment-definition.yml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: myapp-deployment
```

```
  labels:
```

```
    app: myapp
```

```
    type: front-end
```

```
spec:
```

```
  template:
```

```
    metadata:
```

```
      name: myapp-pod
```

```
      labels:
```

```
        app: myapp
```

```
        type: front-end
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx-container
```

```
          image: nginx
```

```
replicas: 3
```

```
selector:
```

```
  matchLabels:
```

```
    type: front-end
```


Namespace

Friday, November 17, 2023 10:03 AM

Important only - and not --

To create pod in a specific namespace :
kubectl run redis --image=redis -n finance

Deploy a redis pod using the redis:alpine image with the labels set to tier=db.

```
kubectl run redis -l tier=db --image=redis:alpine
```

Create a service redis-service to expose the redis application within the cluster on port 6379.

```
on port 8000  
kubectl expose rc nginx --port=80 --target-port=8000
```

```
controlplane ~ * kubectl expose pod redis --port 6379 --name redis-service  
service/redis-service exposed
```

Create a new deployment called redis-deploy in the dev-ns namespace with the redis image. It should have 2 replicas.

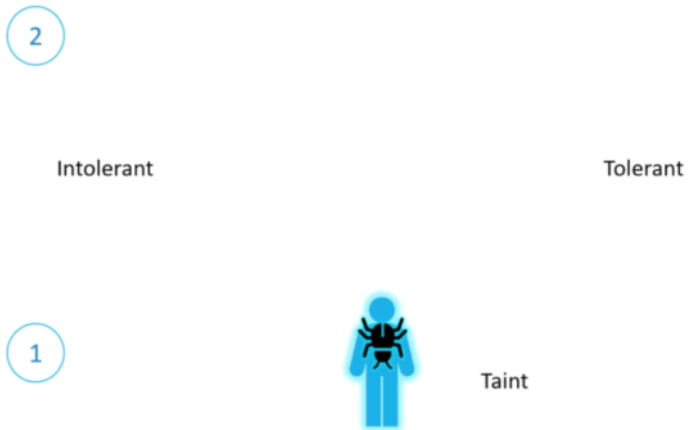
Run the command: kubectl create deployment redis-deploy --image=redis --replicas=2 -n dev-ns

Create a pod called httpd using the image httpd:alpine in the default namespace. Next, create a service of type ClusterIP by the same name (httpd). The target port for the service should be 80.

Run the command: kubectl run httpd --image=httpd:alpine --port=80 --expose

Taint and Tolerant

Monday, November 6, 2023 4:07 PM



Taint - Solution applied
Tolerant - Bug

Person - Node
Bugs - pods

Taints and Tolerant have nothing to do with security, but used to apply restrictions on what pods can be scheduled on a node

No taints



Taint to Node 1 and Tolerant to pod D

A**B****C****D**

2

Taint=blue

1



Node 1



Node 2



Node 3

Taints - Node

```
kubectl taint nodes node-name key=value:taint-effect
```

NoSchedule | PreferNoSchedule | NoExecute

What happens to PODs that do not tolerate this taint?

```
kubectl taint nodes node1 app=blue:NoSchedule
```

✦ Tolerations - PODs

```
kubectl taint nodes node1 :
```

```
pod-definition.yml
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
  tolerations:
    - key: "app"
      operator: "Equal"
      value: "blue"
      effect: "NoSchedule"
```

ClusterIP vs LoadBalancer vs NodePort

Tuesday, December 19, 2023 9:48 AM

ClusterIP vs LoadBalancer vs NodePort

In Kubernetes, services are a fundamental concept used to expose applications running on a set of Pods.

The three primary types of services, ClusterIP, LoadBalancer, and NodePort, differ primarily in how they expose applications to network traffic.

ClusterIP:

Exposes the service internally within the cluster, making it reachable only from within the cluster.

LoadBalancer:

Exposes the service externally using a cloud provider's load balancer, automating the creation of an external IP address.

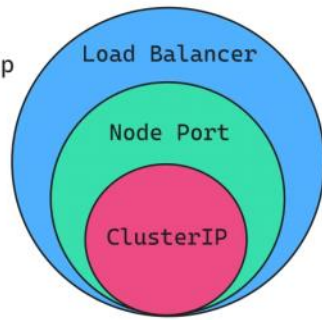
NodePort:

Exposes the service on each Node's IP at a static port, making the service accessible externally through `

The illustration below simplifies the understanding of this concept 

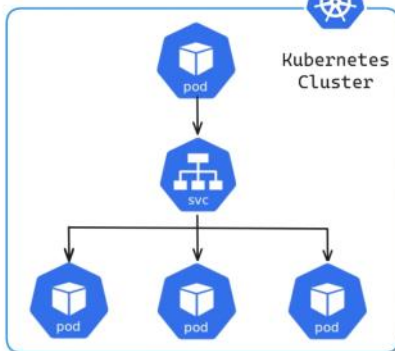
ClusterIP vs LoadBalancer vs NodePort

Basic Relationship

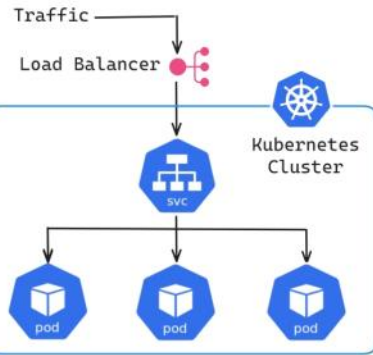


KodeKloud

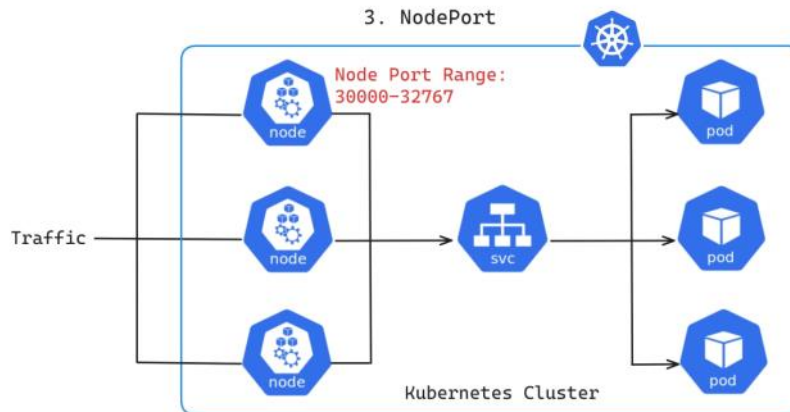
1. ClusterIP



2. LoadBalancer

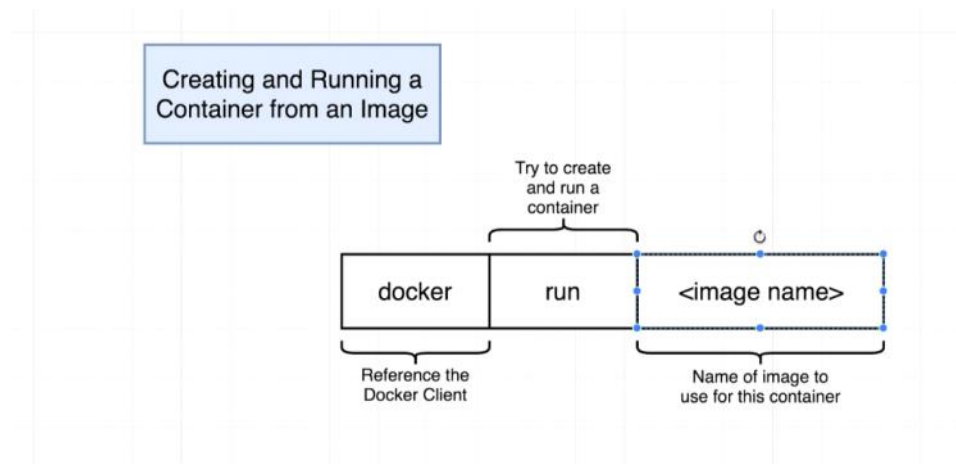


3. NodePort



Docker Stephen Grider

Monday, May 15, 2023 12:21 AM



Docker container run image name

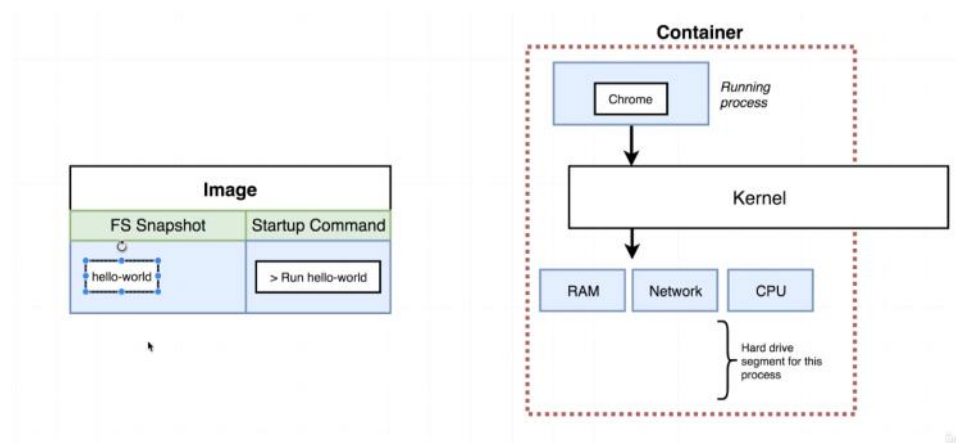
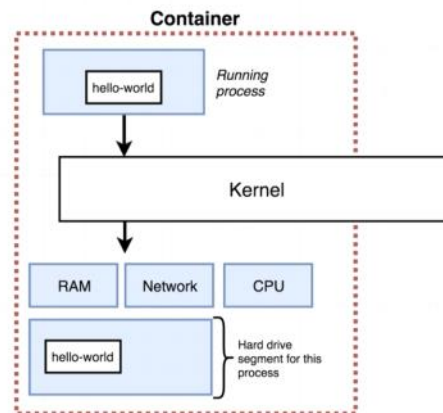
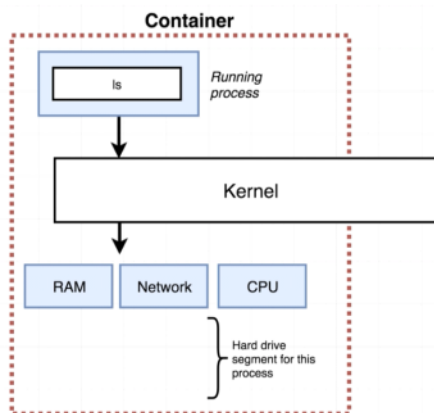


Image	
FS Snapshot	Startup Command
hello-world	> Run hello-world

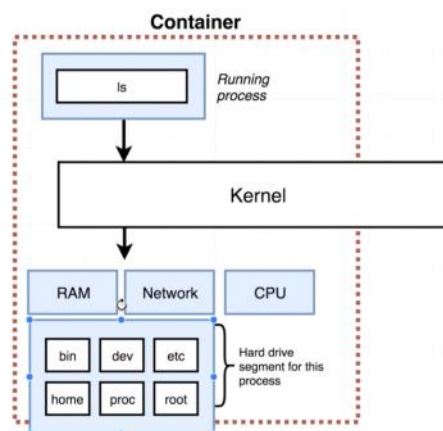


Docker container run busybox ls

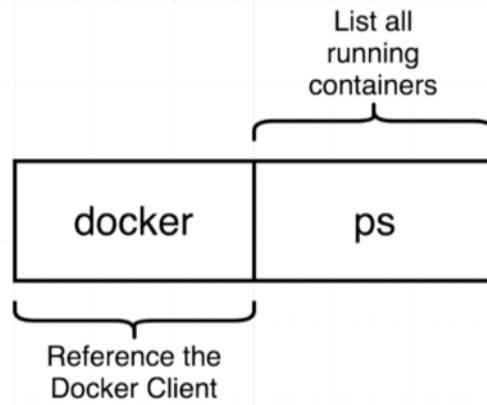
Busybox Image	
FS Snapshot	Startup Command
bin dev etc home proc root	????



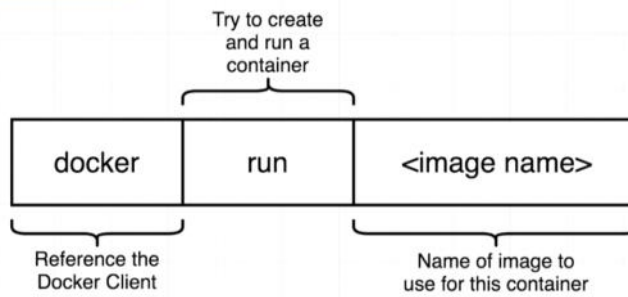
Busybox Image	
FS Snapshot	Startup Command
bin dev etc home proc root	????



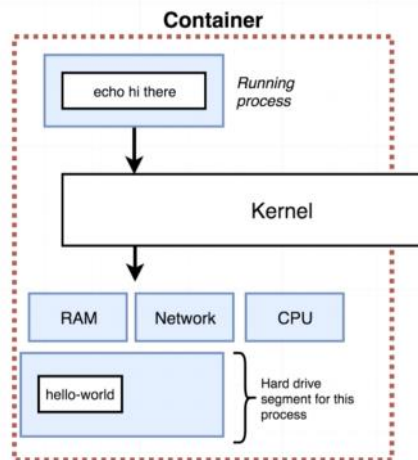
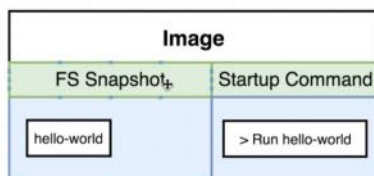
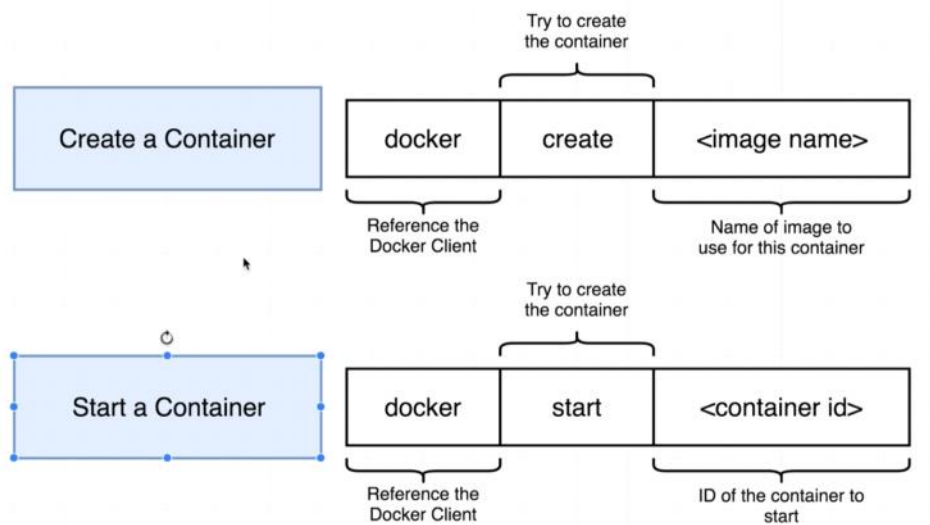
List all running containers



Creating and Running a Container from an Image



`docker run` = `docker create` + `docker start`

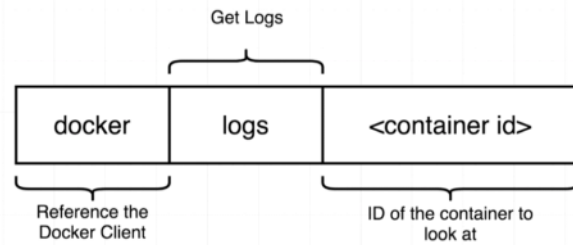


to output logs

`docker start -a containerId`

or `docker container logs ContainerId`

Get logs from a container



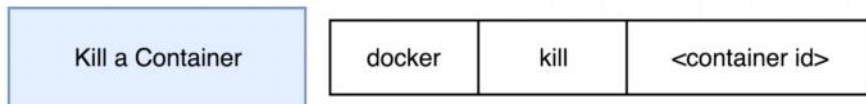
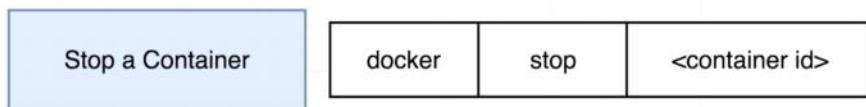
```
→ prod git:(master) docker create busybox echo hi there
a2d69ef73f4dfade7670e093ee8a406116690cbb907cc9007f99de2b56100be1
→ prod git:(master) docker start a2d69ef73f4dfade7670e093ee8a406116690c
99de2b56100be1
a2d69ef73f4dfade7670e093ee8a406116690cbb907cc9007f99de2b56100be1
→ prod git:(master) docker logs a2d69ef73f4dfade7670e093ee8a406116690cb
9de2b56100be1
hi there
→ prod git:(master)
```

SIGTERM -> Terminate Signal -> Gives a little time for the container to shutdown (10 sec), after which it falls back to SIGKILL or kill command

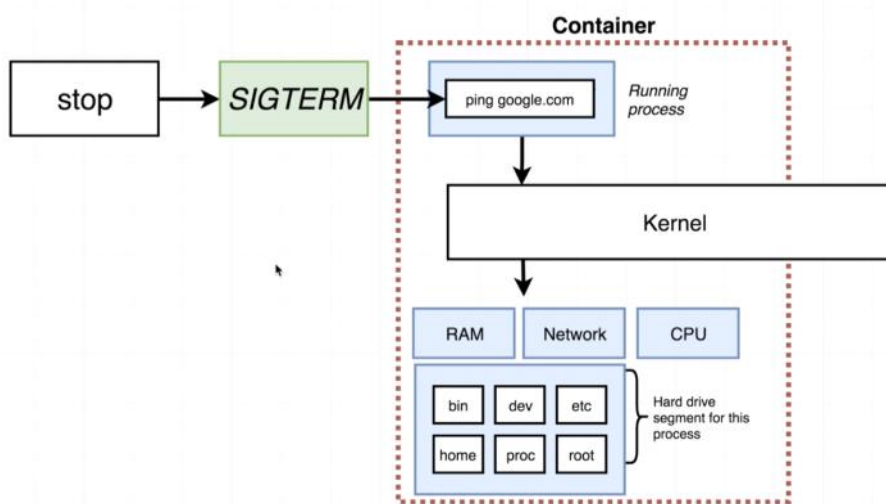
SIGKILL -> Terminate right now

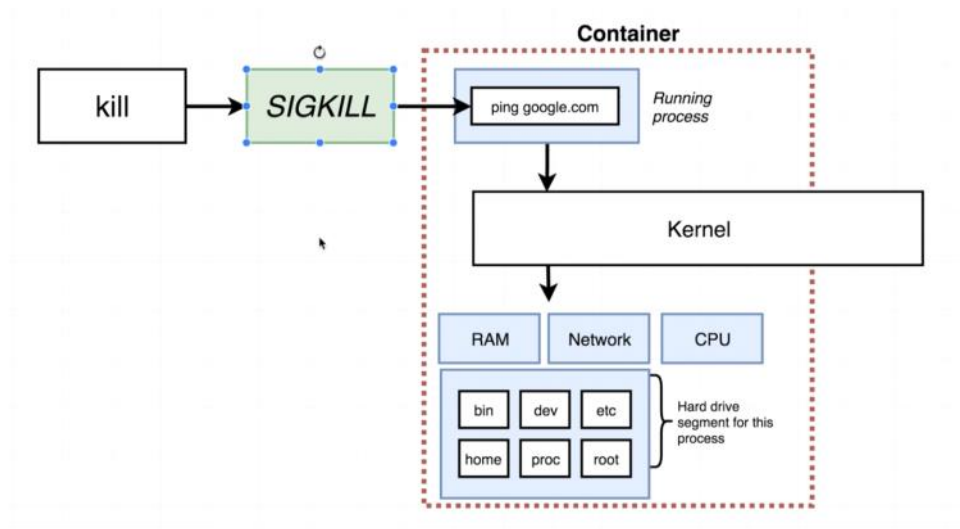
To exit out of a container
Cmd +d

Docker **container** stop containerId

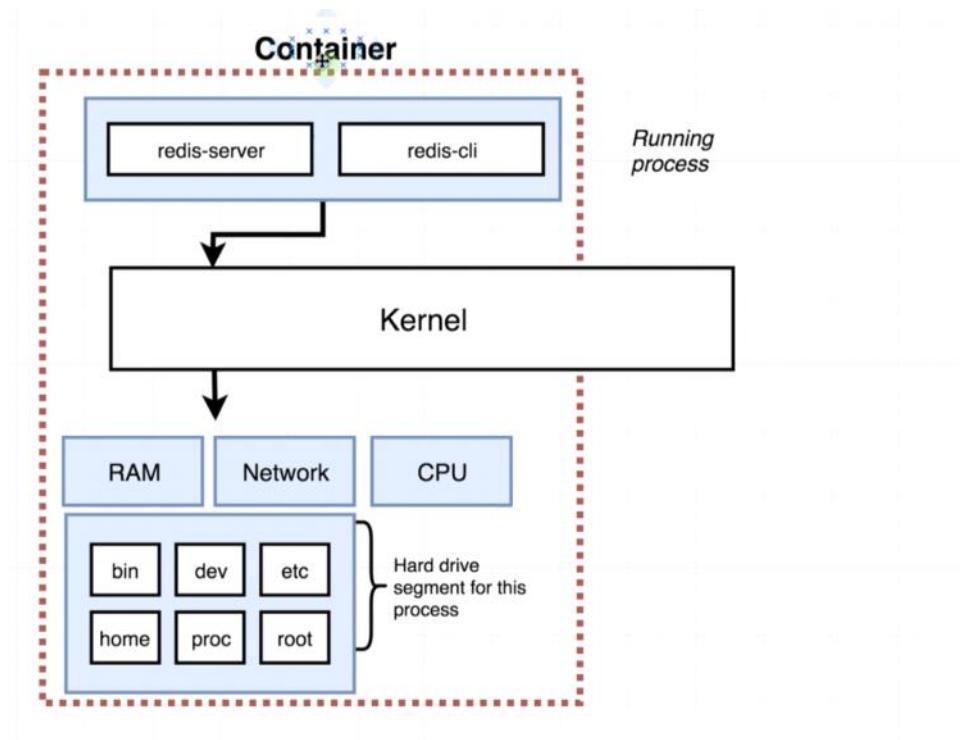


Docker Stop vs Kill



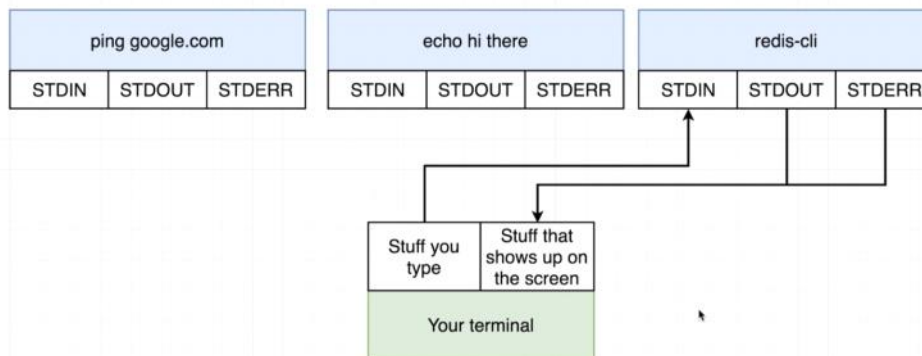
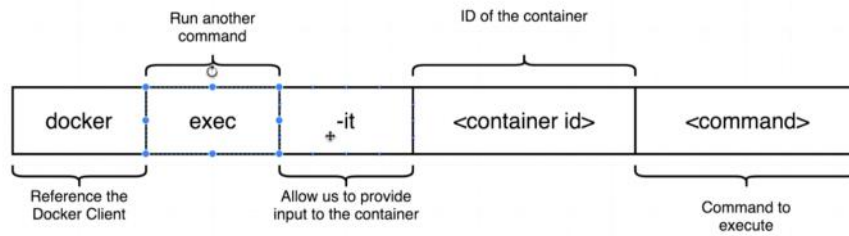


Multi Command container



The purpose of it flag

Execute an additional command in a container

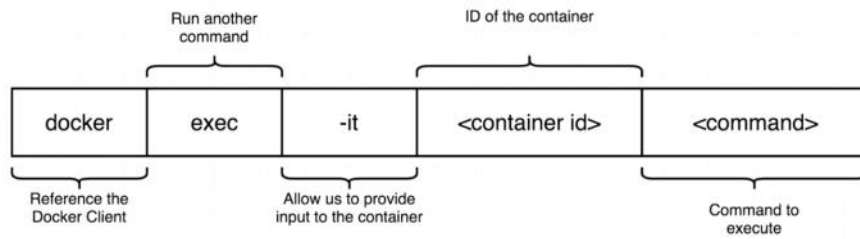


```
→ prod git:(master) ✗ docker exec -i -t
```

-i = Input to container

-t = formatted output to console or connect to a terminal

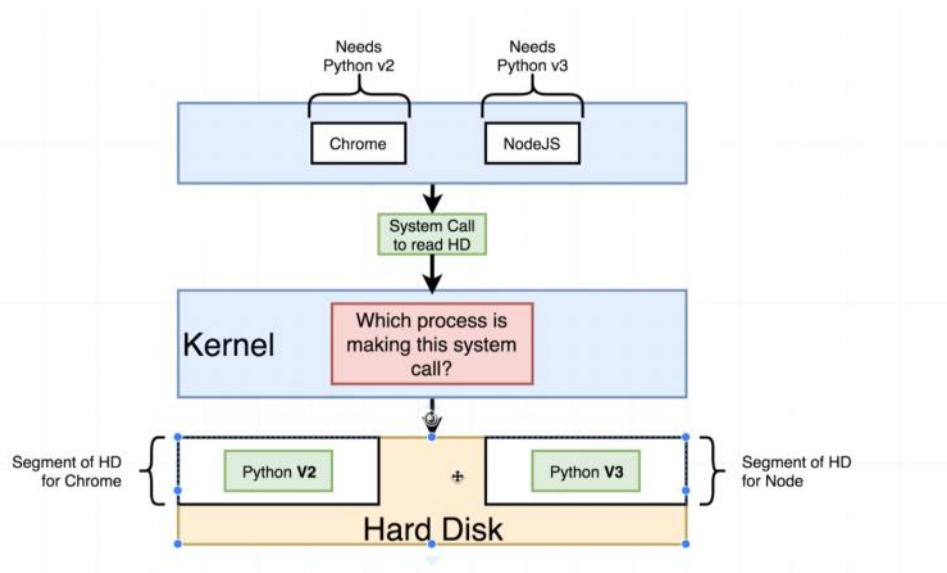
Execute an additional command in a container



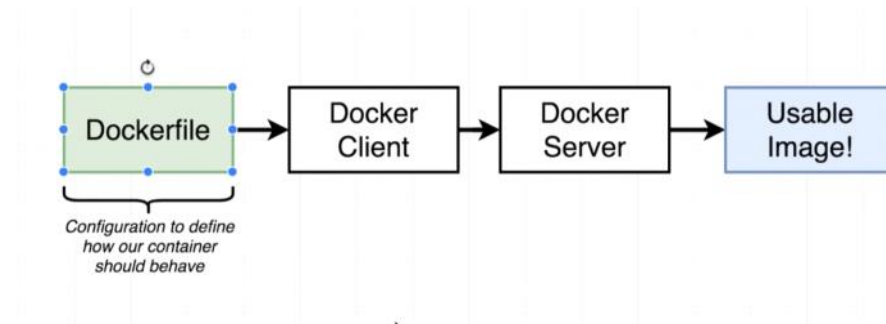
Starting a container with shell

```
→ prod git:(master) docker exec -it 4e3d15293585 sh
#
```

Container Isolation



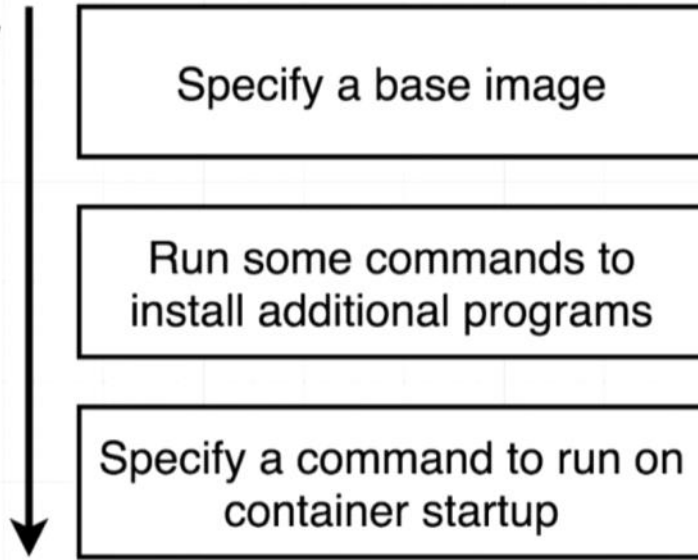
Creating a Docker image



Building a docker file

Creating a Dockerfile

Flow



Create an image that runs redis-server

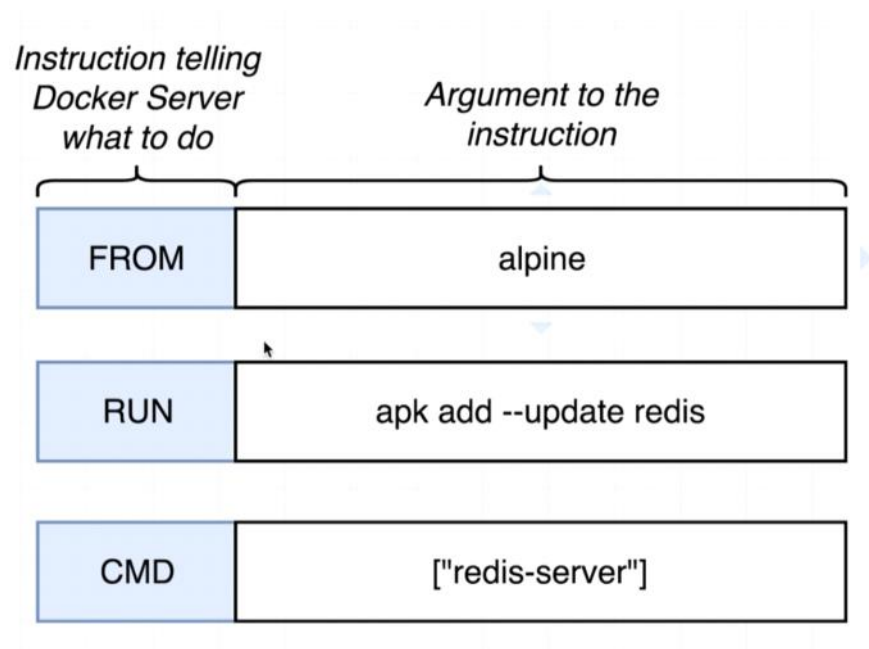
Goal

```
Dockerfile
1 # Use an existing docker image as a base
2
3
4 # Download and install a dependency
5
6
7 # Tell the image what to do when it starts
8 # as a container
```

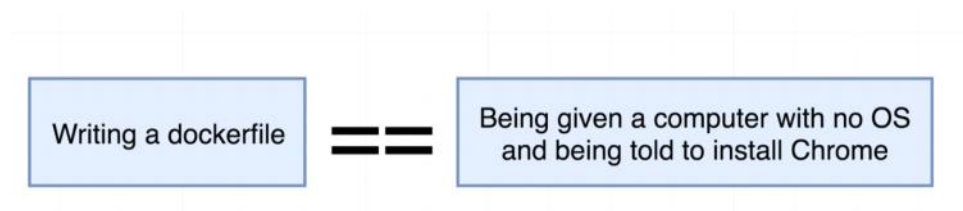
```
Dockerfile x
1 # Use an existing docker image as a base
2 FROM alpine
3
4 # Download and install a dependency
5 RUN apk add --update redis
6
7 # Tell the image what to do when it starts
8 # as a container
9 CMD ["redis-server"]
```

```
redis-image git:(master) docker build .
```

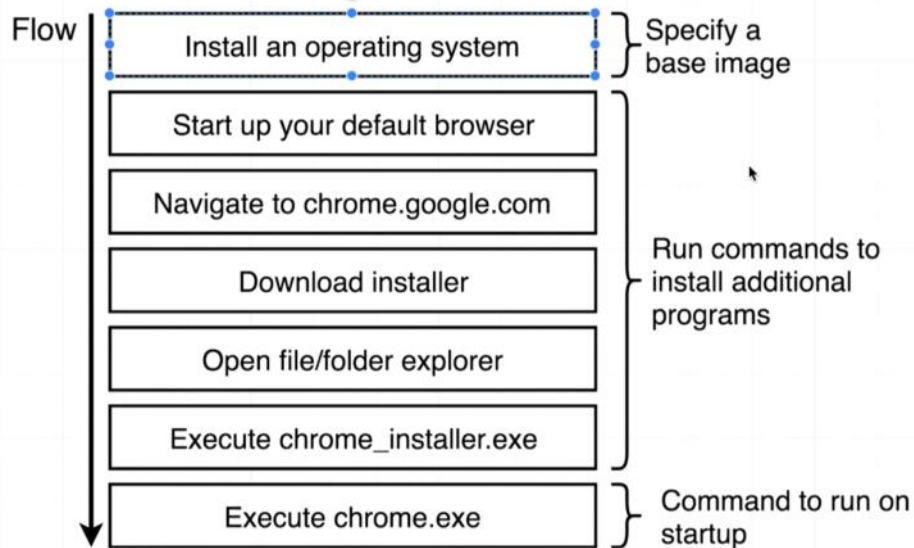
Docker file teardown



What's a Base Image?



How do you install Chrome on a computer with no operating system?

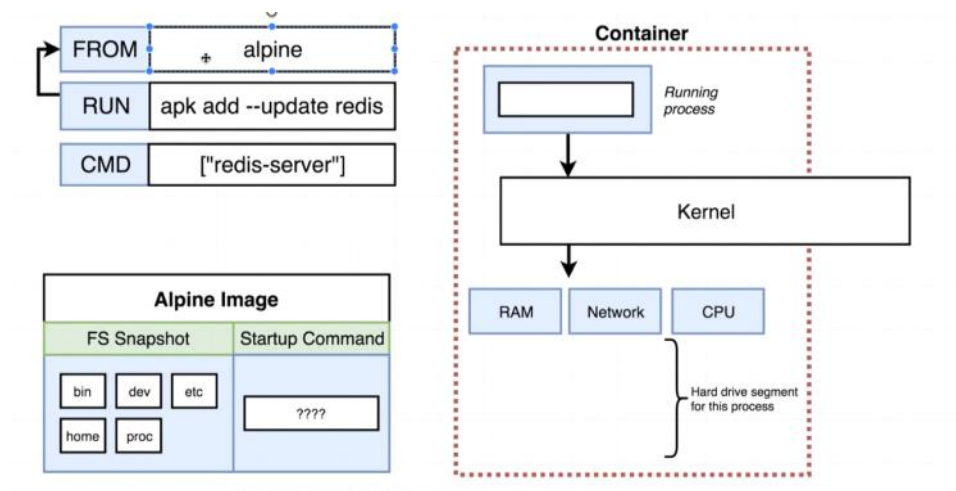


Why did we use alpine as a base image?

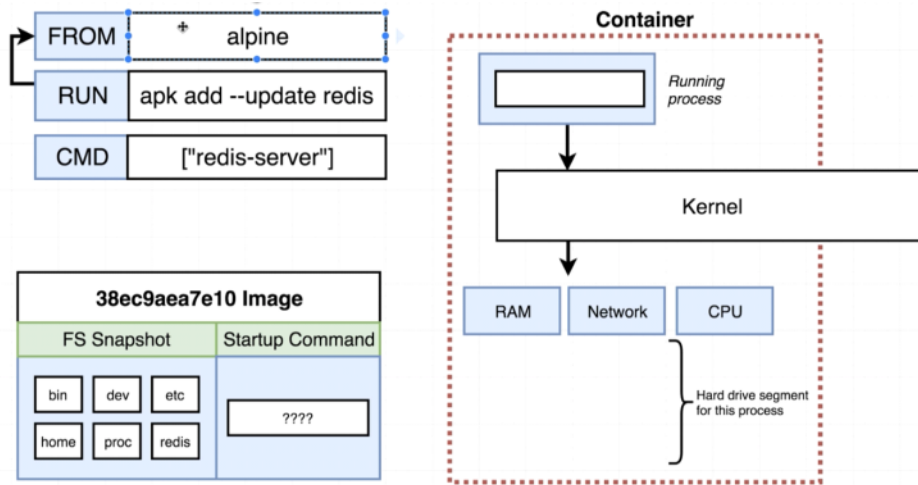
Why do you use Windows, MacOS, or Ubuntu?

They come with a preinstalled set of programs that are useful to you!

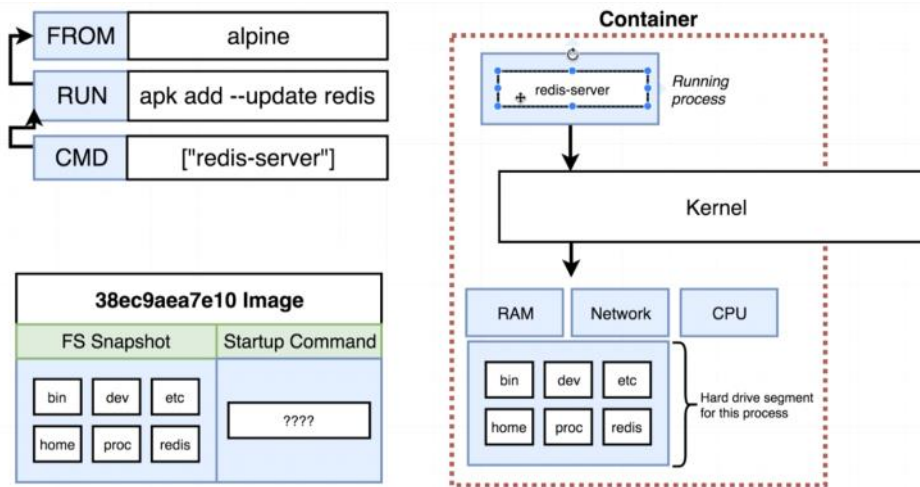
The build process in details

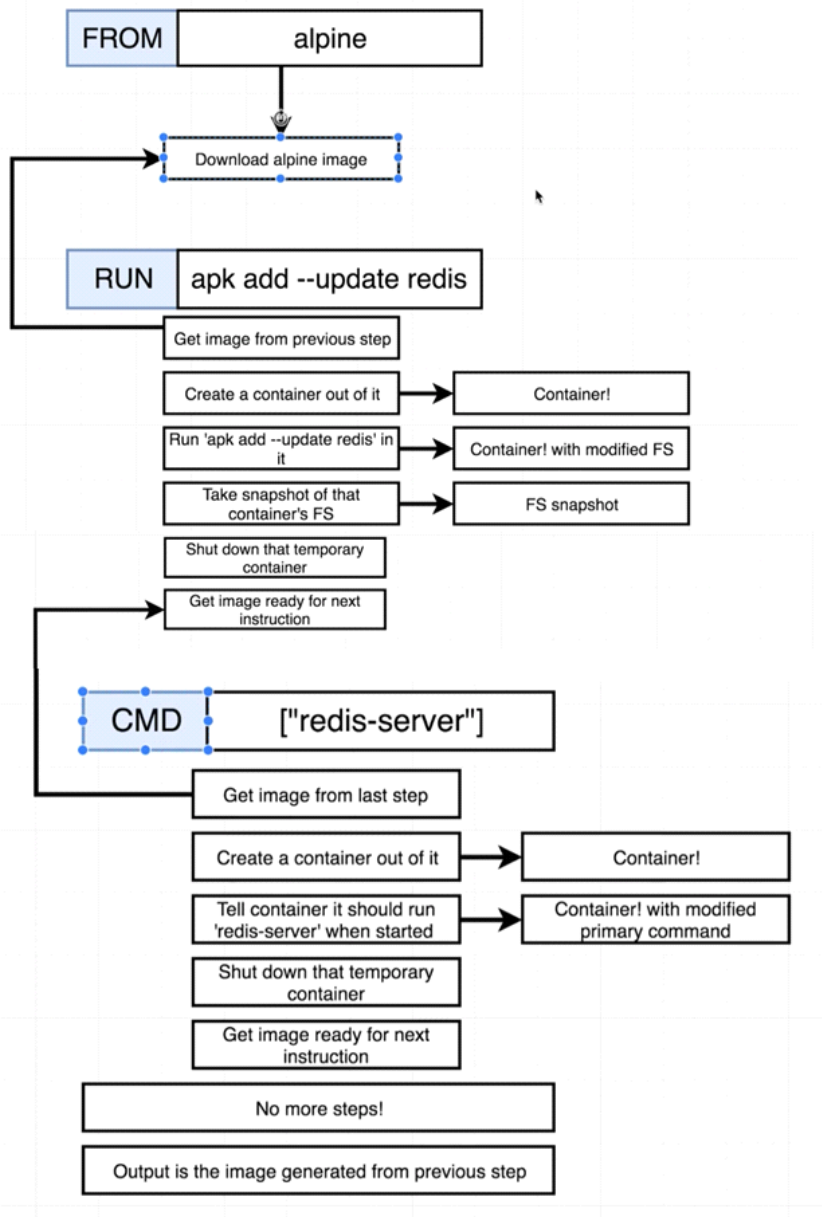


```
---> 58e5de47e10
Step 3/3 : CMD ["redis-server"]
---> Running in e73a22decb35
Removing intermediate container e73a22decb35
---> fc60771eaa08
Successfully built fc60771eaa08
→ redis-image git:(master) X docker run fc60771eaa08
```



Complete recap





Rebuild with cache for any changes made

```

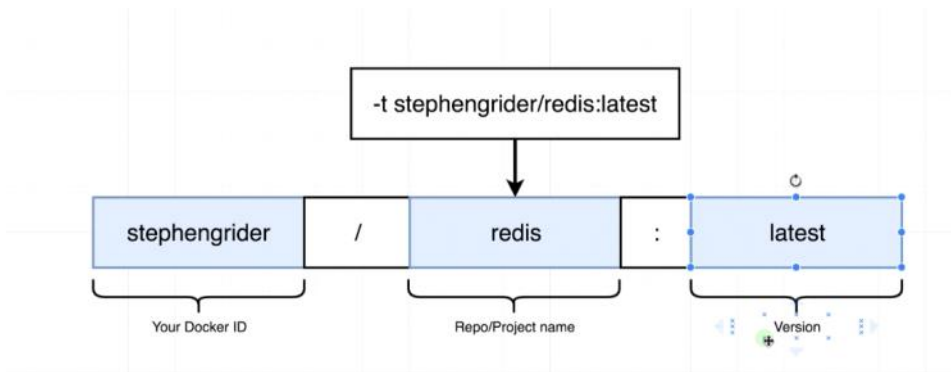
→ redis-image git:(master) ✗ docker build .
Sending build context to Docker daemon 2.048kB
Step 1/4 : FROM alpine
---> 11cd0b38bc3c
Step 2/4 : RUN apk add --update redis
---> Using cache
---> 38ec9aea7e10
Step 3/4 : RUN apk add --update gcc
---> Running in 8340604fcfb4
fetch http://dl-cdn.alpinelinux.org/alpine/v3.8/m

```

Cache using 2nd run

```
→ redis-image git:(master) ✕ docker build .  
Sending build context to Docker daemon 2.048kB  
Step 1/4 : FROM alpine  
---> 11cd0b38bc3c  
Step 2/4 : RUN apk add --update redis  
---> Using cache  
---> 38ec9aea7e10  
Step 3/4 : RUN apk add --update gcc  
---> Using cache  
---> a6d5a254e65b  
Step 4/4 : CMD ["redis-server"]  
---> Using cache  
---> b90e9fb1e7e0  
Successfully built b90e9fb1e7e0  
→ redis-image git:(master) ✕
```

Tagging an Image




```

→ redis-image git:(master) X docker build -t stephengrider/redis:latest .
Sending build context to Docker daemon 2.048kB
Step 1/4 : FROM alpine
--> 11cd0b38bc3c
Step 2/4 : RUN apk add --update gcc
--> Using cache
--> d409bde64c12
Step 3/4 : RUN apk add --update redis
--> Using cache
--> 3266f6626ef3
Step 4/4 : CMD ["redis-server"]
--> Using cache
--> 7dfdfbcf1017
Successfully built 7dfdfbcf1017
Successfully tagged stephengrider/redis:latest
→ redis-image git:(master) X docker run stephengrider/redis

```

Manual Image Generation with Docker Commit

```

→ redis-image git:(master) X docker run -it alpine sh
/ # apk add --update redis
fetch http://dl-cdn.alpinelinux.org/alpine/v3.8/main/x86_64/APKINDEX.ta
r.gz
fetch http://dl-cdn.alpinelinux.org/alpine/v3.8/community/x86_64/APKIND
EX.tar.gz
(1/1) Installing redis (4.0.10-r1)
Executing redis-4.0.10-r1.pre-install
Executing redis-4.0.10-r1.post-install
Executing busybox-1.28.4-r0.trigger
OK: 6 MiB in 14 packages
/ #

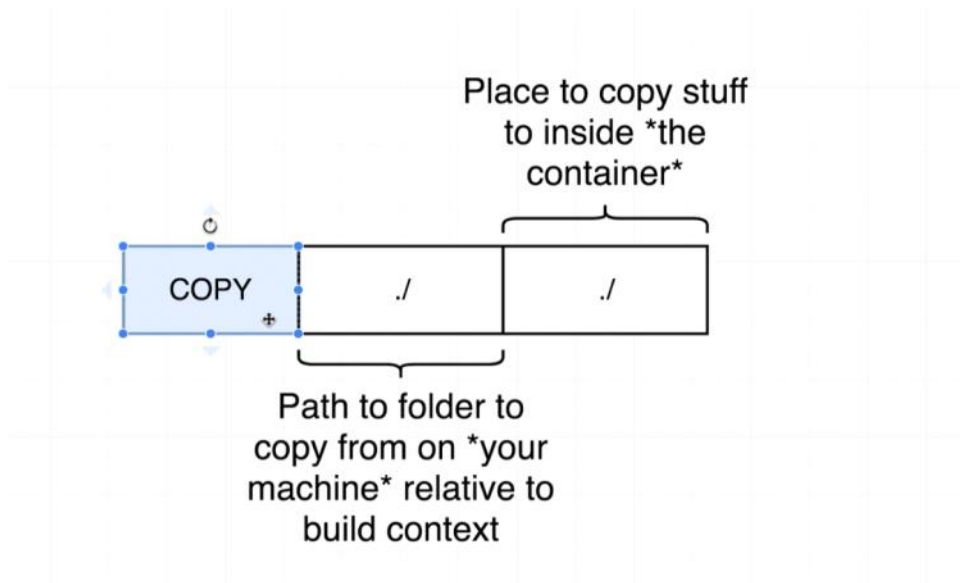
```

```

Last login: Wed Aug 8 13:07:25 on ttys002
→ redis-image git:(master) X docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS            PORTS              NAMES
39075447a383       alpine             "sh"               48 seconds ago
Up 47 seconds      friendly_brown
→ redis-image git:(master) X docker commit -c 'CMD ["redis-server"]' 39075
447a383
sha256:0835898662afae4b5a9d9e9961a0e89c2cbe55d895a0c0fbe6b7664f980d272b
→ redis-image git:(master) X docker run 0835898662af

```

Copy Build files to container

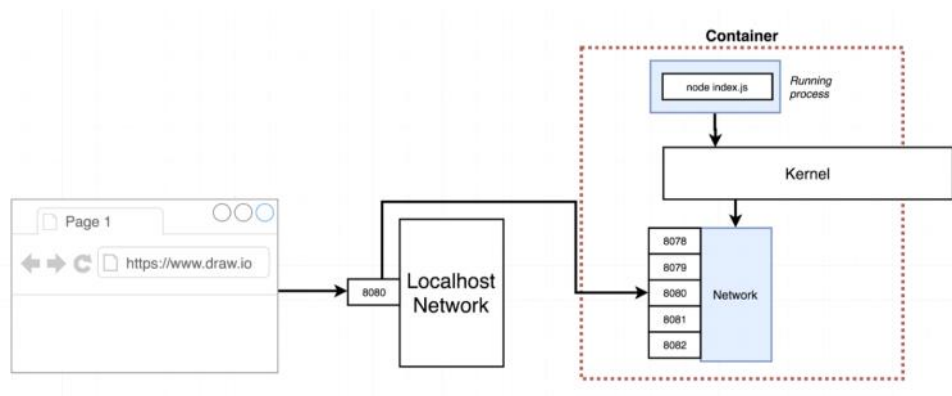


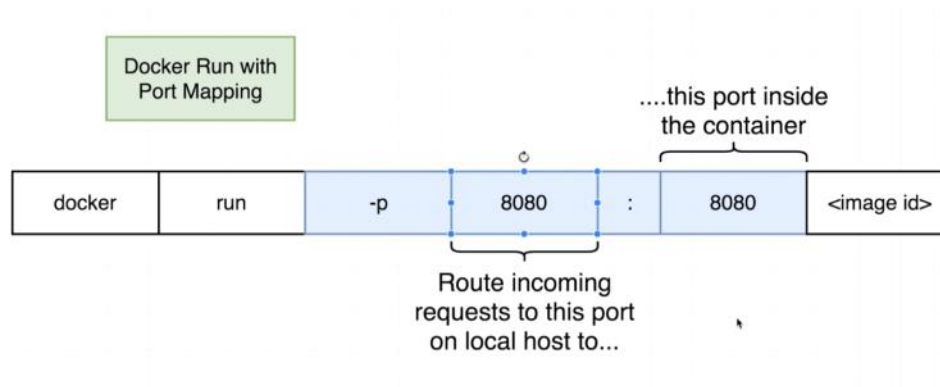
Docker build and run via Docker file

```
❯ docker build -t stephengrider/simpleweb .
```

```
docker run stephengrider/simpleweb
```

Container port mapping





```
docker run -p 8080:8080 stephengrider/simpleweb
```

Overview of EXPOSE instruction

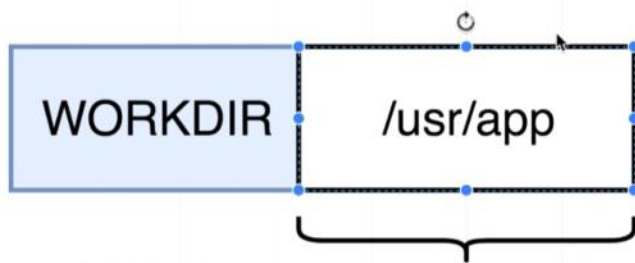
The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime.

The EXPOSE instruction does not actually publish the port.

It functions as a type of documentation between the person who builds the image and the person who runs the container, about which ports are intended to be published.

Example if a spring properties file is set to port 8090, then indicate it in EXPOSE command so as to aid any developers to who uses this app to use this port.

Specifying a work directory

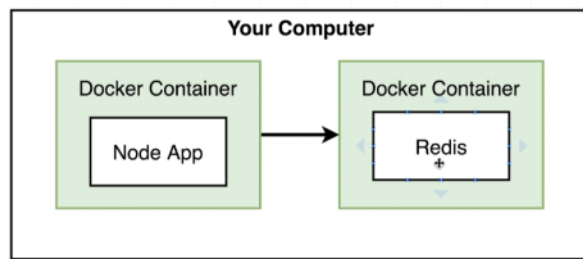


Any following command
will be executed relative
to this path in the
container

Minimizing Cache Busting and Rebuilds

```
Dockerfile x
1  # Specify a base image
2  FROM node:alpine
3
4  WORKDIR /usr/app
5
6  # Install some dependencies
7  COPY ./package.json ./
8  RUN npm install
9  COPY ./ ./
10
11 # Default command
12 CMD ["npm", "start"]
```

Docker Compose – Working with Multiple container



Options for Connecting These Containers

Use Docker CLI's Network Features

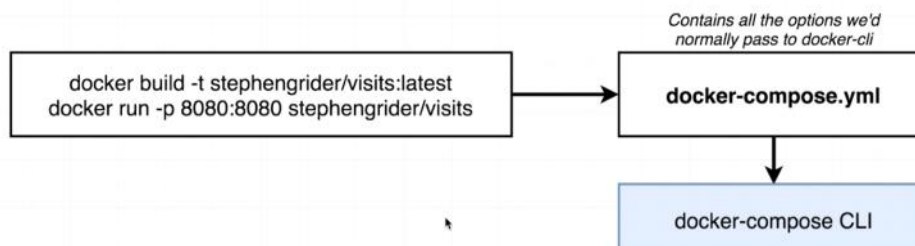
Use Docker Compose

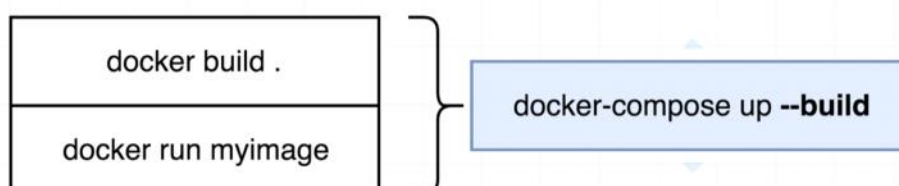
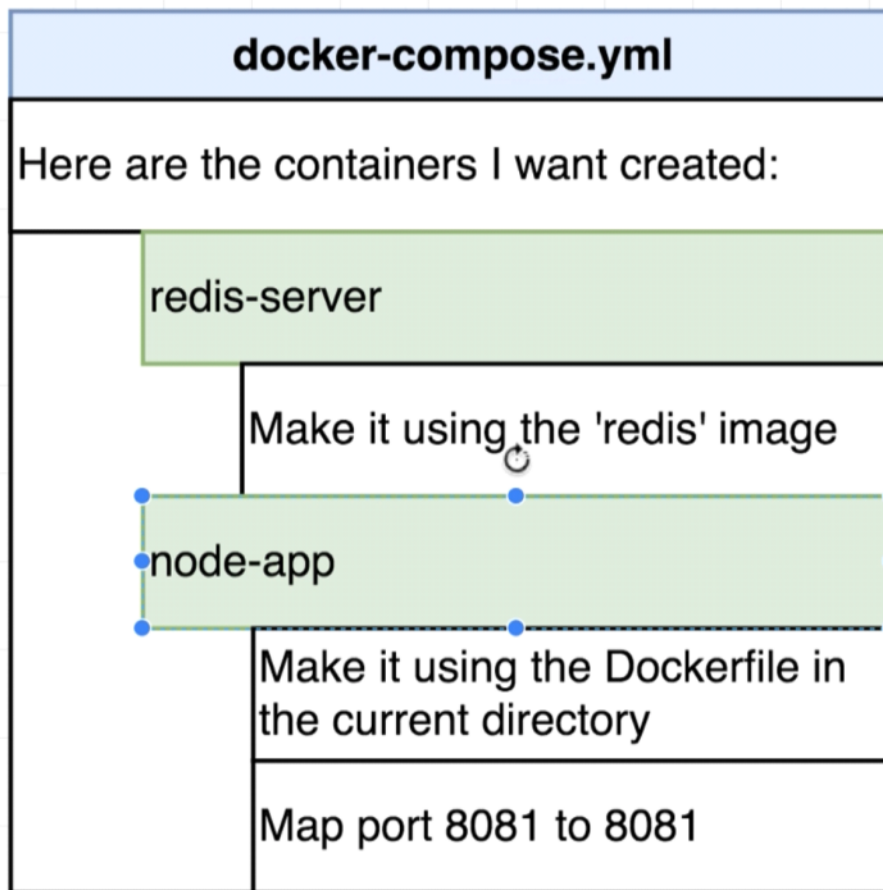
Docker Compose

Separate CLI that gets installed along with Docker

Used to start up multiple Docker containers at the same time

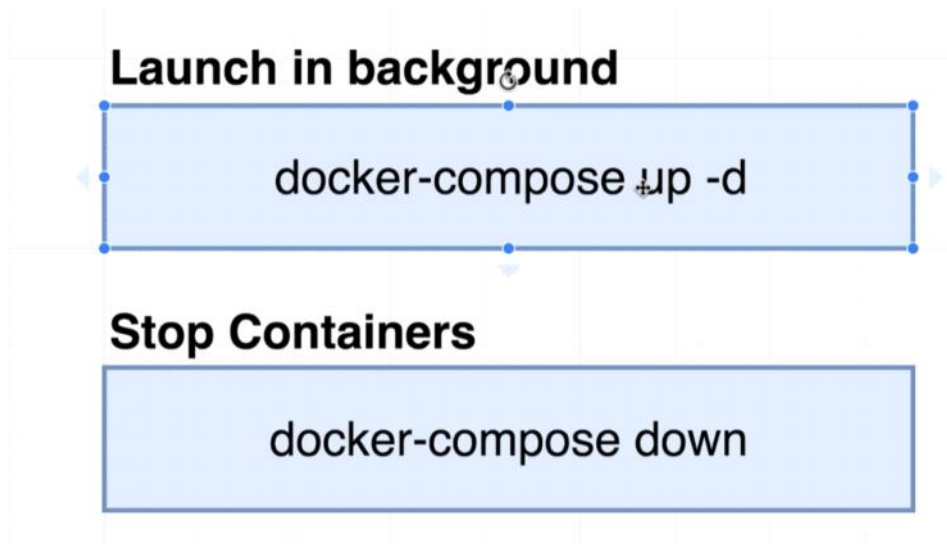
Automates some of the long-winded arguments we were passing to 'docker run'





```
Creating network "visits_default" with the default driver
Building node-app
Step 1/6 : FROM node:alpine
---> 27d9cbdc7319
Step 2/6 : WORKDIR '/app'
---> Using cache
---> 2768c90197ec
Step 3/6 : COPY package.json .
---> ca9d7356d937
Step 4/6 : RUN npm install
---> Running in 512ffffdaaa55
```

Stopping Docker compose containers



Docker restart policies

Restart Policies

"no"	Never attempt to restart this . container if it stops or crashes
always	If this container stops *for any reason* always attempt to restart it
on-failure	Only restart if the container stops with an error code
unless-stopped	Always restart unless we (the developers) forcibly stop it

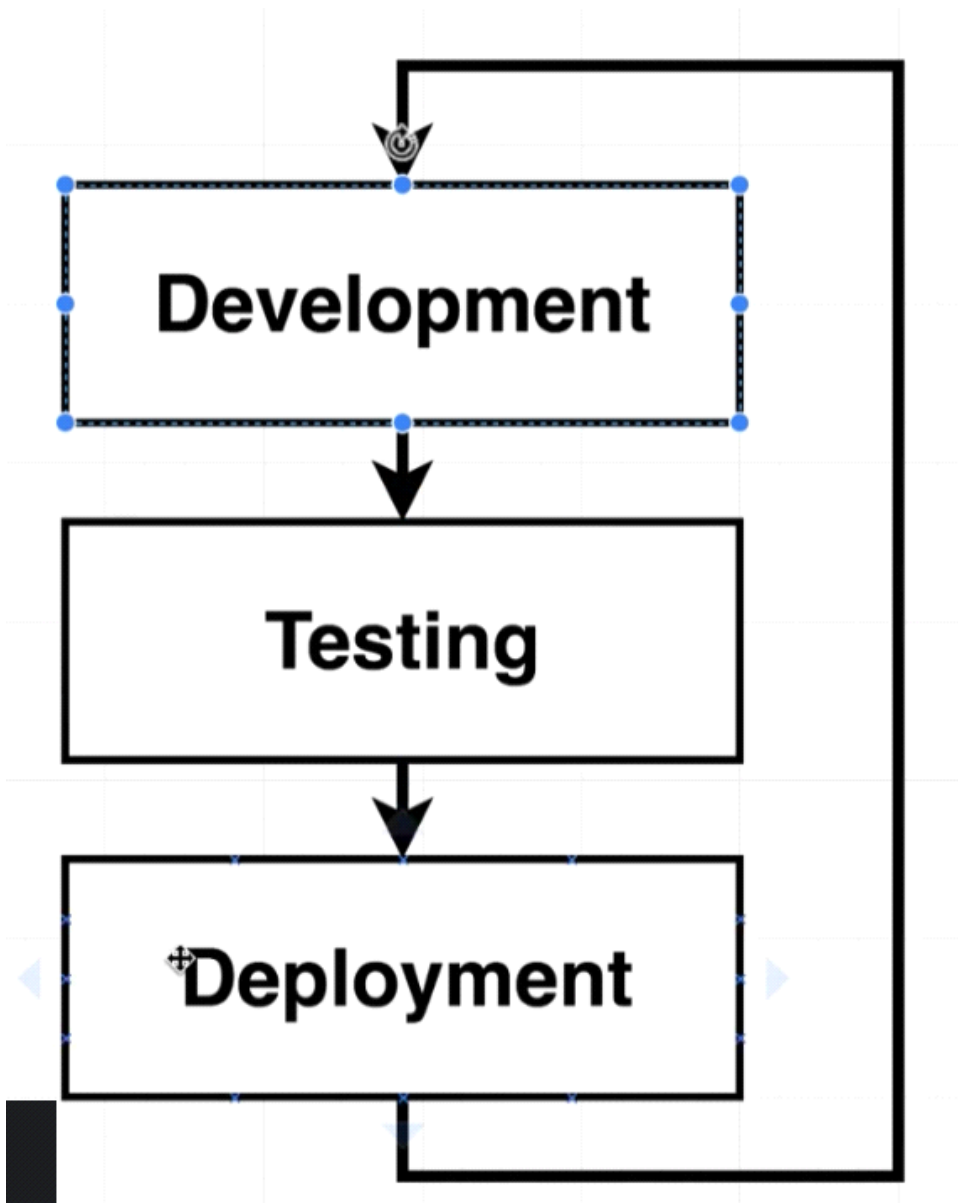
To apply restart policies

```
docker-compose.yml ●
version: '3'
services:
  redis-server:
    image: 'redis'
  node-app:
    restart: always
    build: .
    ports:
      - "4001:8081"
```

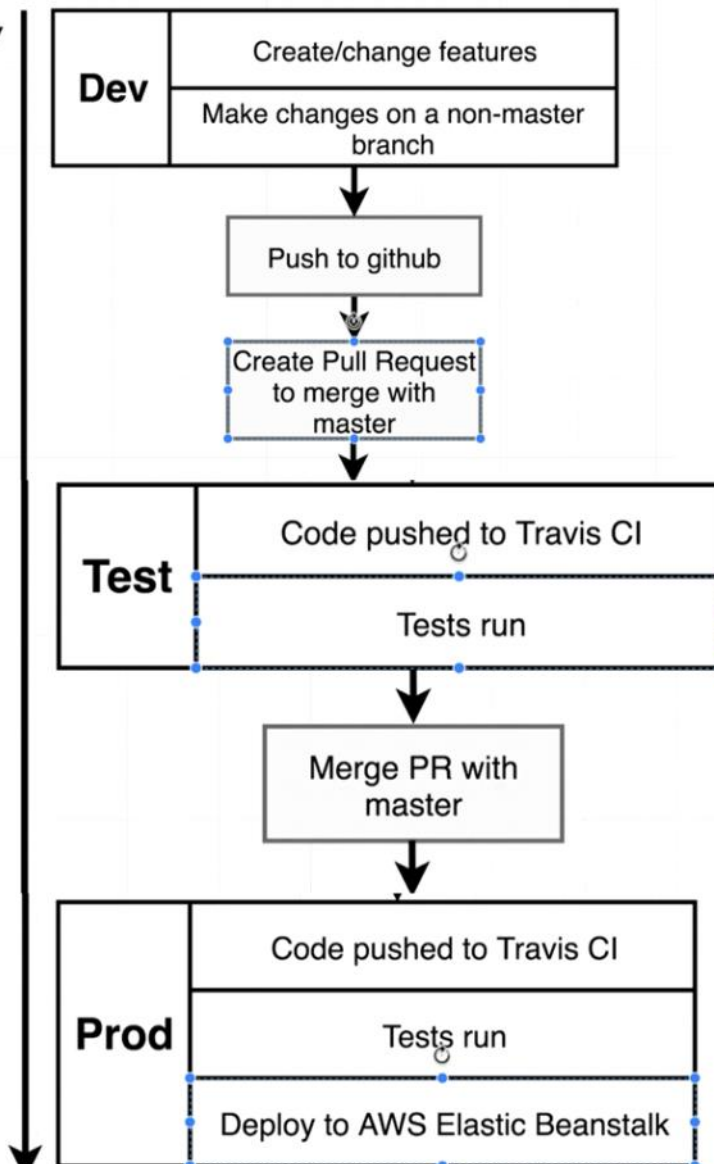
Container Status with Docker Compose

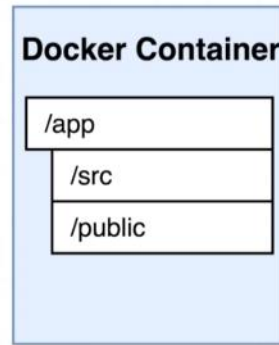
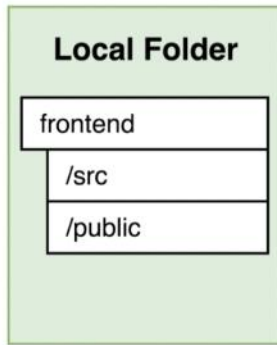
```
→ visits git:(066-stats) x docker-compose ps
-----
Name                Command              State      Ports
-----
visits_node-app_1    npm start            Up         0.0.0.0:4001->8081/t
visits_redis-server_1 docker-entrypoint.sh Up         6379/tcp
redis ...
→ visits git:(066-stats) x
```

Production Grade workflow

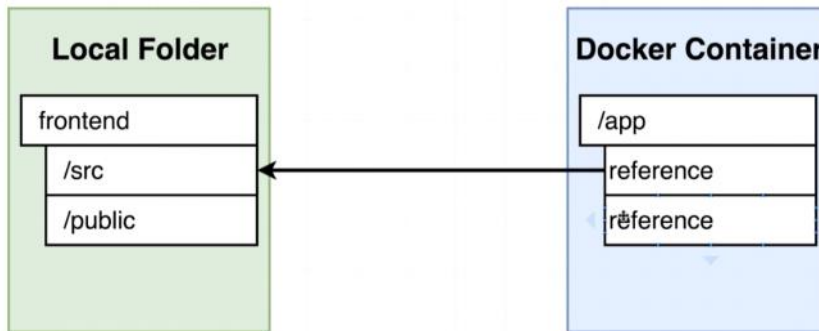


Flow

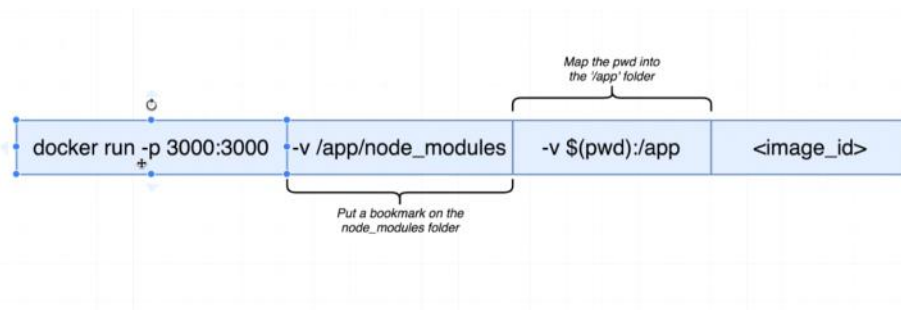


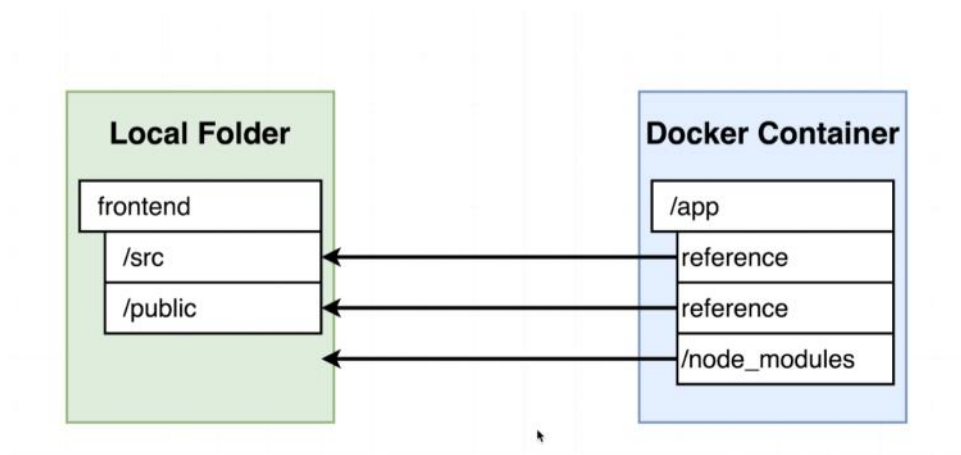


Docker volume



We need to put a bookmark `-v /app/node_modules` as the folder is not present in the local(deleted), so this reference is used to indicate to ignore the `node_modules` folder during volume mapping





```
→ frontend git:(070-flow) ✗ docker run -p 3000:3000 -v /app/node_modules -  
v $(pwd):/app 68dabe9a308e
```

docker container run -p 8081:8091 -v C:/apps:/apps/kpnsnbm

```
version: '3'
services:
  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - /app/node_modules
      - ./app
```

```
version: '3'
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile.dev
    ports:
      - "3000:3000"
    volumes:
      - /app/node_modules
      - ./app
```

Docker commands

- docker image pull image_name

- docker image ls
- docker container ls -a
- docker container --help
- docker container start/stop/rm containerId
- docker container prune
- docker container logs -f containerId
- docker container inspect container_id
- docker volume ls
- docker network ls

Docker Compose Commands

- docker-compose up -d --build
- docker-compose logs -f <name_of_the_service>
- docker-compose config
- docker-compose events -> to trace all the events
- docker-compose images
- docker-compose container
- docker-compose top -> gives all the process that runs in a container
- docker-compose pause -> pause all the containers

Stop all container

- docker compose down
- docker compose up -> won't restart the running container, it will just start the stopped container
- To scale docker container -> docker-compose up -d --scale servicename=3

```
PS C:\>
PS C:\> docker container commit -a "Richard.Chesterwood_contact@virtualpairprogrammers.com" myjdkimage
Error response from daemon: No such container: myjdkimage
PS C:\> docker container commit -a "Richard.Chesterwood_contact@virtualpairprogrammers.com" 5f myjdkimage
sha256:47e1625f67b6c14cb735c31fbbda06100dd79264cf6bab4fe59ea2f3f8b750de
PS C:\>
```

Management Commands:		
checkpoint	Manage	checkpoints
config	Manage	Docker configs
container	Manage	containers
image	Manage	images
network	Manage	networks
node	Manage	Swarm nodes
plugin	Manage	plugins
secret	Manage	Docker secrets
service	Manage	services
stack	Manage	Docker stacks
swarm	Manage	Swarm
system	Manage	Docker
volume	Manage	volumes

Running java images

```

1 FROM ubuntu:latest
2
3 MAINTAINER Richard Chesterwood
  "contact@virtualpairprogrammers.com"
4
5 RUN apt-get update && apt-get install -y openjdk-8-jdk
6
7 WORKDIR /usr/local/bin/
8
9 COPY test-program.jar .
10
11 CMD ["java", "-jar", "test-program.jar"]
12

```

Copy vs Add

Add has some extra features, like working with remote URI and unzip files

But developer prefers COPY as it is simpler

CMD vs ENTRYPOINT

Entry point ignores the command following docker run, whereas CMD executes the command followed by docker run.

```

docker container run -it jdk-image-from-dockerfile /bin/bash

```


Labels vs MAINTAINER

- Maintainer is deprecated
- Labels use case is to identify the owner in case of any issues during Twistlock scanning

Docker Compose file

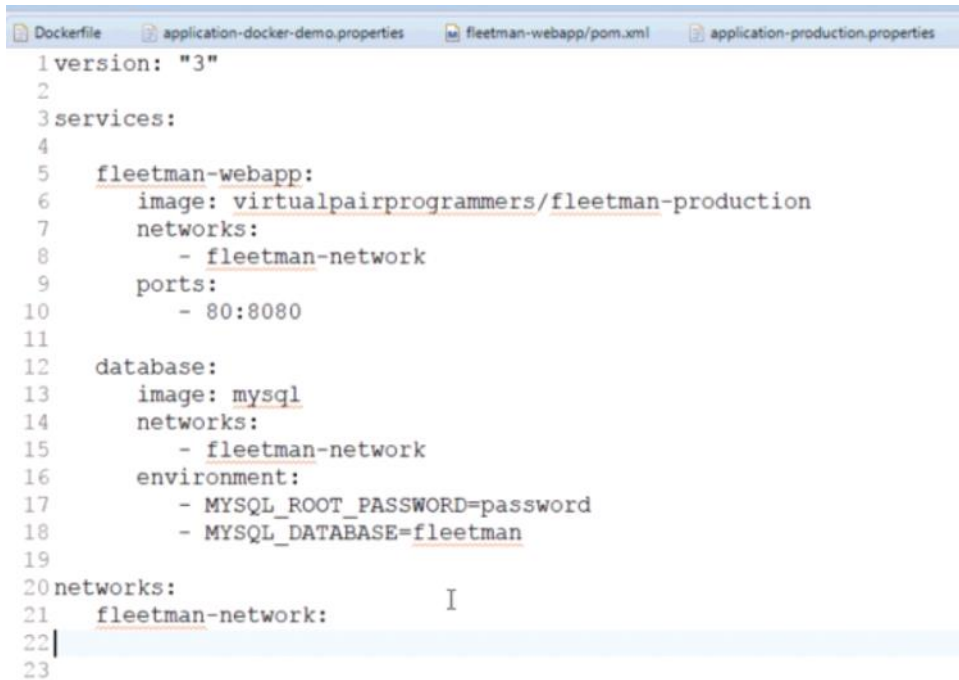
A screenshot of a code editor showing a Docker Compose file. The editor has four tabs at the top: 'Dockerfile', 'application-docker-demo.properties', 'fleetman-webapp/pom.xml', and 'application-production.properties'. The 'Dockerfile' tab is active, displaying a YAML configuration. The configuration defines two services: 'fleetman-webapp' and 'database'. 'fleetman-webapp' uses the 'virtualpairprogrammers/fleetman-production' image, connects to the 'fleetman-network', and maps port 80 to 8080. 'database' uses the 'mysql' image, connects to the 'fleetman-network', and sets environment variables for the root password and database name. A 'networks' section at the bottom defines the 'fleetman-network'.

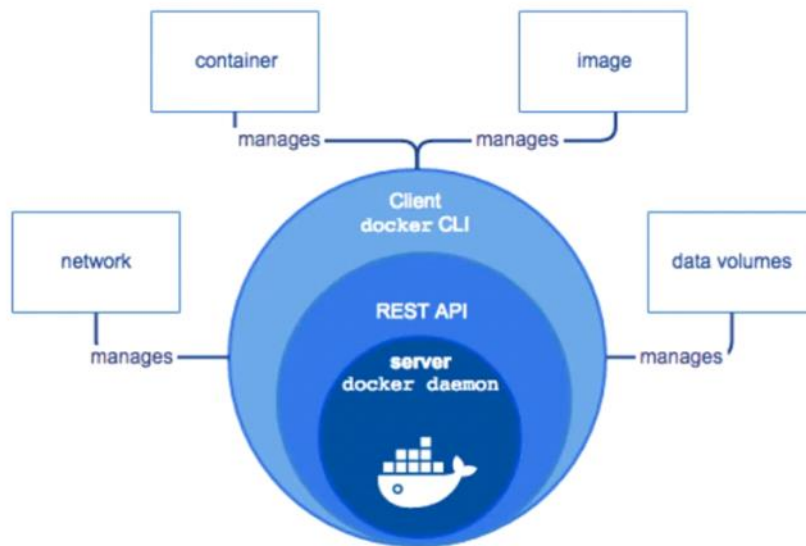
Image and Container

Image = Static version

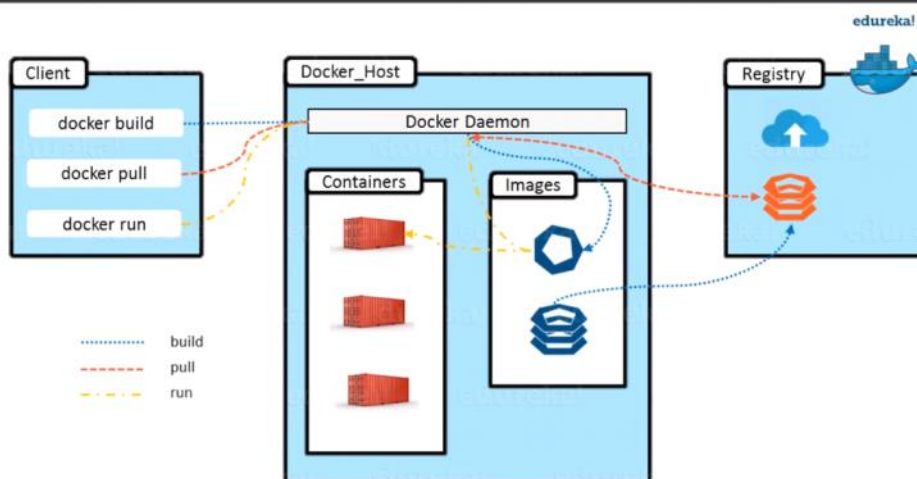
Container = Running version

Image formed via Build

Docker Engine



Docker Architecture



Create a generic reusable Dockerfile

From this



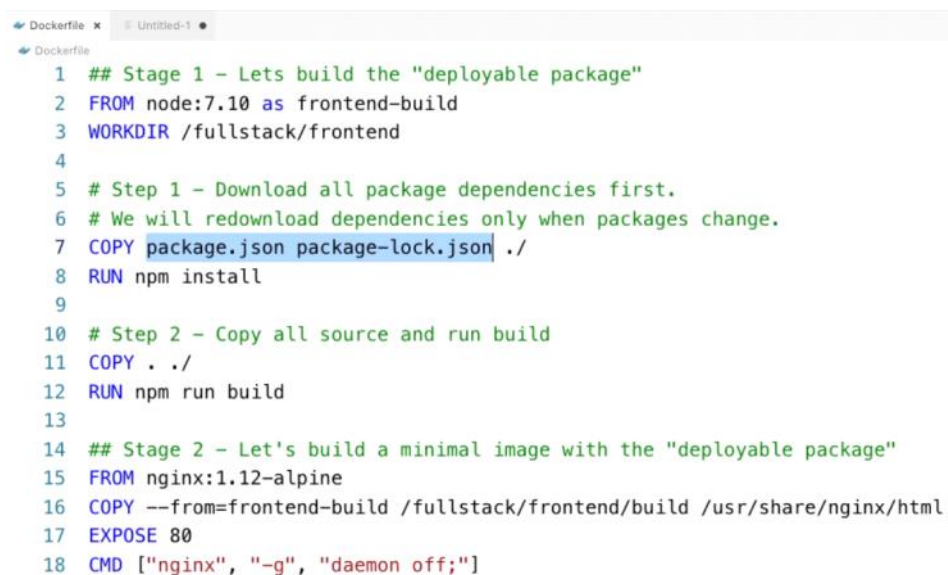
```
1 FROM openjdk:8-jdk-alpine
2 EXPOSE 8080
3 ADD target/hello-world-rest-api.jar hello-world-rest-api.jar
4 ENTRYPOINT ["sh", "-c", "java -jar /hello-world-rest-api.jar"]
```

To this -> Generic



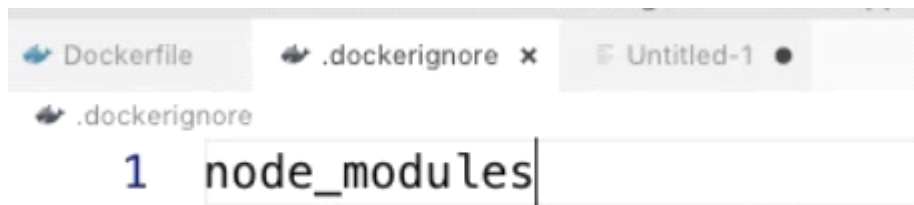
```
1 FROM openjdk:8-jdk-alpine
2 EXPOSE 8080
3 ADD target/*.jar app.jar
4 ENTRYPOINT ["sh", "-c", "java -jar /app.jar"]
```

Creating Multistage Docker Build for React Frontend Code



```
1 ## Stage 1 - Lets build the "deployable package"
2 FROM node:7.10 as frontend-build
3 WORKDIR /fullstack/frontend
4
5 # Step 1 - Download all package dependencies first.
6 # We will redownload dependencies only when packages change.
7 COPY package.json package-lock.json ./
8 RUN npm install
9
10 # Step 2 - Copy all source and run build
11 COPY . ./
12 RUN npm run build
13
14 ## Stage 2 - Let's build a minimal image with the "deployable package"
15 FROM nginx:1.12-alpine
16 COPY --from=frontend-build /fullstack/frontend/build /usr/share/nginx/html
17 EXPOSE 80
18 CMD ["nginx", "-g", "daemon off;"]
```

Improve Front End Docker Build - dockerignore



Using Multistage Docker Build for Java REST API Backend

Here, the building of image is done in the container

```
##### Stage 1 - Lets build the "deployable package"

FROM maven:3.6.1-jdk-8-alpine as backend-build
WORKDIR /fullstack/backend

### Step 1 - Copy pom.xml and download project dependencies

# Dividing copy into two steps to ensure that we download dependencies
# only when pom.xml changes
COPY pom.xml .
# dependency:go-offline - Goal that resolves all project dependencies,
# including plugins and reports and their dependencies. -B -> Batch mode
RUN mvn dependency:go-offline -B

### Step 2 - Copy source and build "deployable package"
COPY src src
RUN mvn install -DskipTests

# Unzip
RUN mkdir -p target/dependency && (cd target/dependency; jar -xf ../*.jar)

##### Stage 2 - Let's build a minimal image with the "deployable package"
FROM openjdk:8-jdk-alpine
VOLUME /tmp
ARG DEPENDENCY=/fullstack/backend/target/dependency
COPY --from=backend-build ${DEPENDENCY}/BOOT-INF/lib /app/lib
COPY --from=backend-build ${DEPENDENCY}/META-INF /app/META-INF
COPY --from=backend-build ${DEPENDENCY}/BOOT-INF/classes /app
ENTRYPOINT ["java", "-cp", "app:app/lib/*", "com.in28minutes.rest.webservices.restfulwebservises.RestfulWebServicesApplication"]
```

Docker Compose for Spring boot MySQL application

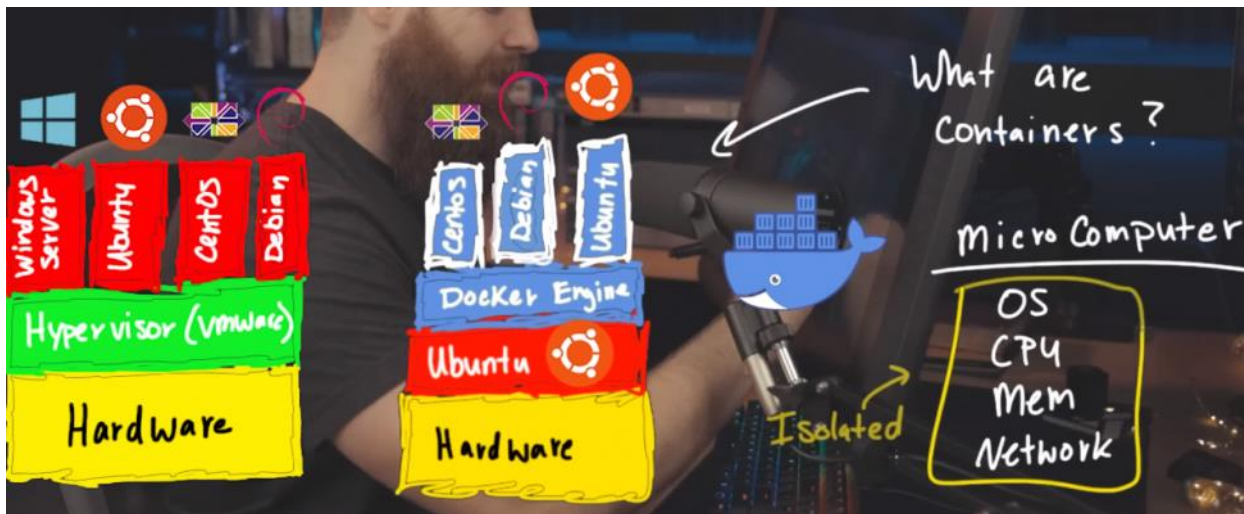
```
services:
  todo-web-application:
    image: in28min/todo-web-application-mysql:0.0.1-SNAPSHOT
    #build:
      #context: .
      #dockerfile: Dockerfile
    ports:
      - "8080:8080"
    restart: always
    depends_on: # Start the depends_on first
      - mysql
    environment:
      RDS_HOSTNAME: mysql
      RDS_PORT: 3306
      RDS_DB_NAME: todos
      RDS_USERNAME: todos-user
      RDS_PASSWORD: dummytodos
    networks:
      - todo-web-application-network

  mysql:
    image: mysql:5.7
    ports:
      - "3306:3306"
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_ROOT_PASSWORD: dummyspassword
      MYSQL_USER: todos-user
      MYSQL_PASSWORD: dummytodos
      MYSQL_DATABASE: todos
    volumes:
      - mysql-database-data-volume:/var/lib/mysql
    networks:
      - todo-web-application-network

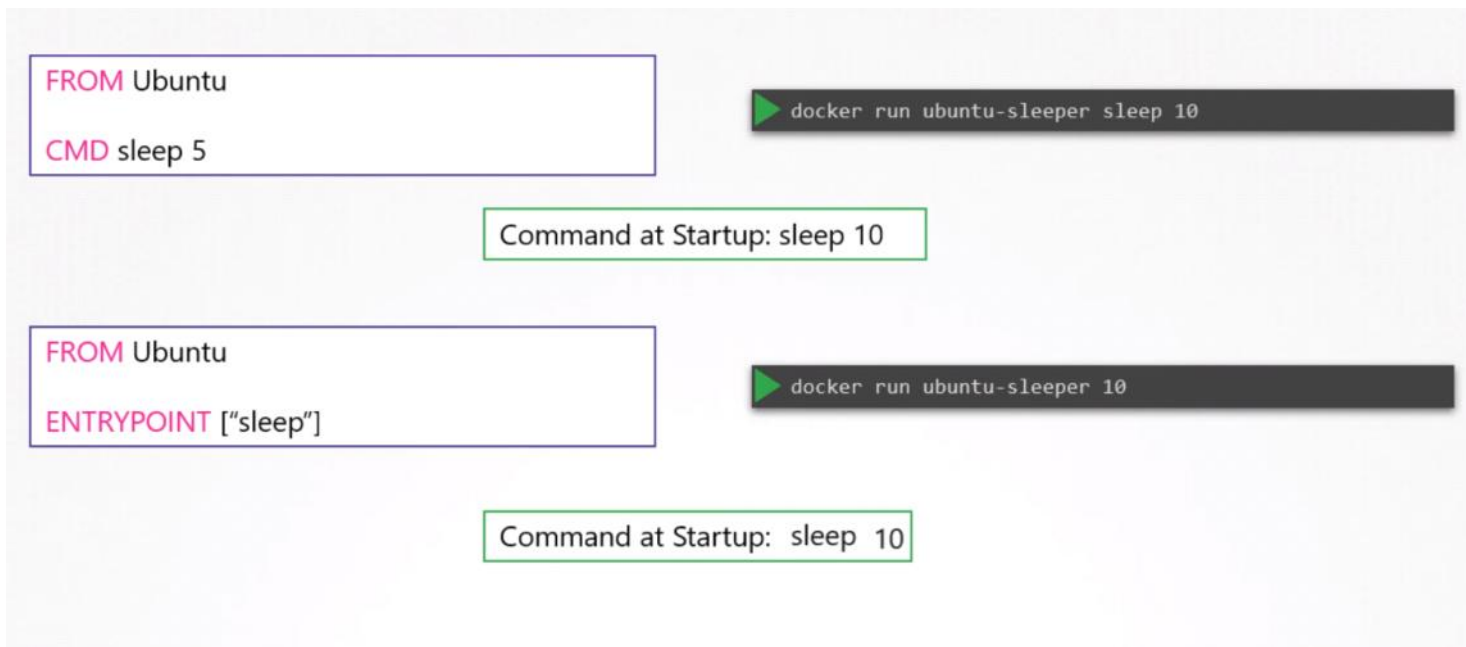
# Volumes
volumes:
  mysql-database-data-volume:
```

Docker

Wednesday, March 15, 2023 11:11 PM



To get docker statistics like CPU, memory, RAM use `docker stats`



Default value for Entry point

FROM Ubuntu

ENTRYPOINT ["sleep"]

CMD ["5"]

```
▶ docker run ubuntu-sleeper
```

```
sleep: missing operand  
Try 'sleep --help' for more information.
```

Command at Startup: sleep 5

```
▶ docker run ubuntu-sleeper 10
```

Command at Startup: sleep 10

Passing arguments via Kubernetes

FROM ubuntu

ENTRYPOINT ["sleep"]

CMD ["5"]

pod-definition.yml

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: ubuntu-sleeper-pod  
spec:  
  containers:  
    - name: ubuntu-sleeper  
      image: ubuntu-sleeper  
      command: ["sleep2.0"]  
      args: ["10"]
```

What is Virtualization?

Virtualization is a creation of logical object version in place of an actual version.

Few examples are virtual computer hardware platform, virtual storage, and virtual LANs.

Hardware virtualization means creating `virtual machine` that acts like a real physical computer with an OS.

For example, a `virtual machine (VM)` hosted on a computer with `Microsoft Windows` may behave like `Ubuntu` and `Ubuntu` supported software may run on the VM.

Need for a Virtual Machine

Virtual machine setup has the following benefits.

- **Multiple operating systems** can be hosted on the same machine simultaneously with complete isolation.
- **Multiple VMs** can be deployed on the same physical box. This reduces the total number of physical machines.
- **Easy maintenance**, app provisioning, and quick recovery.

What are Containers?

Containers are multiple isolated services that are run on a single control host (underlying infrastructure) and they access a single kernel.

Container based virtualization is an OS-level virtualization method for deploying and running distributed applications without launching an entire VM for each application.

They isolate applications from one another.

Docker - Introduction



Docker is a tool intended to make the process of creating, deploying and running applications easier by using **container based virtualization technology**.

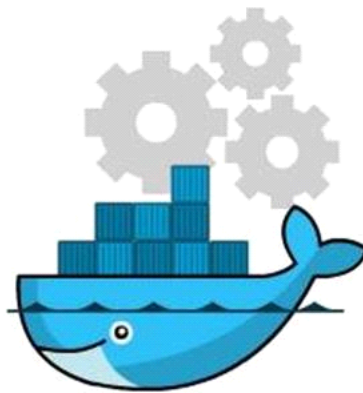
Docker is an open source container technology that provisions far more apps running on the same old servers compared to traditional VMs.

Docker Engine

Docker engine is the Docker core component that is responsible for creating Docker Images and running them as services.

Let us learn in detail about Docker Images in the next topic.

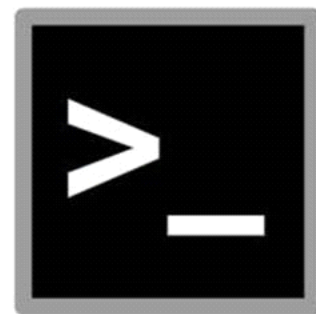
Docker Core Components



Docker Daemon



REST API



Docker Client

Docker Engine Core Components:

- **Docker Daemon**

Continuous running program (daemon process) that manages the service and other docker objects tied to it.

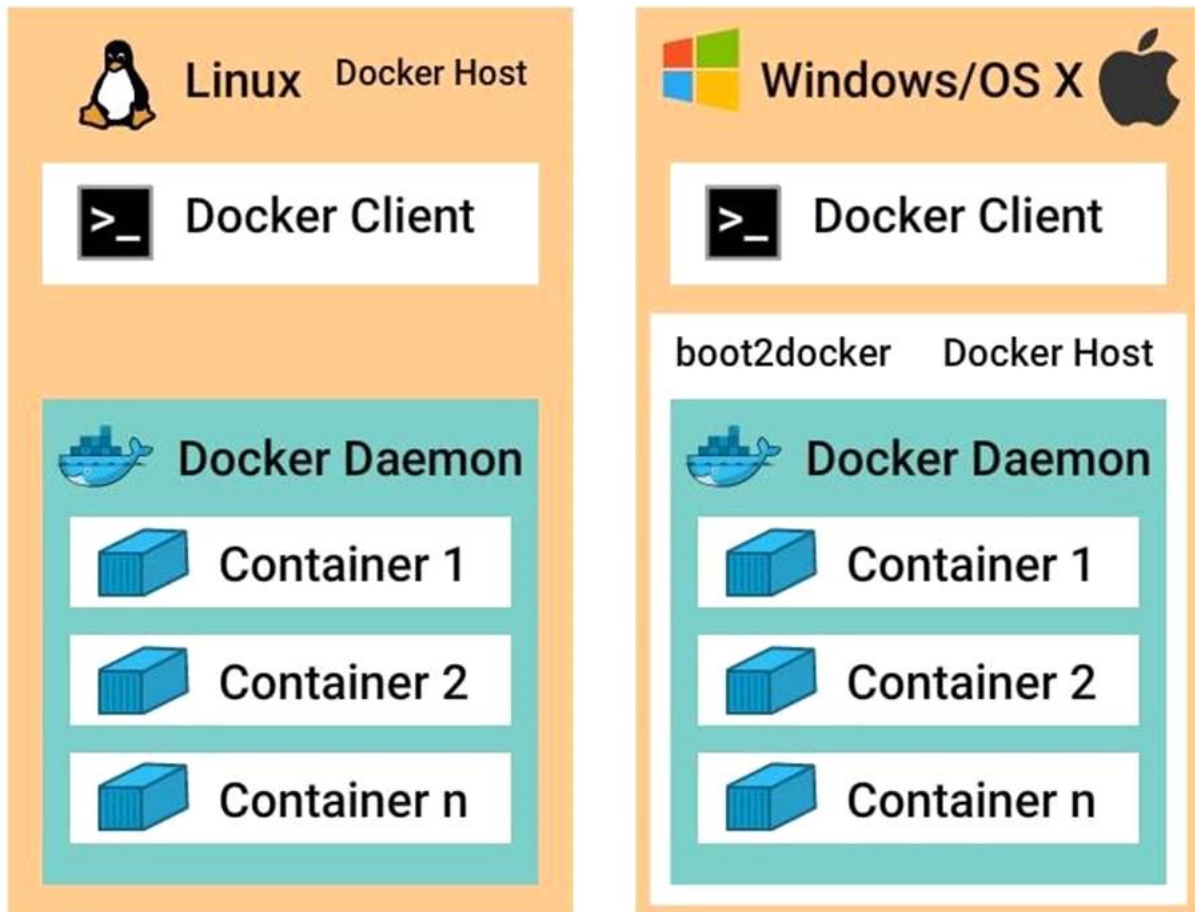
- **REST API**

Specifies **interfaces**, that programs can utilize to speak to the daemon and direct it what to do.

- **Docker Client**

CLI is utilized to interact with the daemon (docker command).

Docker on Linux and non-Linux Kernel



Docker hosted on Linux:

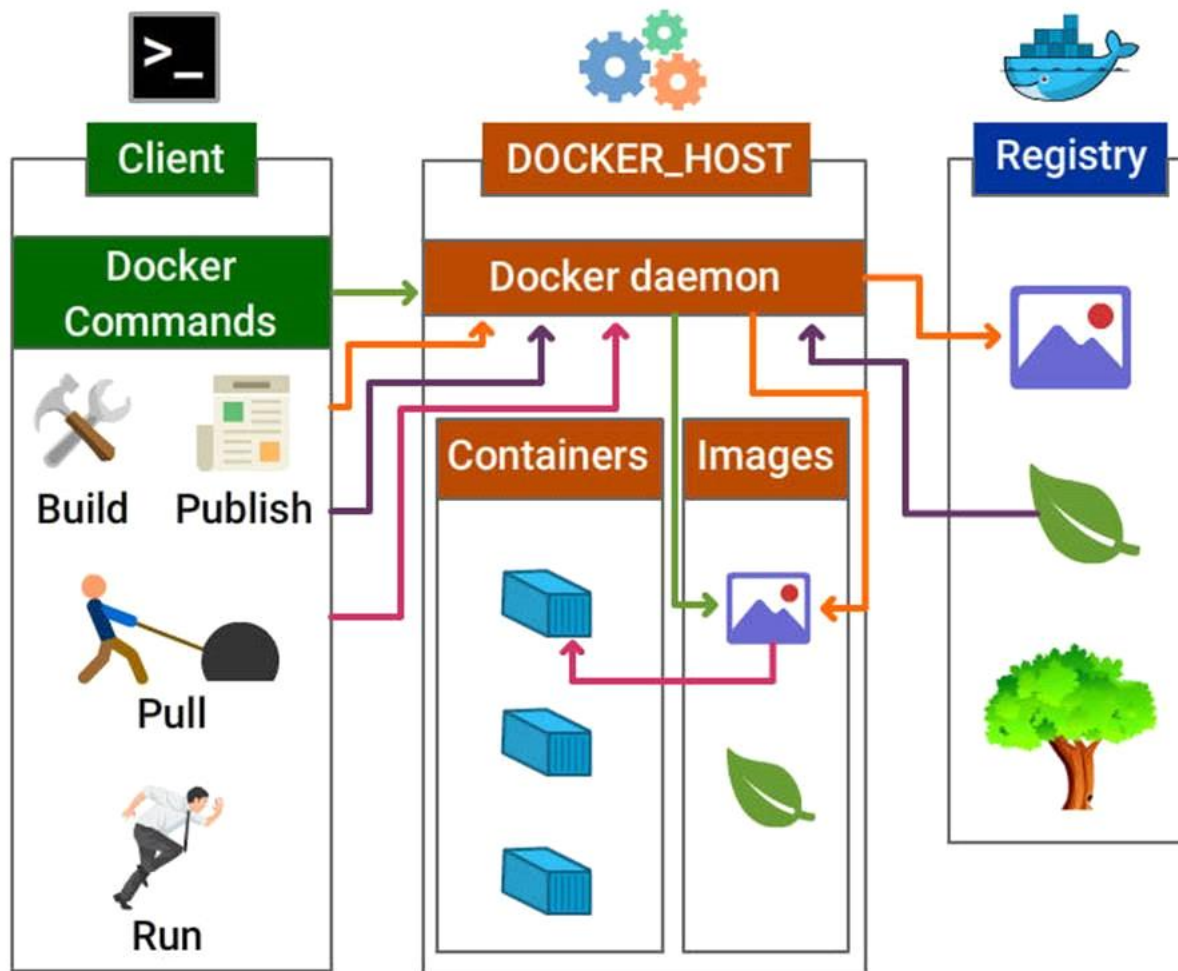
This requires just a Docker client and Docker daemon.

Docker hosted on non-Linux:

- Docker desktop for mac uses [HyperKit](#) VM, which handles virtualization.
- Docker desktop for windows uses **Microsoft Hyper-V** to manage virtualization.
-
-

4 of 4

Docker Architecture



Docker Architecture Block Diagram

Let us discuss in detail on the Docker components and workflow process in the upcoming cards.

Docker Components

Docker components include

- Docker daemon
- Docker client
- Docker Objects
 - Images
 - Containers / Services
 - Network
 - Volumes
- Docker Registry

Docker Daemon and Docker Client

Docker Daemon

Docker daemon is the docker process that receives requests from docker client and is responsible for **managing docker objects** such as containers, images, networks, and volumes.

Docker Client

Docker client **communicates with the docker daemon** through Rest API calls. A docker client can send a request to many docker daemons.

Docker Image

Docker image is the collection of **all files, libraries, binaries and other dependencies** forming an executable software application, which can run everywhere without glitches.

An image is an **inert, immutable** file.

Here are the points to be noted about Docker Image:

- Docker image is **read-only**, i.e., the image and **its content cannot be altered**.
- Although the alteration is not allowed in Docker, we are allowed to **add the new layer with the changes**.
- After having many alterations, a docker image may be visualized as **several layers one above another**.

Docker Image...

Parent and Child Images

The layering concept in docker images leads to the addition of required capabilities efficiently by adding a new layer to the existing one resulting in a new image.

Hence, **the image has the parent-child relationship where the original image is termed as the base image upon which several child images are added**.

Docker Service / Container

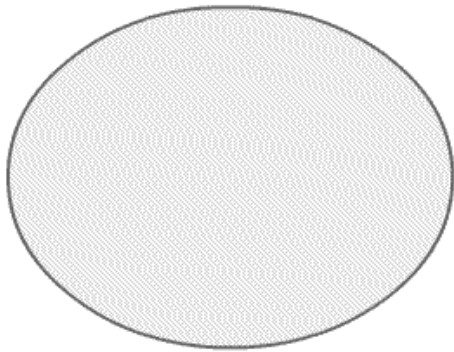
Containers are **run-time instances** of Docker images that can be run using the

Docker run command.

The fundamental purpose of Docker is to run containers.

You can run a docker image to create as many docker containers as you want.

Docker Container / Service...



Create docker image

Docker Container Lifecycle

This example explains the container life cycle stages and traversal from one stage to another.

Docker Network

The concept of networking in Docker comes into account when working with Docker in a real time scenario at a large scale.

Docker Networking helps us to share data across various containers.

Host and containers in Docker are tied with **1:N relationship**, which means one host can command multiple containers.

Modes of Networking

Various modes for networking is all about how we manage connections between containers.

- Bridge mode Networking
- Host Mode Networking
- Container Mode Networking
- No Networking

Docker File

Dockerfile is a **script**, formed of different **arguments** and **commands** (instructions) listed successively to automatically execute actions on a base image to form or create a new one.

A Docker File is a simple text file with instructions on [how to build your images](#).

-
-

Docker Registry

Docker Registry (Docker Repository) is a **storage house** for the Docker Images. It can be accessed publicly or privately by developers across the world.

- Docker images can be **sent to registry** by using `docker push` subcommand.
- Docker images can be **downloaded** from the registry using `docker pull` subcommand.

Following are the places where Docker registry can be hosted:

1. **Docker Hub**
2. **AWS Container Registry**
3. **Google Container Registry** and lot more

Docker Hub

Docker hub is one of the repositories of images which can be accessed at index.docker.io.

Docker Hub is the official repository by Docker development community. Any third party images can be pulled from the repository.

e.g.:

```
docker pull thedockerbook/helloworld
```

Docker Storage

Storage drivers

Container layer contains a **very thin writable layer**, unlike images that have read only layers.

Each container has its storage layer whereas they **share the read only image layer** across containers in the same host.

Docker uses **storage drivers** that will manage the data using copy-on-write mechanism.

Docker Storage...

Copy-on-write mechanism

Docker engine **does not copy the whole image** when we try to launch it. Instead, it uses Copy-on-write mechanism by which it uses a single copy of shared data until the data within the image is modified.

This saves a lot of disk volume, and the startup process is quick.

Volumes

Volume is a **directory mounted** in the container that is created using docker command.

They are used to share data between containers by using the same volume across various containers.

Docker Command

Monday, May 15, 2023 12:23 AM

docker create hellow-world

// docker run would install and run the container
docker run hello-world

docker run hello-world echo hi there -> attach the running process
(this is a one time and cant be overridden via start)

docker start -a imageid -> outputs logs
docker start imageid -> dont outputs logs

delete container / network
docker system prune

list all container
docker ps -all or docker ps -a

//logs
docker logs containerID

//stop container

docker stop container id // sends SIGTERM signal to clean and stop with a minimal time 10s
// if it don't stop by then then it calls kill

docker kill containerID // sends SIGKILL and terminate right away

// executing commands in running container
// example
docker run redis
docker exec -it <container id> redis-cli
-it = -i -it

//Interactive logs
Docker exec -it container id or name /bin/bash or /bin/sh or sh

// Starting with shell

docker run -it hello-world sh

// Creating images using docker file

from base image

run install software
CMD [Assign startup command when built]

To build the image from docker file

docker build .

// To run the container with a name

docker build -t <dockerid>/projectname:version

// example for tagging a name

docker build -t mj/admin-service:1.0.0

//now to run

docker run mj/admin-service:1.0.0

// in case of community images just projectname is enough

example

docker run hello-world

// alpine is a term in the docker world, for an image which is as small as possible

// For running a Java app

Dockerfile

```
FROM openjdk:8-jdk-alpine
VOLUME /tmp
ADD target/hello-docker-0.0.1-SNAPSHOT.jar hello-docker-app.jar
EXPOSE 8085
ENV JAVA_OPTS=""
ENTRYPOINT [ "java", "-jar" , "hello-docker-app.jar" ]
```

How to run

Build the maven project
build the image

docker build -f Dockerfile -t my-first-java-app .

Check if image got created docker images
Run the docker build

docker run -p 8080:8080 my-first-java-app

// To run a multiple container - better leverage docker-compose.xml

// To run an image
docker-compose up

// To rebuild and run an image

docker-compose up --build

Advantage of docker-compose is it will create the network between containers

// to launch in background

docker-compose up -d

// to stop the container

docker-compose down

// status of docker container - this cmd must be run from docker-compose file directory

docker-compose ps

Command to Stop/Remove All Container

Monday, May 15, 2023 12:24 AM

Docker container stop `$(docker container ls -aq)`

Docker container rm `$(docker container ls -aq)`

More on **aq**

-a: This flag, short for --all, instructs Docker to list all containers, including those that are currently running and those that have stopped or exited.

-q: This flag, short for --quiet, instructs Docker to only display the unique container IDs (the first 12 characters of each container's long ID) rather than the full container information. This is useful when you want to obtain a list of container IDs for further processing or scripting.

Docker Commit

Monday, May 15, 2023 12:26 AM

Docker container commit `CONTAINER_id` `NewImageName`

Example if a container has added a new feature during runtime and you want to create an image out of this new container, then this command is used.

Moving Images Across Hosts

Monday, May 15, 2023 12:31 AM

Docker save imagename > imageName.tar

Send over this file over email

Docker load < ImageName.tar

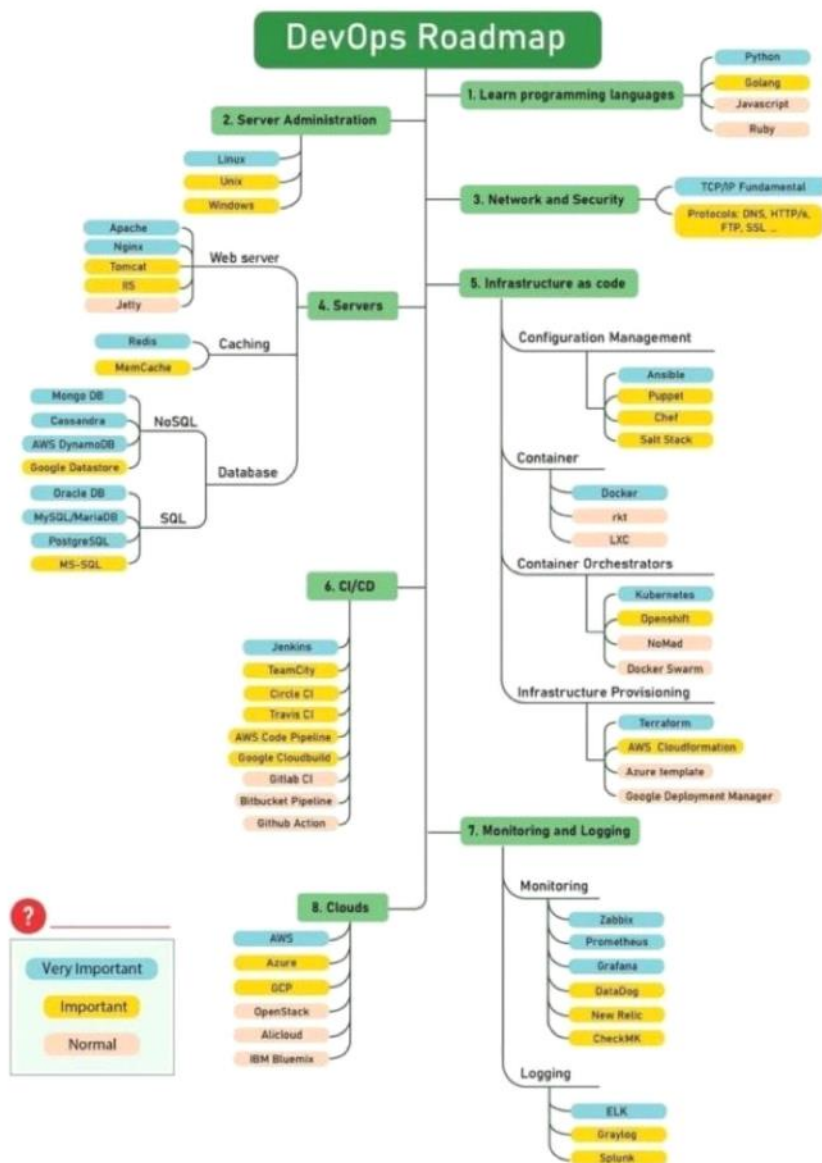
DevOps Interview for 4 Years+ with DevOps Architect

Wednesday, July 5, 2023 4:44 AM

1. Give me brief information about you.
2. Can you define your rolls and responsibility in our previous company?
3. How much you rate yourself in Ansible, Terraform, Docker and Jenkins?
4. Why did Devops method is popular as compared to Agile and waterfall method?
5. Can you tell difference between git fetch and git pull?
6. Can you differentiate git bash and git merge?
7. What is cherry-pick in git?
8. How do you create a new branch?
9. What command you use to delete branch and change the branch in git?
10. How to resolve merge conflict?
11. How git works/area of git working?
12. What is difference between SCM and VCS?
13. What is git stash?
14. What are status code 200, 404, 501,502, etc?
15. How to undo changes in local repository?
16. Differentiate between git revert and git reset?
17. In what team you working?
18. Is it possible to push the code from one local repo to another local repo?
19. Do you know shell script?
20. what is mean by \$\$, \$*, \$# ?
21. could you please share your screen and write the shell script to print 1 to 10?
22. what is bin directory?
23. What is softlink and hardlink?
24. Can you explain Maven lifecycle?
25. Can you explain Kubernetes architecture?

Devops Roadmap

Sunday, April 16, 2023 11:22 AM



What is Stash / Shelve in git

Sunday, August 6, 2023

7:52 PM



what is stash and shelve in git?



In Git, "stash" and "shelve" are both features that allow you to temporarily store changes you have made to your code without committing them to the repository. These features are useful when you want to switch branches or work on a different task without committing unfinished changes.



1. Git Stash:

"Stash" is a Git command that allows you to save changes that are not ready to be committed yet, but you want to switch to another branch or work on something else. When you run `git stash`, Git will take all the modified and staged changes in your working directory and save them into a stack of "stash" entries. This reverts your working directory to the last committed state.

You can later apply the stashed changes using `git stash apply` or `git stash pop`. The difference between these two commands is that `apply` leaves the stash intact, whereas `pop` removes the stash after applying the changes. If you have multiple stashes, you can specify which stash to apply using the stash index, e.g., `git stash apply stash@{2}`.

Docker Command

Thursday, October 19, 2023 11:18 AM

```
docker inspect image_name
```

Ansible

Tuesday, January 31, 2023 10:43 AM

Hi Raja/Vignesh,

I looked up Ansible and following are its use case..

Ansible, is a tool to automate IT tasks, it's more like a Devops tool, something like Jenkins or BigFix or Crowstrike but very powerful. It's mainly used for application deployment, updates on workstations and servers, cloud provisioning, configuration management, intra-service orchestration, and nearly anything a systems administrator does. Refer for more [here](#).

Ansible is so popular due to the following reason.

1. **Ansible is agentless**, meaning it doesn't require any software to be installed in its target servers.
 - As it can connect with Linux via SSH
 - And it can connect to Windows machines via Windows Remote Management (WinRM)
- Please note that with most versions of Windows, WinRM ships in the box but isn't turned on by default. So, we must enable the m. Refer for more [here](#).
2. **Ansible is written using YML language**, so we don't need any programming skills, we just need knowledge on Ansible, and the learning curve is very minimal(One week)
3. It can connect to 1000s or more number of server at the sametime to execute instructions.
4. It is cross-platform, meaning it can be used across windows, Linux, Mac, docker, database etc.

Use case in SupportPlus.

One of the use case I see for Ansible in supportplus is, we can use this tool to Login and Monitor for any services in windows and Linux with simple scripts. I'm not sure if this tool can be used to launch and validate an service

Conclusion :

Current team is using ansible as Scheduler, which can be done via any tool. Ansible is built to do far more powerful features in short span of time and its use case can be expanded to any no of servers. Refer more [here](#).

#Sample Inventory File

```
web  ansible_host=server1.company.com ansible_connection=ssh
db   ansible_host=server2.company.com ansible_connection=winrm
mail ansible_host=server3.company.com ansible_connection=ssh
web2 ansible_host=server4.company.com ansible_connection=winrm
```



Inventory Parameters:

- ansible_connection – ssh/winrm/localhost
- ansible_port – 22/5986
- ansible_user – root/administrator
- ansible_ssh_pass - Password

Complex Ansible Playbook

- Deploy 50 VMs on Public Cloud
- Deploy 50 VMs on Private Cloud
- Provision Storage to all VMs
- Setup Network Configuration on Private VMs
- Setup Cluster Configuration
- Configure Web server on 20 Public VMs
- Configure DB server on 20 Private VMs
- Setup Loadbalancing between web server VMs
- Setup Monitoring components
- Install and Configure backup clients on VMs
- Update CMDB database with new VM Information

playbook.yml

```
-
  name: Play 1
  hosts: localhost
  tasks:
    - name: Execute command 'date'
      command: date

    - name: Execute script on server
      script: test_script.sh

    - name: Install httpd service
      yum:
        name: httpd
        state: present

    - name: Start web server
      service:
        name: httpd
        state: started
```

playbook.yml

```
-
  hosts: localhost
  name: Play 1
  tasks:
    - name: Execute command 'date'
      command: date

    - name: Execute script on server
      script: test_script.sh

-
  name: Play 2
  hosts: localhost
  tasks:
    - name: Start web server
      service:
        name: httpd
        state: started

    - name: Install web service
      yum:
        name: httpd
```

module

playbook.yml

```
-
  name: Play 1
  hosts: localhost
  tasks:
    - name: Execute command 'date'
      command: date

    - name: Execute script on server
      script: test_script.sh

    - name: Install httpd service
      yum:
        name: httpd
        state: present

    - name: Start web server
      service:
        name: httpd
        state: started
```

Devops Interview

Tuesday, March 21, 2023 10:44 AM

Here are the Some interview Questions for the DevOps Engineer which I faced during my last interview . and I Mentioned Topic wise question Below .

✓ Asking About Me And Projects

- 1) Can You Please Introduce Yourself
- 2) Please Tell me About Last Project Ex -: E-commerce
- 3) Which Technologies did You Used In Your last Project
- 4) What Your Roles and Responsibilities in Your Project or current Project ?
- 5) What are the Challenges did You Faced During the Project ?

✓ Asking About Git !

- 1) What are the Branching Strategies you Follows in your Project ?
- 2) In Which Branch Production Happened ?
- 3) Which Branch You Used in Production ? ex - Master , Main
- 4) What is the Difference Between Master and Main Branch ?
- 5) Assume That I have a Git client in My system and i forget my password and I want to update my password in git client how can I do ?

✓ Asking about Linux !

- 1) Which Linux Version you used in Your Project ?
- 2) what is the difference between amazon Linux and Ubuntu ?
- 3) what is AWK Command in Linux ?
- 4) How did You Check Free space in Linux Machine ?
- 5) What is the difference between wget and Curl command ?
- 6) Can we check which user are belong to the particular group ?

✓ Asking about Sonar ?

1) What is Quality Profile in Sonar ?

2) Supposed my code Quality Get failed what will you do ?

✓ Asking about Jenkins !

1) What is Master and Slave in Jenkins ?

2) If Master Node Failed what will happen ?

3) Suppose there are 10 Jobs are Running Suddenly all the jobs get failed Its Possible to Run a Job again and if Yes How ?

4) Can You Write a Simple Jenkins pipeline ?

5) In Your Project what are the stages Available in Jenkins ?

6) Let's Take a Scenario Even if my First job get Failed I don't want to stop my job I want to continue with the remaining stages in the pipeline

✓ Asking About Docker !

1) Can You Explain me DockerFile ?

2) What is Difference between a RUN and CMD ?

3) Explain me what is Docker Images and Docker Container ?

✓ Asking About K8S !

1) what is POD ?

2) Can You Write a POD Manifest ?

3) How you creates a POD in Project ?

4) what is statefulset in kubernetes ?

5) What is the Purpose of statefulset ?

hope it will really help everyone who ever will Give the interview in future

Docker Interview Question

Monday, April 10, 2023 11:45 PM

Copy vs Add

Add has some extra features, like working with remote URI and unzip files

But developer prefers COPY as it is simpler

CMD vs ENTRYPOINT

Entrypoint ignores the command following docker run, whereas CMD executes the command followed by docker run.

```
docker container run -it jdk-image-from-dockerfile /bin/bash
```

DockerFile vs K8s Manifest Perspective

Docker Field Name	Kubernetes Field name	Description
ENTRYPOINT	command	Command that will be run by container.
CMD	args	Argument passed to the container.

Labels vs MAINTAINER

- Maintainer is depreciated
- Labels use case is to identify the owner in case of any issues during Twistlock scanning

Docker Stop vs Kill

SIGTERM -> Terminate Signal -> Gives a little time for the container to shutdown

(10 sec), after which it falls back to SIGKILL or kill command

SIGKILL -> Terminate right now

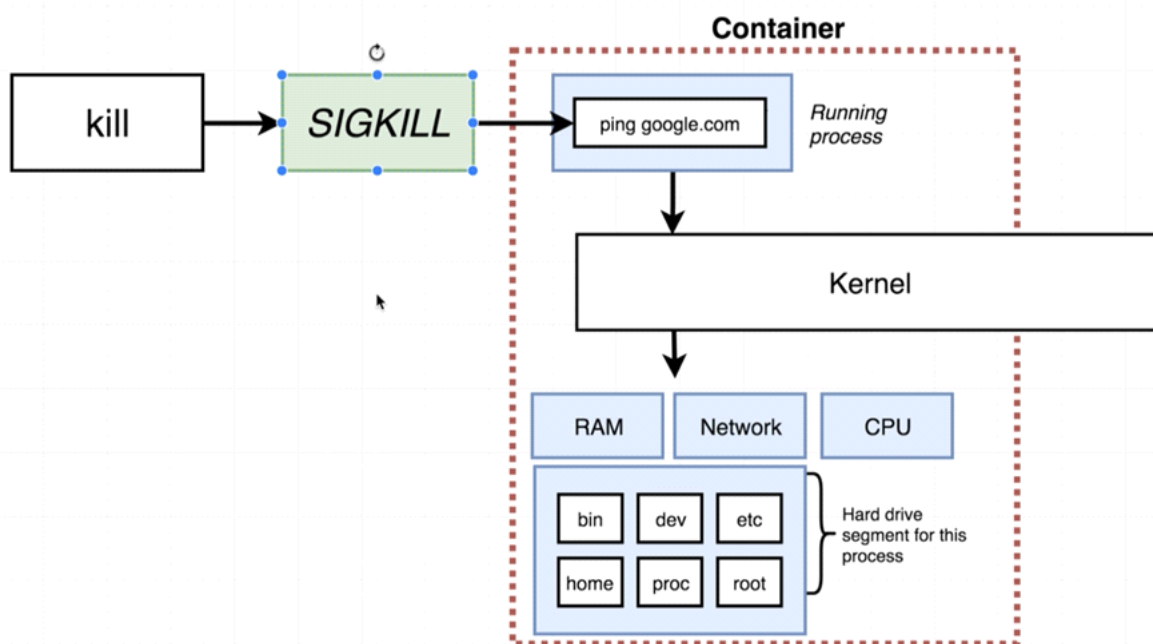
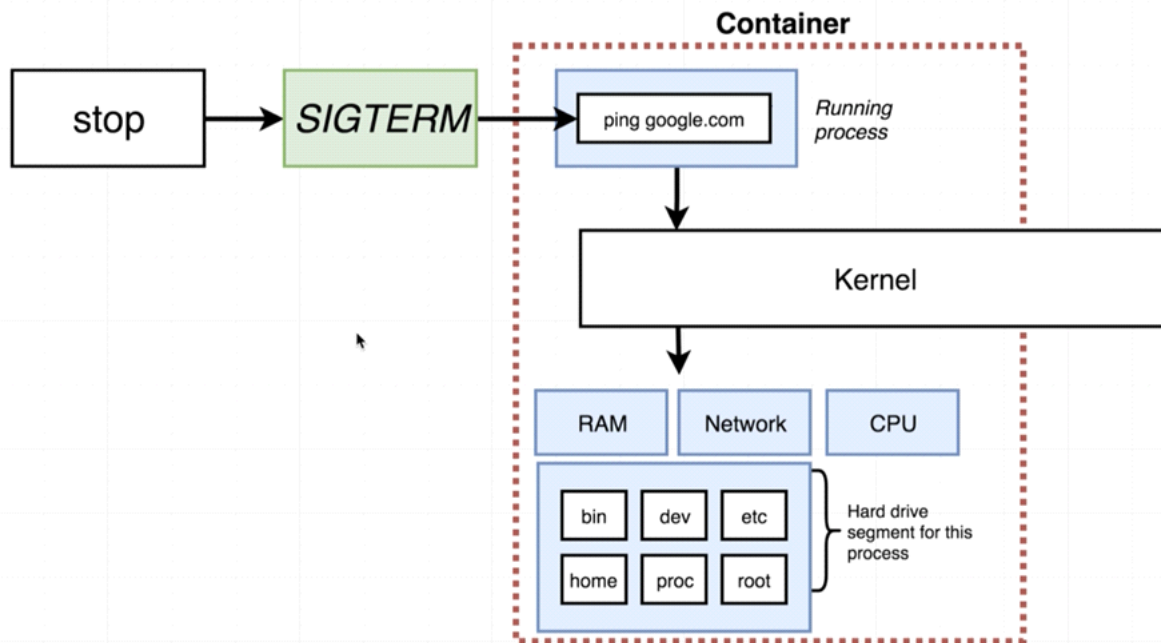
To exit out of a container

Cmd +d

Docker **container** stop containerId



Docker Stop vs Kill



Tuesday, July 25, 2023 1:04 AM

- ## What is Splunk? >

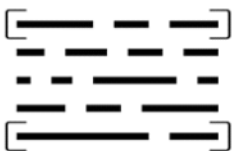


How Splunk Stores Data>



Index

- A repository for Splunk data
- Splunk transforms incoming data into events, and stores it in indexes
- An event is a single row of data



Event

- A single row of data
- Data is specified by fields (key=value pairs)
- Splunk adds default fields to all events

Timestamp, host, source, sourcetype

How Splunk Stores Data>

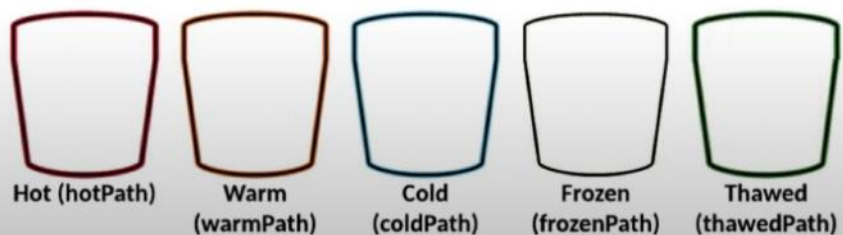


- A single row of data
- Data is specified by fields (key=value pairs)
- Splunk adds default fields to all events

How Splunk Stores Data >



Indexes

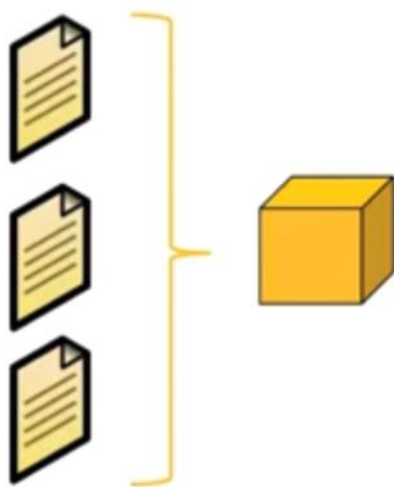


Frozen - Archived

Thawed - Data retrieved from Frozen

Splunk Apps >

- An app is a collection of Splunk configuration files

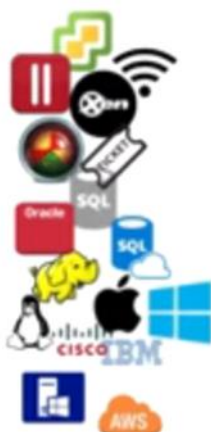


Splunk Apps >

- Apps are downloaded from Splunkbase.com
- Most apps are free, however there are a few premium apps for which you'll need a license.
- These include:
 - Splunk App for Enterprise Security
 - Splunk App for PCI Compliance
 - Splunk App for VMware
 - Splunk App for Microsoft Exchange

How Splunk Consumes Data >

- Virtual Machines
- Physical Machines
- Servers
- IoT
- Communications



- Logs
- Configurations
- Scripts
- Tickets
- Alerts

Building a Search

```
index=web OR index=security | stats sum(bytes) as Total_Bytes | eval Total_Bytes = tostring(Total_Bytes, "commas")
```

Pull data from disk

Set your command

Determine your functions

Call your arguments

Name your indexes and
meta fields

What are we trying to do

Do we need to calculate
results?

What fields are needed?

Basic Commands

table

rename

fields

dedup

sort

Refined Search

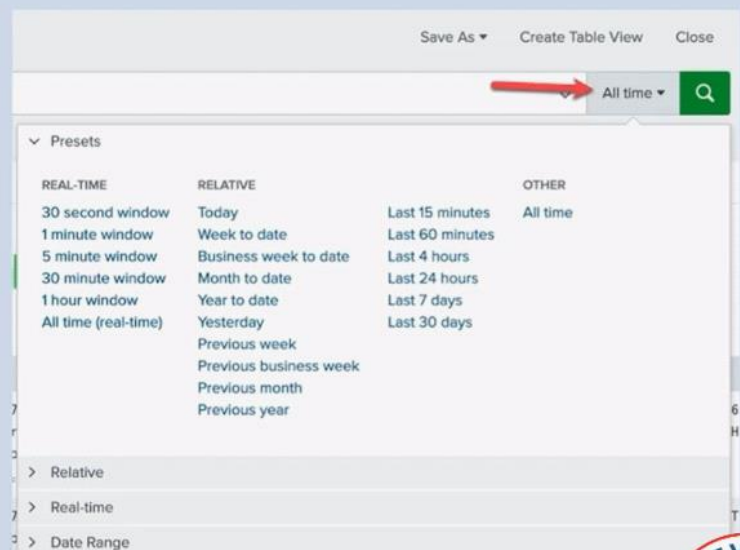
Timeline Overview

- Drill down based on time
- Identify trend analysis by number of events over time



Time Picker

- Most valuable search feature
- Precedence of time picker comes second to the SPL bar
- Real time searching vs static searching



Basic Searching >

Basic search terms (building blocks)

- Keywords
 - `failed, error`
- Phrases
 - `"failed login"`
- Fields
 - Key value pairs
 - `user=user1.domain.com`
- Wildcards
 - `*ailed, fail*, user=*`
- Booleans
 - Case sensitive (upper)
 - AND, OR, NOT

Basic Searching >

Basic search commands

- `chart / timechart`
 - Returns results in tabular output for charting
- `rename`
 - Renames a specific field
- `sort`
 - Sorts results by specified fields
- `stats`
 - Provides statistics
- `eval`
 - Calculates an expression
- `dedup`
 - Removes duplicates
- `table`
 - Builds a table with the specified fields

Dealing with Time >

Time Conversion

- Time can be converted from Splunk's default to a format of your choice using the `strftime()` eval function

```
| eval time=strftime(_time, "%H:%M")
```

16:34

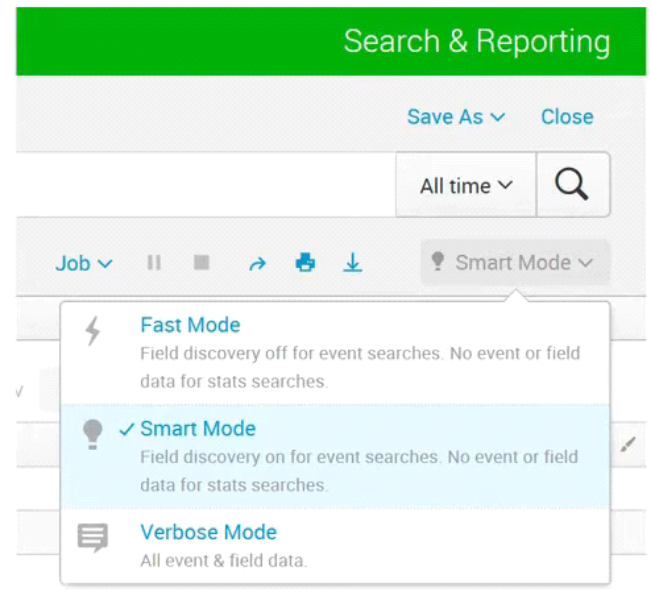
Time Conversion

Time variable	Description
%H	Hour (24 hour clock)
%I	Hour (12 hour clock)
%M	Minute
%S	Second
%p	AM or PM

Search Modes, Fields, Discovery >

The Search App has three modes

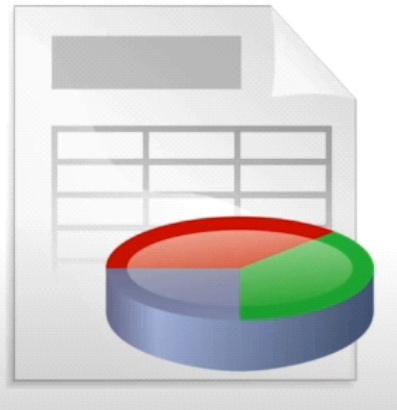
1. Fast
2. Smart
3. Verbose



Intermediate Searching >

Let's look at some transforming commands

- top
- rare
- stats



Top

- Finds the top common values of a field in a table
- Top 10 results by default
- Can use with arguments

Rare

- Finds the least common values of a field in a table
- Opposite of top

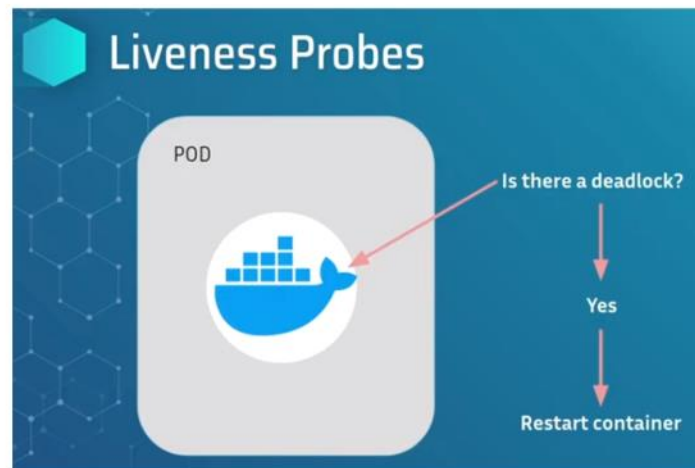
Stats

- Calculate statistics
- count, dc, sum, avg, list, values, etc.

Liveness vs Readiness

Saturday, July 29, 2023

4:04 PM





Liveness Probes

```
livenessProbe:
  httpGet:
    path: /hello
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
```



Liveness Probes

```
livenessProbe:
  httpGet:
    path: /hello
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
```

Health endpoint of application

Port on which it is running

The first time when liveness will test

Regular interval after the application startup

What is Readiness Probe



Chapter -2

Readiness Probes



Is POD ready to serve traffic?

No

Remove the POD from service,loadbalancer



Readiness Probes

```
readinessProbe:  
  httpGet:  
    path: /hello  
    port: 8080  
  initialDelaySeconds: 15  
  periodSeconds: 10
```



Readiness Probes

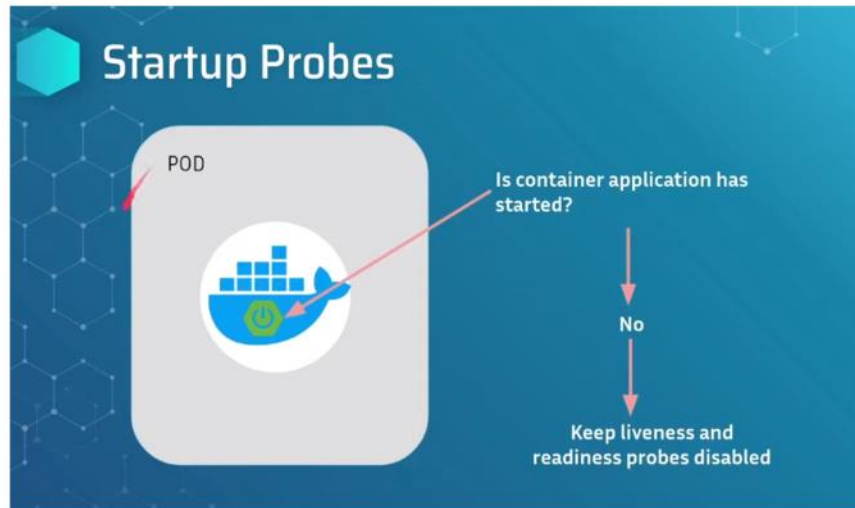
```
readinessProbe:  
  httpGet:  
    path: /hello  
    port: 8080  
  initialDelaySeconds: 15  
  periodSeconds: 10
```

Health endpoint of application

Port on which it is running

The first time when Readiness will test

Regular interval after the application startup



Startup Probes

```
startupProbe:
  httpGet:
    path: /hello
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```


Startup Probes

```
startupProbe:
  httpGet:
    path: /hello
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

Health endpoint of application

Port on which it is running

K8s will try in case probe fails

How often the check is performed

```
startupProbe:
  httpGet:
    path: /healthz
    port: liveness-port
  failureThreshold: 30
  periodSeconds: 10
```

Thanks to the startup probe, the application will have a maximum of 5 minutes ($30 * 10 = 300s$) to finish its startup. Once the startup probe has succeeded once, the liveness probe takes over to provide a fast response to container deadlocks. If the startup probe never succeeds, the container is killed after 300s and subject to the pod's `restartPolicy`.

Liveness vs Readiness

Saturday, July 29, 2023 3:57 PM

Readiness - Responsible for the Pod

Liveness - Responsible for the docker container

Startup - Responsible for the application deployed in the docker container.

KUBERNETES PROBES

- Liveness
 - Application is broken and needs a restart
- Readiness
 - Application is ready to serve traffic

```
management.endpoint.health.show-details=always  
management.endpoint.health.probes.enabled=true
```

Use case - 1

- Want to make sure there is no deadlocks in the application and its always running
 - "Liveness Probe"

Use case - 2

- If the application is temporarily down, and you don't want to serve traffic. Eg connectivity issues with Database
 - Add "Readiness Probe" as restarting application container will be of no use

Use case - 3

- When the startup time is slow
 - Add "Startup Probe"

Official definition of Kubernetes

- > Open source **container orchestration tool**
- > Developed by **Google**
- > Helps you **manage containerized applications** in different **deployment environments**

physical

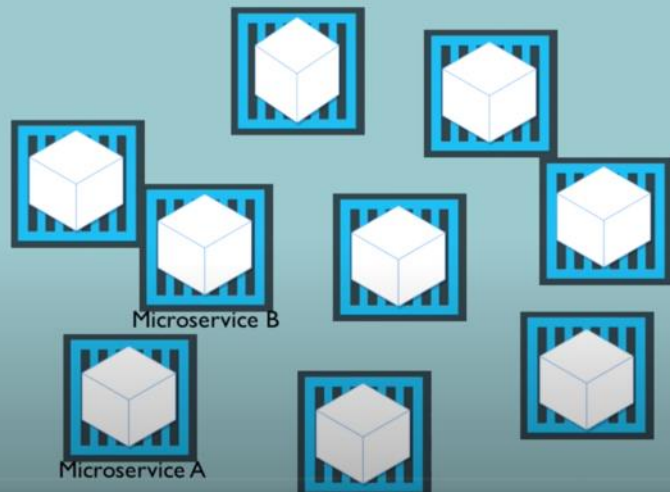
cloud

virtual



The need for a container orchestration tool

- > Trend from **Monolith** to **Microservices**
- > Increased usage of **containers**



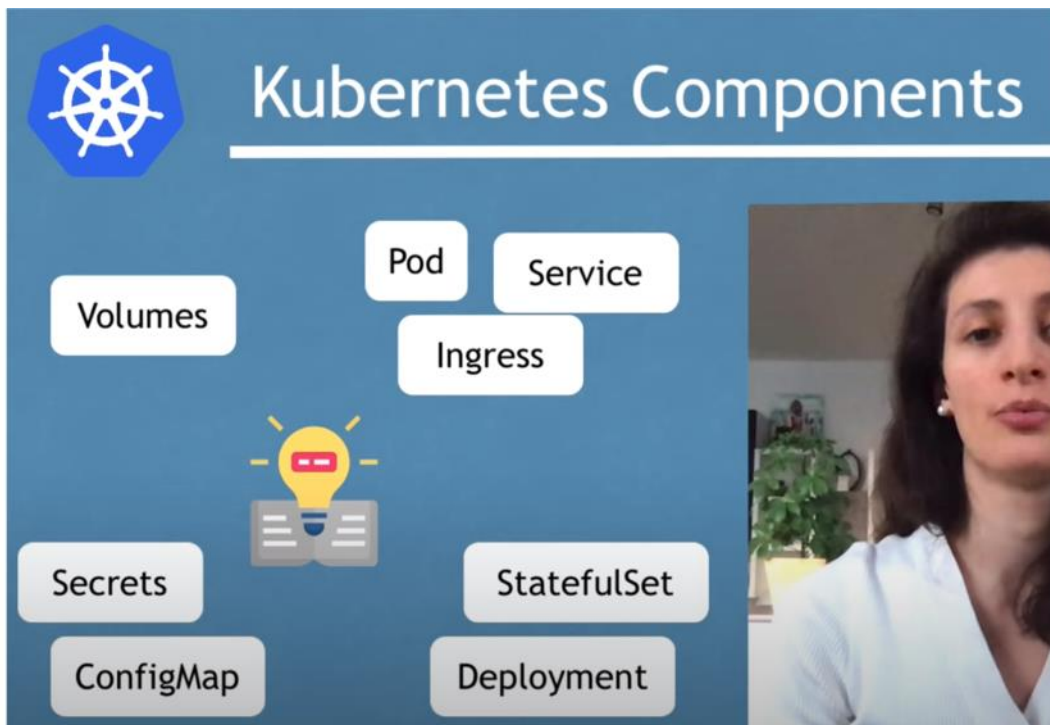
The need for a container orchestration tool

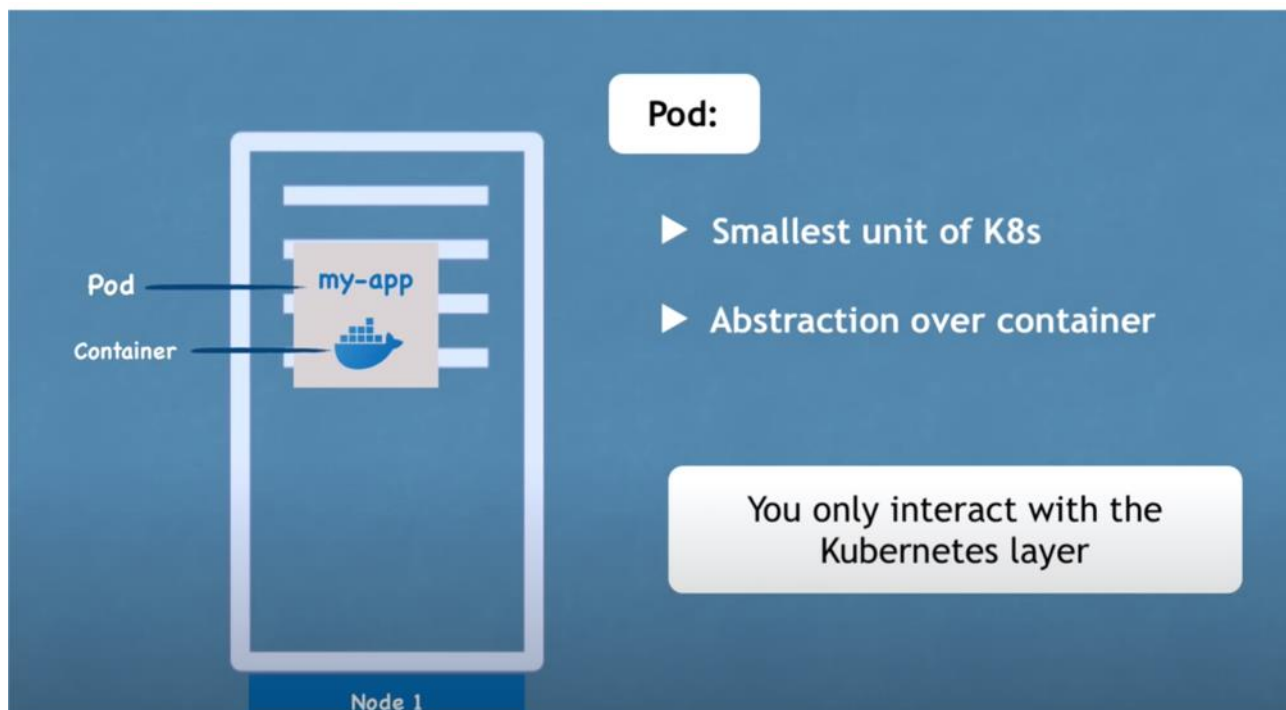
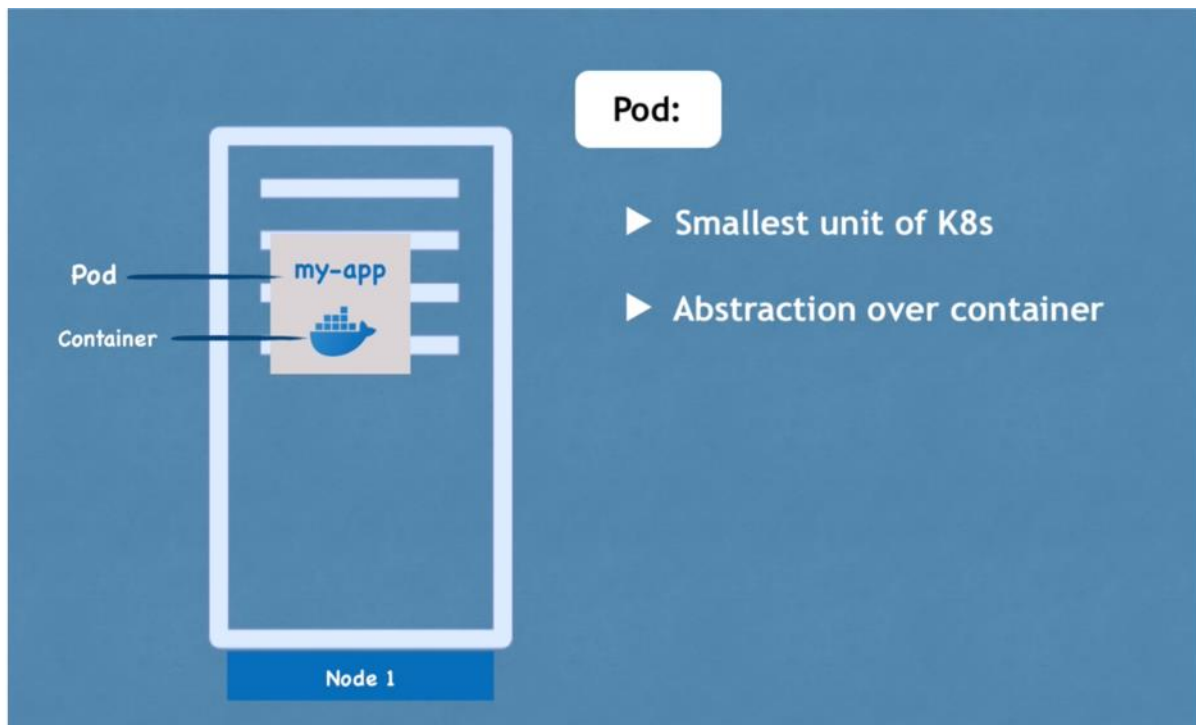
- > Trend from **Monolith** to **Microservices**
- > Increased usage of **containers**
- > Demand for a **proper way** of **managing** those hundreds of containers

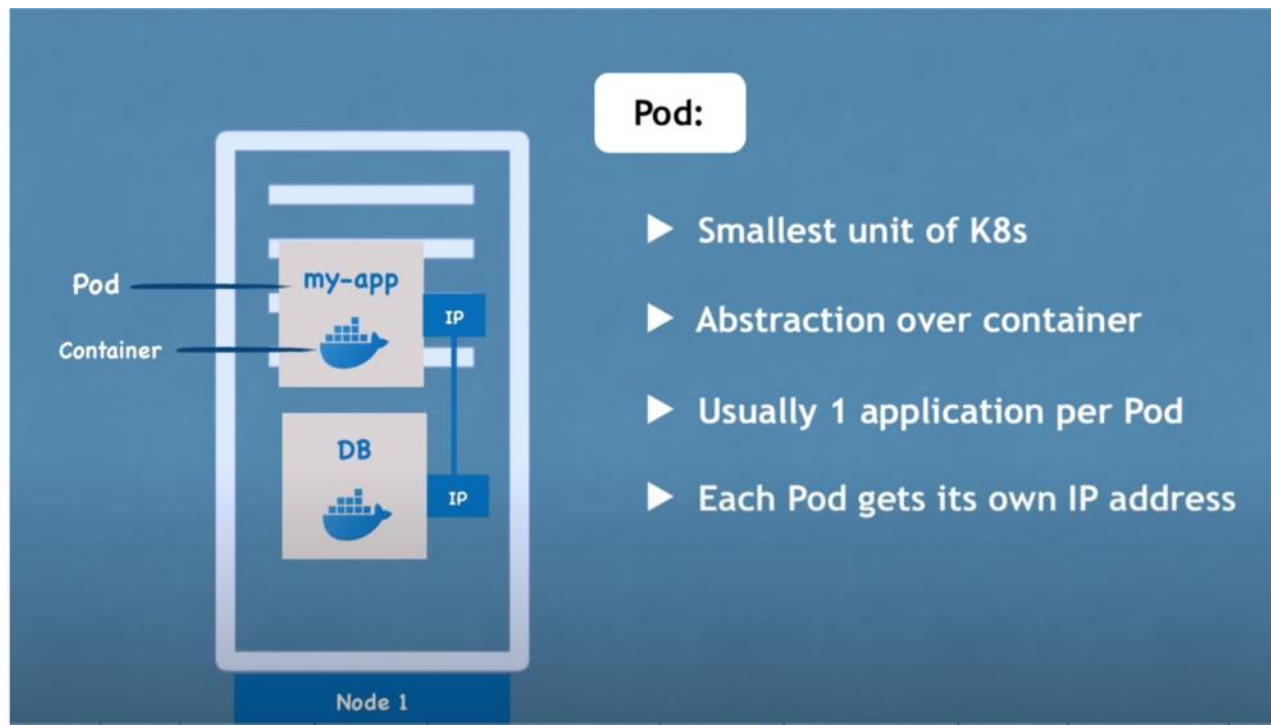
What features do orchestration tools offer?

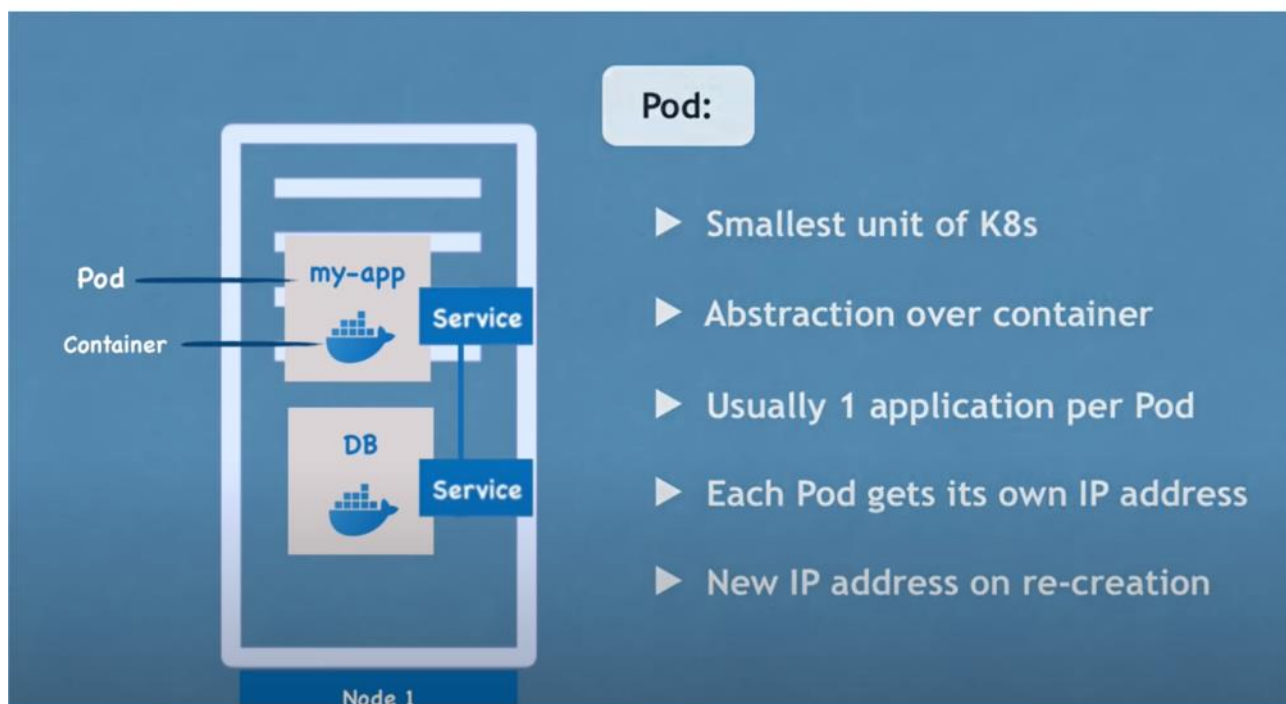


- > **High Availability** or no downtime
- > **Scalability** or high performance
- > **Disaster recovery** - backup and restore



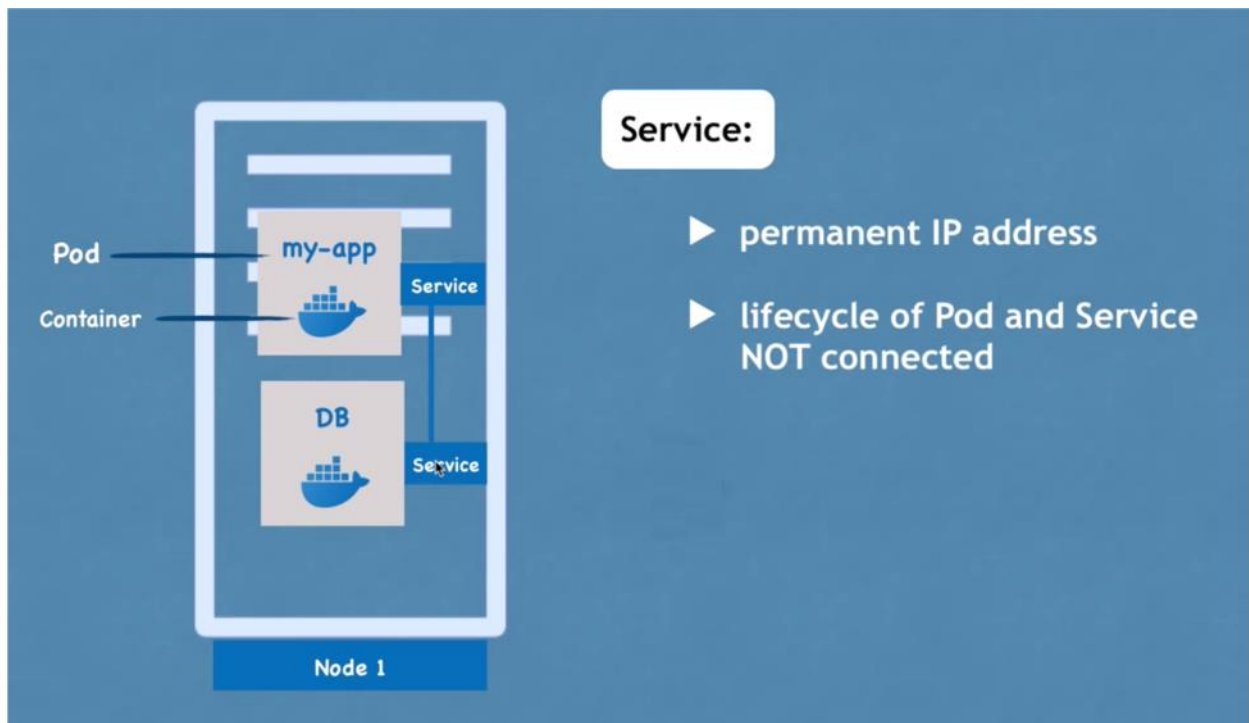


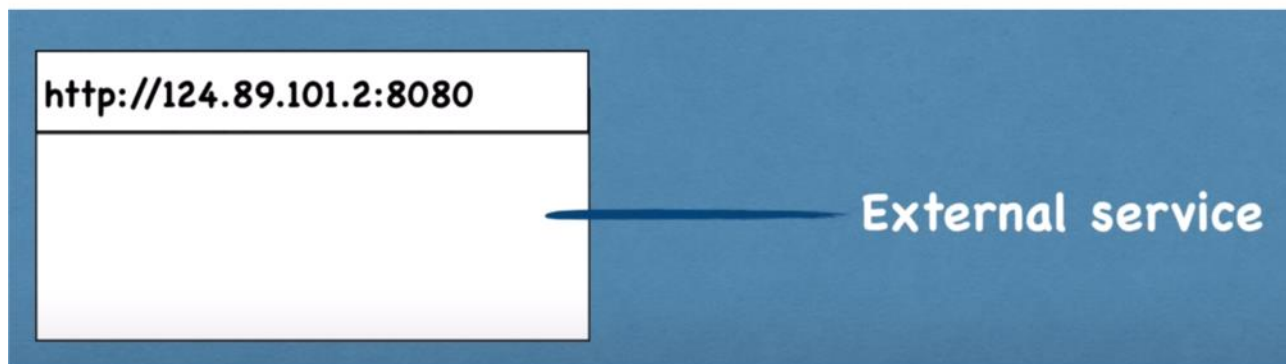
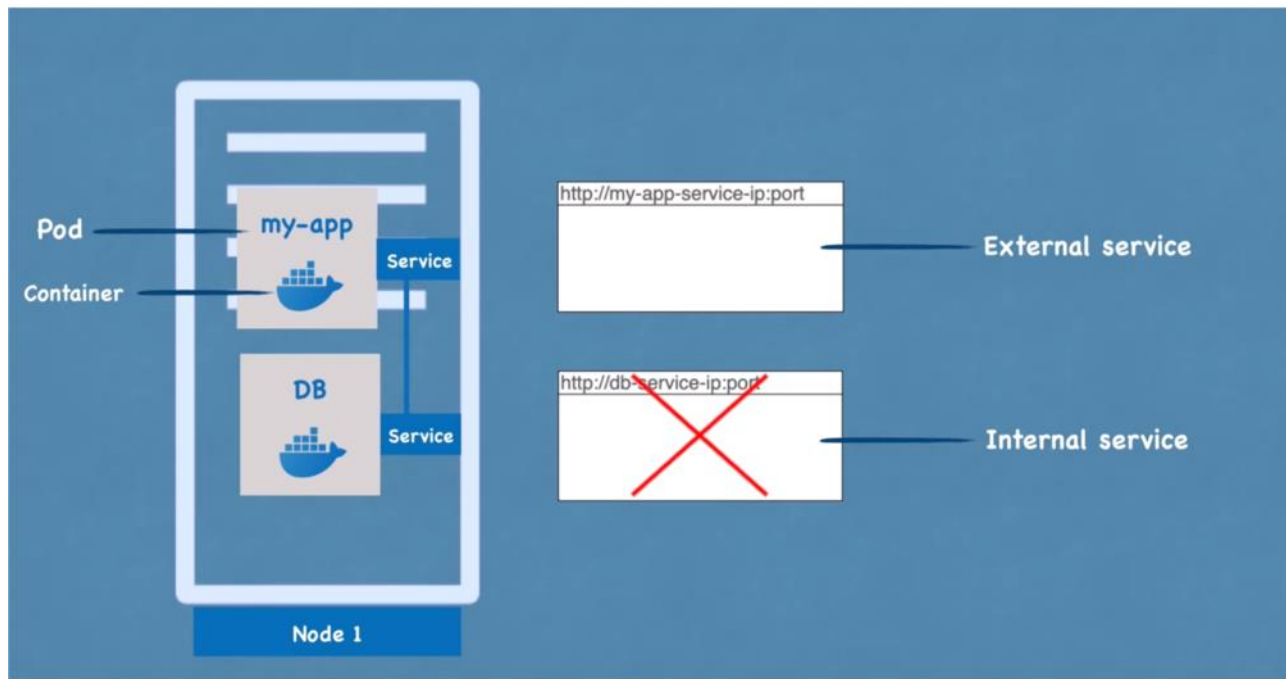


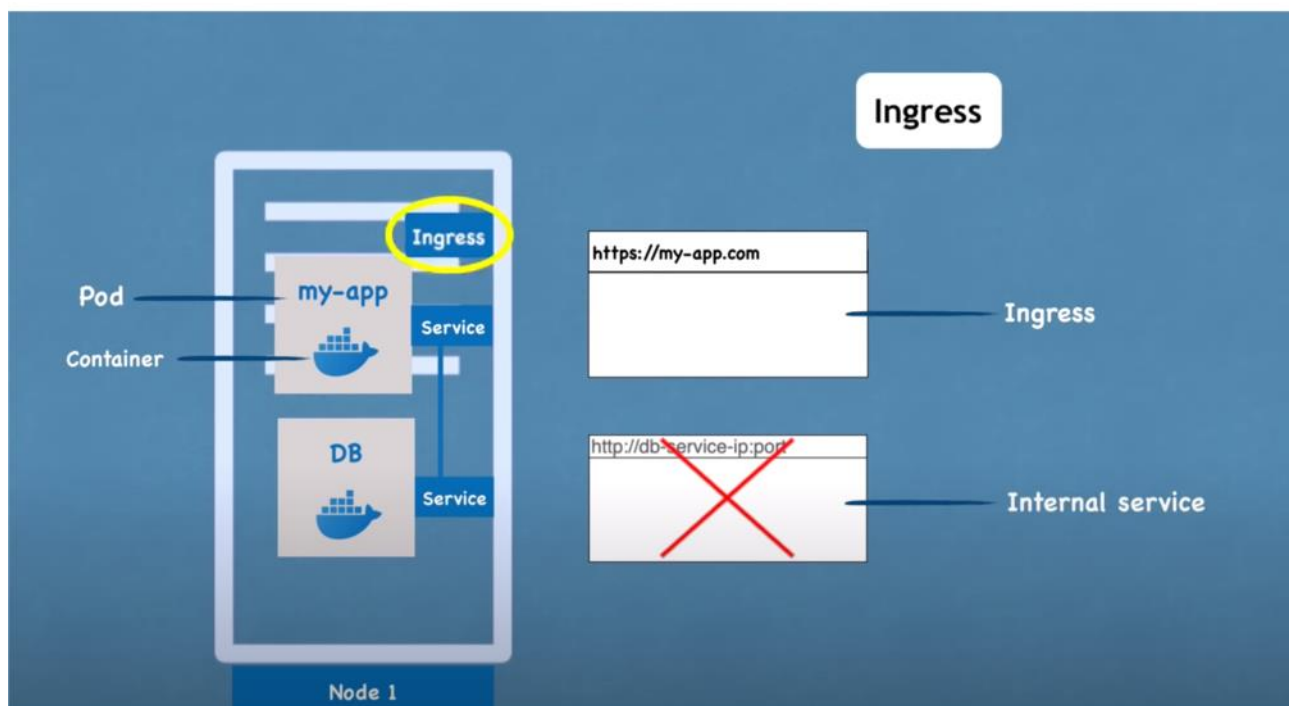
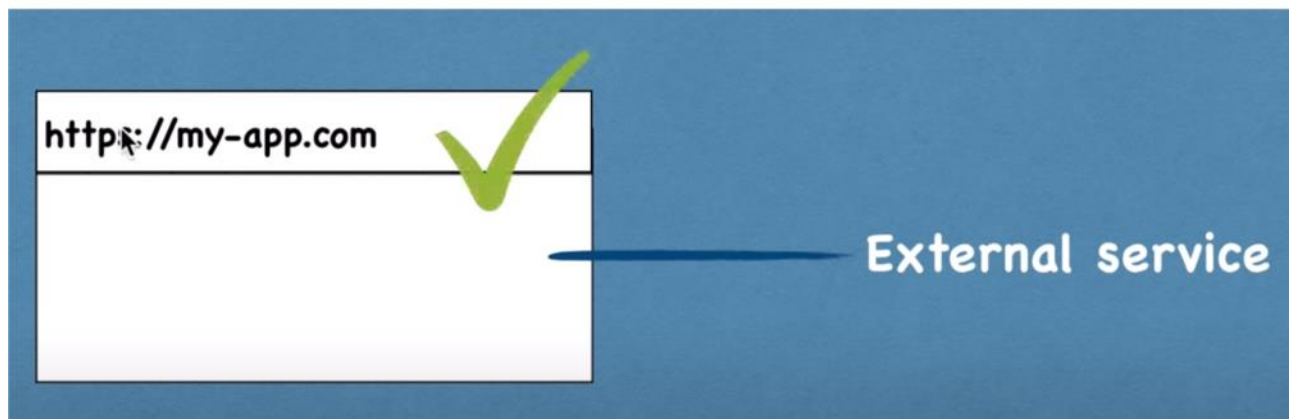


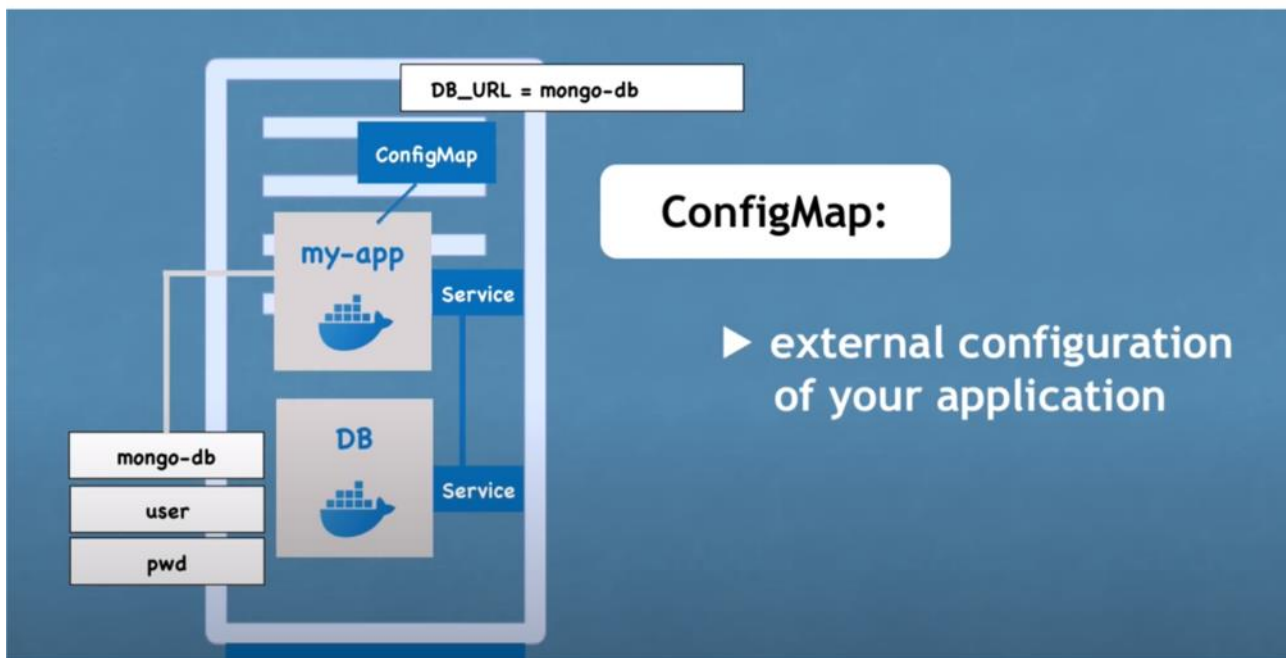
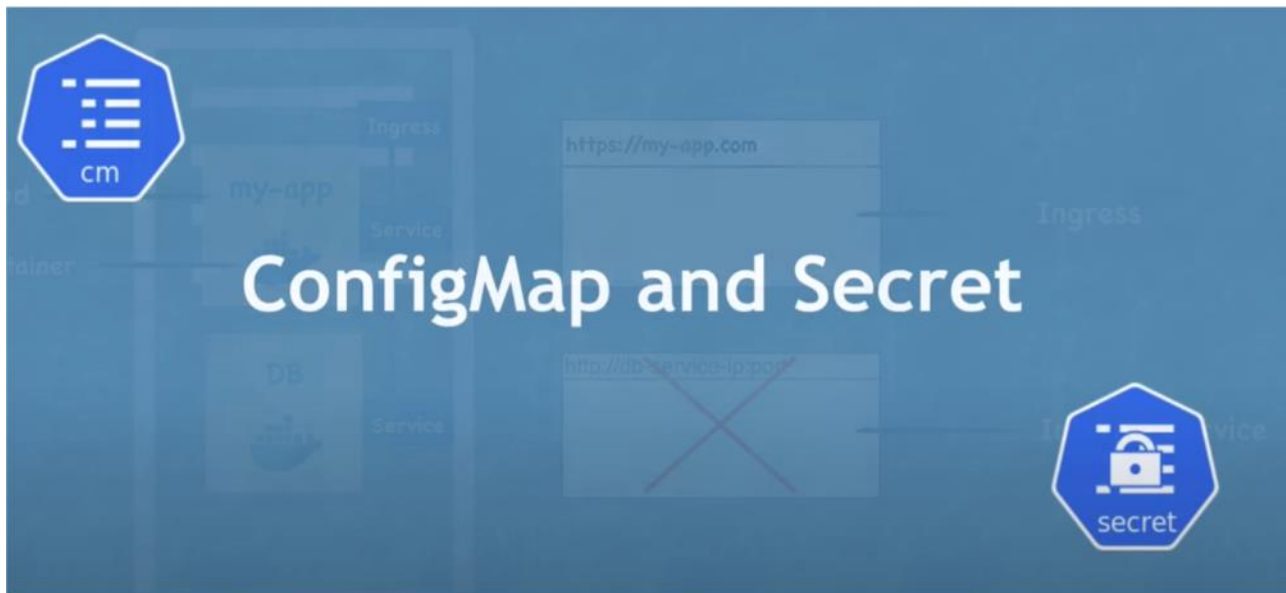


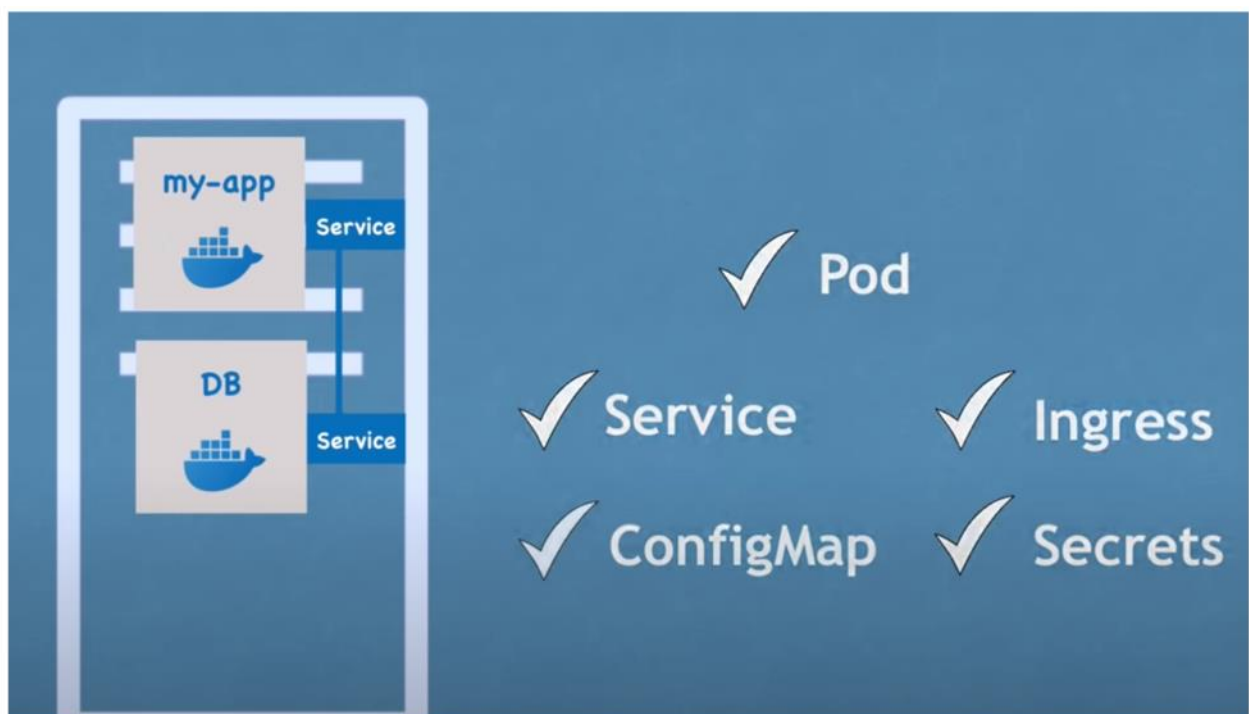
Service and Ingress

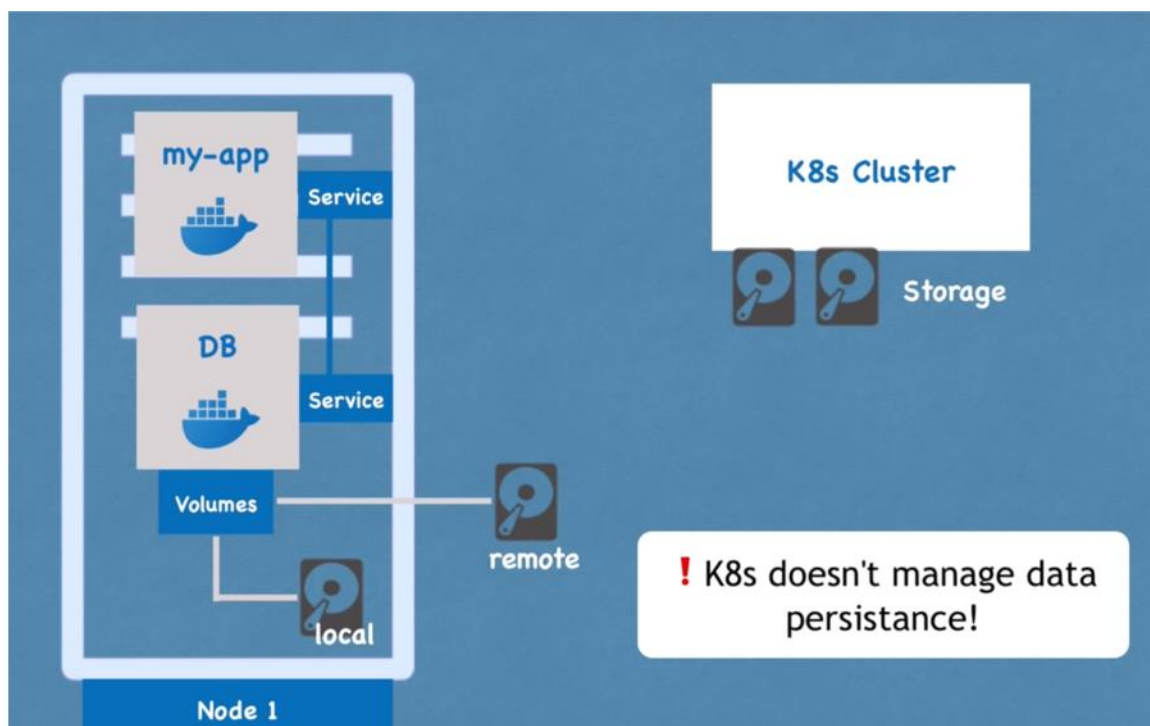
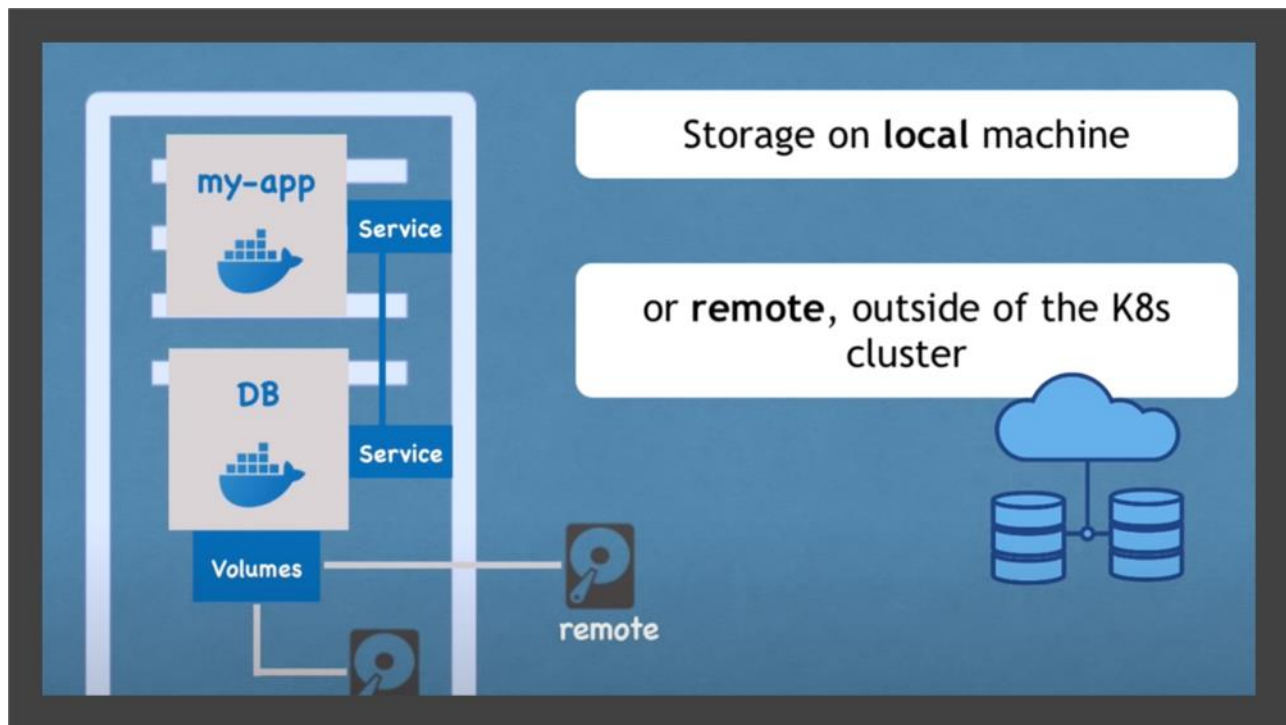


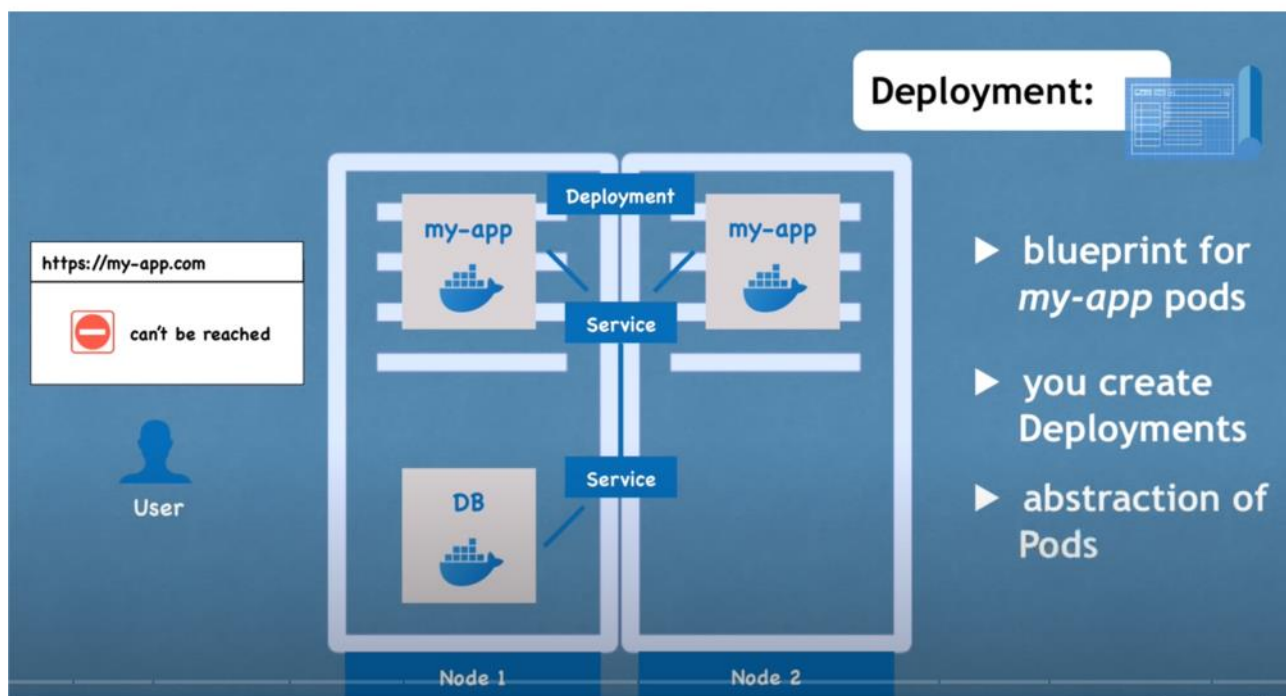
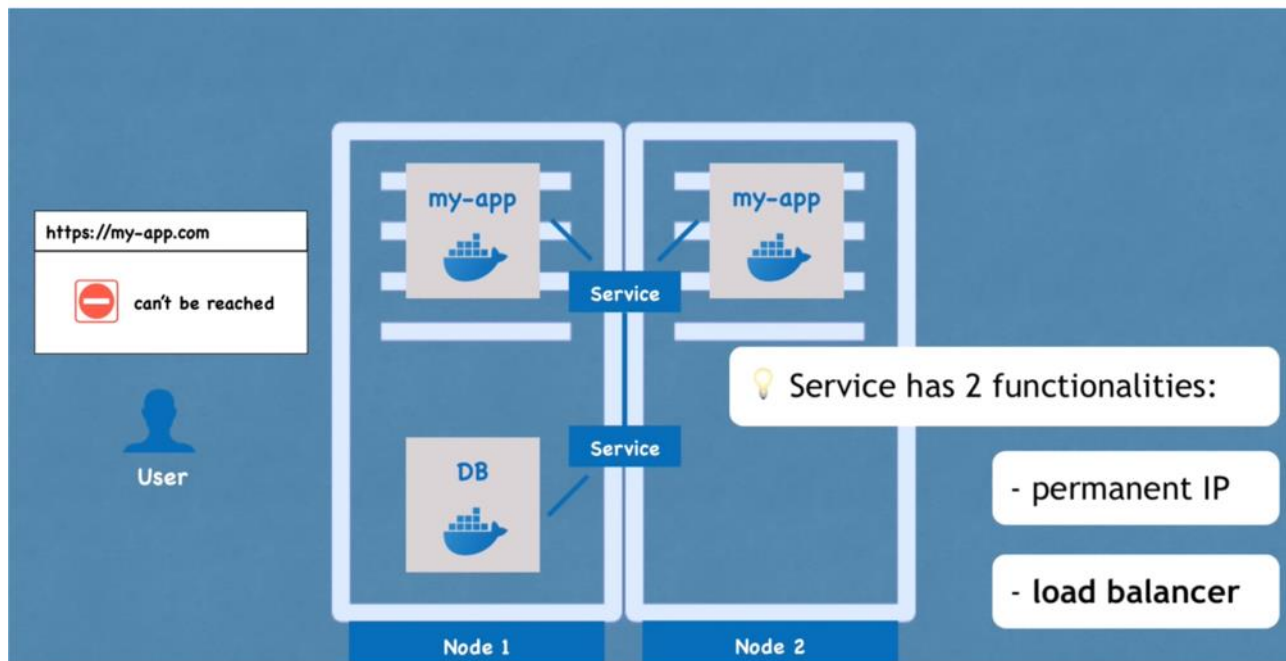


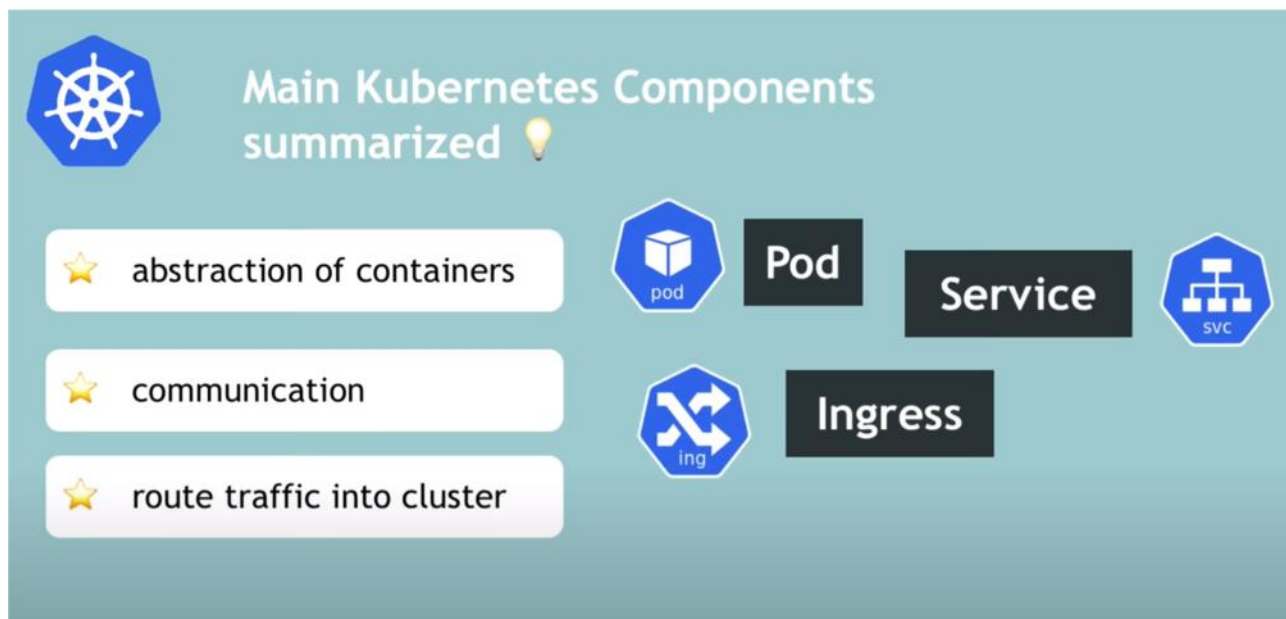
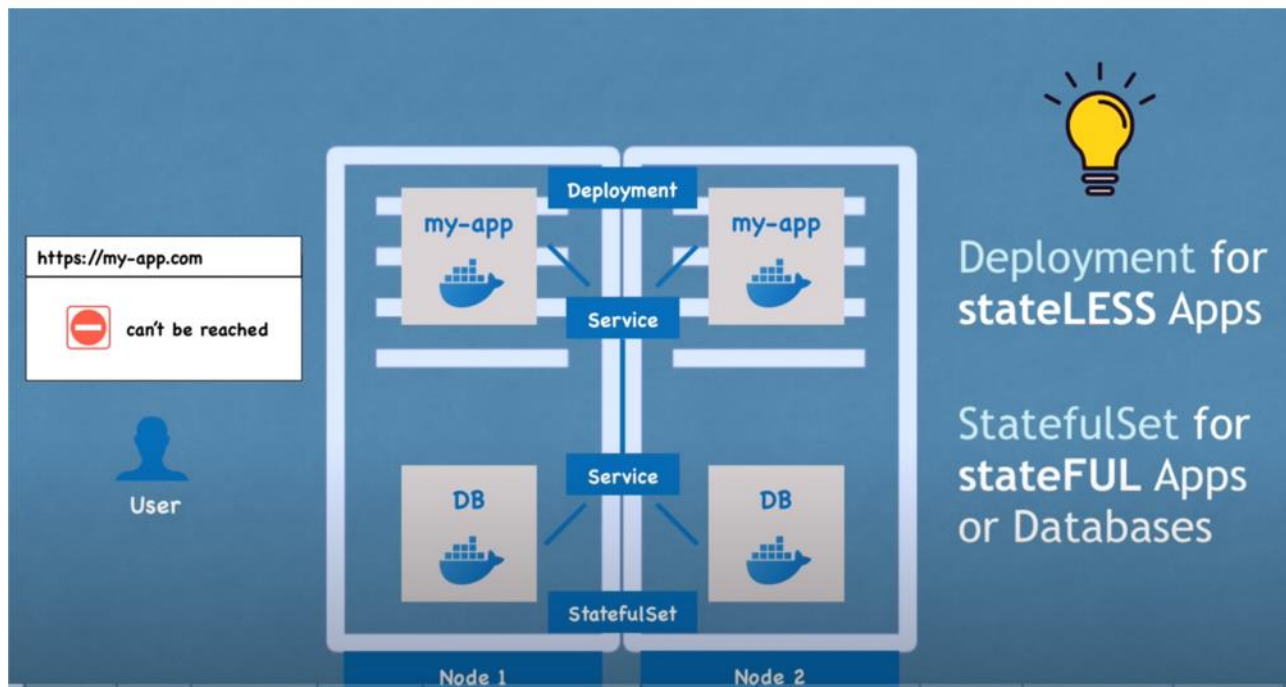














Main Kubernetes Components summarized 💡



external configuration

ConfigMap



Secrets



Pod

Service

Ingress



Main Kubernetes Components summarized 💡



Volumes



Pod

Service

Ingress

ConfigMap

Secrets



Kubernetes Architecture





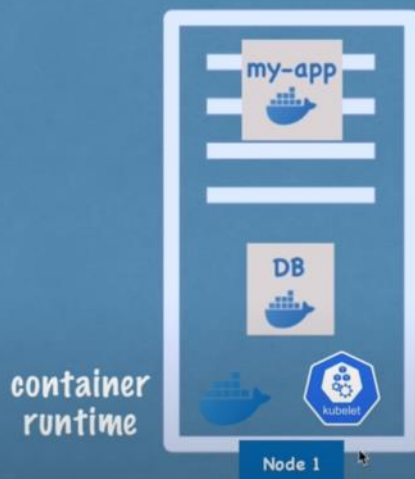
Worker machine in K8s cluster



- ▶ each Node has multiple Pods on it
- ▶ 3 processes must be installed on every Node
- ▶ Worker Nodes do the actual work

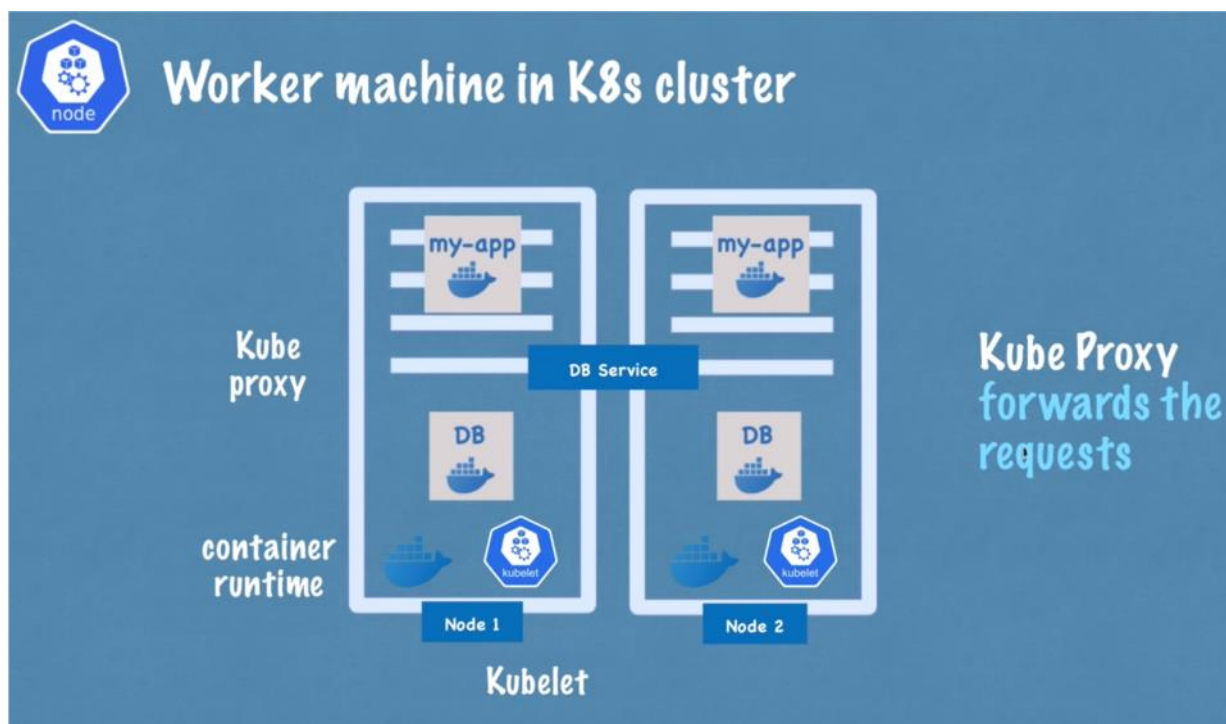
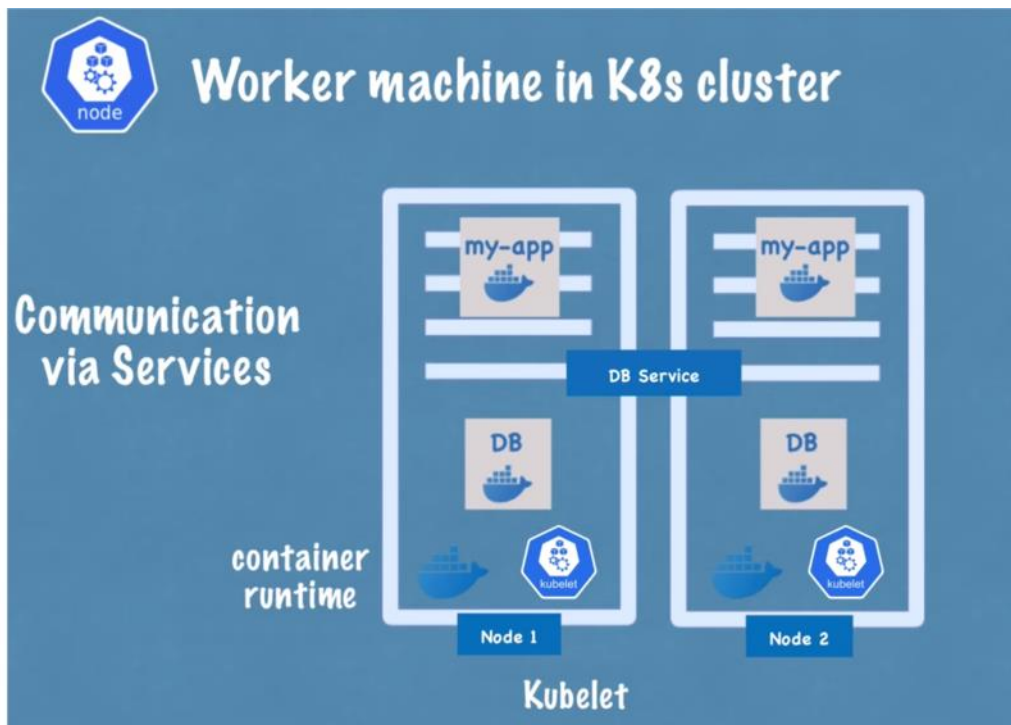


Worker machine in K8s cluster



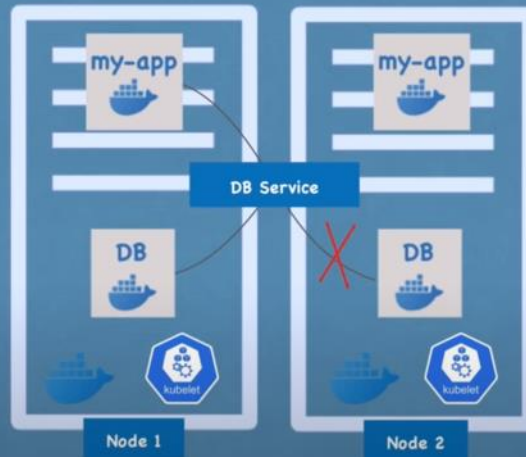
Kubelet interacts with both - the container and node

Kubelet starts the pod with a container inside





Worker machine in K8s cluster



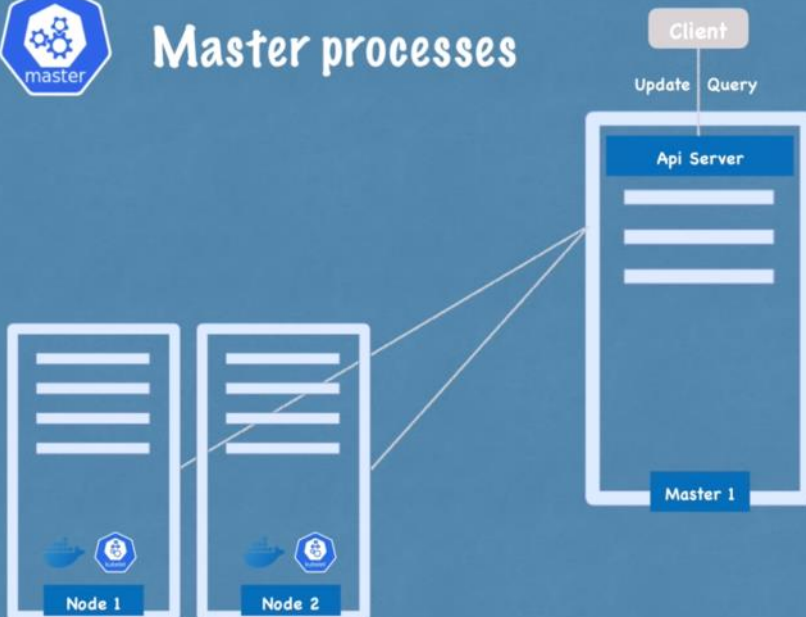
3 Node processes:



1. Kubelet
2. Kube Proxy
3. Container runtime

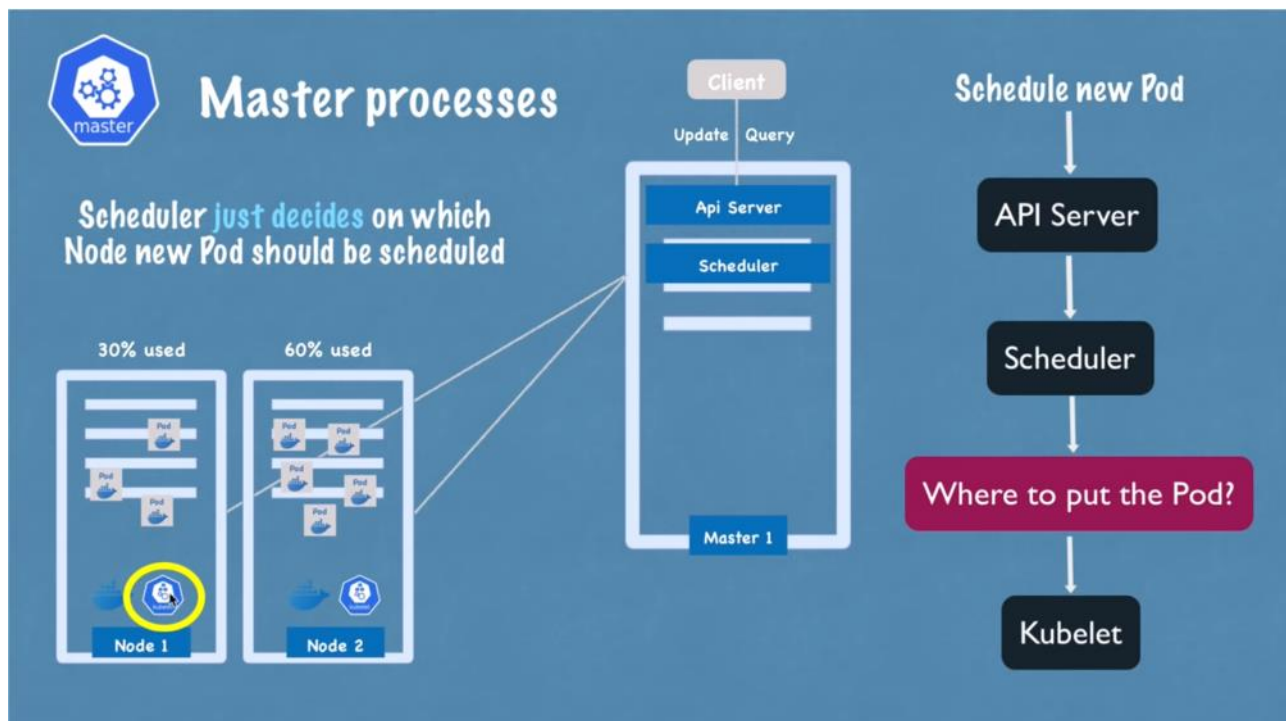
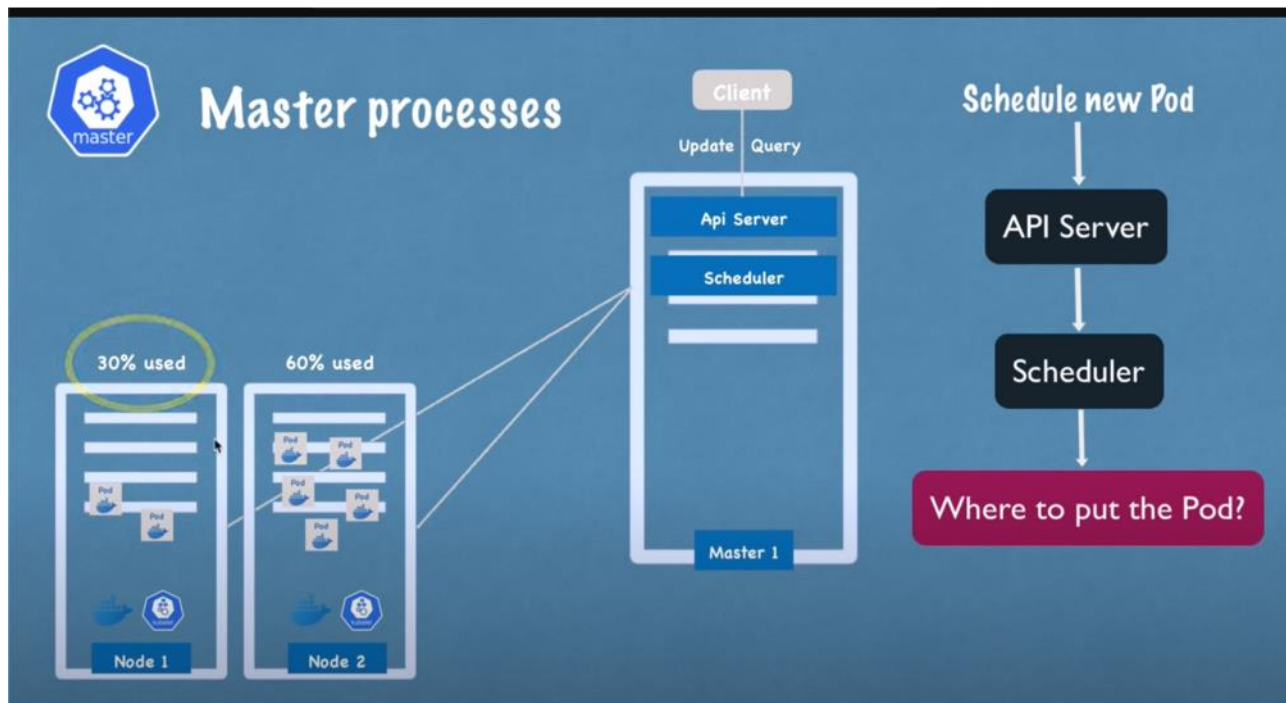


Master processes



- ▶ cluster gateway
- ▶ acts as a gatekeeper for authentication!





Interview - Why EKS over ECS

Wednesday, July 10, 2024 11:42 PM

- **Multi-cloud Support:** Kubernetes is cloud-agnostic and supports multi-cloud deployments. This means you can potentially move workloads between different cloud providers or manage hybrid deployments more easily compared to ECS, which is more tightly integrated with AWS.
- **Declarative Configuration:** Kubernetes uses YAML manifests to define the desired state of applications and infrastructure, which promotes versioning, reproducibility, and easier management of complex application deployments.

Dynatrace Dashboard

Saturday, July 29, 2023 4:06 PM



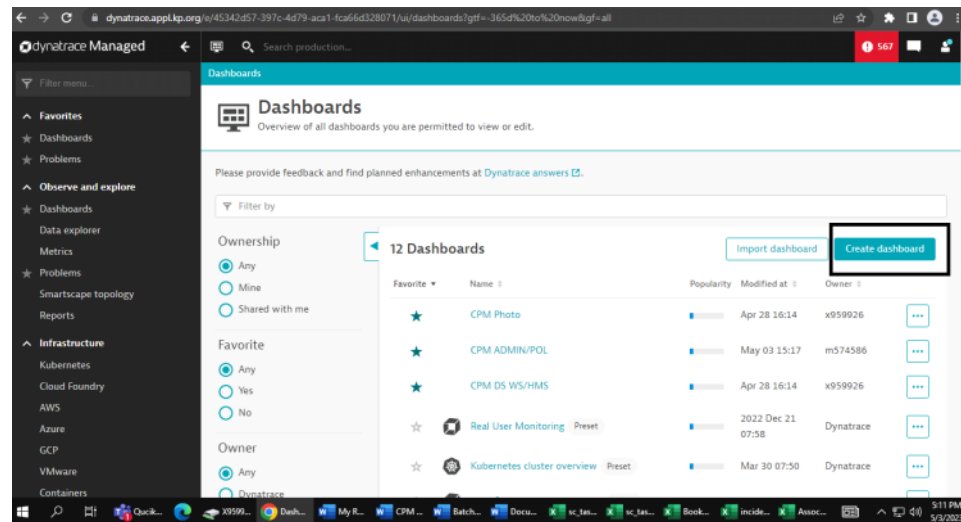
Dynatrace
Dashboar...

Dynatrace Dashboard creation – SOP

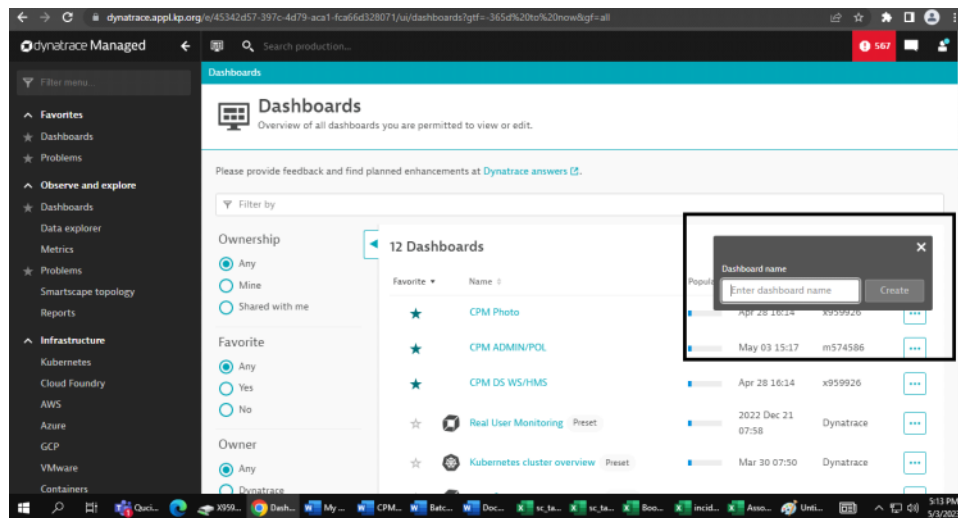
Step 1: <https://dynatrace.appl.kp.org/e/45342d57-397c-4d79-aca1-fca66d328071/ui/dashboards?gtf=-365d%20to%20now&gf=all>

Log in using NUID and CS domain password

Step 2: Click on Create Dashboard

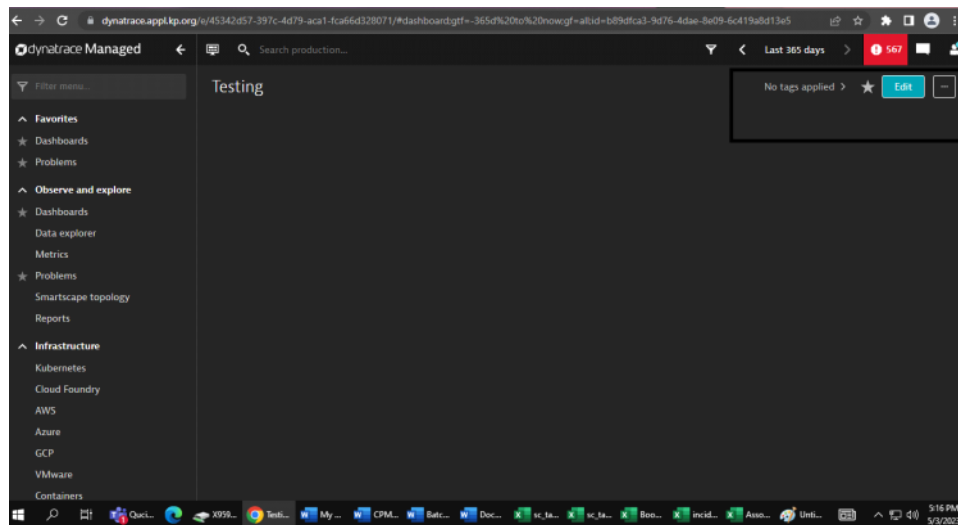


Step 3: Enter the dashboard name which you want to create, and click on **create**



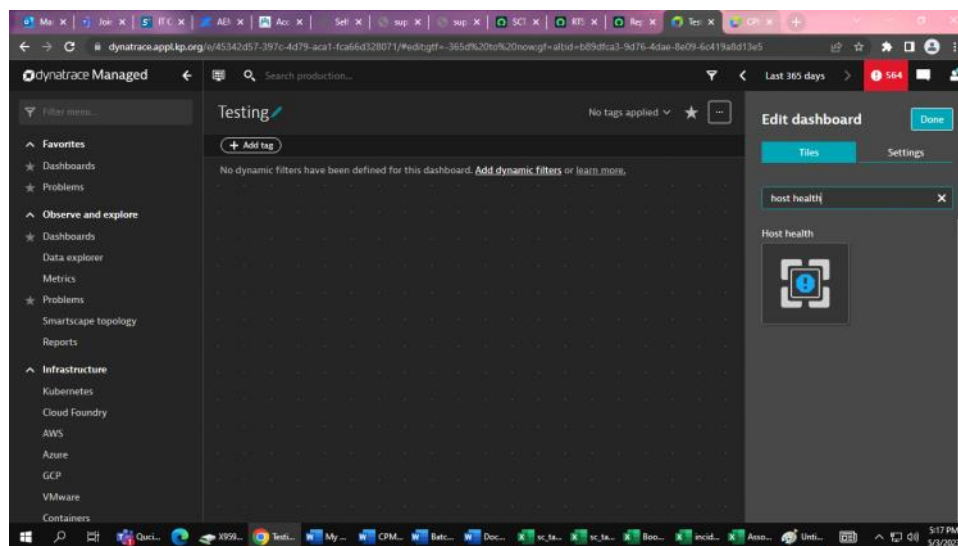
Step 4:

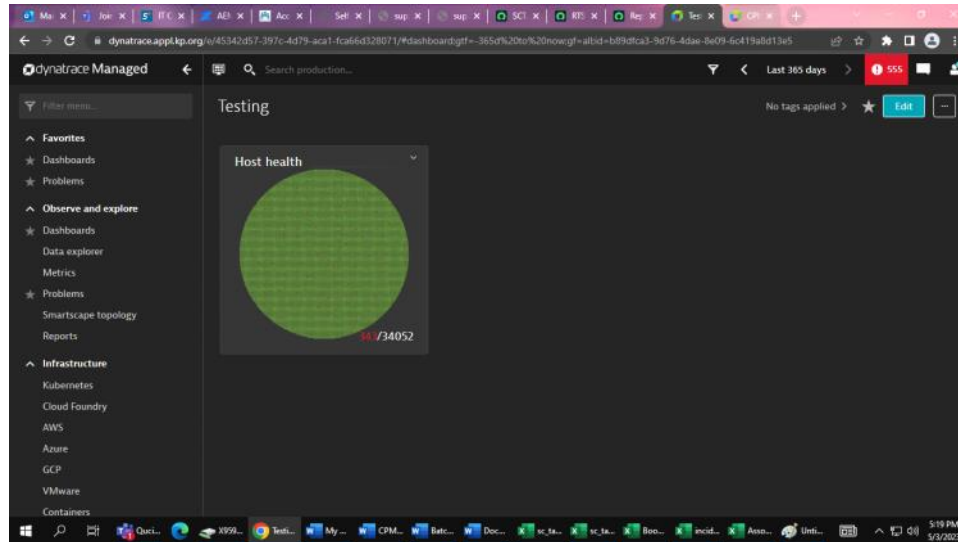
Click on edit



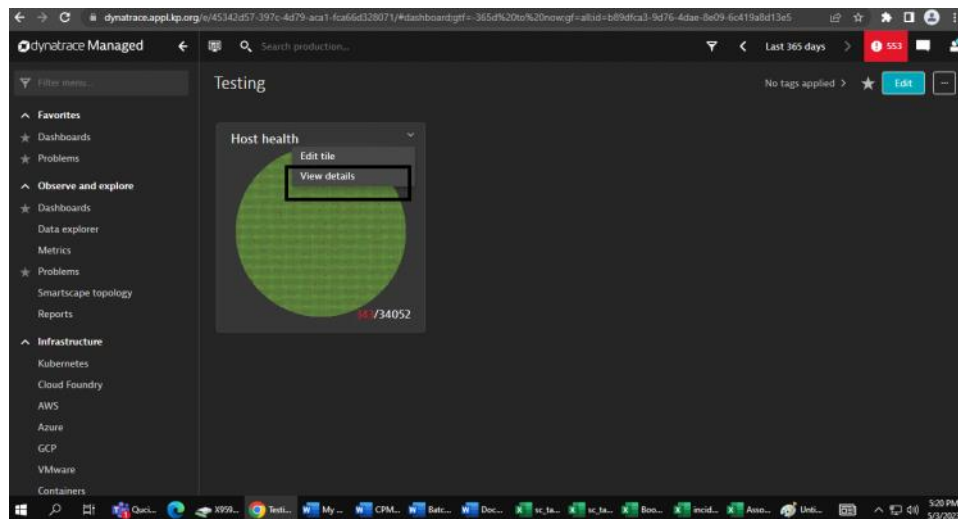
Step 5:

In filter tiles type "host health" and drag and drop it in the dashboard

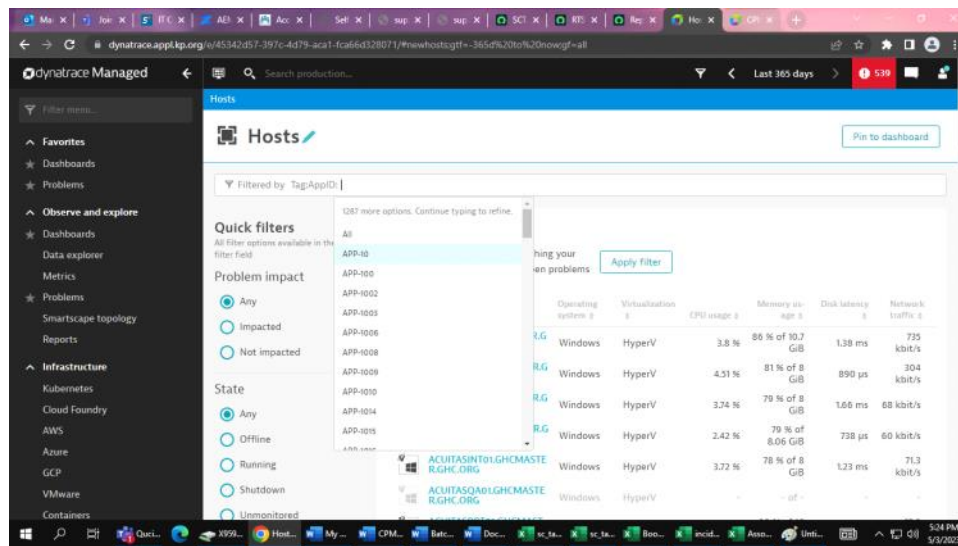




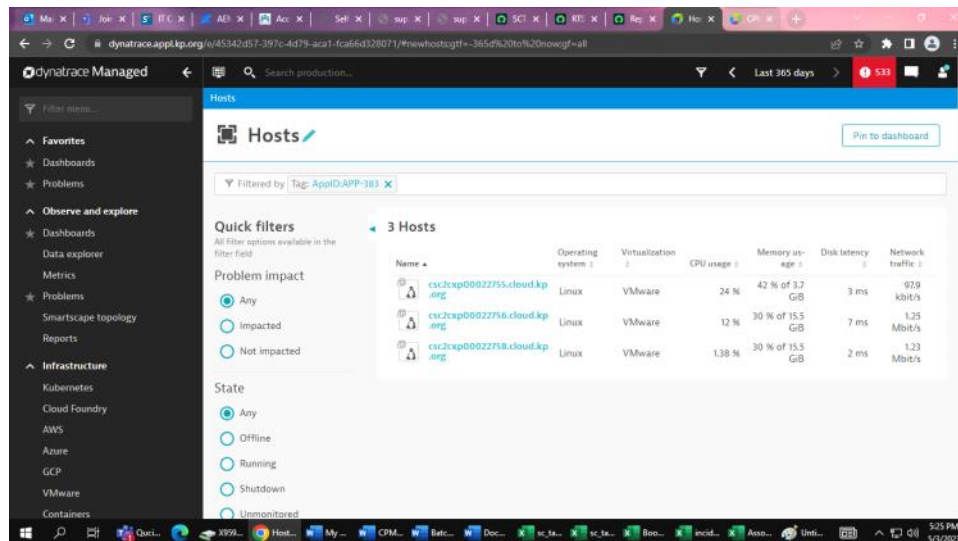
Step 6: Click the drop down and click on **view details**



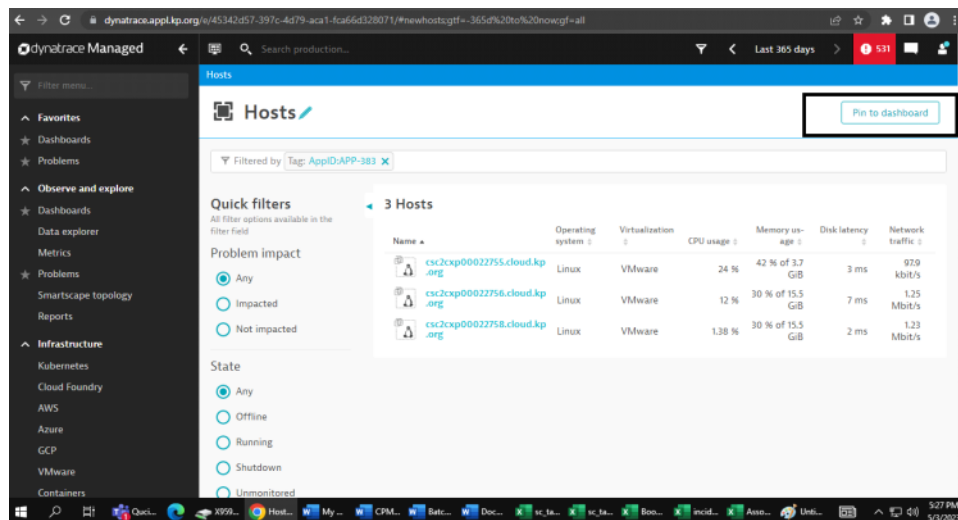
Step 7: Filtered by **tag** and AppID and enter your application ID and click enter



The application server will appear after tagged the AppID in the host

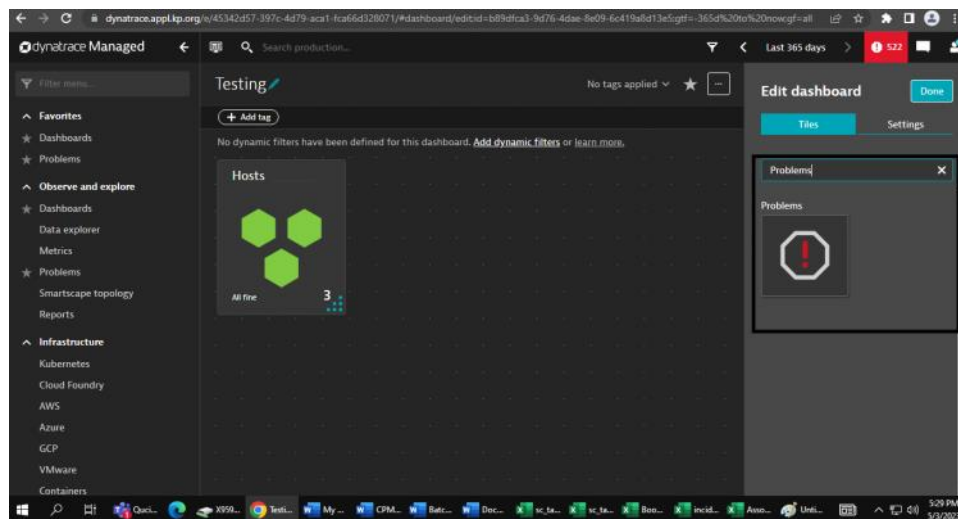


Click on **Pin to Dashboard**

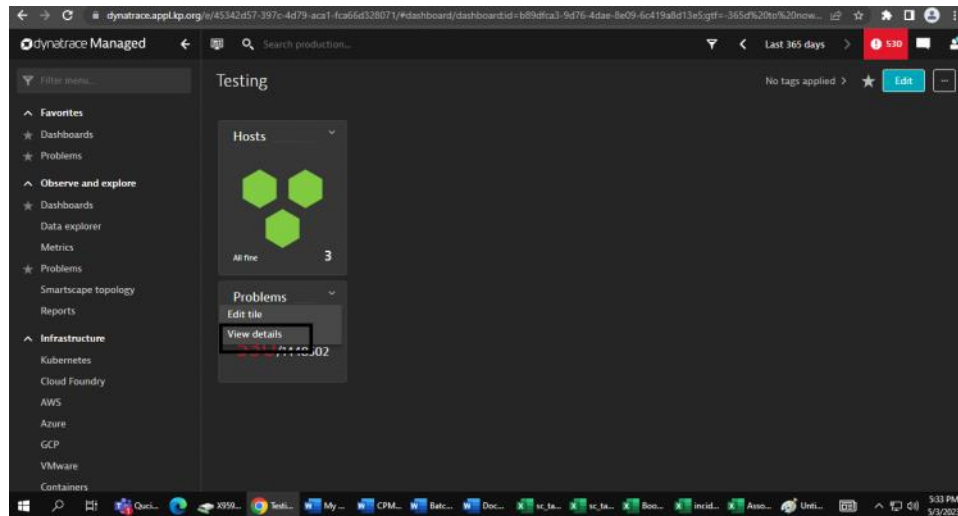


Step 8:

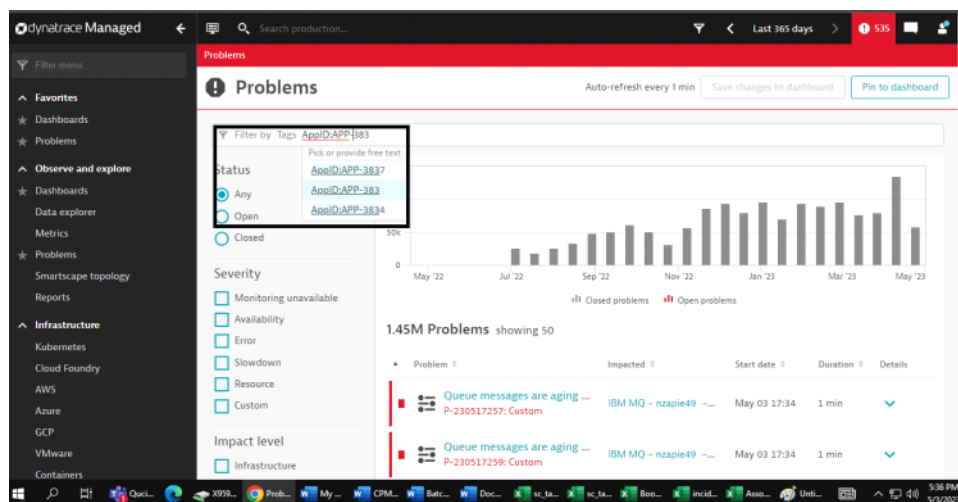
In filter tiles type **problems** and drag and drop it in the dashboard



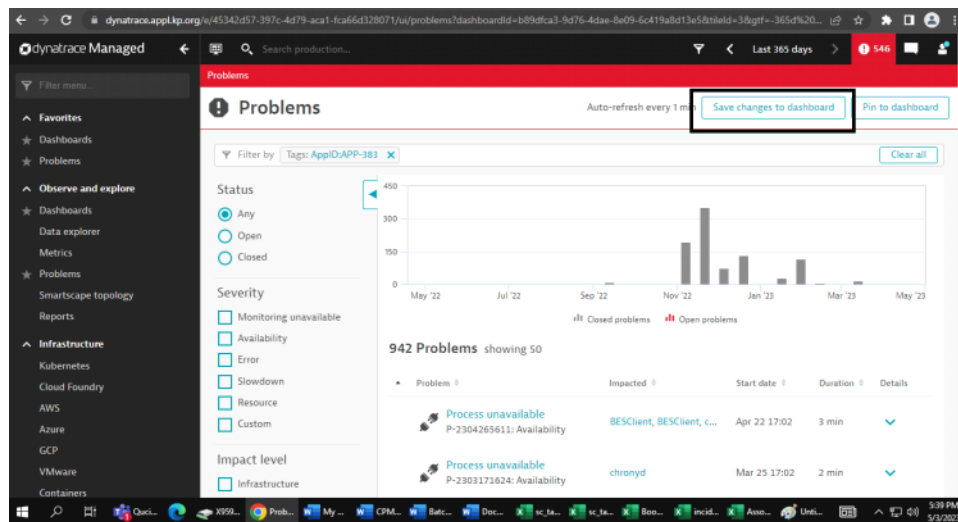
Click the drop down and click on **view details**



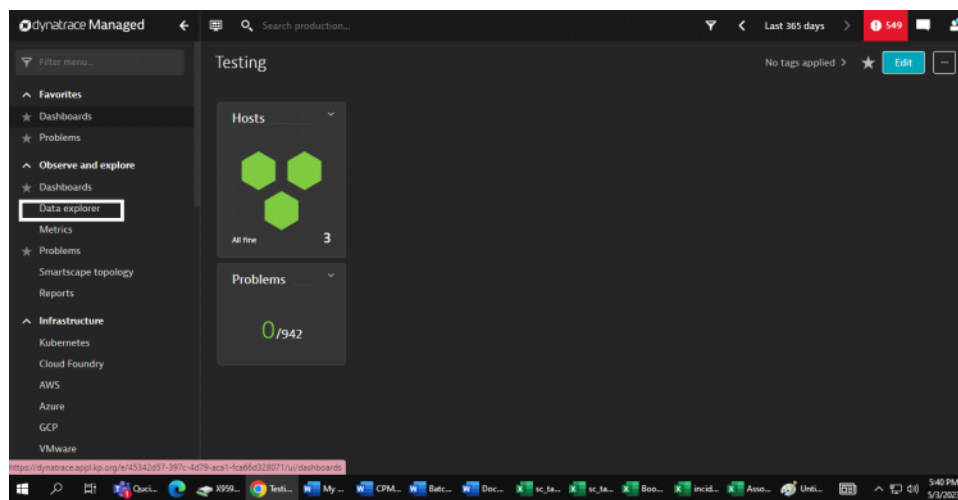
Filtered by tags, AppID: APP- (APP ID) and choose your respective app id and click on enter



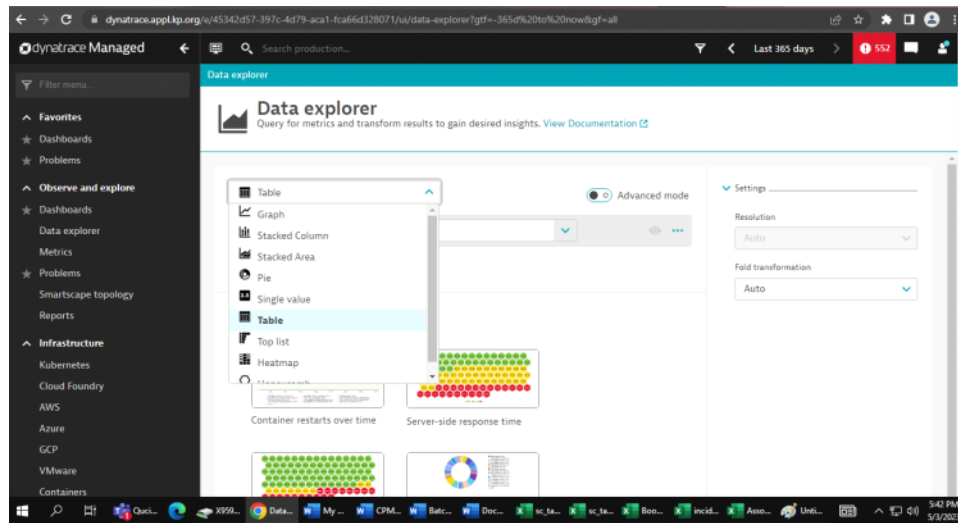
Click on **save changes to dashboard**



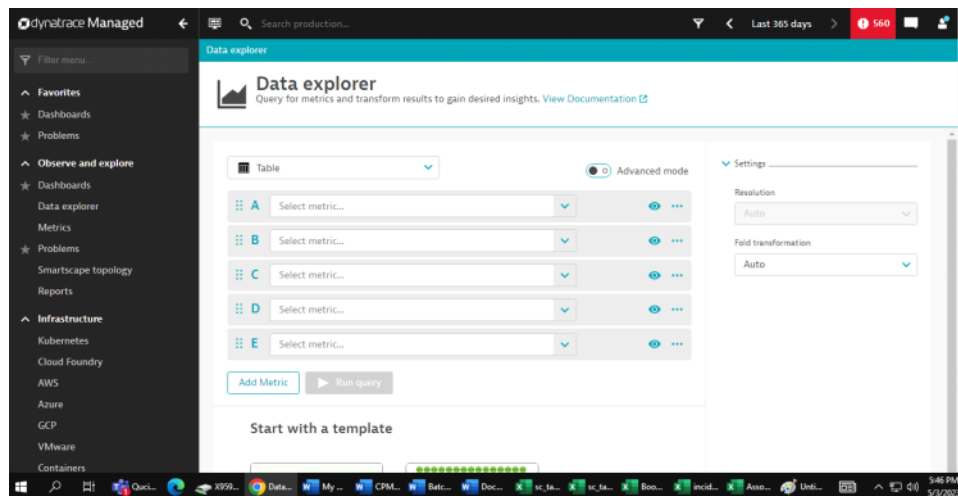
Step 9: Observe and explorer → Data Explorer



Choose table



Create a 5 Metric



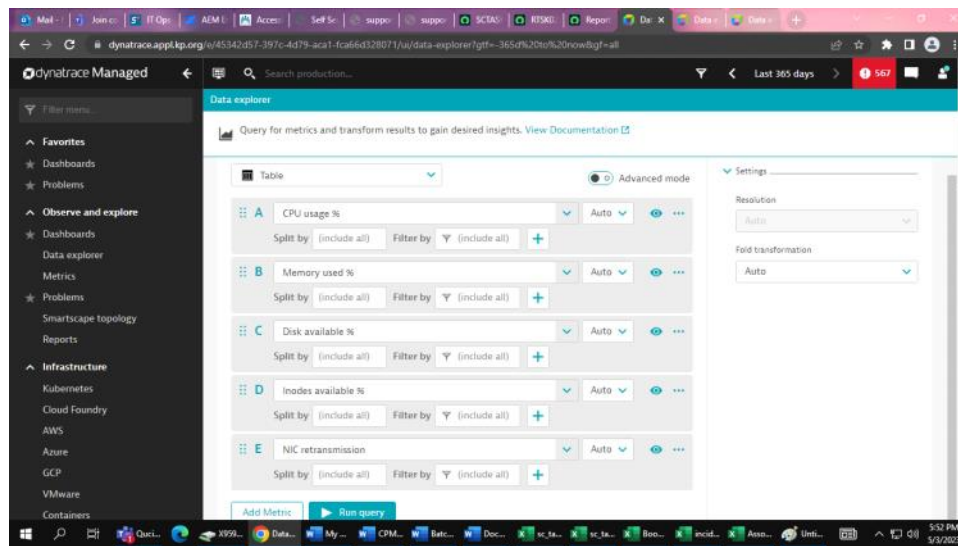
Metric A: CPU Usage %

Metric B: Memory Used %

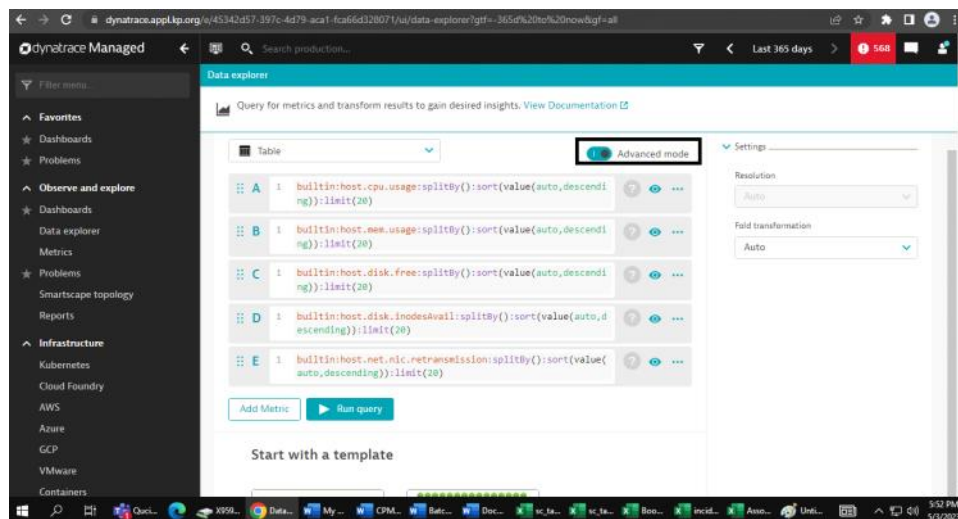
Metric C: Disk Available %

Metric D: Inodes available %

Metric E: NIC retransmission %



Change to **Advanced mode**



Write a query inside each metric and replace the server name with your respective application

- **Metric A: CPU Usage %**

```
builtin:host.cpu.usage:filter(
  and(
    or(
      in("dt.entity.host",entitySelector("type(host),entityName.in(
        ~"csc2cxp00022749.cloud.kp.org~",
        ~"csc2cxp00022750.cloud.kp.org~",
        ~"csc2cxp00022751.cloud.kp.org~"
      )))
    )
  )
):splitBy("dt.entity.host"):sort(value(auto,descending)):limit(100)
```

- **Metric B: Memory Used %**

```
builtin:host.mem.usage:filter(
  and(
    or(
      in("dt.entity.host",entitySelector("type(host),entityName.in(
        ~"csc2cxp00022749.cloud.kp.org~",
        ~"csc2cxp00022750.cloud.kp.org~",
        ~"csc2cxp00022751.cloud.kp.org~"
      )))
    )
  )
):splitBy("dt.entity.host"):sort(value(auto,descending)):limit(100)
```

- **Metric C: Disk Available %**

```
builtin:host.disk.free:filter(
  and(
    or(
      in("dt.entity.host",entitySelector("type(host),entityName.in(
        ~"csc2cxp00022749.cloud.kp.org~",

```

```

        ~"csc2cxp00022750.cloud.kp.org~",
        ~"csc2cxp00022751.cloud.kp.org~"
    ))))
)
)
):splitBy("dt.entity.host"):sort(value(auto,descending)):limit(100)

```

- **Metric D:** Inodes available %

```

builtin:host.disk.inodesAvail:filter(
    and(
        or(
            in("dt.entity.host",entitySelector("type(host),entityName.in(
                ~"csc2cxp00022749.cloud.kp.org~",
                ~"csc2cxp00022750.cloud.kp.org~",
                ~"csc2cxp00022751.cloud.kp.org~"
            )))
        )
    )
):splitBy("dt.entity.host"):sort(value(auto,descending)):limit(100)

```

- **Metric E:** NIC retransmission %

```

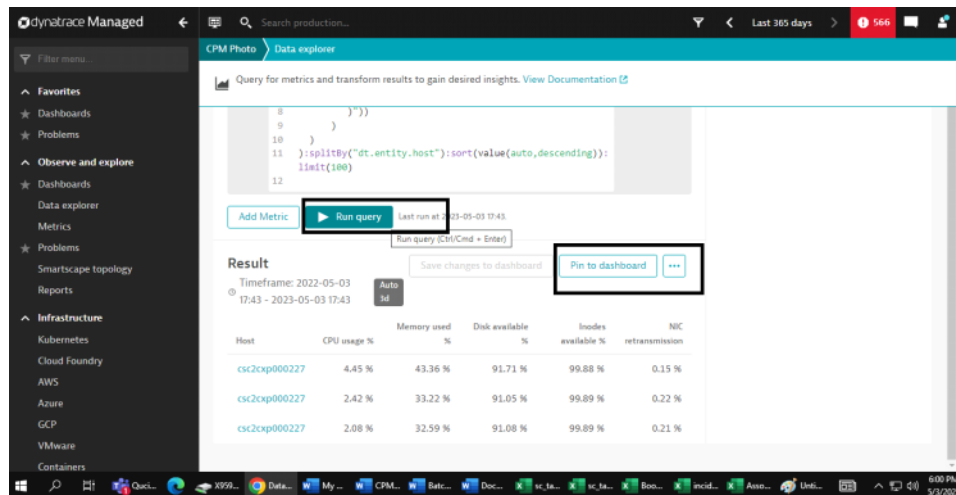
builtin:host.net.nic.retransmission:filter(
    and(
        or(
            in("dt.entity.host",entitySelector("type(host),entityName.in(
                ~"csc2cxp00022749.cloud.kp.org~",
                ~"csc2cxp00022750.cloud.kp.org~",
                ~"csc2cxp00022751.cloud.kp.org~"
            )))
        )
    )
)

```

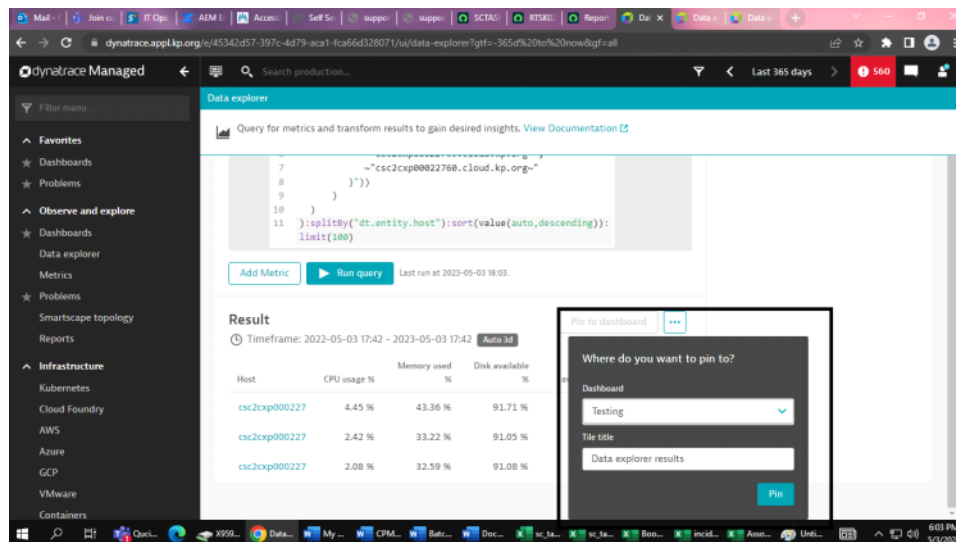
)

```
):splitBy("dt.entity.host"):sort(value(auto,descending)):limit(100)
```

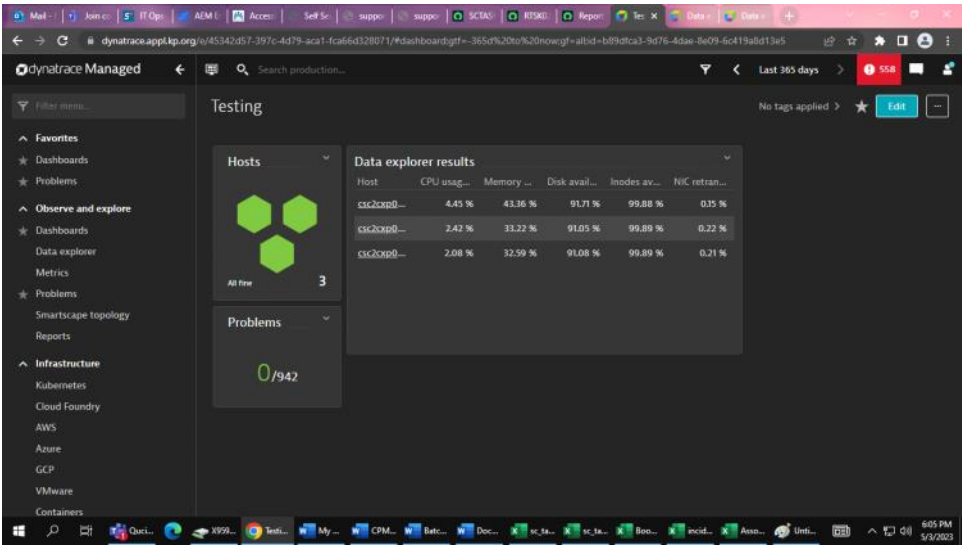
Step 10: Click on **run query** and Click on **Pin to dashboard**



Choose the dashboard and click on pin



Dynatrace Dashboard:



Git

Saturday, July 29, 2023 4:40 PM

The important commands are

git init - This is to initialize an empty repository

git add - This is to add files to the staging area from your working directory

git commit - This is to commit files onto git

git push - This is to push changes onto a remote repository

git pull - This is to pull changes from a git repository

git push -u origin master

git branch - This is for listing, creating and deleting branches

git checkout - You can switch between branches with this command

git checkout commitHash

To change email

```
C:\devops>git config --global user.name "Alan Rodrigues"
C:\devops>git config --global user.email "techsup1000@gmail.com"
```

git status

git log

more FielA.txt

New Branch

git branch - master main

git checkout master

git merge feature

Squash merge -- to ignore commits from feature branch when merged into master

git merge --squash feature

Publish all branch

git push --all origin

Horizontal vs vertical architecture

Tuesday, December 19, 2023 11:31 AM

Horizontal Scaling (Scale-Out):

- **When to Choose:**

1. **Increased Load:** Choose horizontal scaling when your application's load is increasing, and you need to distribute the load across multiple servers.
2. **Distributed Workloads:** If your application can be easily partitioned into independent units of work that can run concurrently, horizontal scaling is effective.
3. **Cost Efficiency:** Horizontal scaling often allows you to add more commodity hardware, which can be cost-effective compared to upgrading a single powerful machine.

- **Factors:**

1. **Ease of Horizontal Scaling:** Some applications are designed to scale horizontally more easily than others. Consider the architecture of your application and how well it supports distributed scaling.
2. **Network Overhead:** Be mindful of potential increased network communication between horizontally scaled instances.

Vertical Scaling (Scale-Up):

- **When to Choose:**

1. **Increased Resource Requirements:** Choose vertical scaling when your application's resource requirements, such as CPU, memory, or storage, are growing, and adding more resources to a single machine is a feasible solution.
2. **Simplicity:** Vertical scaling can be simpler to implement initially, especially if your application is not designed for distributed architectures.

- **Factors:**

1. **Hardware Limitations:** Vertical scaling has hardware limitations. If you reach the maximum capacity of a single server, you'll need to invest in more powerful hardware, which might become expensive.
2. **Single Points of Failure:** A single large server can become a single point of failure. If it goes down, the entire application may be affected.

Considerations for Both:

1. **Load Characteristics:** Understand the nature of your application's load. If it's variable and unpredictable, a combination of horizontal and vertical scaling might be appropriate.
2. **Cost:** Evaluate the cost implications of both scaling approaches. Horizontal scaling might be more cost-effective in some cases, but it depends on factors like cloud provider pricing and the efficiency of your application's scaling.
3. **Fault Tolerance:** Consider the impact of failures on your application. Horizontal scaling can provide better fault tolerance as the workload is distributed across multiple instances.
4. **Flexibility:** Consider how quickly and easily you need to scale. Horizontal scaling allows for more flexibility and agility in responding to changing demands.
5. **Resource Utilization:** Vertical scaling may lead to underutilization of resources if your application doesn't fully utilize the upgraded hardware. Horizontal scaling allows for more granular resource allocation.

Challenges of using Microservice

Tuesday, December 19, 2023 11:33 AM

Microservices architecture, while offering many benefits such as scalability, flexibility, and technology independence, also comes with its set of challenges. Here are some typical challenges associated with microservices and strategies to address them:

1. Complexity of Distributed Systems:

- **Challenge:** Microservices involve multiple independent services that need to communicate with each other. Managing this complexity can be challenging.
- **Solution:** Use tools for service discovery, API gateways, and centralized logging. Implement thorough monitoring and tracing to troubleshoot issues.

2. Data Management:

- **Challenge:** Each microservice has its own database, and managing data consistency and integrity across services can be complex.
- **Solution:** Choose the right data storage strategy (e.g., eventual consistency), use distributed transactions sparingly, and consider implementing a polyglot persistence approach.

3. Service Coordination and Orchestration:

- **Challenge:** Coordinating actions across multiple services and managing their interactions can be complex, especially when dealing with distributed transactions.
- **Solution:** Implement choreography or orchestration patterns. Tools like Apache Kafka or message queues can help with asynchronous communication and event-driven architectures.

4. Network Latency and Fault Tolerance:

- **Challenge:** As services communicate over the network, latency and failures become more common, impacting the overall system reliability.
- **Solution:** Design for resilience by implementing retries, timeouts, and circuit breakers. Use load balancing to distribute traffic evenly.

5. Security:

- **Challenge:** With multiple services, securing communication between them and managing access control can be challenging.
- **Solution:** Implement proper authentication and authorization mechanisms. Use API gateways with security features and force security best practices for each microservice.

if your db is responding slow then as a SRE what would be your plan of action ?

Tuesday, December 19, 2023 11:37 AM



ChatGPT

As a Site Reliability Engineer (SRE), if you observe that the database is responding slowly, you would typically follow a structured plan of action to identify and address the underlying issues. Here is a general guideline:

1. Monitor and Gather Data:

- Use monitoring tools to gather data on key database metrics such as response time, throughput, and resource utilization (CPU, memory, disk I/O).
- Identify patterns of slow queries or unusual behavior.

2. Isolate the Issue:

- Determine whether the slowness is affecting the entire database or specific queries/operations.
- Check if the issue is isolated to a particular database server, shard, or cluster.

3. Identify the Root Cause:

- Analyze slow query logs and database performance metrics to identify the root cause. Look for inefficient queries, missing indexes, or resource bottlenecks.
- Examine recent changes to the database schema, configuration, or application code that might have contributed to the performance degradation.

4. Optimize Queries:

- If the slow performance is due to specific queries, optimize them. This may involve adding or modifying indexes, rewriting queries, or adjusting the schema.
- Consider using database query optimization tools and techniques.

5. Review Database Configuration:

- Verify that the database server configuration is tuned for optimal performance. Adjust settings related to memory allocation, connection pooling, and query cache based on best practices for your database engine.

6. Scale the Database:

- If the database is under heavy load, consider scaling it horizontally (adding more nodes) or vertically (upgrading hardware resources).
- Evaluate the need for sharding or partitioning data to distribute the workload.

8. **Implement Caching:**

- Introduce caching mechanisms to reduce the load on the database. Consider caching frequently accessed data or query results.
- Evaluate the use of in-memory databases or caching solutions.

9. **Review Indexing Strategy:**

- Ensure that appropriate indexes are in place. Missing or inefficient indexes can significantly impact query performance.
- Regularly review and update index statistics.

Use retention policy

Retention policies are typically associated with data storage and refer to the rules that determine how long data is retained and when it should be deleted. In the context of a database, retention policies might be applied to logs, backups, or other data.

Linux new commands learning

Sunday, October 15, 2023 5:40 PM

3. create link files for tomcat startup.sh and shutdown.sh

```
ln -s /opt/apache-tomcat-<version>/bin/startup.sh /usr/local/bin/tomcatup  
ln -s /opt/apache-tomcat-<version>/bin/shutdown.sh /usr/local/bin/tomcatdown  
tomcatup
```

whereis command

whereis ls

tar -xvzf filename

tree

```
[root@jenkinsserver pipeline_job]# cd webapp/
[root@jenkinsserver webapp]# ls
pom.xml  src  target
[root@jenkinsserver webapp]# tree
.
├── pom.xml
├── src
│   └── main
│       └── webapp
│           ├── index.jsp
│           ├── WEB-INF
│           └── web.xml
└── target
    ├── maven-archiver
    │   └── pom.properties
    ├── surefire
    ├── webapp
    │   ├── index.jsp
    │   ├── META-INF
    │   ├── WEB-INF
    │   │   ├── classes
    │   │   └── web.xml
    └── webapp.war

11 directories, 7 files
[root@jenkinsserver webapp]#
```

adduser vs useradd

Sunday, October 29, 2023 5:15 PM



`adduser` and `useradd` are both Linux commands used for creating user accounts, but they have different purposes and usage.



1. `adduser`:

- `adduser` is a higher-level utility designed for ease of use and user-friendliness. It is a front-end script for creating and adding users in a more interactive and user-friendly manner.
- When you run the `adduser` command, it prompts you for various user details, such as full name, password, and other user information. It also automatically creates a home directory for the user.
- It can set up some default configurations, create the home directory, and copy files from the `/etc/skel` directory to the user's home directory.
- `adduser` is more commonly used on Debian-based systems like Ubuntu.

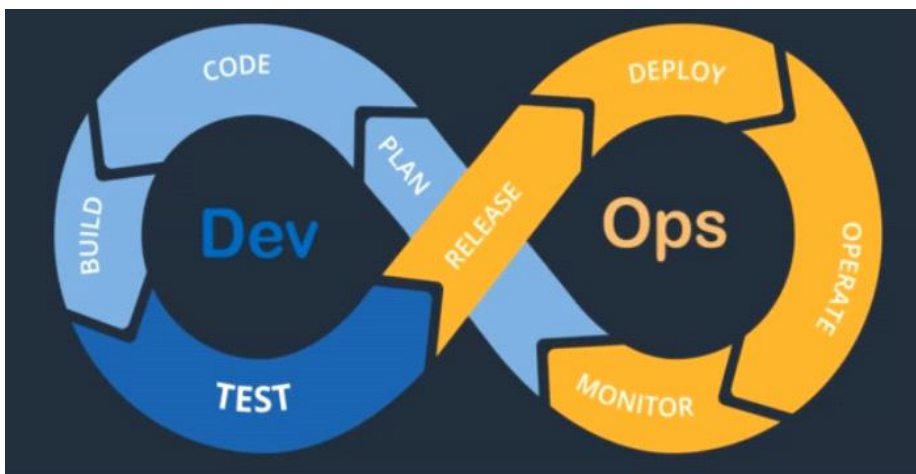
2. `useradd`:

- `useradd` is a lower-level command for creating user accounts and is often used in shell scripts or for system administration tasks where you need more control over the process.
- When you run the `useradd` command, you must specify various user parameters as command-line options. It does not interactively prompt you for user information.
- It doesn't automatically create a home directory or set up user configurations. You need to use additional commands like `passwd` and `mkdir` to set a password and create a home directory for the user, respectively.
- `useradd` is more commonly used on Red Hat-based systems like CentOS and Fedora.

In summary, the main difference between `adduser` and `useradd` is the level of user interaction and automation. `adduser` is more user-friendly and interactive, while `useradd` is a lower-level command that provides more control but requires manual setup of various user attributes. The choice of which command to use may depend on your specific needs and the Linux distribution you are using.

Devops

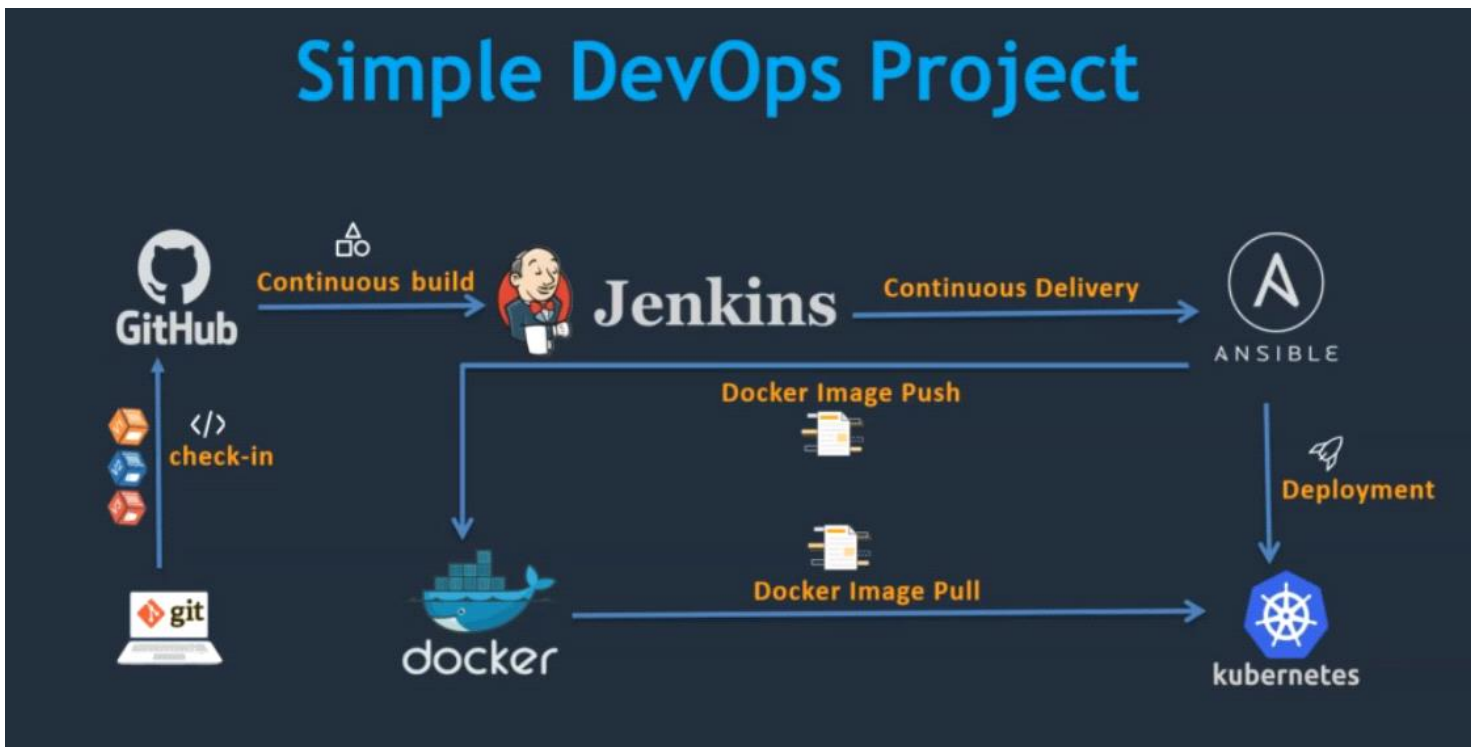
Tuesday, October 10, 2023 2:35 PM



Tools to be used

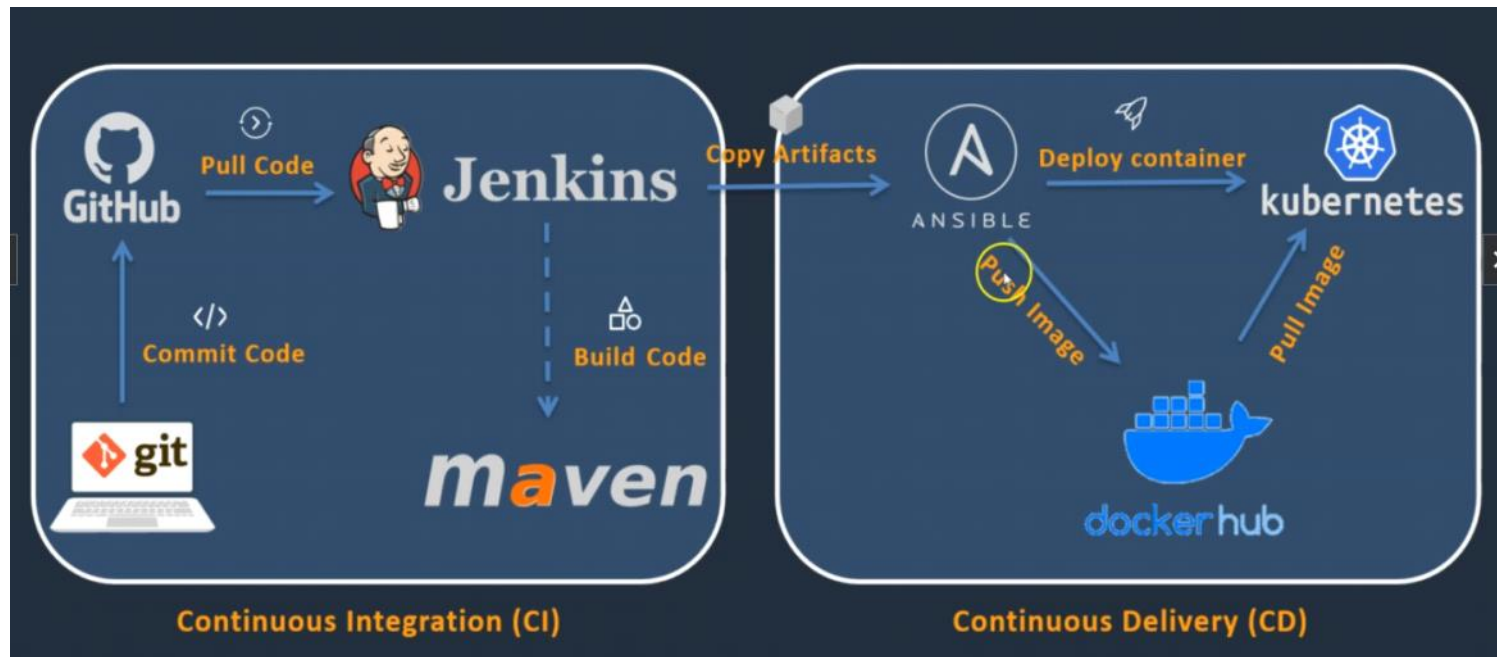


Final end goal



Simple DevOps Project

- Continuous Integration (CI)
- Continuous Delivery (CD)
- Continuous Deployment (CD)



Integrate Jenkins with Github

Sunday, October 15, 2023 5:29 PM

Integrate GitHub with Jenkins

- Install Git on Jenkins Instance
- Install GitHub Plugin on Jenkins GUI
- Configure Git on Jenkins GUI



Integrate Maven in Jenkins

Sunday, October 15, 2023 5:34 PM

Integrate Maven with Jenkins

- Setup Maven on Jenkins Server
- Setup Environment Variables
 - `JAVA_HOME`, `M2`, `M2_HOME`
- Install Maven Plugin
- Configure Maven and Java



Build Code



```
Quick connect... 2. 52.91.26.146 (ec2-user) 3. 52.91.26.146 (ec2-user)

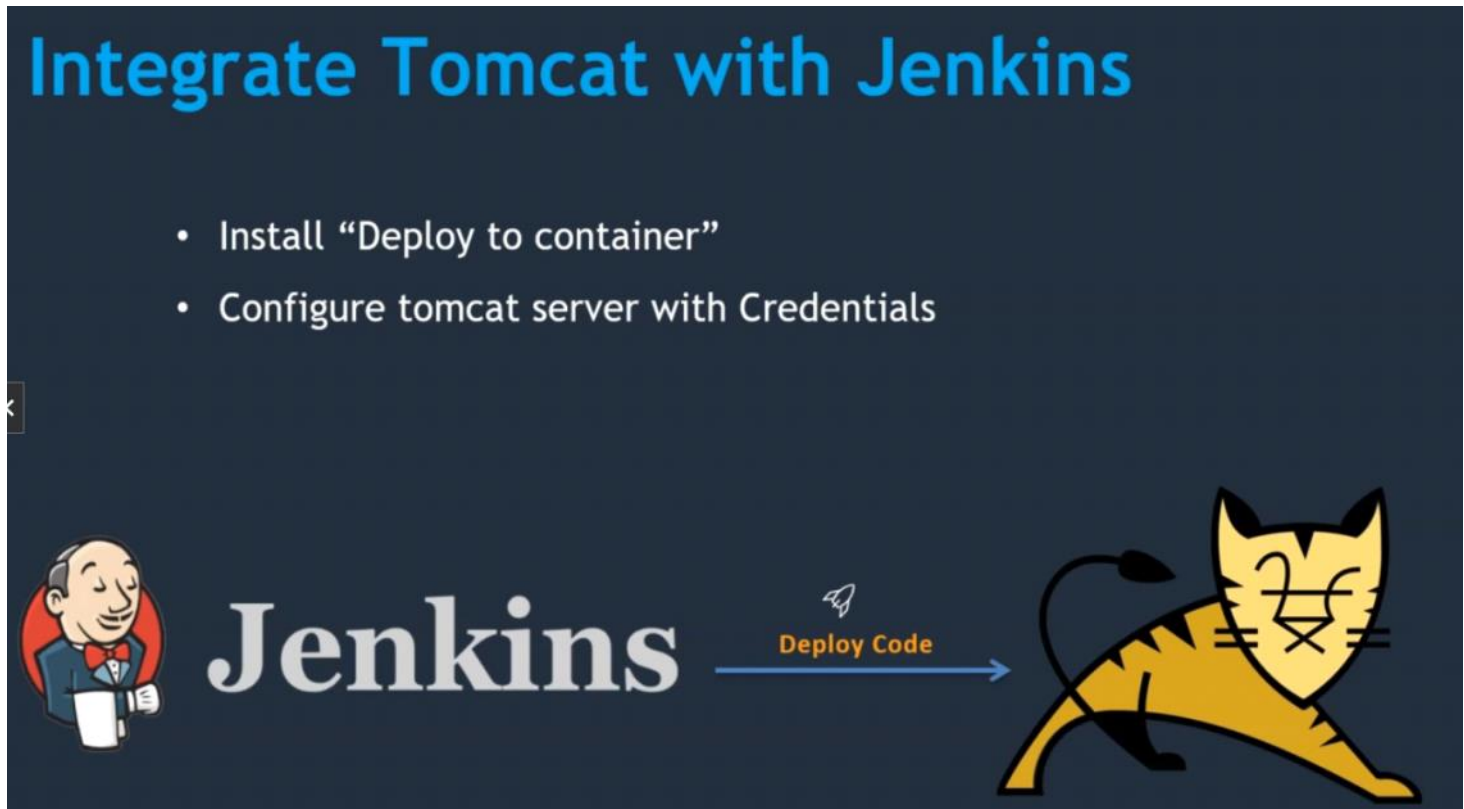
# .bash_profile
# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi
M2_HOME=/opt/maven
M2=/opt/maven/bin
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-11.0.12.0.7-0.amzn2.0.2.x86_64
# User specific environment and startup programs

PATH=$PATH:$HOME/bin:$JAVA_HOME:$M2_HOME:$M2

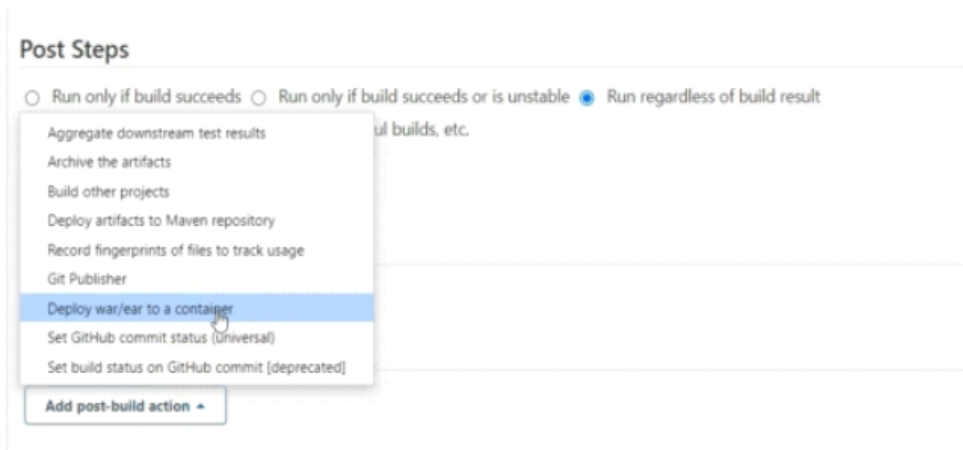
export PATH
```

Integrate Tomcat and Jenkins

Sunday, October 15, 2023 6:22 PM



Once the Deploy to Container plugin is installed, the following option is enabled



Via which you can deploy a war file to tomcat

Post-build Actions

Deploy war/ear to a container

WAR/EAR files ?

**/*.war

War/ear files to deploy. Relative to the workspace root. You can also specify Ant-style GLOBs, like "**/*.war" (from [Deploy to container Plugin](#))

Context path ?

Containers

Tomcat 8.x Remote

Credentials

deployer/***** (tomcat deployer) ▾

Add ▾

Tomcat URL ?

http://52.90.65.63:8080/

Advanced...

Integrate Docker in CI/CD

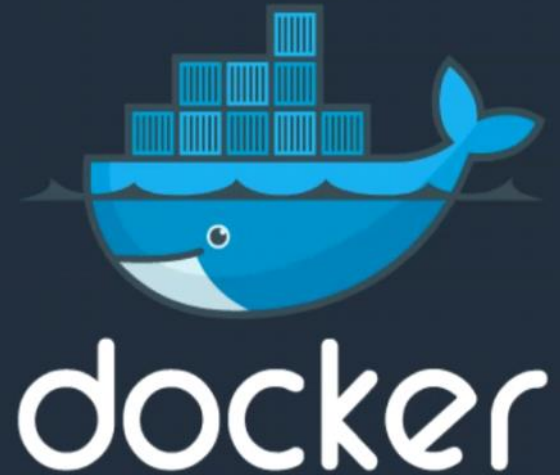
Friday, November 24, 2023 8:55 AM

Setup Docker host

Sunday, October 15, 2023 6:48 PM

Setup Docker Host

- Setup a Linux EC2 Instance
- Install docker
- Start docker services
- Basic docker commands

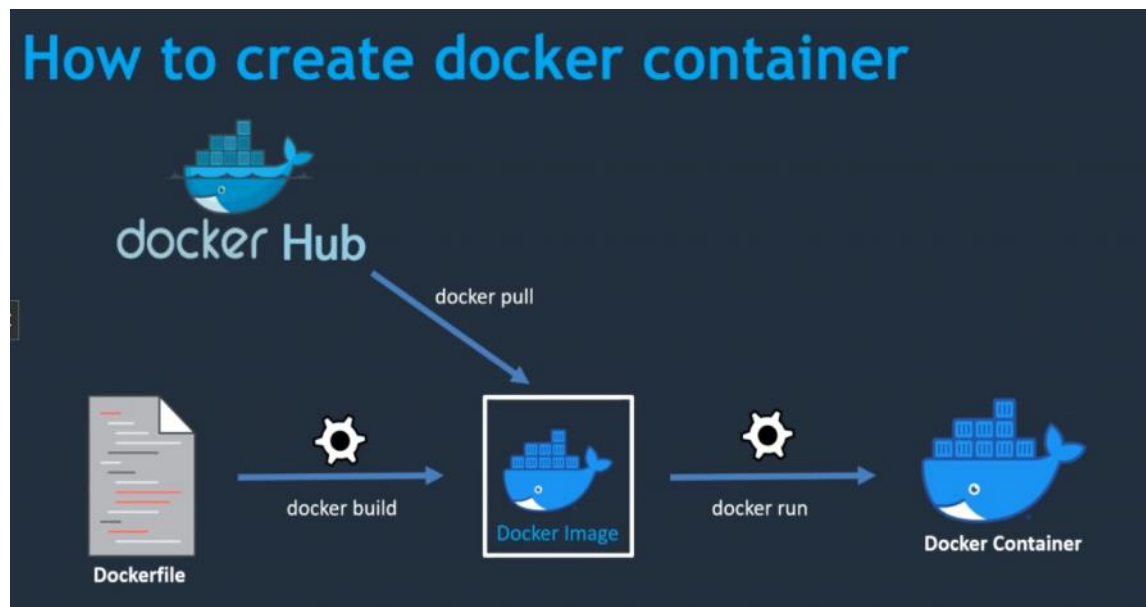


After this setup you will have following three instance

☐	Name	Instance ID
☐	Jenkins_Server	i-0395298512e6388c1
☐	Tomcat_Server	i-013ef454b1094eab5
☒	Docker-Host	i-0515ed458e286eb6b

How to create tomcat docker container

Sunday, October 15, 2023 6:51 PM



Write Your 1st Docker File

- **FROM:** To pull the base image
- **RUN:** To execute commands
- **CMD:** To provide defaults for an executing container
- **ENTRYPOINT:** To configure a container that will run as an executable
- **WORKDIR:** To sets the working directory
- **COPY:** To copy a directory from your local machine to the docker container
- **ADD:** To copy files and folders from your local machine to docker containers
- **EXPOSE:** Informs Docker that the container listens on the specified network ports at runtime
- **ENV:** To set environment variables



Dockerfile

ADD is also used to execute wget command

Install tomcat on Centos

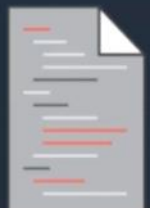
- Pull centos from dockerhub
- Install java
- Create /opt/tomcat directory
- Change work directory to /opt/tomcat
- Download tomcat packages
- Extract tar.gz file
- Rename to tomcat directory
- Tell to docker that it runs on port 8080
- Start tomcat services



Dockerfile

Install tomcat on Centos

- | | |
|--|-------------------|
| • Pull centos from dockerhub | - FROM |
| • Install java | - RUN |
| • Create /opt/tomcat directory | - RUN |
| • Change work directory to /opt/tomcat | - WORKDIR |
| • Download tomcat packages | - ADD /RUN |
| • Extract tar.gz file | - RUN |
| • Rename to tomcat directory | - RUN |
| • Tell to docker that it runs on port 8080 | - EXPOSE |
| • Start tomcat services | - CMD |



Dockerfile

The image shows a terminal window with a dark background and light-colored text. The window title is "Quick connect..." and it shows a connection to IP 2.54.221.91.40. The terminal content is as follows:

```
FROM centos:latest
RUN yum install java -y
RUN mkdir /opt/tomcat
WORKDIR /opt/tomcat
ADD https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.54/bin/apache-tomcat-9.0.54.tar.gz .
RUN tar -xvzf apache-tomcat-9.0.54.tar.gz
RUN mv apache-tomcat-9.0.54/* /opt/tomcat
EXPOSE 8080
CMD ["/opt/tomcat/bin/catalina.sh", "run"]
```

On the left side of the terminal, there is a vertical toolbar with icons for "Sessions", "Tools", "Macros", and "Sftp".

Integrate Docker with Jenkins

Sunday, October 15, 2023 11:05 PM



Integrate Docker with Jenkins

- Create a dockeradmin user
- Install “Publish Over SSH” plugin
- Add Dockerhost to Jenkins “configure systems”



Add dockeradmin(can be any name) user and add it to **Docker** group, only then you can perform all docker related activities

Please note that the copy over Ssh is used to transfer the war file generated in Jenkins to Docker server for deployment

```
[root@dockerhost ~]# useradd dockeradmin
[root@dockerhost ~]# passwd dockeradmin
Changing password for user dockeradmin.
New password:
BAD PASSWORD: The password is shorter than 7 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@dockerhost ~]# id dockeradmin
uid=1001(dockeradmin) gid=1001(dockeradmin) groups=1001(dockeradmin)
[root@dockerhost ~]# usermod -aG docker dockeradmin
[root@dockerhost ~]#
```

Publish over SSH

Jenkins SSH Key

Passphrase



Concealed

Path to key

Key

☐ Disable exec

SSH Servers

Add

Click add new SSH server and configure Docker server

SSH Servers



SSH Server

Name

dockerhost

Hostname

172.31.21.252

Username

dockeradmin

Remote Directory

☐ Use password authentication, or use a different key

Jump host

Port

22

Timeout (ms)

Save

Apply

Now use the configured server and apply the following configurations

Post-build Actions

Send build artifacts over SSH

SSH Publishers

SSH Server

Name ?

dockerhost

Advanced...

Transfers

Transfer Set

Source files ?

webapp/target/*.war

Remove prefix ?

webapp/target

Remote directory ?

//opt/docker

Exec command ?

cd /opt/docker;
docker build -t regapp:v1 .;
docker run -d --name registerapp -p 8087:8080 regapp:v1

Ansible

Saturday, November 25, 2023

10:59 AM



Prepare Ansible Server

- Setup **EC2** instance
- Setup **hostname**
- Create **ansadmin** user
- Add user to **sudoers** file
- Generate **ssh keys**
- Enable **password based** login
- Install **ansible**



Manage DockerHost with Ansible

- **On Docker Host**

- Create ansadmin
- Add ansadmin to sudoers files
- Enable password based login



- **On Ansible Node**

- Add to hosts file
- Copy ssh keys
- Test the connection

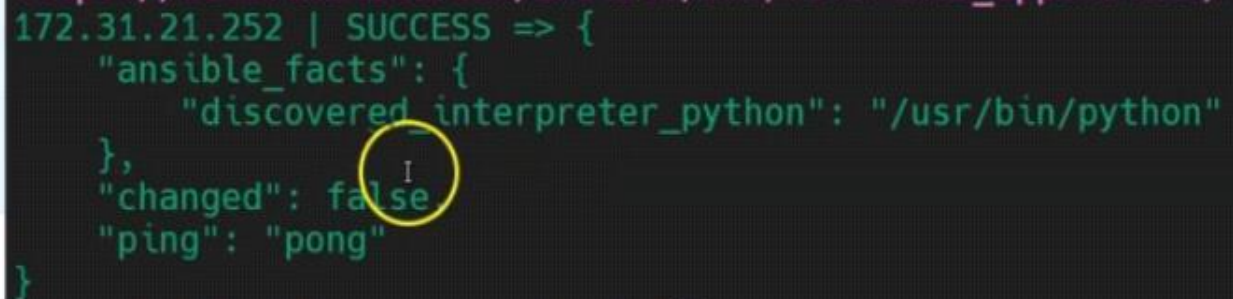


Ansible commands

Saturday, November 25, 2023 11:34 AM

To ping all host that ansible can connect

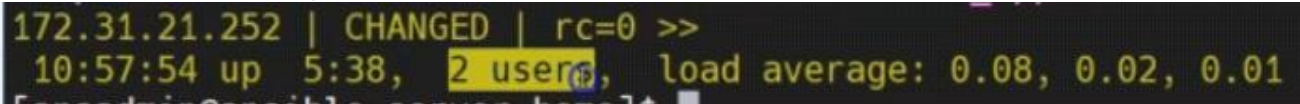
- ansible all -m ping



```
172.31.21.252 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python"
  },
  "changed": false
  "ping": "pong"
}
```

To check uptime of all connected host

- ansible all -m command -a uptime



```
172.31.21.252 | CHANGED | rc=0 >>
10:57:54 up 5:38, 2 users, load average: 0.08, 0.02, 0.01
```

Integrate Ansible with Jenkins

Saturday, November 25, 2023 11:37 AM

Use the same copy over SSH plugin

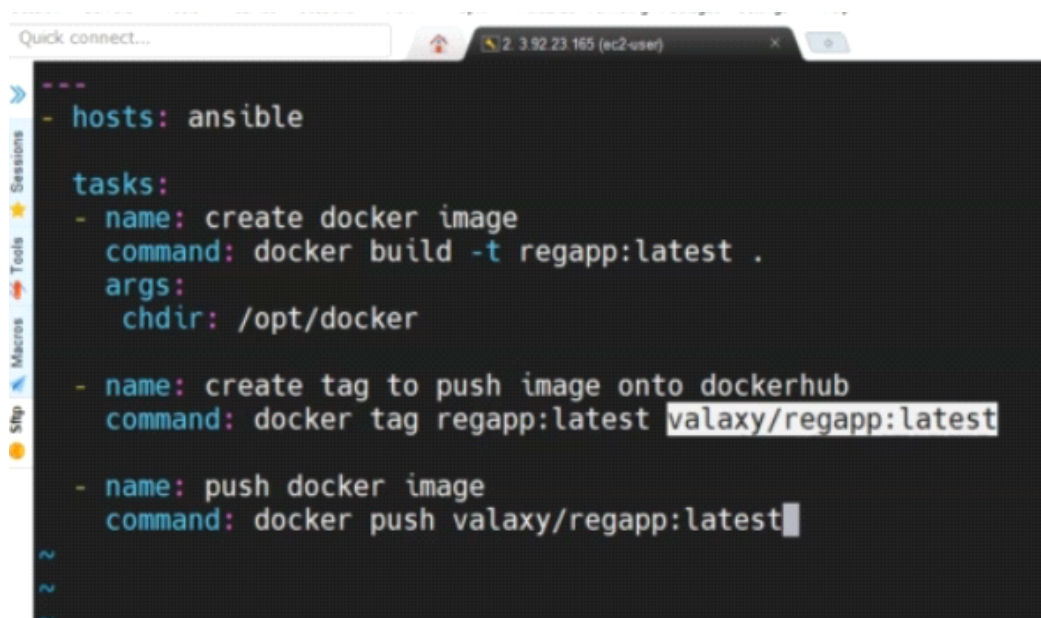
Ansible playbook integration

Saturday, November 25, 2023 10:12 PM

Ansible playbook integration to Upload and download repo to Docker server

Ansible playbook to build and push image to docker repository

Regapp.yaml



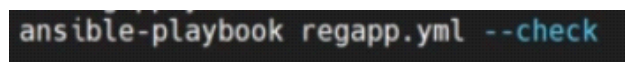
```
---
- hosts: ansible

  tasks:
    - name: create docker image
      command: docker build -t regapp:latest .
      args:
        chdir: /opt/docker

    - name: create tag to push image onto dockerhub
      command: docker tag regapp:latest valaxy/regapp:latest

    - name: push docker image
      command: docker push valaxy/regapp:latest
```

To check if ansible file is without error



```
ansible-playbook regapp.yml --check
```

Deploy to Docker playbook

Deploy-regapp.yaml

```
Quick connect... 2. 3.92.23.165 (ec2-user) x 4. 54.226.49.126 (ec2-user) S
- - -
- hosts: dockerhost

tasks:
- name: stop existing container
  command: docker stop regapp-server
  ignore_errors: yes

- name: remove the container
  command: docker rm regapp-server
  ignore_errors: yes

- name: remove image
  command: docker rmi valaxy/regapp:latest
  ignore_errors: yes

- name: create container
  command: docker run -d --name regapp-server -p 8082:8080 valaxy/regapp:latest

~
```

Kubernetes

Sunday, November 26, 2023 5:07 PM

This open is used as in Docker, there is no option for the application to recover if its down



Deployment & Service file

Monday, November 27, 2023 8:34 AM

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: valaxy-regapp
  labels:
    app: regapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: regapp
  template:
    metadata:
      labels:
        app: regapp
    spec:
      containers:
        - name: regapp
          image: valaxy/regapp
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
      strategy:
        type: RollingUpdate
        rollingUpdate:
          maxSurge: 1
          maxUnavailable: 1
```

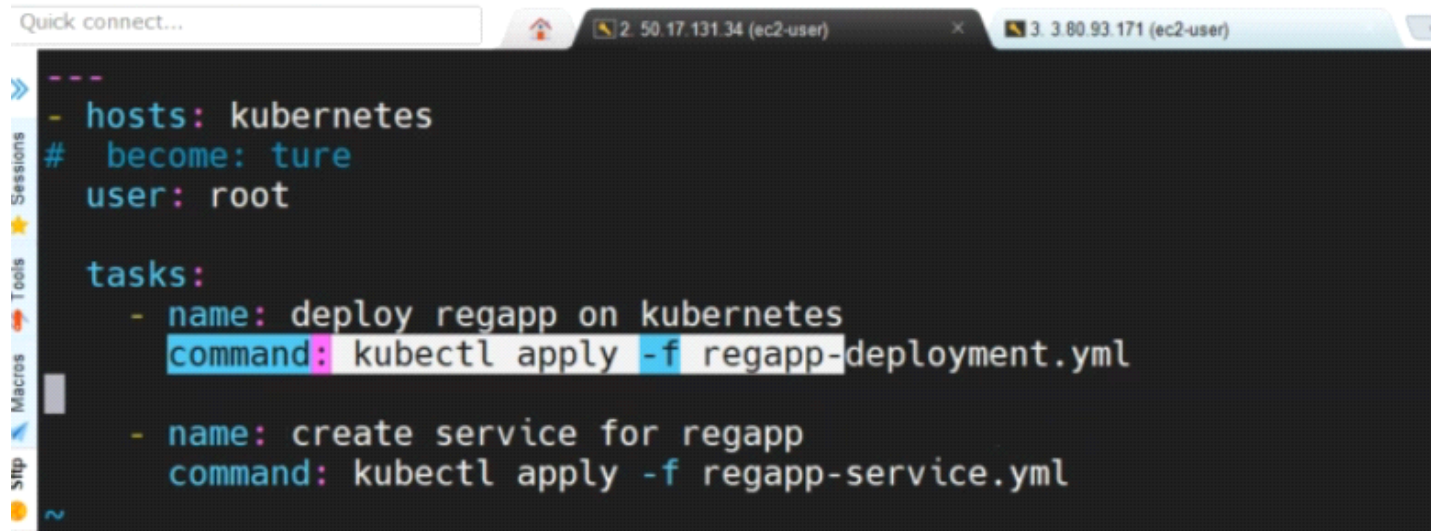
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: valaxy-regapp
  labels:
    app: regapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: regapp
  template:
    metadata:
      labels:
        app: regapp
    spec:
      containers:
        - name: regapp
          image: valaxy/regapp
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
      strategy:
        type: RollingUpdate
        rollingUpdate:
          maxSurge: 1
          maxUnavailable: 1
```

```
apiVersion: v1
kind: Service
metadata:
  name: valaxy-service
  labels:
    app: regapp
spec:
  selector:
    app: regapp
  ports:
    - port: 8080
      targetPort: 8080
  type: LoadBalancer
```



Ansible playbook for K8s Deployment and Service

Monday, November 27, 2023 1:24 PM



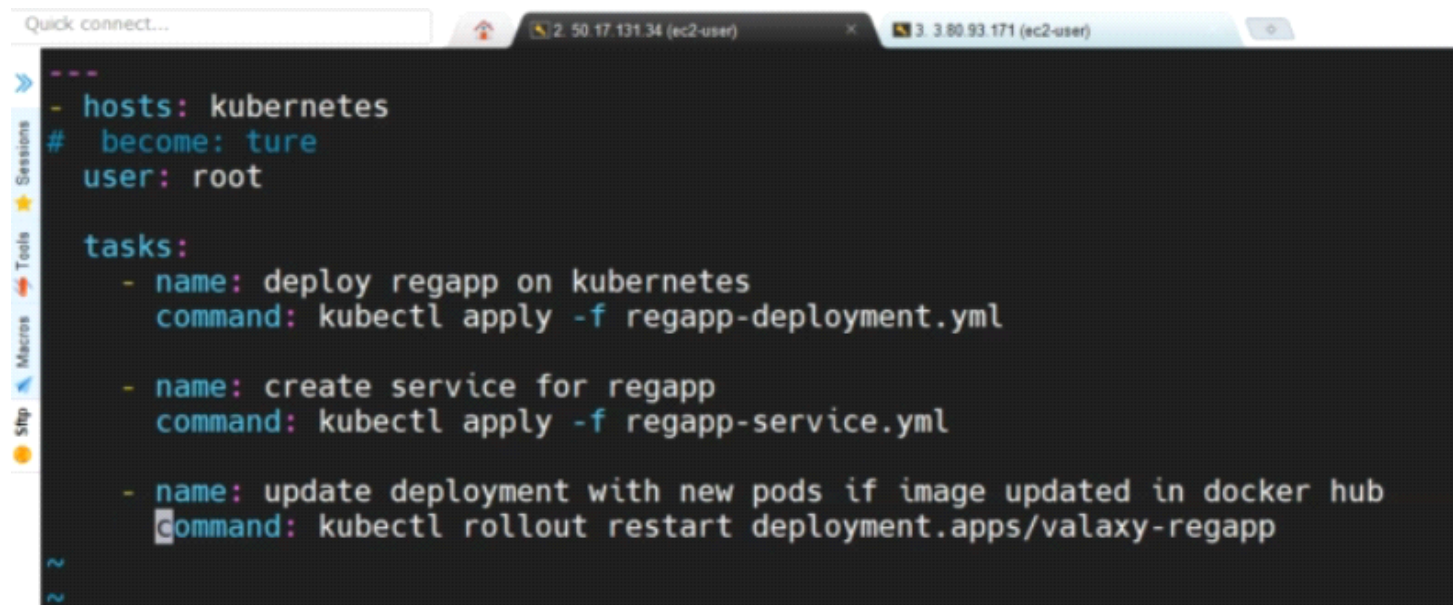
A screenshot of a terminal window showing an Ansible playbook. The terminal has a dark background with light blue and white text. The playbook is written in YAML and includes a 'hosts' section set to 'kubernetes', a 'become' section set to 'ture' (likely a typo for 'true'), and a 'user' section set to 'root'. The 'tasks' section contains two tasks: 'deploy regapp on kubernetes' with the command 'kubectl apply -f regapp-deployment.yml', and 'create service for regapp' with the command 'kubectl apply -f regapp-service.yml'. The terminal window has a 'Quick connect...' search bar at the top and two tabs labeled '2. 50.17.131.34 (ec2-user)' and '3. 3.80.93.171 (ec2-user)'. On the left side, there is a vertical sidebar with icons for 'Sessions', 'Tools', 'Macros', and 'Step'.

```
---
- hosts: kubernetes
  # become: ture
  user: root

  tasks:
    - name: deploy regapp on kubernetes
      command: kubectl apply -f regapp-deployment.yml

    - name: create service for regapp
      command: kubectl apply -f regapp-service.yml
```

To implement rolling update to replace the code with latest image



A screenshot of a terminal window showing an updated Ansible playbook. The terminal has a dark background with light blue and white text. The playbook is written in YAML and includes a 'hosts' section set to 'kubernetes', a 'become' section set to 'ture' (likely a typo for 'true'), and a 'user' section set to 'root'. The 'tasks' section contains three tasks: 'deploy regapp on kubernetes' with the command 'kubectl apply -f regapp-deployment.yml', 'create service for regapp' with the command 'kubectl apply -f regapp-service.yml', and 'update deployment with new pods if image updated in docker hub' with the command 'kubectl rollout restart deployment.apps/valaxy-regapp'. The terminal window has a 'Quick connect...' search bar at the top and two tabs labeled '2. 50.17.131.34 (ec2-user)' and '3. 3.80.93.171 (ec2-user)'. On the left side, there is a vertical sidebar with icons for 'Sessions', 'Tools', 'Macros', and 'Step'.

```
---
- hosts: kubernetes
  # become: ture
  user: root

  tasks:
    - name: deploy regapp on kubernetes
      command: kubectl apply -f regapp-deployment.yml

    - name: create service for regapp
      command: kubectl apply -f regapp-service.yml

    - name: update deployment with new pods if image updated in docker hub
      command: kubectl rollout restart deployment.apps/valaxy-regapp
```

FYI



ChatGPT

The `kubectl rollout restart deployment` command is used to manually trigger a rolling restart of a Kubernetes deployment. This means that it will update the running instances of your application pods with the latest version of the Docker image specified in the deployment configuration.

Pipeline scp

Monday, October 16, 2023 5:36 PM

The screenshot shows the Jenkins Pipeline configuration page for a job named 'pipeline_job'. The 'Pipeline' tab is selected, and the 'Definition' is set to 'Pipeline script'. The script is displayed in a text area with line numbers 1 through 17. The script defines a pipeline with an agent 'any', an environment with a custom PATH, and two stages: 'clone code' and 'build code'. The 'clone code' stage uses the 'git' step with credentials. The 'build code' stage uses the 'sh' step to run 'mvn clean install'. A yellow circle highlights the 'agent any' line in the script.

```
1 pipeline {  
2   agent any  
3   environment {  
4     PATH = "/opt/apache-maven-3.6.3/bin:$PATH"  
5   }  
6   stages {  
7     stage("clone code"){  
8       steps{  
9         git credentialsId: 'git_credentials', url: 'https://github.com:  
10      }  
11    }  
12    stage("build code"){  
13      steps{  
14        sh "mvn clean install"  
15      }  
16    }  
17  }
```

*Stages -> Stage -> Steps

Generate syntax using helper

The screenshot shows the Jenkins Pipeline Syntax helper page for the 'pipeline_job' job. The 'Pipeline Syntax' tab is selected. On the left, there are links to 'Declarative Online Documentation', 'Steps Reference', 'Global Variables Reference', 'Online Documentation', 'Examples Reference', and 'IntelliJ IDEA GDSDL'. On the right, there is a section for 'Steps' with a 'Sample Step' field containing 'sshagent: SSH Agent'. Below this, there is a dropdown menu showing 'ec2-user (tomcat)' and 'ec2-user (deploy-war)' (highlighted with a yellow circle). An 'Add' button is also visible.

```
scp <src_file> username@IP:<dest_path>
```

The screenshot shows a snippet of the Jenkins Pipeline script. It defines a stage 'build code' with a step 'sshagent' that runs the 'scp' command to copy the web application war file to the target server. A yellow circle highlights the 'scp' command in the script.

```
stage("build code"){  
  steps{  
    sshagent(['deploy_user']) {  
      sh "scp webapp/target/webapp.war ec2-user@13.229.183.126:/opt/apache-tomcat-8.5.  
    }  
  }  
}
```

Setup Triggers in Jenkins Pipeline

```

pipeline {
    agent {
        node {
            label 'jenkins-agent-goes-here'
        }
    }
    triggers {
        pollSCM '*/*5 * * * *'
    }
    stages {
        stage('Build') {

```

Alternate Sonar integration

```

stage('Sonar Analysis') {
    steps {
        sh "mvn clean package"
        sh "mvn sonar:sonar -Dsonar.url=http://13.233.155.135:9000/"
    }
}

```

' ' - used to run

"" used to evaluate expression within

DevSecOps

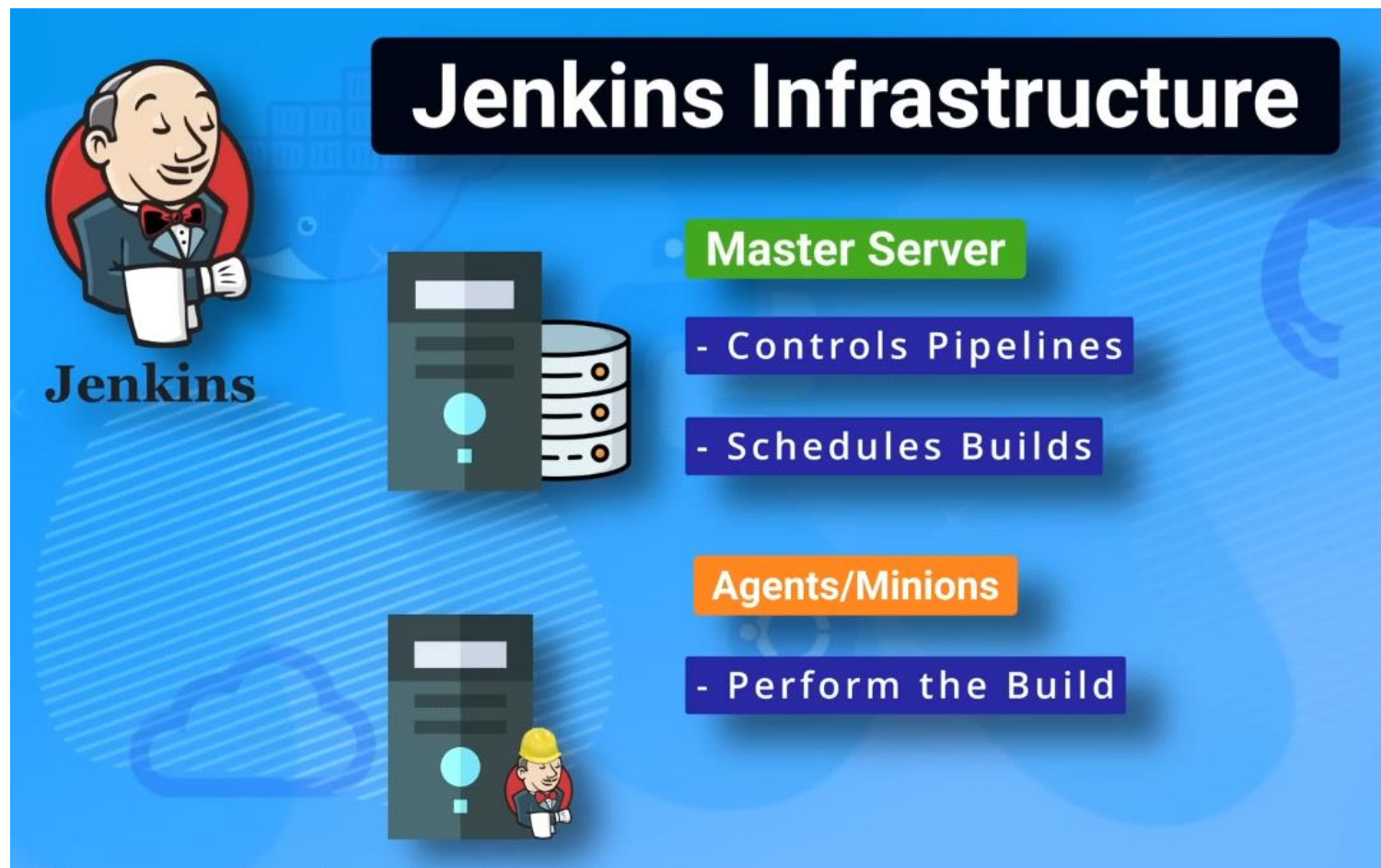
Thursday, October 19, 2023 10:17 AM

<https://github.com/mjameer/DevSecOps/blob/main/Jenkinsfile>

[Jenkins Full Course 2023 | Jenkins Tutorial For Beginners](#)



Jenkins can have One master and multiple slave where in you can control the no of build executions



Jenkins Infrastructure



1. Commit Triggers Pipeline

2. Agent Selected based on Configured Labels

3. Agent Runs Build

Agent Types



Agent Types

Permanent Agents

- Dedicated Servers for Running Jobs



Agent Types



Cloud Agents

- Ephemeral/Dynamic Agents spun up on demand



Build Types

Freestyle Build

- Simplest method to create a build

- 'Feels' like Shell Scripting

Pipelines

- Use the Groovy Syntax

- Use Stages to break down Components of builds

Jenkins Interview Question

Saturday, October 21, 2023 5:50 PM

Always Set Jenkins master executors and agents to 0

Try to use Docker in slave system so that you don't have to maintain separate VM for multiple versions of Java node or maven

Always prefer Declarative Jenkins file over scripted pipeline unless and until you have a strong Jenkins team

What types of agents are you using in Jenkins

- We are using docker agents, since it is light weight and gets deleted after the pipeline completion.

Jenkins Pipeline As Code:

Monday, October 23, 2023 6:06 PM

^ Declarative Pipeline

45 lectures • 3hr 20min

 Overview Of Pipeline Syntax	Preview	01:29
 Hello World Pipeline Script	Preview	01:31
 Pipeline > Agent	Preview	03:59
 Pipeline > Stage > Steps > Script	Preview	02:11
 Pipeline > Stage > Steps > Retry/Timeouts		04:53
 Pipeline > Tools		03:23
 Pipeline > Stage > Options > Error/Retry		03:20
 Pipeline > Stage > Options > Timeout		01:27
 Pipeline > Stage > Options > Timestamps		02:32
 Pipeline > Stage > Options > Timestamps (alternative method)		02:33
 Pipeline > Stage > Options > SkipDefaultCheckout		13:17
 Pipeline > Stage > Environment Credentials		08:38
 Pipeline > Stage > When (Part 1)		12:54
 Pipeline > Stage > When (Part 2) > Branch		03:02
 Pipeline > Stage > When (Part 2) > BuildingTag and Tag		12:34
 Pipeline > Stage > When (Part 2) > Changelog		04:25
 Pipeline > Stage > When (Part 2) > ChangeRequest		11:35
 Pipeline > Stage > When (Part 2) > Changeset		05:37
 Pipeline > Stage > When (Part 3) > BeforeAgent		02:49
 Pipeline > Stage > Parallel/FailFast		03:59
 Pipeline > Stage > Input		08:20
 Pipeline > Stage > Post		12:26
 Pipeline > Options > BuildDiscarder		03:07
 Pipeline > Options > DisableConcurrentBuilds		02:00
 Pipeline > Options > OverrideIndexTriggers		07:00

▶ Pipeline > Options > DisableConcurrentBuilds	02:00
▶ Pipeline > Options > OverrideIndexTriggers	07:09
▶ Pipeline > Options > SkipStagesAfterUnstable	02:55
▶ Pipeline > Options > CheckoutToSubdirectory	02:17
▶ Pipeline > Options > NewContainerPerStage	03:10
▶ Pipeline > Parameters	03:11
▶ Pipeline > Pipeline Triggers > Cron	06:00
▶ Pipeline > Pipeline Triggers > PollSCM	04:32
▶ Pipeline > Pipeline Triggers > Upstream	04:57
▶ Pipeline > Agent > Docker > Image	02:27
▶ Pipeline > Agent > Docker > Args	01:36
▶ Pipeline > Agent > Docker > AlwaysPull	01:59
▶ Pipeline > Agent > Docker > CustomWorkspace	01:36
▶ Pipeline > Agent > Docker > RegistryUrl/CredentialsId	06:24
▶ Pipeline > Agent > Docker > Label	02:05
▶ Pipeline > Agent > Docker > ReuseNode	03:30
▶ Pipeline > Agent > Dockerfile	02:53
▶ Pipeline > Agent > Dockerfile > Dir	02:40
▶ Pipeline > Agent > Dockerfile > Filename	02:18
▶ Pipeline > Agent > Dockerfile > AdditionalBuildArgs	02:25
▶ Pipeline > Agent > Dockerfile > CustomWorkspace	02:48
▶ Groovy Script Outside of Declarative Pipeline Script	01:19

^ Shared Libraries

- ▶ Global Libraries
- ▶ Importing Libraries

^ Shared Libraries

- ▶ Global Libraries
- ▶ Importing Libraries
- ▶ Importing Class Libraries
- ▶ Importing Static Libraries
- ▶ Custom Steps
- ▶ Closures
- ▶ Importing Third Party Libraries
- ▶ Loading Resources
- ▶ Defining Pipeline Script In Call Method
- 💡 Test Your Knowledge!

^ Scripted Pipeline

- ▶ Introduction
- ▶ Docker > Image
- ▶ Docker > Args
- ▶ Docker > AlwaysPull
- ▶ Docker > CustomWorkspace
- ▶ Docker > RegistryUrl/CredentialsId
- ▶ Dockerfile
- ▶ Dockerfile > Dir
- ▶ Dockerfile > Filename
- ▶ Dockerfile > AdditionalBuildArgs
- ▶ Dockerfile > CustomWorkspace
- ▶ Stage > Options > Error/Retry

- ▶ Dockerfile > CustomWorkspace
- ▶ Stage > Options > Error/Retry
- ▶ Stage > Options > Timeout
- ▶ Stage > Options > Timestamps
- ▶ Stage > Environment Credentials
- ▶ Stage > When (Part 1)
- ▶ Stage > When (Part 2) > Branch
- ▶ Stage > When (Part 2) > BuildingTag and Tag
- ▶ Stage > When (Part 2) > Changelog
- ▶ Stage > When (Part 2) > ChangeRequest
- ▶ Stage > When (Part 2) > ChangeSet
- ▶ Stage > Parallel/FailFast
- ▶ Stage > Input
- ▶ Stage > Post
- ▶ Options > BuildDiscarder
- ▶ Options > DisableConcurrentBuilds

- ▶ Options > OverrideIndexTriggers
- ▶ Options > SkipStagesAfterUnstable
- ▶ Options > CheckoutToSubdirectory
- ▶ Options > NewContainerPerStage
- ▶ Parameters
- ▶ Tools
- ▶ Triggers > Cron
- ▶ Triggers > PollSCM
- ▶ Triggers > Upstream

Jenkins Declarative Pipeline

Monday, October 23, 2023 7:54 AM

Build and wait plugin has options of timeout

Error/Retry

This code will be retried 3 times due to error call

```
pipeline{
  agent any
  stages{
    stage("Build"){
      options{
        retry(3)
      }
      steps{
        echo "Before error statement"
        error "Error statement just got executed"
        echo "After error statement"
      }
    }
  }
}
```

Option/Timeout

Definition

Pipeline script

Script

```
1 pipeline{
2   agent any
3
4   stages {
5     stage('Build') {
6       options {
7         timeout(time: 1, unit: 'SECONDS') //values: NANOSECONDS, MICROSECONDS, MILLISECONDS, SECONDS, MINUTES, HOURS, DAYS
8       }
9       steps {
10        echo 'Hello World'
11        sleep 2
12      }
13    }
14  }
15 }
16
```

Timestamp

Definition

Pipeline script

Script

```
1 pipeline{
2   agent any
3
4   options {
5     timestamps()
6   }
7
8   stages{
9     stage('Build'){
10
11     steps{
12       echo "Hello World"
13       echo "Hello World Again"
14     }
15   }
16 }
```

Console Output

```
[Pipeline] echo
12:57:59 Hello World
[Pipeline] echo
12:57:59 Hello World Again
```



Environment :

Definition

Pipeline script

```
Script
1 pipeline {
2     agent any
3     environment {
4         name1 = "Jeff"
5         name2 = "John"
6     }
7     stages {
8         stage('Build') {
9             environment {
10                 name3 = "Jamie"
11             }
12         }
13         steps {
14             echo "name1 ${name1}"
15             echo "name2 ${name2}"
16         }
17     }
18 }
```

Credentials :

Definition

Pipeline script

```
Script
1 pipeline {
2     agent any
3     stages {
4         stage('Build') {
5             environment {
6                 SOME_USERNAME_PASSWORD = credentials('some_username_password')
7                 SOME_SECRET = credentials('some_secret')
8             }
9             steps {
10                 echo "SOME_USERNAME_PASSWORD ${SOME_USERNAME_PASSWORD}" //SOME_USERNAME_PASSWORD is in the format of username:password
11                 echo "SOME_USERNAME_PASSWORD_USR ${SOME_USERNAME_PASSWORD_USR}"
12                 echo "SOME_USERNAME_PASSWORD_PSW ${SOME_USERNAME_PASSWORD_PSW}"
13                 echo "SOME_SECRET ${SOME_SECRET}"
14             }
15         }
16     }
17 }
18 }
```

When :

Definition

Pipeline script

```
Script
1 pipeline {
2     agent any
3     environment {
4         DEPLOY_TO='production'
5     }
6     stages {
7         stage('Build') {
8             when {
9                 environment name: 'DEPLOY_TO', value: 'production'
10             }
11         }
12         steps {
13             echo 'Deploying'
14         }
15     }
16 }
```

Example 2

```

agent any
environment{
    some_name="jeff"
}
stages {
    stage('Build') {
        when {
            equals expected: "jeff", actual: some_name
        }
        steps {
            echo "${some_name}"
        }
    }
}

```

Example 3: when not

```

pipeline {
    agent any
    environment{
        some_name="jeff"
    }
    stages {
        stage('Build') {
            when {
                not{
                    equals expected: "jeff", actual: some_name
                }
            }
            steps {
                echo "${some_name}"
            }
        }
    }
}

```

Example 4: All of

Pipeline script

Script	Code
1	pipeline {
2	agent any
3	environment{
4	version="1.0"
5	some_name="jeff"
6	}
7	stages {
8	stage("Build") {
9	when {
10	allOf {
11	environment name: "version", value: "1.0"
12	environment name: "some_name", value: "jeff"
13	}
14	}
15	steps {

Example 5: Branch (keyword)

```

pipeline {
    agent any
    stages {
        stage('Build Master') {
            when {
                branch 'master'
            }
            steps {
                echo 'Building master'
            }
        }
        stage('Build Dev') {
            when {
                branch 'dev'
            }
            steps {
                echo 'Building dev'
            }
        }
    }
}

```


Example 6: buildingTag

```
Branch: master | pipeline-stage-when-buildingTag-tag / Jenkinsfile

pipelinescode pipeline-buildingTag-tag
0 contributors

16 lines (13 sloc) | 218 Bytes

1 pipeline {
2   agent any
3
4   stages {
5     stage('Build') {
6
7         when(
8             buildingTag()
9         )
10
11     steps {
12         echo 'Hello world building tag'
13     }
14 }
15 }
16 }
```

Example 6: Changelog

```
pipeline {
    agent any

    stages {
        stage('Build') {

            when(
                changing "%some.txt%"
            )

            steps {
                echo 'Hello World changelog'
            }
        }
    }
}
```

Post

Can be at end of the stage or end of the pipeline

```
post{
    //Only runs if the current Pipeline's or stage's run has a "success" status.
    success{
        echo 'post->success is called'
    }
    //Only runs if the current Pipeline's or stage's run has a "Failed" status
    failure{
        echo 'post->failure is called'
    }
    //Runs regardless of the completion status of the Pipeline's or stage's run.
    always{
        echo 'post->always is called'
    }
}
```

Disable Concurrent Build

```
agent any
options {
    disableConcurrentBuilds()
}
```

Trigger

Monday, October 23, 2023 5:34 PM

```
pipeline {
    agent any
    triggers {
        //minute, hour, day of month, month, and day of week (0 and 7 are sundays)
        cron('0 1 2 3 1-7')

        //cron('H */2 1-3 1,3 2-4')

        //@yearly, @annually, @monthly, @weekly, @daily, @midnight, @hourly
        //cron('@daily')

        //cron('TZ=Europe/London\n0 1 * * 1-7')
    }
}
```

```
pipeline {
    agent any
    triggers {
        pollSCM('* * * * *')
    }
    stages {
        stage('Build') {
            steps {
                checkout([$class: 'GitSCM',
                    branches: [[name: "origin/master"]],
                    userRemoteConfigs: [[
                        url: 'https://github.com/pipelineascodecourse/pipeline-triggers-pollSCM.git']]
            )
        }
    }
}
```

Upstream jobs

```
pipeline {
  agent any
  triggers {
    upstream(upstreamProjects: 'pipeline-triggers-upstream-job1,pipeline-triggers-upstream-job2',
      threshold: hudson.model.Result.SUCCESS)//UNSTABLE, FAILURE, NOT_BUILT, ABORTED
  }
  stages {
    stage('Build') {
      steps {
        echo 'pipeline-triggers-upstream executed'
      }
    }
  }
}
```

Jenkins Container

Monday, October 23, 2023 4:45 PM

New Container Per stage

```
pipeline {
  agent {
    docker {
      image 'ubuntu:latest'
    }
  }
  options {
    newContainerPerStage()
  }
  stages {
    stage('Build') {
      steps {
        sh 'cat /etc/lsb-release'
```

```
pipeline {
    agent {
        docker {
            image 'maven:3.5.3-jdk-10-slim'
        }
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn -v'
            }
        }
    }
}
```

Jenkins Shared Lib

Tuesday, October 24, 2023 11:19 AM

Your recently pushed branches:

🔗 dev (9 minutes ago)

Branch: master ▼

New pull request

🔗 pipelineascodecourse shared-lib

📁 resources/com/acceleratedskillup

📁 src/com/acceleratedskillup

📁 vars

Branch: master ▼

pipeline-shared-lib / vars /

🔗 pipelineascodecourse shared-lib

..

📄 execBuildOrDeploy.groovy

📄 execMaven.groovy

📄 executeInScript.groovy

📄 swissknife.groovy

📄 swissknife.txt

Contains
Documentation

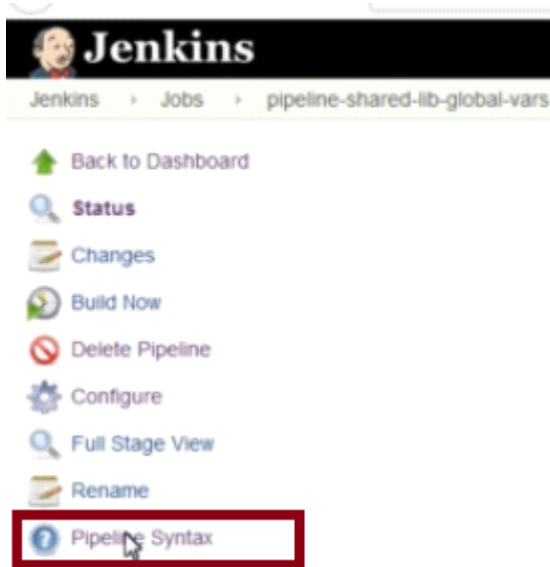
To configure -> Manage Jenkins -> Configuration -> Global configuration -> Refer the Github repo here

```
@Library("pipeline-shared-lib") _

pipeline {
    agent any
    stages{
        stage("Build"){
            steps{
                script{
                    swissknife.printName 'John'
                }
            }
        }
    }
}
```

_ in the first line indicates to include all the Jenkins library within the file

To view the Jenkins Lib - use the following options



Now if you have added the documentation in form of a txt file, then it will appear here

Configure Multiple shared Lib :

```
@Library(['pipeline-shared-lib', 'pipeline-shared-other-lib'])
```

[swissknife](#)

```
printName(name)  
prints out a person's name
```

Lib under Src folder

Tuesday, October 24, 2023 11:37 AM

Branch: master ▾ pipeline-shared-lib / src / com / acceleratedskillup / Calculator.groovy

 pipelineascodecourse shared-lib

0 contributors

6 lines (4 sloc) | 106 Bytes

```
1 package com.acceleratedskillup;
2
3 def add(firstNumber, secondNumber) {
4     return firstNumber+secondNumber
5 }
```

```
@Library("pipeline-shared-lib")
import com.acceleratedskillup.*;
```

```
pipeline {
    agent any
    stages{
        stage('Build'){
            steps{
                script{
                    def calculator = new Calculator()
                    println calculator.add(1,2)
                }
            }
        }
    }
}
```


Class in Src

Tuesday, October 24, 2023 11:38 AM

Branch: master ▾ pipeline-shared-lib / src / com / acceleratedskillup / Car.groovy

 pipelineascodecourse shared-lib

0 contributors

21 lines (16 sloc) | 345 Bytes

```
1  package com.acceleratedskillup;
2
3  class Car {
4      int numberOfDoors
5      String brand
6      String model
7      def script
8
9      Car(script) {
10         this.script = script
11     }
12
13     void printOutCar() {
14         script.echo "using script.echo"
15         script.echo "numberOfDoors:${numberOfDoors}"
16         script.echo "brand:${brand}"
17         script.echo "model:${model}"
18     }
19
20 }
```

Pipeline script

Script

```
1 @Library("pipeline-shared-lib")
2 import com.acceleratedskillup.*;
3
4 pipeline {
5     agent any
6     stages{
7         stage('Build'){
8             steps{
9                 script{
10                     def car = new Car(10) {
11                         car.numberOfDoors = 4
12                         car.brand="brandX"
13                         car.model="modelY"
14                         car.printOutCar()
15                     }
16                 }
17             }
18         }
19     }
20 }
```

Import static lib

Tuesday, October 24, 2023 11:42 AM

Branch: master ▾ [pipeline-shared-lib](#) / [src](#) / [com](#) / [acceleratedskillup](#) / [ValidatorUtils.groovy](#)

 pipelineascodecourse shared-lib

0 contributors

11 lines (6 sloc) | 147 Bytes

```
1 package com.acceleratedskillup;
2
3 class ValidatorUtils {
4
5     static def validateInput(script) {
6         script.echo "in static validateInput()"
7     }
8
9 }
10
```

Pipeline script

Script

```
1 @Library("pipeline-shared-lib")
2 import static com.acceleratedskillup.ValidatorUtils.*;
3
4 pipeline {
5     agent any
6     stages{
7         stage('Build'){
8             steps{
9                 script{
10                     validateInput(this)
11                 }
12             }
13         }
14     }
15 }
```

Custom Scripts

Tuesday, October 24, 2023 11:44 AM

Branch: master ▾

[pipeline-shared-lib](#) / [vars](#) / [execMaven.groovy](#)

 [pipelineascodecourse](#) shared-lib

0 contributors

3 lines (3 sloc) | 59 Bytes

```
1 def call(String options = '-v') {  
2     sh "mvn ${options}"  
3 }
```

Pipeline script

Script

```
1 @Library("pipeline-shared-lib") _  
2  
3 pipeline {  
4     agent any  
5     stages{  
6         stage('Build'){  
7             steps{  
8                 execMaven '--help'  
9                 execMaven()  
10            }  
11        }  
12    }  
13 }  
14 }
```

Loading resource

Tuesday, October 24, 2023 12:29 PM

pipelineascodecourse / pipeline-shared-lib

Code

Issues 0

Pull requests 0

Projects 0

Branch: master

pipeline-shared-lib / resources / com / acce

pipelineascodecourse shared-lib

0 contributors

4 lines (4 sloc) 30 Bytes

```
1 {
2   name:"test",
3   height:1.55
4 }
```

Pipeline

Definition

Pipeline script

Script

```
1 @Library("pipeline-shared-lib") _
2
3 pipeline {
4   agent any
5   stages{
6     stage('Build'){
7       steps{
8         script{
9           def request = libraryResource "com/acceleratedskillup/request.json"
10          echo request
11        }
12      }
13    }
14  }
15 }
```

Use Groovy Sandbox

Scripted Pipeline

Tuesday, October 24, 2023 12:34 PM

Scripted pipeline is a more powerful way to create jenkins pipeline as it allows the use of groovy in Jenkins

Docker -> Image

```
node {
    stage('Build'){
        def mavenImage = docker.image('maven:3.5.3-jdk-10-slim')

        mavenImage.inside{
            sh 'mvn -v'
        }
    }
}
```

Docker -> Args

```
node {
    stage('Build'){
        def mavenImage = docker.image('maven:3.5.3-jdk-10-slim')

        mavenImage.inside('-e someEnv=dev', {
            sh 'echo $someEnv'
        })
    }
}
```

Always Pull

```
node {
    stage('Build'){
        def mavenImage = docker.image('maven:3.5.3-jdk-10-slim')

        mavenImage.pull()

        mavenImage.inside('-e someEnv=dev', {
            sh 'echo $someEnv'
        })
    }
}
```

Docker with Registry

```
node {
    stage('Build'){
        def mavenImage = docker.withRegistry('https://quay.io', 'quay.io'){
            docker.image('quay.io/pipelineascodcourse/myfirstrepo:latest').inside {
                sh 'printenv'
            }
        }
    }
}
```

With Dockerfile

pipelineascodecourse / pipeline-agent-dockerfile

[Code](#)[Issues 0](#)[Pull requests 0](#)[Projects 0](#)

Branch: master ▾

[pipeline-agent-dockerfile / Dockerfile](#) pipelineascode Dockerfile

0 contributors

1 lines (1 sloc) | 18 Bytes

1 FROM ubuntu:latest

Pipeline

Definition

Pipeline script

Script

```
1 node {
2     stage('Build'){
3         checkout([$class: 'GitSCM',
4             branches: [[name: "origin/master"]],
5             userRemoteConfigs: [[
6                 url: 'https://github.com/pipelineascodecourse/pipeline-agent-dockerfile.git']]
7         ])
8     }
9     def myCustomUbuntuImage = docker.build("my-ubuntu:my-latest")
10
11     myCustomUbuntuImage.inside {
12         sh 'cat /etc/lsb-release'
13     }
14 }
15 }
```

☒ Use Groovy Sandbox

Sample Jenkins Template

Thursday, December 7, 2023 10:33 PM

```
pipeline{
  agent any
  stages{
    stage('Build'){
      steps{
        sh'mvn clean install'
      }
    }
    stage('Test'){
      steps{
        sh'mvn test'
      }
    }
    stage('Deploy'){
      when{
        branch'master'
      }
      steps{
        sh'mvn deploy'
      }
    }
  }
  post{
    always{
      sh"echo pipeline completed"
    }
    success{
      sh"echo pipeline succeeded"
    }
    failure{
      sh"echo pipeline failed"
    }
  }
}
```

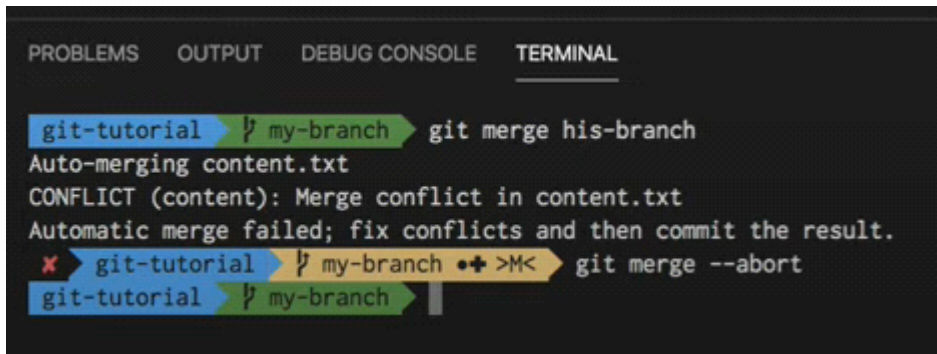
```
FROM alpine:latest
RUN apk add --update nodejs npm
WORKDIR /app
COPY ./app
Run npm install
EXPOSE 3000
CMD["npm","start"]
```

```
version: '3'
services:
  app:
    restart: always
    image: adslocal.artifactory-pharmacy.kp.org/smt_api:${IMAGE_TAG}
    container_name: ${CONTAINER_NAME}
    volumes:
      - ${LOG_DIR}:/usr/local/tomcat/logs
      - ${CERT_DIR}:/usr/local/tomcat/cert.pfx
    ports:
      - ${HTTP}:8080
      - ${HTTPS}:8443
    environment:
      - TZ=PST8PDT
      - ADDITIONAL_JAVA_OPTS=-Dtomcat.keystore.filename=${KS_FLNAME} -Dtomcat.keystore.password=${KS_PASS} -Dtomcat.keystore.type=${KS_TV}
```

ENV CATALINA_OPTS="-Xms512m -Xmx1024m"

Git merge

Tuesday, December 19, 2023 10:08 AM



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, and TERMINAL. The TERMINAL tab is active. The terminal output shows a Git merge command being executed, followed by an auto-merge attempt for content.txt. A conflict is detected, and the user is prompted to resolve it. The user then aborts the merge using `git merge --abort`.

```
git-tutorial my-branch git merge his-branch
Auto-merging content.txt
CONFLICT (content): Merge conflict in content.txt
Automatic merge failed; fix conflicts and then commit the result.
git-tutorial my-branch git merge --abort
git-tutorial my-branch
```

Commit resolved changes

Use IDE or Online

Git Branches

Sunday, June 11, 2023 4:57 PM

feature - members to work on different feature

release - Branch to be deployed to prod

hotfix - For Immediate prod fix

master - Should always be up to date with all of the above branch.

Git MERGE vs REBASE

Tuesday, December 19, 2023 10:14 AM



ChatGPT

``git merge`` and ``git rebase`` are both Git commands used to integrate changes from one branch into another, but they do so in different ways. Here's an overview of the key differences between them:

1. ``git merge``:

- Merges changes from a source branch into a target branch.
- Creates a new commit that has two parent commits – one from the target branch and one from the source branch.
- Preserves the commit history of both branches.

bash

Copy code

```
# Example usage of git merge
git checkout target_branch
git merge source_branch
```

SCSS

Copy code

```
o---o---o---o---o (target_branch)
 \
  o---o---o (source_branch)
```

2. `git rebase`:

- Integrates changes by moving or combining a sequence of commits from one branch onto another.
- Results in a linear commit history, as it appears as if the changes in the source branch were made directly on top of the target branch.
- Can be used to create a cleaner, more linear history but should be used with caution on shared branches, as it rewrites commit history.

bash

 Copy code

```
# Example usage of git rebase
git checkout source_branch
git rebase target_branch
```

SCSS

 Copy code

```
o---o---o---o---o (target_branch)
      \
      o---o---o (source_branch)
```

When to use `git merge`:

- Use `git merge` when you want to preserve the existing commit history of both branches.
- It's a safe option for shared branches where commit history is important, as it avoids rewriting history.

When to use `git rebase`:

- Use `git rebase` when you want a clean and linear commit history.
- It's useful for feature branches or when preparing a branch for integration into a shared repository, but it should be avoided on branches that are shared and already pushed.

Considerations:

- **Avoid rebasing already shared branches:** If you've already pushed the branch to a shared repository, it's generally not recommended to rebase it, as it can cause confusion and conflicts for collaborators.
- **Rebasing is rewriting history:** Be cautious with rebasing, as it rewrites commit history. If others are working on the same branch, coordinating and communicating is crucial to avoid conflicts.

In summary, both `git merge` and `git rebase` are valuable tools, and the choice between them depends on the specific use case and the desired outcome for your Git repository's history.